

PVR Cloud

03/22/2026 12:58:09

Thought process: Orchestrated comprehensive cloud overview covering platforms,...

Thought process: Orchestrated comprehensive Azure Data Factory curriculum...

Thought process: Audited PDF content comprehensively and identified multiple...

Thought process: Orchestrated comprehensive Databricks curriculum spanning...

Thought process: Orchestrated comprehensive Azure Synapse curriculum aligned...

Thought process: Orchestrated comprehensive Microsoft Fabric curriculum mapping...

Thought process: Orchestrated comprehensive cloud overview covering platforms, Azure, and big data fundamentals.

The user wants me to explain the "Overview of Cloud" section from the PDF in depth with real-world examples. Let me cover all the topics under Overview of Cloud:

1. Basics of Cloud Computing
2. Cloud Computing Platforms/Vendors
3. Introduction to Azure
4. Overview of Azure Resources/Services
5. Introduction to BigData

Let me provide a comprehensive, in-depth explanation.

Overview of Cloud — Complete In-Depth Guide

Basics of Cloud Computing

What is Cloud?

Cloud computing means using computing resources — servers, storage, databases, networking, software — **over the internet**, instead of owning and maintaining physical hardware yourself.

Real-World Analogy: Think of electricity. You don't build your own power plant at home — you simply plug into the grid and pay for what you use. Cloud computing works the same way. You "plug into" a provider's infrastructure and pay only for the resources you consume.

Before Cloud (Traditional IT):

- A company needed to buy physical servers
- Set up a data center with cooling, power, security
- Hire IT staff to maintain it
- If traffic spiked (e.g., Black Friday), they had to pre-buy extra servers that sat idle 11 months a year

With Cloud:

- No upfront hardware cost
 - Scale up/down instantly
 - Pay only for what you use
 - Access from anywhere in the world
-

◆ **Types of Cloud Deployment Models**

A. Private Cloud

A cloud environment **dedicated exclusively to one organization**. The infrastructure is either on-premises or hosted by a third party, but it is not shared with anyone else.

Real-World Example: A large bank like HDFC or JPMorgan runs its own private cloud. Banking regulations require that customer financial data never leaves their controlled environment. So they build a private cloud within their own data centers.

Characteristics:

- Full control over security & compliance
 - High cost (you maintain everything)
 - Best for: regulated industries — banking, healthcare, government
-

B. Public Cloud

Cloud resources are **owned and operated by a third-party provider** (like Microsoft Azure, AWS, GCP) and **shared across multiple customers** over the internet. Each customer's data is logically isolated.

Real-World Example: Netflix runs almost entirely on AWS public cloud. When millions of users stream simultaneously, Netflix doesn't own those servers — AWS provides them on demand. Netflix scales from 1 million to 100 million streams within minutes.

Characteristics:

- No hardware to manage
 - Pay-as-you-go pricing
 - Massive scalability
 - Best for: startups, SaaS products, web applications
-

C. Hybrid Cloud

A **combination of private and public cloud**, where data and applications can move between them.

Real-World Example: A hospital stores patient records on a private cloud (for HIPAA compliance), but uses the public cloud (Azure) for running AI diagnostics on anonymized data and for their patient appointment booking website.

Characteristics:

- Sensitive data stays private
 - Non-sensitive workloads use public cloud
 - Best for: enterprises with compliance needs but wanting cloud flexibility
-

◆ Types of Cloud Services

A. IaaS — Infrastructure as a Service

The cloud provider gives you **raw infrastructure**: virtual machines, storage, networking. You manage the OS, middleware, runtime, and everything above it.

Real-World Analogy: Renting an empty apartment. The building (infrastructure) is maintained by the landlord, but you furnish and decorate it yourself.

Example: Azure Virtual Machines (VMs). A company rents a VM on Azure, installs Windows Server, SQL Server, and their application on it. They manage patches, configurations, and scaling.

You manage: OS, middleware, runtime, applications, data **Provider manages:** Virtualization, servers, storage, networking

B. PaaS — Platform as a Service

The cloud provider manages the **infrastructure + OS + runtime**. You just bring your **code and data**.

Real-World Analogy: Renting a fully furnished apartment. You just move in with your clothes — no need to buy furniture or worry about plumbing.

Example: Azure SQL Database. You don't manage any server, OS, or SQL Server installation. You just create a database, connect, and run queries. Azure handles patching, backups, high availability automatically.

Another Example in Data Engineering: Azure Data Factory, Azure Databricks — you write pipelines and notebooks, Azure manages all the underlying infrastructure.

You manage: Applications, data **Provider manages:** OS, runtime, middleware, infrastructure

C. SaaS — Software as a Service

The cloud provider delivers a **complete, ready-to-use software application** over the internet.

Real-World Analogy: Staying in a hotel. Everything is handled — room, cleaning, food. You just show up and use it.

Examples:

- Gmail / Outlook 365 — you don't manage any server, just use the email service
- Salesforce CRM — ready-made CRM, you just enter your customer data
- Microsoft Teams / Zoom — video conferencing, fully managed

You manage: Just your data and user configurations **Provider manages:** Everything else

Quick Comparison Table

Model	You Manage	Provider Manages	Example
IaaS	OS, Apps, Data	Hardware, Network	Azure VMs
PaaS	Apps, Data	OS, Runtime, HW	Azure SQL DB, ADF
SaaS	Data only	Everything	Office 365, Gmail

Cloud Computing Platforms / Vendors

There are three dominant public cloud providers in the world:

1. Microsoft Azure

- Launched: 2010
- Owned by: Microsoft
- Market share: ~25% (2nd largest)

- **Strength:** Enterprise integration (Active Directory, Office 365, Windows Server), hybrid cloud, data analytics (ADF, Databricks, Synapse)
 - **Real-World Use:** Most Fortune 500 enterprises that already use Microsoft products (Windows, SQL Server) migrate to Azure naturally
 - **Key Data Engineering Services:** Azure Data Factory, Azure Databricks, Azure Synapse Analytics, Microsoft Fabric
-

◆ 2. AWS — Amazon Web Services

- Launched: 2006
 - Owned by: Amazon
 - Market share: ~33% (largest)
 - **Strength:** Widest service catalog, most mature platform, dominant in startups and tech companies
 - **Real-World Use:** Netflix, Airbnb, Uber, NASA all use AWS
 - **Key Data Engineering Services:** AWS Glue (similar to ADF), Amazon EMR (similar to Databricks), Amazon Redshift (similar to Synapse)
-

◆ 3. GCP — Google Cloud Platform

- Launched: 2011
- Owned by: Google
- Market share: ~11% (3rd)
- **Strength:** AI/ML capabilities (TensorFlow, Vertex AI), BigQuery (fastest serverless data warehouse), data analytics
- **Real-World Use:** Twitter, Spotify, PayPal use GCP
- **Key Data Engineering Services:** BigQuery, Dataflow, Dataproc, Pub/Sub

🔥 Why Azure for Data Engineering?

- Seamlessly integrates with on-premise SQL Server and Windows-based systems
 - Azure Active Directory for enterprise-level security
 - ADF + Databricks + Synapse forms a complete, powerful data pipeline ecosystem
 - Most large enterprises (banks, insurance, healthcare) prefer Azure for compliance reasons
-

3 Introduction to Azure

◆ Azure Portal Walkthrough

The **Azure Portal** (portal.azure.com) is a web-based, unified console where you can manage all your Azure resources.

Real-World Analogy: Think of the Azure Portal as the dashboard of a car. All controls — engine (compute), fuel (storage), navigation (networking) — are accessible from one place.

Key sections in Azure Portal:

- **Dashboard** — Customizable home screen with your pinned resources
 - **All Resources** — View all Azure services you've created
 - **Cost Management** — Track and optimize spending
 - **Marketplace** — Browse and deploy pre-built solutions
 - **Cloud Shell** — Built-in command line (bash/PowerShell) right in the browser
-

◆ What is a Subscription?

A **Subscription** is a **billing and access boundary** in Azure. It ties your Azure usage to a payment method.

Real-World Analogy: Think of a subscription like a mobile phone plan. Your phone number (resources) operates under a specific plan (subscription) that determines what you can use and how much you're billed.

Key Points:

- All Azure resources you create belong to a subscription
- One Azure account can have **multiple subscriptions** (e.g., Dev subscription, Test subscription, Production subscription)
- Billing is tracked per subscription
- Access policies and spending limits can be set at the subscription level

Example in a Company:

- Subscription 1: "Development" — Dev team creates test pipelines here, budget capped at \$500/month
- Subscription 2: "Production" — Live pipelines run here, strictly controlled access

◆ What is a Resource Group?

A **Resource Group** is a **logical container** that holds related Azure resources for a solution.

Real-World Analogy: Like a folder on your computer. You create a folder called "Project_Sales" and put all files related to that project inside it. Similarly, a Resource Group named "RG-DataPipeline-Prod" holds all ADF, Storage, SQL, and Databricks resources for your data pipeline project.

Key Points:

- Resources in a group share the same lifecycle — you can deploy, update, and delete them all at once

- Resources in different regions can be in the same Resource Group
- Useful for cost tracking, access control, and management

Example:

```
Resource Group: RG-AzureDataEngineering-Dev
├─ Azure Data Factory: adf-sales-pipeline
├─ Azure Data Lake Storage Gen2: adlsstoragedatalake
├─ Azure SQL Database: sqlldb-salesdata
├─ Azure Databricks: databricks-workspace
└─ Azure Key Vault: kv-secrets-dev
```

◆ What is a Resource?

A **Resource** is any **individual service instance** you create in Azure.

Examples of Resources:

- One Azure Data Factory instance = 1 Resource
- One Azure SQL Database = 1 Resource
- One Azure Storage Account = 1 Resource
- One Azure Databricks Workspace = 1 Resource

Real-World Analogy: If the Subscription is the city, the Resource Group is the building, and Resources are the individual rooms/apartments inside that building.

4 Overview of Azure Resources / Services

These are the core Azure services you'll work with throughout the Data Engineering course:

◆ 1. Azure Data Factory (ADF)

ADF is Azure's **cloud-based data integration service** — like an orchestration engine for moving and transforming data.

Real-World Use Case: An e-commerce company has sales data in 10 different sources: Oracle DB, SAP, CSV files in FTP, REST APIs. Every night at midnight, ADF pulls data from all these sources, transforms it, and loads it into Azure Data Lake. This is classic **ETL/ELT orchestration**.

Think of ADF as: The conductor of an orchestra — it doesn't play an instrument itself but coordinates when and how each instrument (service) plays.

◆ 2. Azure Databricks

A **cloud-based big data processing and analytics platform** built on Apache Spark. Used for large-scale data transformation, machine learning, and streaming.

Real-World Use Case: A telecom company has 10 TB of call detail records (CDRs) daily. Azure Databricks processes all this data in parallel using Spark clusters, applies transformations, detects fraud patterns, and writes results to Delta Lake — all in under 30 minutes.

Think of Databricks as: A supercharged, distributed computing engine that can process terabytes of data by splitting the work across hundreds of machines simultaneously.

◆ 3. BLOB Storage, Data Lake Storage Gen1 and Gen2

Azure Blob Storage: Object storage for any type of unstructured data — images, videos, logs, CSVs, backups.

Azure Data Lake Storage Gen2 (ADLS Gen2): Built on top of Blob Storage but optimized for **big data analytics** with a hierarchical namespace (folder structure), fine-grained access

control, and high throughput.

Real-World Use Case: A healthcare company stores raw patient data files (HL7, CSV) in ADLS Gen2. The Bronze layer holds raw files, Silver holds cleaned data, Gold holds aggregated reports — this is the **Medallion Architecture**.

◆ 4. Azure SQL Server & SQL Database

Azure SQL Server: The logical server that hosts SQL databases in the cloud. **Azure SQL Database:** A fully managed relational database (PaaS) — Microsoft handles patches, backups, and high availability automatically.

Real-World Use Case: A retail company uses Azure SQL Database to store their processed sales transactions. ADF copies data from ADLS Gen2 into Azure SQL Database every hour, which Power BI then reads for dashboards.

◆ 5. Azure Key Vault

A **secure, centralized secrets management service** that stores passwords, connection strings, API keys, certificates.

Real-World Use Case: Your ADF pipeline needs to connect to a SQL Server database. Instead of hardcoding the password in the pipeline (a serious security risk), you store the password in Key Vault as a secret, and ADF retrieves it securely at runtime. If the password changes, you update it in one place — Key Vault — and all pipelines instantly use the new password.

Think of Key Vault as: A bank vault for your digital secrets — only authorized applications and users can access what's inside.

◆ 6. Logic Apps

A **low-code workflow automation service** used to integrate systems and automate business processes.

Real-World Use Case in Data Engineering: When an ADF pipeline fails, a Logic App is triggered that automatically sends an email notification to the data engineering team with details of the failure, the pipeline name, error message, and timestamp — without writing any custom code.

5 Introduction to Big Data

◆ What is Data?

Data is **raw facts and figures** without context. On its own, data has no meaning until processed and analyzed.

Examples:

- **28** — just a number, meaningless alone
 - **Temperature: 28°C, City: Mumbai, Date: 2024-06-01** — now it's meaningful data
-

◆ What is Big Data?

Big Data refers to datasets that are **so large, fast-moving, or complex** that traditional database tools (like MySQL, SQL Server) cannot process them efficiently.

Real-World Examples:

- Facebook generates ~4 petabytes (4,000 TB) of data daily
- YouTube: 500 hours of video uploaded every minute
- A jet engine generates ~1 TB of sensor data per flight

- A stock exchange processes millions of transactions per second

Why Traditional Databases Fail: A standard MySQL database on a single server can handle maybe 100GB–1TB efficiently. When you have 10TB, 100TB, or petabytes of data — you need distributed processing systems like Spark (Databricks).

◆ Data Sources of Big Data

Big data comes from everywhere:

Source	Example
Social Media	Facebook posts, tweets, Instagram photos
IoT Sensors	Smart meters, factory sensors, GPS trackers
Log Files	Web server logs, application logs
Transactional Systems	POS systems, banking transactions
Clickstream Data	User behavior on websites/apps
Machine Data	Industrial machinery sensor readings

◆ Characteristics of Big Data — The 3 V's

Volume

The **sheer size** of the data — terabytes, petabytes, exabytes.

Real-World Example: Walmart collects over 2.5 petabytes of customer transaction data every hour from its stores worldwide. Storing and querying this in a traditional database is impossible — you need distributed storage like ADLS Gen2 and processing like Databricks.

Velocity

The **speed** at which data is generated and needs to be processed.

Real-World Example: A fraud detection system at a bank must analyze each credit card transaction **within milliseconds** of it occurring to approve or block it. This requires real-time/streaming processing. In Azure, this is handled by Databricks Structured Streaming or Azure Event Hubs.

Variety

The **different types and formats** of data.

Real-World Example: An e-commerce platform deals with:

- Structured data: Order table in SQL Database (rows & columns)
- Semi-structured data: Customer reviews in JSON format, product catalog in XML
- Unstructured data: Product images (JPEG), promotional videos (MP4), customer support call recordings (MP3)

All these different formats must be ingested and processed together.

◆ Types of Data

A. Structured Data

Data that has a **predefined schema** — organized in rows and columns.

Examples:

- SQL Database tables (customer_id, name, email, purchase_date)
- Excel spreadsheets
- CSV files with consistent headers

Processing Tools: SQL Server, Azure SQL Database, Synapse Dedicated SQL Pool

B. Semi-Structured Data

Data that **doesn't fit neatly into rows/columns** but has some organizational structure using tags or markers.

Examples:

- **JSON:** `{"customer_id": 101, "name": "Ravi", "orders": [{"id": 1, "amount": 500}]}`
- **XML:** Used in healthcare (HL7), banking (SWIFT messages)
- **Avro, Parquet, ORC:** Columnar file formats used in big data pipelines

Processing Tools: Azure Databricks, ADF Data Flows, Azure Synapse Spark

C. Unstructured Data

Data with **no predefined structure** — the vast majority of data in the world (~80-90%).

Examples:

- Images (product photos, X-rays)
- Videos (surveillance footage, YouTube content)
- Audio (call center recordings, podcasts)
- Free text (customer reviews, emails, social media posts)
- PDF documents

Processing Tools: Azure Cognitive Services (AI), Azure Databricks with ML, Azure Blob Storage for storage

Summary: How Everything Connects

Data Sources (Structured/Semi/Unstructured)



Azure Data Factory ↔ Azure Key Vault (secrets)
(Orchestrate & Ingest)



Azure Data Lake Storage Gen2
(Raw data stored in BLOB/ADLS)



Azure Databricks (Spark)
(Transform, process big data)



Azure Synapse / SQL Database
(Serve processed data)



Power BI / Reports
(Business insights)

All of this runs on **Azure (Public Cloud)**, under a **Subscription**, organized in **Resource Groups**, with each service being a **Resource** — and sensitive credentials securely stored in **Azure Key Vault**.

 **Overview of Cloud section complete.** All 5 major topics — Cloud Basics, Cloud Vendors, Azure Introduction, Azure Services, and Big Data — have been covered with real-world examples and deep conceptual explanations. Ready to proceed to the next section: **Azure Data Factory** whenever you are!

17/03/2026, 21:21:51

Thought process: Orchestrated comprehensive Azure Data Factory curriculum spanning twenty-one interconnected topics.

The user wants me to continue with the next section from the PDF, which is **Azure Data Factory**. Let me cover every single concept from the PDF in depth with proper examples and good formatting.

Let me go through all the ADF topics from the PDF:

- 1) Azure Data Factory (What is ADF, Architecture, Portal UI, Top-level concepts)
- 2) Pipeline
- 3) Linked Service
- 4) DataSets
- 5) Activities
- 6) Triggers
- 7) Integration Runtime (IR)
- 8) Source control
- 9) Global parameters
- 10) Credentials
- 11) Monitoring ADF Jobs
- 12) Alerts
- 13) Send Failure Notifications using Logic Apps

- 14) Data Flows
- 15) Azure Devops
- 16) SDLC
- 17) Agile Methodology
- 18) ADF Interview Questions
- 19) ADF Resume Preparation
- 20) End TO End ADF Project
- 21) ADF Exercises

Now I'm moving into the remaining topics—Data Flows, Azure DevOps integration, SDLC practices, Agile methodology, interview prep, resume tips, and a complete end-to-end project with exercises. I'll provide comprehensive explanations with practical examples and proper formatting for each section.



Azure Data Factory — Complete In-Depth Guide

1 Azure Data Factory — Core Fundamentals

◆ What is Azure Data Factory?

Azure Data Factory (ADF) is Microsoft Azure's **fully managed, serverless cloud-based data integration and orchestration service**. It allows you to create, schedule, and manage data pipelines that can **ingest, transform, and load (ETL/ELT)** data from hundreds of sources to destinations.

In simple terms: ADF is the **plumber of your data system** — it moves water (data) from one place (source) through pipes (pipelines) to another place (destination), sometimes filtering or treating the water (transforming data) along the way.

Real-World Business Problem ADF Solves:

Imagine you work as a Data Engineer at a large retail company. Every single day:

- Sales transactions come in from **Oracle Database** (on-premise)
- Customer data arrives as **CSV files** dropped on an **FTP server**
- Marketing campaign results come from a **REST API**
- Web clickstream data lands in **Azure Blob Storage** as JSON files
- Inventory data sits in **SAP** (on-premise ERP system)

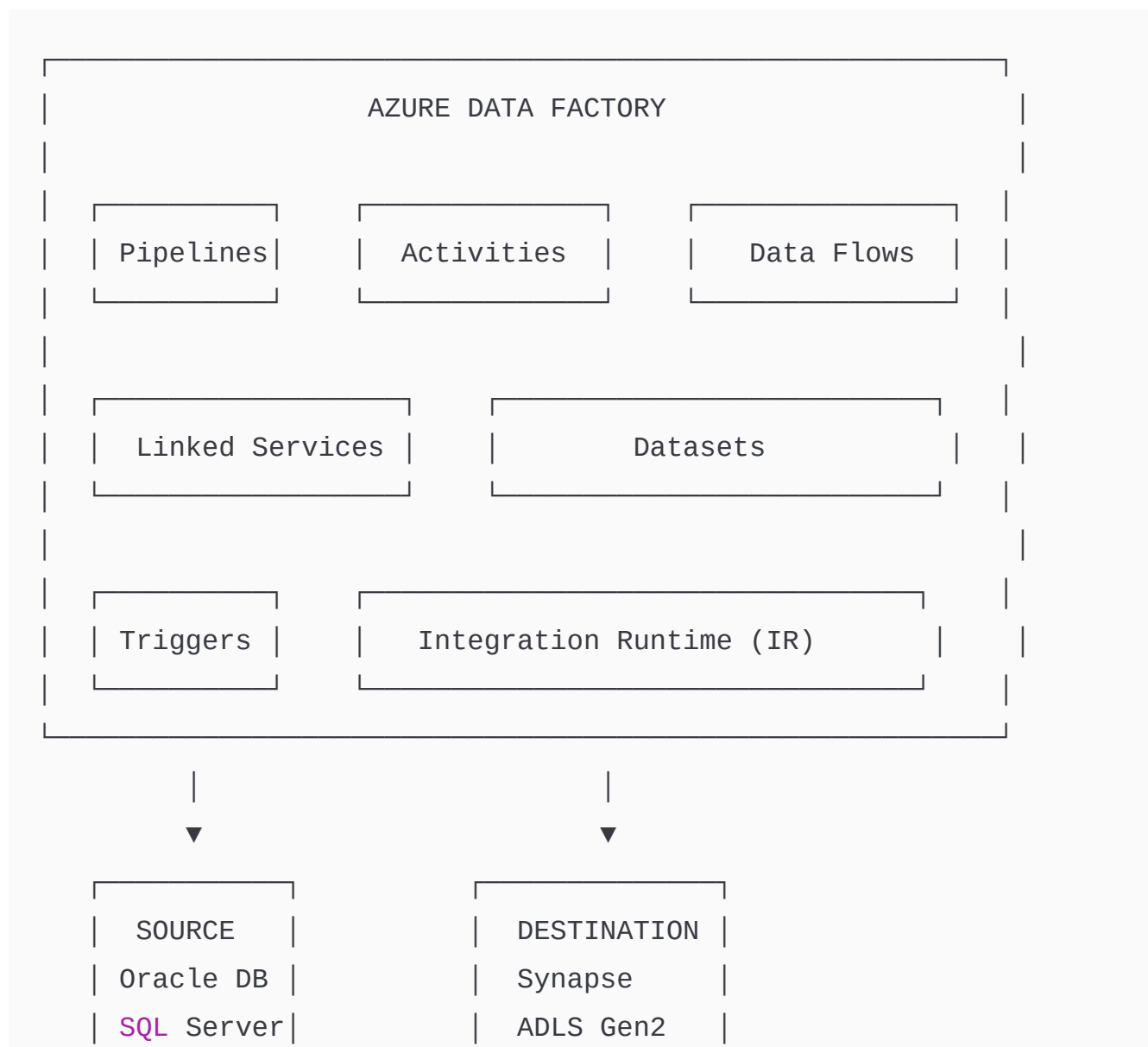
You need to pull all this data, clean it, transform it, join it, and load it into **Azure Synapse Analytics** for reporting — every night at midnight, automatically.

Without ADF: You'd write custom Python scripts on a VM, manage scheduling via cron jobs, handle errors manually, monitor failures yourself.

With ADF: You build visual pipelines that do all of this automatically — with built-in monitoring, alerts, retry logic, and 90+ connectors — zero infrastructure management.

◆ Azure Data Factory Architecture

Understanding ADF's architecture is critical before building pipelines.



CSV Files	SQL DB
REST API	Blob

Key Architecture Layers:

Control Plane (ADF Service): The brain of ADF — manages metadata, monitors execution, schedules triggers. Runs in Azure's managed infrastructure.

Data Plane (Integration Runtime): The actual execution engine — this is what physically moves data. Can run in Azure cloud or on your own network (Self-Hosted IR).

Storage Layer: ADF doesn't store data itself — it just moves it between your source and destination systems.

◆ Azure Data Factory Portal UI

The ADF Portal (adf.azure.com) has several key sections:

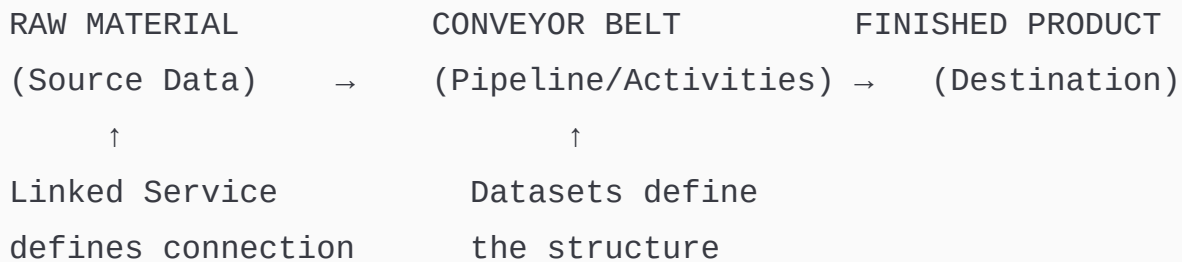
ADF Studio UI

- | 📁 Author → Build pipelines, linked services, datasets, **data** flows
- | 📊 Monitor → Track pipeline runs, activity runs, trigger runs
- | 🔧 Manage → Linked services, IR, triggers, git config, key vault
- | 🏠 Home → Quick access, recent resources, templates

Real-World Tip: The ADF Studio is completely browser-based — no software installation needed. Your entire pipeline code is stored as **JSON under the hood**, even though you build it visually.

◆ Top-Level Concepts in ADF

Think of ADF like a **factory assembly line**:



Let's now explore each concept in depth:

A. Pipelines

A **Pipeline** is a **logical grouping of activities** that together perform a task. It's the top-level container in ADF.

Real-World Analogy: A pipeline is like a recipe. The recipe (pipeline) contains multiple steps (activities): chop vegetables → boil water → cook → serve.

Example:

```
Pipeline: "Daily_Sales_ETL"
├─ Activity 1: GetMetadata (check if file arrived)
├─ Activity 2: If Condition (if file exists → proceed)
├─ Activity 3: Copy Data (move CSV from FTP to ADLS)
├─ Activity 4: Databricks Notebook (run transformations)
└─ Activity 5: Stored Procedure (update audit log)
```

B. Activities

Activities are the **individual steps/tasks** inside a pipeline. Each activity performs a specific action.

Categories of Activities:

- **Data Movement:** Copy Data activity
- **Data Transformation:** Databricks Notebook, Data Flow, Stored Procedure
- **Control Flow:** If Condition, ForEach, Until, Switch, Filter

Real-World Example: In a pipeline that processes daily sales reports, activities might be: check if today's file exists → copy the file from SFTP → transform it in Databricks → load to SQL Database → send success email.

C. Linked Services

Linked Services are **connection strings** — they define how ADF connects to external resources (databases, storage, APIs).

Real-World Analogy: Like saving a contact in your phone. Instead of dialing the full 10-digit number every time, you save "Ravi - SQL Server" and just tap to call. A Linked Service is that saved contact for your data source.

Example Linked Services:

Linked Service 1: LS_AzureSQLDB_Production

- Server: myserver.database.windows.net
- Database: SalesDB
- Auth: Key Vault secret

Linked Service 2: LS_ADLS_Gen2_DataLake

- Account: mydatalakestorage
- Auth: Managed Identity

D. Datasets

Datasets represent the **structure and location of data** — they point to the specific table, file, or folder you want to work with, using a Linked Service for the connection.

Real-World Analogy: If a Linked Service is the address of a building (SQL Server), then a Dataset is the specific room number (specific table or file) within that building.

```
Linked Service: LS_ADLS_Gen2 → (Connection to storage account)
Dataset: DS_SalesCSV → (Points to
/raw/sales/2024/*.csv)
```

E. Triggers

Triggers determine **when a pipeline runs** — they define the schedule or event that fires the pipeline.

Types:

- **Schedule Trigger:** Run every day at midnight
- **Tumbling Window Trigger:** Process hourly time windows of data
- **Storage Event Trigger:** Run when a new file arrives in ADLS

Real-World Example: A trigger fires at 11:00 PM every night to start the overnight data loading pipeline.

F. Data Flows

Data Flows are the **visual data transformation feature** in ADF — like SSIS or Informatica but in the cloud. They use Apache Spark under the hood for execution.

Real-World Use Case: You receive a raw CSV with duplicate customer records, inconsistent date formats, and missing values. A Data Flow can filter duplicates, standardize dates, fill

nulls, and join with another dataset — all visually, without writing code.

G. Integration Runtimes

The **Integration Runtime (IR)** is the **compute infrastructure** that ADF uses to actually run activities and move data.

Types:

- **Azure IR:** Runs in Azure cloud — for cloud-to-cloud data movement
- **Self-Hosted IR:** Installed on your own server — for accessing on-premise databases
- **Azure-SSIS IR:** For running SSIS packages in the cloud

Real-World Example: To copy data from an on-premise Oracle database (inside your company's firewall) to Azure, you install a Self-Hosted IR agent on a machine inside your company network. This agent acts as a secure bridge between your internal network and ADF.

2 Pipeline — Deep Dive

◆ What is a Pipeline?

A Pipeline is the **core unit of work** in ADF. It's a directed acyclic graph (DAG) of activities — meaning activities can run sequentially or in parallel, but there are no circular dependencies.

Real-World Analogy: A pipeline is like an airport. Multiple aircraft (activities) land and take off in a coordinated sequence. Some flights wait for others; some run simultaneously on parallel runways.

◆ Create a New Pipeline

Steps to create a pipeline in ADF Studio:

1. Go to ADF Studio → **Author** tab
2. Click "+" → **Pipeline** → **New Pipeline**
3. Name your pipeline (e.g., `PL_Daily_SalesIngestion`)
4. Drag activities from the Activities panel onto the canvas
5. Connect them with arrows to define execution order

JSON behind the scenes:

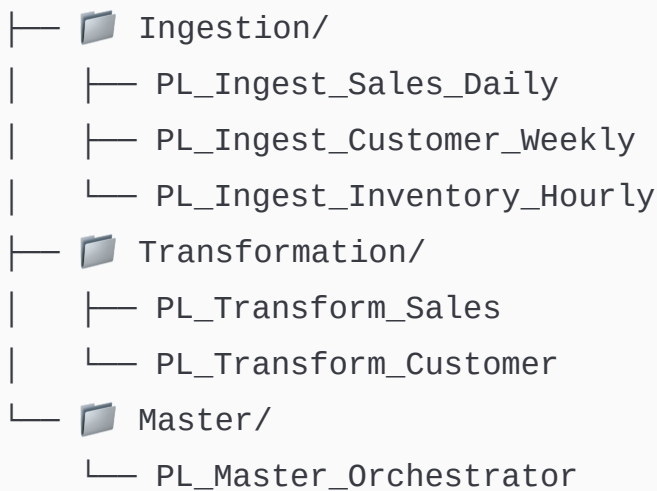
```
{
  "name": "PL_Daily_SalesIngestion",
  "properties": {
    "activities": [
      {
        "name": "CopySalesData",
        "type": "Copy",
        "inputs": [{ "referenceName": "DS_Source_CSV" }],
        "outputs": [{ "referenceName": "DS_Sink_ADLS" }]
      }
    ]
  }
}
```

◆ Organize Pipelines into Folders

As your ADF grows, you might have 50–100 pipelines. Organizing them into folders keeps things manageable.

Best Practice Folder Structure:

ADF Pipelines



Real-World Example: A bank's ADF might have 200+ pipelines — organized by department (Retail Banking, Corporate Banking, Risk), then by data domain (Ingestion, Transform, Load).

◆ Debug Pipeline

Debug mode allows you to **test your pipeline without publishing it** — great for development and troubleshooting.

How it works:

- Click **"Debug"** button in ADF Studio
- Pipeline runs immediately using current (unpublished) configuration
- Results appear in the **Output** tab at the bottom
- Green checkmark = success, Red X = failure with error details

Real-World Tip: Always debug with a small data sample first. If you have 10 million rows in a table, use a query like `SELECT TOP 100 * FROM sales` as your source during debugging to save cost and time.

◆ Publish Pipeline

Publishing saves your pipeline to **Azure Resource Manager (ARM)** and makes it live. Before publishing, changes only exist in draft mode (if using live mode) or in your Git branch.

Best Practice Workflow:

Developer A: Creates pipeline in Feature Branch

- Commits to Git
- Raises Pull Request

Developer B: Reviews and approves

- Merges to Main branch
- Publish from Main branch to ADF Production

◆ Pipeline Parameters

Parameters make pipelines **reusable and dynamic**. Instead of hardcoding values like file names or table names, you pass them as parameters at runtime.

Real-World Example:

Without parameters — you need 12 separate pipelines for 12 months of data:

PL_Load_Jan, PL_Load_Feb, PL_Load_Mar... (bad practice!)

With parameters — one pipeline handles all months:

Pipeline: PL_Load_Monthly_Data

Parameter: p_month (**string**) → pass "Jan", "Feb", "Mar" etc.

Parameter: p_year (**string**) → pass "2024", "2025"

Source path dynamically becomes:

```
/raw/sales/{pipeline().parameters.p_year}/{pipeline().parameters.p
```

How to define parameters:

1. Click on the empty canvas in pipeline
2. Go to **Parameters** tab
3. Click "+ New" → Name: `p_month`, Type: `String`, Default: `Jan`

3 Linked Service — Deep Dive

◆ What is a Linked Service?

A **Linked Service** is a **connection definition** that contains the connectivity information ADF needs to connect to external data stores or compute resources. It's reusable across multiple datasets and activities.

Real-World Analogy: A Linked Service is like a Wi-Fi saved network profile on your laptop. You save "OfficeWiFi" with the password once — and anytime you're in the office, your laptop automatically connects using that saved profile. Similarly, once you define a Linked Service for your SQL Server, all your pipelines use it without re-entering credentials.

◆ Creating Linked Services

A. Linked Service for BLOB Storage

Name: LS_AzureBlob_RawData

Type: Azure Blob Storage

Authentication: Account `Key` / Managed Identity / SAS URI

Storage Account: mystorageaccount

Use Case: Copy raw CSV files uploaded by the business team from Blob Storage into ADLS for processing.

B. Linked Service for Azure SQL Database

```
Name: LS_AzureSQLDB_SalesDB
Type: Azure SQL Database
Server: myserver.database.windows.net
Database: SalesDB
Authentication: SQL Authentication
Username: sqladmin
Password: @Microsoft.KeyVault(SecretUri=...) ← from Key Vault!
```

Use Case: Load transformed sales data from ADLS into a SQL Database for Power BI reporting.

C. Linked Service for SQL Server (On-Premise)

```
Name: LS_OnPrem_SQLServer
Type: SQL Server
Server: 192.168.1.100 (internal IP)
Database: LegacyDB
Integration Runtime: SelfHostedIR01 ← REQUIRED for on-prem!
Authentication: Windows Authentication
```

Key Point: On-premise connections **always require a Self-Hosted Integration Runtime** because ADF (in the cloud) cannot directly reach servers behind your corporate firewall.

D. Linked Service for Data Lake Storage Gen1 & Gen2

```
Name: LS_ADLS_Gen2_DataLake
Type: Azure Data Lake Storage Gen2
Account: mydatalakestorage.dfs.core.windows.net
Authentication: Managed Identity (recommended!)
```

Why Managed Identity is Best: Instead of storing passwords, Azure automatically manages the identity. Your ADF instance gets a system-assigned identity, and you grant it "Storage Blob Data Contributor" role on the storage account — zero passwords to manage!

◆ Linked Service Parameterization

Just like pipelines, Linked Services can be parameterized to make them dynamic.

Real-World Use Case: You have 5 different SQL databases (one per region: India, US, UK, Germany, Singapore). Instead of creating 5 separate Linked Services, create **one parameterized Linked Service**:

```
Name: LS_SQLDatabase_Parameterized
Parameter: p_serverName → Type: String
Parameter: p_databaseName → Type: String

Server: @{linkedService().p_serverName}
Database: @{linkedService().p_databaseName}
```

Now pass different server and database names at runtime — one Linked Service serves all 5 regions!

4 Datasets — Deep Dive

◆ What is a Dataset?

A **Dataset** is a **named view of data** that simply points to the data you want to use in your activities. It references a Linked Service for the connection and adds specifics about location and structure.

Real-World Analogy: If a Linked Service is your Amazon account (connection), a Dataset is like a specific item in your cart (the specific product/file you want to work with).

```
Linked Service: LS_ADLS_Gen2  ← HOW to connect (connection
details)
Dataset: DS_RawSales_CSV      ← WHAT to access (specific
file/table)
```

◆ Creating Datasets

A. File-Based Datasets (CSV, JSON, Parquet, Avro, Excel, etc.)

CSV Dataset Example:

```
Name: DS_Source_SalesCSV
Linked Service: LS_ADLS_Gen2
Format: DelimitedText (CSV)
File Path: raw/sales/2024/sales_data.csv
First Row as Header: Yes
Column Delimiter: ,
Encoding: UTF-8
```

Parquet Dataset Example:

```
Name: DS_Silver_SalesParquet
Linked Service: LS_ADLS_Gen2
Format: Parquet
```

File Path: silver/sales/

Compression: Snappy

Why Parquet? Parquet is a columnar format — 10x faster for analytics queries, 70% smaller file size compared to CSV. Industry standard in big data pipelines.

B. Table-Based Datasets (SQL Database, SQL Server, Oracle)

SQL Table Dataset:

Name: DS_SQLTable_Customers

Linked Service: LS_AzureSQLDB_SalesDB

Table Name: [dbo].[Customers]

Query-Based Dataset (more flexible):

Name: DS_SQL_RecentOrders

Linked Service: LS_AzureSQLDB_SalesDB

Use Query: **SELECT** * **FROM** Orders **WHERE** OrderDate >= '2024-01-01'

◆ Dataset Parameterization

Real-World Use Case: You process daily sales files. File path changes every day:

/raw/sales/2024/01/sales_20240101.csv

/raw/sales/2024/01/sales_20240102.csv

/raw/sales/2024/01/sales_20240103.csv

Instead of creating 365 datasets, create **one parameterized dataset**:

Dataset: DS_DailySalesCSV_Param

Parameter: p_date → Type: String

File Path:

raw/sales/@{dataset().p_date}/sales_@{dataset().p_date}.csv

Now pass `p_date = "20240101"` at runtime → automatically accesses the right file!

5 Activities — Complete Deep Dive

Activities are the **building blocks** of every ADF pipeline. Let's cover each one thoroughly.

◆ 1. Wait Activity

Pauses pipeline execution for a specified number of seconds.

Real-World Use Case: After triggering a stored procedure that kicks off a long database rebuild, you add a **Wait activity for 60 seconds** before the next activity checks if the rebuild is complete — giving the DB time to finish.

Activity: Wait

Wait Time in Seconds: 60

◆ 2. Variables

Variables store **temporary values** during pipeline execution — like variables in any programming language.

Create a Variable:

In the Pipeline canvas → Variables tab → Click "+ New"

```
Name: v_fileName
Type: String
Default Value: ""
```

Set Variable Activity:

Assigns a value to a pipeline variable.

```
Activity: Set Variable
Variable Name: v_fileName
Value: @concat('sales_', formatDateTime(utcnow(), 'yyyyMMdd'),
'.csv')
Result: v_fileName = "sales_20240615.csv"
```

Append Variable Activity:

Adds values to an Array-type variable — useful for building lists.

Real-World Use Case:

```
v_processedTables = [] (starts empty)
```

```
ForEach iteration 1: Append "Customers" → v_processedTables =
["Customers"]
```

```
ForEach iteration 2: Append "Orders" → v_processedTables =
["Customers", "Orders"]
```

```
ForEach iteration 3: Append "Products" → v_processedTables =
["Customers", "Orders", "Products"]
```

```
After loop: Log all processed tables to audit table
```

◆ 3. Copy Data Activity — Most Important Activity

The **Copy Data activity** is the heart of ADF — it moves data from a **Source** to a **Sink** (destination) with optional mapping and transformations.

Internal Architecture of Copy Activity:

```
SOURCE → [Read] → [Staging (optional)] → [Write] → SINK
           ↑
    Integration Runtime executes this
```

A. General Tab:

Name: Copy_Sales_CSV_to_ADLS
Timeout: 0.12:00:00 (12 hours max)
Retry: 3 (retry 3 times on failure)
Retry Interval: 30 seconds

B. Source Tab:

Defines WHERE to read data from.

Source Dataset: DS_OnPrem_SalesCSV
File Path Type: Wildcard
Wildcard Folder Path: /exports/sales/
Wildcard File Name: sales_*.csv ← picks all sales CSV files
Recursively: Yes ← includes subfolders

Available Source Options:

- **Query:** Override dataset with a custom SQL query
- **Stored Procedure:** Use a stored procedure as source
- **Partition Option:** For large tables, read in parallel partitions (physical/logical/dynamic range)

C. Sink Tab:

Defines WHERE to write data to.

Sink Dataset: DS_ADLS_RawSales

Copy Behavior: PreserveHierarchy / FlattenHierarchy / MergeFiles

Write Batch Size: 10000 rows per batch

Copy Behaviors explained:

- **PreserveHierarchy:** Maintains folder structure from source
- **FlattenHierarchy:** All files land in destination root folder
- **MergeFiles:** Merges all source files into one destination file

D. Mapping Tab:

Maps source columns to destination columns — especially when column names differ between source and destination.

Source Column	→	Destination Column
CUST_ID	→	customer_id
CUST_NM	→	customer_name
ORD_DT	→	order_date
AMT	→	order_amount

You can also add **data type conversions** here (e.g., string → datetime).

E. Settings Tab:

Fault Tolerance: Skip incompatible rows ← logs bad rows, continues pipeline

Enable Staging: Yes ← for large data loads using PolyBase (Synapse)

Data Consistency Verification: Yes ← verifies row count after copy

F. User Properties:

Custom key-value pairs added to activity for monitoring purposes.

```
pipeline_name: @{pipeline().Pipeline}  
run_date: @{formatDateTime(utcnow(), 'yyyy-MM-dd')}  
source_system: Oracle_ERP
```

These appear in the monitoring dashboard, making it easy to debug which pipeline/date caused an issue.

◆ 4. Copy File(s) Between BLOB Containers

A. Copy One File from a Folder:

```
Source: /raw/sales/sales_20240601.csv (specific file)  
Sink: /archive/sales/sales_20240601.csv
```

B. Copy All Files from a Folder:

```
Source Dataset: Wildcard File Path = /raw/sales/*.csv  
Sink: /processed/sales/  
Copy Behavior: PreserveHierarchy
```

C. Copy All Files and Folders Recursively:

```
Source: /raw/ (entire directory tree)  
Recursive: TRUE  
Sink: /backup/  
Copy Behavior: PreserveHierarchy
```

Real-World Use Case: Daily backup job that copies the entire `/raw/` data lake zone to a `/backup/` container recursively.

◆ 5. Copy Data from BLOB to SQL Database / ADLS Gen2

Scenario: CSV files land in BLOB every morning → ADF copies them to Azure SQL Database for reporting.

Source:

Linked Service: `LS_BlobStorage`

Dataset: `DS_RawCSV`

File Path:

`/incoming/sales_{formatDateTime(utcnow(),'yyyyMMdd')}.csv`

Sink:

Linked Service: `LS_AzureSQLDB`

Dataset: `DS_SQL_SalesTable`

Write Behavior: `Upsert` ← Insert new, update existing

Key Columns: `[sale_id]`

As Parquet to ADLS Gen2 (more common in modern pipelines):

Source: `SQL Database query result`

Sink: `ADLS Gen2 - Parquet format`

Path: `/silver/sales/year=2024/month=06/`

Partition: `By date` ← Hive-style partitioning for Spark

◆ 6. Databricks Notebook Activity

Runs an **Azure Databricks notebook** from within an ADF pipeline.

Activity: Databricks Notebook

Linked Service: LS_AzureDatabricks_Workspace

Notebook Path: /notebooks/transform_sales_data

Parameters:

input_path: /raw/sales/2024/

output_path: /silver/sales/2024/

process_date: @{formatDateTime(utcnow(), 'yyyy-MM-dd')}

Real-World Use Case: ADF orchestrates the overall flow → when it's time to transform data, it hands off to Databricks (which has the heavy Spark processing power) → Databricks runs the notebook → returns control back to ADF → ADF continues to the next activity.

◆ 7. Azure Function Activity

Calls an **Azure Function** (serverless code) from your pipeline.

Real-World Use Case: After copying data, call an Azure Function that:

- Sends a Teams notification: "Sales data loaded successfully for 2024-06-15"
- Or validates that row counts match between source and destination
- Or calls a third-party REST API to notify downstream systems

Activity: Azure Function

Linked Service: LS_AzureFunction_Notifications

Function Name: SendPipelineNotification

Method: POST

Body: {

"pipeline": "@{pipeline().Pipeline}",

"status": "Success",

"rows_copied": "@{activity('CopyData').output.rowsCopied}"

}

◆ 8. Lookup Activity

Lookup reads data from a source and **returns the result to the pipeline** as an object — used to dynamically fetch configuration or list of items to process.

Real-World Use Case — Dynamic Table List:

You have a **control table** in SQL Database:

```
-- table: dbo.ControlTable
table_name      | is_active | source_schema
-----|-----|-----
Customers       | 1         | dbo
Orders          | 1         | dbo
Products        | 0         | dbo ← skip this one
Inventory       | 1         | dbo
```

Lookup activity reads this table:

```
Activity: LookupActiveTables
Source: Azure SQL Database
Query: SELECT table_name FROM dbo.ControlTable WHERE is_active =
1
```

Output:

```
{
  "firstRow": {"table_name": "Customers"},
  "value": [
    {"table_name": "Customers"},
    {"table_name": "Orders"},
    {"table_name": "Inventory"}
  ],
}
```

```
"count": 3
}
```

This output feeds into a **ForEach** activity to copy only active tables — making your pipeline 100% dynamic!

◆ Stored Procedure Activity

Executes a **SQL stored procedure** at any point in your pipeline.

Real-World Use Cases:

- Log pipeline execution status to an audit table
- Call a stored procedure that merges staging data into production table
- Execute post-load data quality checks

Activity: SP_UpdateAuditLog

Linked Service: LS_AzureSQLDB

Stored Procedure Name: [dbo].[usp_UpdatePipelineLog]

Parameters:

pipeline_name: @{{pipeline().Pipeline}}

run_id: @{{pipeline().RunId}}

status: "Success"

rows_loaded: @{{activity('CopyData').output.rowsCopied}}

end_time: @{{utcnow()}}

◆ 9. Get Metadata Activity

Retrieves **metadata about a file or folder** — such as file name, size, last modified date, child items list.

Real-World Use Case — Check if File Arrived Before Processing:

Activity: GetMetadata_CheckFile

Dataset: DS_ExpectedFile

Field List: [exists, size, lastModified, itemName, childItems]

Output:

```
{
  "exists": true,
  "size": 1048576,           ← 1 MB
  "lastModified": "2024-06-15T23:45:00Z",
  "itemName": "sales_20240615.csv",
  "childItems": [...]
}
```

Then use an **If Condition**: if `exists = true` → proceed with copy, else → send alert email.

◆ Delete Activity

Deletes files or folders from Azure Storage (BLOB or ADLS).

Real-World Use Cases:

- Delete source files after successful copy (archival cleanup)
- Delete files older than 30 days from the raw zone
- Clean up staging files after data is loaded

Activity: DeleteSourceFiles

Dataset: DS_ProcessedFiles

Recursive: True

Wildcard File Name: *.csv

Best Practice: Always use Delete **after** confirming the copy was successful — use it as the last step, not the first!

◆ 10. Execute Pipeline Activity

Calls **another pipeline** from within a pipeline — enables **modular pipeline design**.

Real-World Use Case — Master-Child Pipeline Pattern:

```
PL_Master_OvernightETL (runs at midnight)
├─ Execute Pipeline → PL_Ingest_SalesData
├─ Execute Pipeline → PL_Ingest_CustomerData
├─ Execute Pipeline → PL_Ingest_InventoryData
├─ Execute Pipeline → PL_Transform_AllData
└─ Execute Pipeline → PL_Load_Synapse
```

Each child pipeline can also run independently for testing/reprocessing.

Configuration:

```
Activity: RunIngestionPipeline
Invoked Pipeline: PL_Ingest_SalesData
Wait on Completion: YES ← master waits for child to finish
Parameters:
  p_date: @{formatDateTime(utcnow(), 'yyyy-MM-dd')}
```

◆ 11. Validation Activity

Waits until a **file or folder exists** before continuing — prevents pipeline failure when source isn't ready.

Real-World Use Case: The upstream system uploads a file "sales_done.flag" to signal that all sales data has been delivered. Your pipeline waits for this flag file before starting data processing:

```
Activity: WaitForSalesFile
Dataset: DS_FlagFile ← points to sales_done.flag
Timeout: 4 hours ← wait max 4 hours, then fail
Sleep: 60 seconds ← check every 60 seconds
Minimum Size: 0 bytes ← file just needs to exist
```

◆ Fail Activity

Intentionally fails a pipeline with a custom error message — used for business rule violations.

```
Activity: FailPipeline_NoData
Fail Message: @{{concat('No data file found for date: ',
formatDateTime(utcnow(), 'yyyy-MM-dd'))}}
Error Code: ERR_NO_SOURCE_FILE
```

When to use: If a critical file is missing, rather than silently succeeding (which would produce wrong reports), use Fail activity to raise an explicit, clearly named error.

◆ 12. Iteration & Conditionals

These are **control flow activities** that add programming logic to your pipelines.

A. Filter Activity

Filters an array based on a condition — like WHERE clause for pipeline arrays.

```
Activity: Filter_ActiveTables
Items: @{{activity('LookupTables').output.value}}
Condition: @{{equals(item().is_active, 1)}}

Output: Only tables where is_active = 1
```

B. ForEach Activity

Loops through an array and executes inner activities for each item — most commonly used with Lookup output.

```
Activity: ForEach_ProcessTables
Items: @{{activity('Filter_ActiveTables').output.value}}
Batch Count: 4 ← process 4 tables in parallel
Sequential: No ← run in parallel (faster!)

Inner Activity: Copy Data
  Source Table: @{{item().table_name}}
  Sink Path:    /processed/@{{item().table_name}}/
```

Real-World Example: Instead of writing separate copy pipelines for 50 tables, one ForEach loop iterates through all 50 and copies them — cutting pipeline count from 50 to 1!

C. If Condition Activity

Executes different activities based on a **True/False condition**.

```
Activity: CheckFileSize
Condition: @{{greater(activity('GetMetadata').output.size, 0)}}
```

If True Activities:

→ Copy Data to ADLS

If False Activities:

→ Send Alert Email

→ Fail Pipeline

Real-World Use Case: Check if file is non-empty before processing. An empty file (0 bytes) would cause downstream failures — catch it early!

D. Switch Activity

Like a **switch/case statement** — routes to different activity branches based on a value.

Activity: RouteByRegion

On: `@{pipeline().parameters.p_region}`

Cases:

"INDIA": → Copy to India ADLS container

"US": → Copy to US ADLS container

"UK": → Copy to UK ADLS container

Default: → Log unknown region warning

E. Until Activity

Loops **until a condition becomes true** — like a do-while loop.

Real-World Use Case: Poll an API every 30 seconds until a long-running job (like Databricks job) completes:

Activity: WaitForDatabricksJob

Condition: @equals(variables('v_jobStatus'), 'COMPLETED')}

Timeout: 2 hours

Delay: 30 seconds

Inner Activities:

1. Call REST API to check Databricks job status
2. Set Variable: v_jobStatus = API response

6 Triggers — Deep Dive

◆ What is a Trigger?

A **Trigger** is the mechanism that **fires (starts) a pipeline**. Without a trigger, a pipeline can only be run manually.

Real-World Analogy: A trigger is like an alarm clock — the alarm (trigger) fires at a specific time (or event), waking you up (starting the pipeline).

◆ Types of Triggers

A. Schedule Trigger

Runs a pipeline on a **fixed time schedule** — most commonly used trigger.

Trigger Name: TRG_Daily_SalesETL

Start Time: 2024-01-01 00:00:00 UTC

Recurrence: Every 1 Day

Timezone: India Standard Time (UTC+5:30)

End Time: (none - runs indefinitely)

Real-World Examples:

- Daily at midnight: Run nightly data warehouse load
- Every 15 minutes: Refresh near-real-time sales dashboard
- Every Monday at 8 AM: Generate weekly executive reports

Important: One Schedule Trigger can attach to **multiple pipelines**. Or one pipeline can have **multiple triggers** (e.g., daily + manual).

B. Tumbling Window Trigger

Processes data in **fixed-sized, non-overlapping time windows** — ideal for **reprocessing historical data** or **ensuring exactly-once processing**.

```
Trigger Name: TRG_Hourly_Sales_Window
Start Time: 2024-01-01 00:00:00 UTC
Period: 1 Hour
Max Concurrency: 4 ← run 4 windows in parallel
Retry: 3
Delay: 5 minutes ← wait 5 min after window end before firing
```

What makes it different from Schedule Trigger:

- Tumbling window tracks **which windows have run successfully**
- If a window fails, it retries that specific window
- You can **backfill** — run all past windows (e.g., reprocess Jan 1 to today)

Real-World Use Case: A financial system needs to ensure **every single hour of transaction data** is processed exactly once. If processing for 3 PM–4 PM fails at 4:15 PM, the tumbling window trigger retries only that specific hour — it never skips or double-processes.

Trigger Parameters available inside pipeline:

```
@{triggerBody().window.windowStart} → "2024-06-15T03:00:00Z"  
@{triggerBody().window.windowEnd}   → "2024-06-15T04:00:00Z"
```

C. Storage Event Trigger

Fires when a **file is created or deleted in ADLS Gen2 or Blob Storage** — enables event-driven pipelines.

```
Trigger Name: TRG_Event_NewSalesFile  
Storage Account: mydatalakestorage  
Container: raw  
Blob Path Begins With: sales/incoming/  
Blob Path Ends With: .csv  
Event: BlobCreated ← fires when new CSV arrives
```

Real-World Use Case: The upstream system drops a file

`/raw/sales/incoming/sales_20240615.csv` → **immediately** (within seconds), ADF detects the new file and starts the ingestion pipeline — no waiting for the next scheduled run!

Use Cases:

- Process files immediately as they arrive (real-time ingestion feel)
- Trigger different pipelines for different file types (.csv vs .json)
- Event-driven architecture — upstream and downstream are decoupled

◆ Triggers with Parameters

Triggers can pass **parameter values to the pipeline** at runtime:

```
Schedule Trigger: TRG_Daily_Load
```

```
Pipeline Parameters Passed:
```

```
  p_loadDate:    @{{formatDateTime(trigger().scheduledTime, 'yyyy-MM-dd')}}
```

```
  p_loadType:    "FULL"
```

```
  p_source:      "Oracle_ERP"
```

Storage Event Trigger passes file details:

```
p_fileName:      @{{triggerBody().fileName}}
```

```
p_folderPath:    @{{triggerBody().folderPath}}
```

7 Integration Runtime (IR) — Deep Dive

◆ What is Integration Runtime?

The **Integration Runtime** is the **compute engine** of ADF — the actual machine/cluster that executes activities and moves data.

Real-World Analogy: If ADF is the brain (control plane), the Integration Runtime is the body/muscles (execution plane) — it does the actual physical work.

◆ Types of Integration Runtime

A. Azure AutoResolve Integration Runtime

- **Default IR** — automatically created when you create ADF
- Runs in Microsoft-managed Azure data centers
- **Location: Auto-resolve** — ADF automatically picks the nearest Azure region to your source and sink

- No configuration needed — works out of the box

Use Cases:

- Cloud-to-cloud data movement
- Azure Blob ↔ Azure SQL Database ↔ ADLS Gen2
- All resources are already in Azure

B. Azure Managed Virtual Network Integration Runtime

- An Azure IR that runs inside a **Managed Virtual Network (VNet)**
- Allows **private endpoints** — connects to Azure services without going over the public internet
- Provides enhanced security — data never leaves Microsoft's backbone network

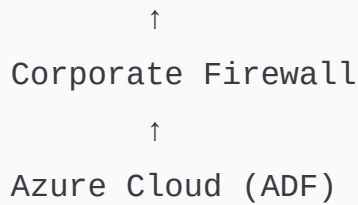
Real-World Use Case: A bank's Azure SQL Database is configured to reject all public internet connections. You use Managed VNet IR with a private endpoint — ADF connects through Azure's internal network, satisfying security compliance requirements.

C. Self-Hosted Integration Runtime (SHIR)

The most critical IR for enterprises — installed **on your own on-premise server or VM** to bridge on-premise systems with ADF.

On-Premise Network





Setup Steps:

1. Create SHIR in ADF Studio (Manage → Integration Runtimes → New → Self-Hosted)
2. Copy the authentication key
3. On your on-premise server: Download SHIR installer from Microsoft
4. Install and paste the authentication key → It registers with ADF
5. Now ADF can reach your on-premise SQL Server, Oracle, files, etc.

Requirements:

- Windows machine (VM or physical server)
- Outbound HTTPS port 443 open to Azure
- Minimum: 4 CPU cores, 8GB RAM (production: 8+ cores, 16+GB RAM)

Real-World Example: A manufacturing company keeps its SAP ERP data on-premise for security reasons. They install SHIR on a server inside their network. ADF uses this SHIR to extract daily production data from SAP and load it into Azure Data Lake for analytics.

D. Linked Self-Hosted Integration Runtime

Allows **multiple ADF instances to share one Self-Hosted IR.**

Company has:

ADF Dev Instance → uses "LinkedSHIR" → points to SHIR on server-01

ADF Test Instance → uses "LinkedSHIR" → points to SHIR on server-01

ADF Prod Instance → uses "LinkedSHIR" → points to SHIR on

server-01

All 3 ADF instances share 1 SHIR installation!

Benefit: Save cost — one SHIR serves multiple ADF environments instead of installing SHIR 3 times.

8 Source Control — Git Integration

◆ Why Source Control for ADF?

Without Git integration, ADF pipelines are saved directly to the ADF service — this means:

- No version history (can't undo bad changes)
- No collaboration (two developers overwriting each other)
- No code review before deploying to production
- No rollback if a deployment breaks production

With Git integration, all ADF resources (pipelines, datasets, linked services) are stored as **JSON files in a Git repository** — giving you full DevOps capabilities.

◆ Git Configuration

Supported Git Providers:

- Azure DevOps Repos (recommended for enterprises)
- GitHub

Setup:

Author → Git Configuration:

Repository Type: Azure DevOps Git

Azure DevOps Account: mycompany.visualstudio.com

Project Name: DataEngineering

Repository Name: ADF-Pipelines

Collaboration Branch: main

Publish Branch: adf_publish

Root Folder: /

How it works after setup:

Developer works in → Feature Branch (e.g., feature/add-customer-pipeline)

- Commits changes (JSON files)
- Creates Pull Request to main branch
- Code reviewed by team lead
- Merged to main
- Published to ADF service (live)

◆ ARM Templates — Export/Import

ARM (Azure Resource Manager) Template is a JSON file that describes all ADF resources — it captures the entire ADF configuration.

Export Use Cases:

- Backup your entire ADF configuration
- Deploy the same ADF to a different environment (Dev → Test → Prod)
- Share ADF configuration with a new team

Export:

```
ADF Studio → Manage → ARM Template → Export ARM Template
Downloads: ARMTemplateForFactory.json (full config)
           ARMTemplateParametersForFactory.json (environment-
           specific values)
```

Import (deploy to new environment):

```
Azure Portal → Create Resource → Deploy a Custom Template
Upload: ARMTemplateForFactory.json
Parameters: ARMTemplateParametersForFactory.json (edit for target
env)
```

Real-World Use Case: Development ADF is fully built and tested. You export the ARM template, update the parameter file with production server names, storage accounts, and key vault references, then deploy to Production ADF — exact same configuration, different environment values.

◆ Azure DevOps Repos Integration

With Azure DevOps, your entire ADF code lives in Repos as JSON:

```
ADF-Pipelines/ (Git Repository)
├─ pipeline/
│   ├─ PL_Daily_SalesETL.json
│   ├─ PL_Customer_Load.json
│   └─ PL_Master_Orchestrator.json
├─ dataset/
│   ├─ DS_SalesCSV.json
│   └─ DS_SQL_Customers.json
├─ linkedService/
│   ├─ LS_ADLS_Gen2.json
│   └─ LS_AzureSQLDB.json
```

```
|— trigger/
|   |— TRG_Daily_Schedule.json
|— dataflow/
|   |— DF_TransformSales.json
```

9 Global Parameters

Global Parameters are pipeline parameters that are **available across all pipelines** in an ADF — like environment-level constants.

Real-World Use Case:

```
Global Parameter: gp_environment      = "Production"
Global Parameter: gp_dataLakeURL      =
"https://prodlake.dfs.core.windows.net"
Global Parameter: gp_emailAlertGroup = "dataengg-
team@company.com"
```

All pipelines can reference these:

```
@pipeline().globalParameters.gp_environment
@pipeline().globalParameters.gp_dataLakeURL
```

Huge benefit during environment promotion: When moving from Dev to Prod, update just the Global Parameters — all 100 pipelines automatically use production values!

10 Credentials

Credentials in ADF enable secure authentication using **Managed Identity or Service Principal** instead of storing passwords.

Types:

- **System-assigned Managed Identity:** ADF automatically gets an identity in Azure AD — grant this identity access to storage/SQL
- **User-assigned Managed Identity:** You create a custom identity and assign it to ADF
- **Service Principal:** App registration in Azure AD with client ID + secret

Best Practice: Always use **Managed Identity** — no passwords to rotate, no secrets to leak, fully managed by Azure.

11 Monitoring ADF Jobs

◆ Monitor Tab in ADF Studio

The Monitor section gives you **complete visibility** into all pipeline and activity runs.

Pipeline Runs View:

Pipeline Name	Status	Start Time	Duration
Triggered By			
-----	-----	-----	-----

PL_Daily_SalesETL	Succeeded	2024-06-15 00:00	12m 45s
TRG_Daily_Schedule			
PL_Customer_Load	Failed	2024-06-15 00:13	2m 10s
TRG_Daily_Schedule			
PL_Inventory_Refresh	Running	2024-06-15 00:25	-
Manual			

Activity Runs View (drilling into a pipeline run):

Activity Name	Type	Status	Duration	
Output				
-----	-----	-----	-----	-----
GetMetadata_CheckFile	GetMetadata	Succeeded	3s	
exists:true, size:2MB				
CopySalesData	Copy	Succeeded	5m 22s	
rowsCopied:142,850				
RunDatabricksNotebook	Databricks	Succeeded	6m 10s	
notebook:completed				
SP_UpdateAuditLog	SqlScript	Succeeded	1s	-

Useful monitoring features:

- **Re-run failed pipelines** directly from monitor — with same or new parameters
- **Cancel running pipelines** that are stuck
- **Filter** by date range, pipeline name, status
- **Gantt chart view** for pipeline execution timeline

1 2 Alerts

ADF integrates with **Azure Monitor** to send alerts when pipelines fail, succeed, or take too long.

Creating an Alert:

```
Alert Rule: ADF_Pipeline_Failure_Alert
Condition: Pipeline runs failed > 0 (in last 5 minutes)
Severity: Sev 1 (Critical)
Action Group: DataEngg_Team
  → Email: dataengg@company.com
```

- SMS: +91-98765-43210
- Teams Webhook: (teams channel URL)

Real-World Use Case: Production pipeline fails at 2 AM. Alert fires immediately → entire data engineering team gets an email and SMS → on-call engineer is woken up → fixes the issue before 9 AM business hours → business reports are ready on time.

13 Send Failure Notifications Using Logic Apps

Logic Apps provides a more customizable notification system than basic Azure Monitor alerts — you can send **rich, detailed emails** with formatted HTML content.

Architecture:

ADF Pipeline Fails

↓

Web Activity in ADF calls Logic App HTTP trigger

↓

Logic App receives JSON payload

↓

Logic App sends formatted email via Office 365 connector

ADF Web Activity Configuration:

```
Activity: NotifyFailure (in pipeline's failure path)
URL: https://prod-xx.logic.azure.com/workflows/.../triggers/...
Method: POST
Body:
{
  "pipeline_name": "@{pipeline().Pipeline}",
  "run_id": "@{pipeline().RunId}",
  "failed_activity": "@{activity('CopyData').Error.message}",
```



```
"error_code": "@{activity('CopyData').Error.errorCode}",
"run_time": "@{utcnow()}",
"environment": "Production"
}
```

Email received by team:

Subject: ✖ ADF Pipeline FAILED - PL_Daily_SalesETL

Pipeline: PL_Daily_SalesETL

Environment: Production

Failed Activity: CopyData

Error: File not found at /raw/sales/sales_20240615.csv

Error Code: UserErrorFileNotFound

Run Time: 2024-06-15 00:02:34 UTC

Run ID: abc123-def456-...

[\[Click here to view in ADF Monitor\]](#)

14

Data Flows — Deep Dive

◆ What is Data Flow?

Data Flows provide a **visual, code-free way to transform data** at scale inside ADF. Under the hood, Data Flows run on **Apache Spark clusters** — so they can handle large datasets efficiently.

When to use Data Flow vs Copy Activity:

	Copy Activity	Data Flow
Purpose	Move data as-is	Transform data

	Copy Activity	Data Flow
Transformations	Basic column mapping	Complex aggregations, joins, pivots
Coding	None	None (visual)
Engine	ADF copy engine	Apache Spark
Debugging	Run & check output	Live data preview at each step

◆ Mapping Data Flow

Mapping Data Flow is the main type — visually maps transformations using a canvas.

Example Transformation Flow:

```

Source (ADLS CSV)
  ↓
Filter (remove rows where sales_amount <= 0)
  ↓
Derived Column (create new column: tax = sales_amount * 0.18)
  ↓
Aggregate (group by region, sum sales_amount)
  ↓
Join (join with region_master table for region names)
  ↓
Select (pick only needed columns)
  ↓
Sink (write to Azure SQL Database)

```

◆ Data Flow Debug

Debug mode spins up a small Spark cluster and lets you **preview data at every transformation step** — like a debugger for your data transformations.

After **Filter** transformation → Click "Data Preview"

Shows: **142,850 rows** passed **filter** (**0** negative sales removed)

After Aggregate transformation → Click "Data Preview"

Shows: **5 rows** (**one per region with** summed sales)

Cost Note: Debug cluster runs continuously while debugging — **turn it off when done** to avoid unnecessary charges!

◆ Transformations — Complete Reference

A. Filter

Removes rows that don't meet a condition — like SQL WHERE clause.

```
Condition: sales_amount > 0 && region != 'UNKNOWN'
```

B. Aggregate

Groups data and computes aggregations — like SQL GROUP BY.

```
Group By: region, product_category
```

```
Aggregations:
```

```
total_sales = sum(sales_amount)
```

```
avg_sales   = avg(sales_amount)
```

```
order_count = count(order_id)
```

C. Join

Combines two data streams — like SQL JOIN.

```
Left Stream: SalesData
Right Stream: RegionMaster
Join Type: Inner Join
Condition: SalesData.region_code == RegionMaster.region_code
```

D. Conditional Split

Like a SQL CASE or IF-ELSE — routes rows to different output streams.

```
Stream 1 (HighValue):    sales_amount >= 100000
Stream 2 (MediumValue):  sales_amount >= 10000 && sales_amount <
100000
Stream 3 (LowValue):     Default (everything else)
```

Real-World Use Case: Route high-value transactions to a premium customer table, others to regular.

E. Derived Column

Creates new columns or modifies existing ones — like SQL computed columns.

```
full_name      = concat(first_name, ' ', last_name)
year_month     = toString(order_date, 'yyyy-MM')
tax_amount     = sales_amount * 0.18
category_upper = upper(product_category)
```

F. Exists

Checks if rows from one stream exist in another — like SQL EXISTS / NOT EXISTS.

```
Exists Type: Exists (keep only matching)
Left Stream: NewOrders
Right Stream: ExistingOrders
Condition: NewOrders.order_id == ExistingOrders.order_id
Use: Find orders that are new (not already in warehouse)
```

G. Union

Combines multiple data streams vertically — like SQL UNION ALL.

```
Stream 1: SalesData_India
Stream 2: SalesData_US
Stream 3: SalesData_UK
Output: All rows combined into one dataset
```

H. Lookup

Enriches data by looking up values from a reference table.

```
Primary Stream: SalesTransactions
Lookup Stream: CustomerMaster
Match: sales.customer_id == customer.customer_id
Output: All sales columns + customer name/email from lookup
```

I. Sort

Orders data by one or more columns — like SQL ORDER BY.

```
Sort By: order_date DESC, sales_amount DESC
```

J. GroupBy

Groups rows for aggregation (similar to Aggregate but focused on grouping).

K. Pivot

Converts **rows to columns** — transforms category values into column headers.

```
Group By: region
Pivot Column: product_category (Electronics, Clothing, Food)
Aggregation: sum(sales_amount)

Result:
```

region	Electronics	Clothing	Food
North	500000	200000	150000
South	300000	250000	180000

L. Unpivot

Opposite of Pivot — converts **columns to rows**.

Input: Electronics=500000, Clothing=200000, Food=150000 (in one row)
 Output: Three rows: Electronics/500000, Clothing/200000, Food/150000

M. Flatten

Expands **nested arrays/structs in JSON** into flat rows.

Input JSON: {"customer": "Raj", "orders": [{"id": 1}, {"id": 2}, {"id": 3}]}

Flatten: orders array

Output: 3 rows, one per order

N. Parse & Stringify

- **Parse:** Converts a string column into a complex type (JSON struct)
- **Stringify:** Converts a complex type back to a string

O. Alter Row

Sets **DML policies** on rows — tells the sink how to handle each row.

Alter Row Conditions:

Insert If: `isNull(existing_id)`

Update If: `!isNull(existing_id) && changed_flag == true`

```
Delete If: status == 'DELETED'
Upsert If: true (always upsert)
```

Real-World Use Case: Implementing SCD Type 1 — automatically upsert changed records.

P. Schema Validate & Schema Drift

- **Schema Validate:** Enforces that incoming data matches expected schema
- **Schema Drift:** Allows extra/unexpected columns to flow through without breaking the pipeline

Real-World Use Case: The upstream system sometimes adds new columns to CSV files. With Schema Drift enabled, ADF handles this gracefully instead of throwing errors.

Q. Remove Duplicate Rows

Using **Aggregate transformation** to deduplicate:

```
Group By: All columns (or key columns like customer_id)
Aggregate: row_number = first(order_id) ← keeps first occurrence
Then Filter: row_number == 1
```

R. Flowlet

A **reusable data flow component** — encapsulates common transformation logic that's used across multiple data flows.

Real-World Use Case: Every data flow in your company needs to: add audit columns (load_date, source_system, batch_id). Instead of repeating this in all 20 data flows, create a Flowlet "AddAuditColumns" and reuse it everywhere.

15 Azure DevOps

◆ Azure DevOps Repos

As covered earlier, Azure DevOps Repos is a **Git-based version control system** integrated with ADF for tracking all pipeline code changes.

Complete SDLC Flow with ADF + DevOps:

1. Developer creates feature branch **in** Repos
2. Builds/modifies pipelines **in** ADF Dev environment (linked **to** feature branch)
3. Commits JSON files **to** feature branch
4. Raises Pull Request → Team Lead reviews
5. Approved → Merges **to** main branch
6. CI/CD Pipeline **in** Azure DevOps:
 - a. Triggers **on** merge **to** main
 - b. Exports ARM template **from** ADF Dev
 - c. Deploys ARM template **to** ADF Test
 - d. Runs smoke tests
 - e. Deploys **to** ADF Production (**with** approval gate)

16 SDLC (Software Development Life Cycle)

The **SDLC** for ADF projects typically follows these phases:

1. REQUIREMENTS GATHERING
 - Business analyst documents data requirements
 - Source **system** details, transformation rules, target schema
2. DESIGN
 - Data architect designs pipeline architecture
 - Bronze-Silver-Gold layer design
 - Error handling strategy
3. DEVELOPMENT

- Data engineers build pipelines **in** ADF Dev
- Unit test **with** sample data
- Peer code review via Pull Requests

4. TESTING

- Deploy **to** ADF Test environment
- Integration testing **with full** dataset
- Performance testing (run **time**, throughput)
- UAT (**User** Acceptance Testing) **by** business

5. DEPLOYMENT

- Deploy **to** ADF Production
- Deployment during maintenance **window**
- Parallel run (**old** + **new** pipelines run together initially)

6. MAINTENANCE

- Monitor daily runs
- Handle failures **and** data quality issues
- Enhance pipelines **as** requirements change

17 Agile Methodology in ADF Projects

Agile delivers ADF projects in **short iterations (sprints)** rather than one big delivery.

Sprint Structure (2-week sprint example):

Sprint **1** (Week **1-2**):

- Build ingestion pipelines **for** Sales **module**
- Unit test, deploy **to** Dev
- Demo **to** business stakeholders

Sprint **2** (Week **3-4**):

- Build transformation pipelines for Sales
- Address Sprint 1 feedback
- Deploy Sales module to Test

Sprint 3 (Week 5-6):

- Build Customer module pipelines
- Deploy Sales module to Production
- Sprint review + retrospective

Agile Ceremonies:

- **Daily Standup:** 15-min sync — "What did I do yesterday? What will I do today? Any blockers?"
- **Sprint Planning:** Define tasks for next 2 weeks from backlog
- **Sprint Review:** Demo completed pipelines to stakeholders
- **Sprint Retrospective:** What went well / what to improve

Tools used:

- **Azure DevOps Boards:** Kanban board for tracking pipeline development tasks
- **Azure DevOps Repos:** Code repository
- **Azure DevOps Pipelines:** CI/CD for automated deployment

18 ADF Resume Preparation

Key Skills to Highlight:

Technical Skills:

- ✓ Azure Data Factory (Pipelines, Activities, Data Flows, Triggers, IR)
- ✓ Azure Data Lake Storage Gen2
- ✓ Azure Databricks (PySpark, Delta Lake)

- ✓ Azure Synapse Analytics
- ✓ Azure SQL Database / SQL Server
- ✓ Azure Key Vault
- ✓ Azure DevOps (Repos, CI/CD Pipelines)
- ✓ Microsoft Fabric

Projects to mention:

- ✓ End-to-end ETL pipeline from Oracle/SAP to Azure
- ✓ Medallion architecture implementation (Bronze/Silver/Gold)
- ✓ Real-time streaming pipeline using Event Trigger + Databricks
- ✓ On-premise to cloud data migration using SHIR

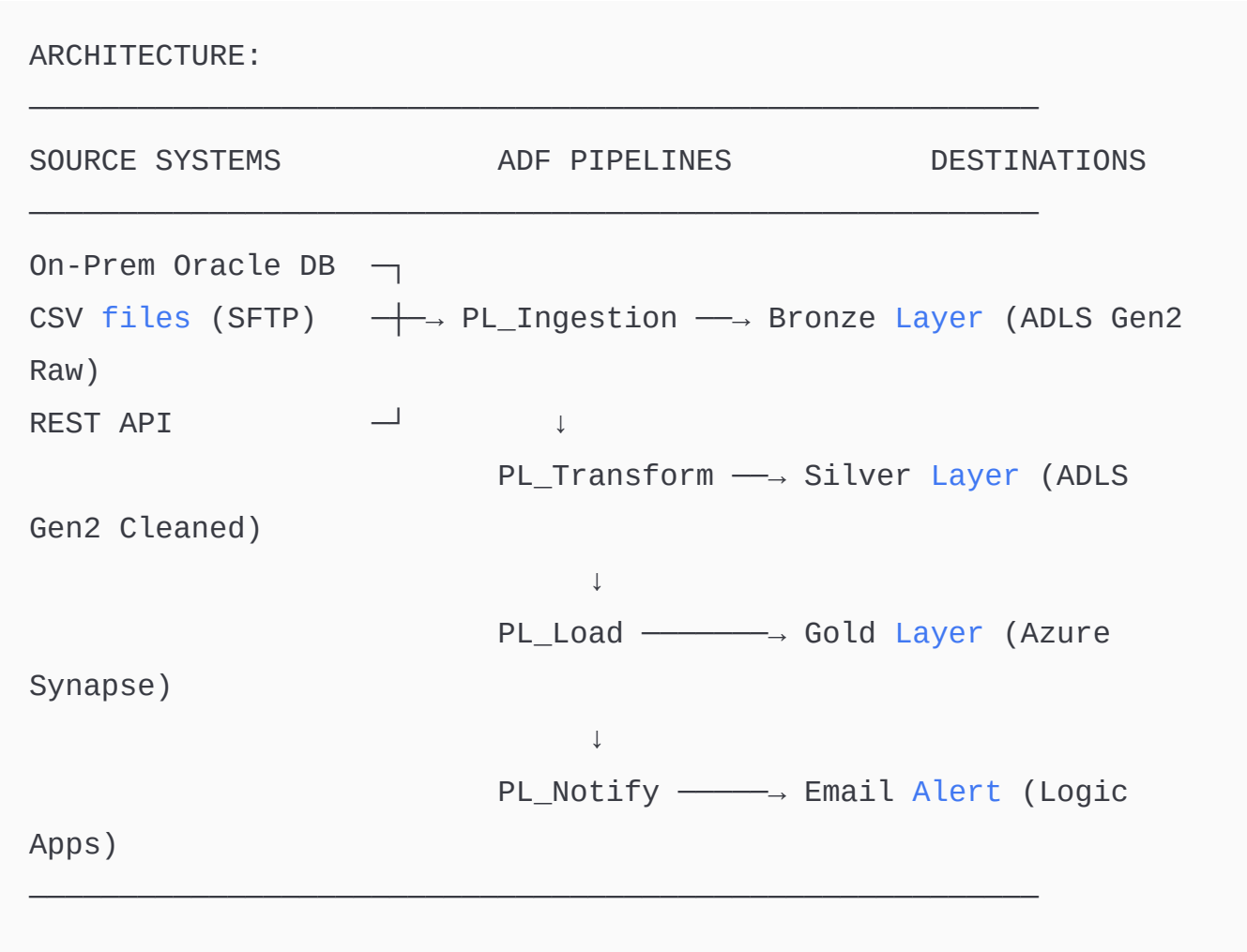
Sample Resume Points:

- Designed and implemented end-to-end ADF pipelines to ingest 50+ tables
from on-premise SQL Server to ADLS Gen2 using Self-Hosted Integration Runtime
- Built parameterized, reusable ADF pipelines with dynamic dataset
parameterization, reducing pipeline count from 50 to 1
- Implemented Delta Lake Medallion Architecture
(Bronze/Silver/Gold)
using ADF + Azure Databricks, processing 500GB daily
- Set up CI/CD deployment using Azure DevOps and ARM templates
for
Dev → Test → Production pipeline promotion
- Integrated Azure Key Vault with ADF for secure credential
management,
eliminating all hardcoded passwords from pipelines

19

End-to-End ADF Project

Project: Daily Sales Analytics Pipeline



Master Pipeline Flow:

1.

VALIDATE:

Check all source files arrived (Validation + GetMetadata)
2.

INGEST:

Copy all sources to Bronze (Copy Activity + ForEach)
3.

TRANSFORM:

Run Databricks notebooks (Databricks Notebook Activity)
4.

LOAD:

Copy Silver data to Synapse Gold (Copy Activity)
5.

AUDIT:

Log success to audit table (Stored Procedure)
6.

NOTIFY:

Send completion report (Logic Apps via Web Activity)

Error Handling (failure path **on every** activity):

- Log error details **to** error **table**
- Send failure alert email **with** error details
- **Set** pipeline status **to** FAILED

20 ADF Exercises Summary

The exercises in the course cover all real-world scenarios:

Beginner Exercises:

- Create pipeline variables, use Set Variable activity
- Build If Condition for branching logic
- Copy files from Blob to Blob (single file, all files, recursive)
- Create linked services for BLOB, ADLS, SQL

Intermediate Exercises:

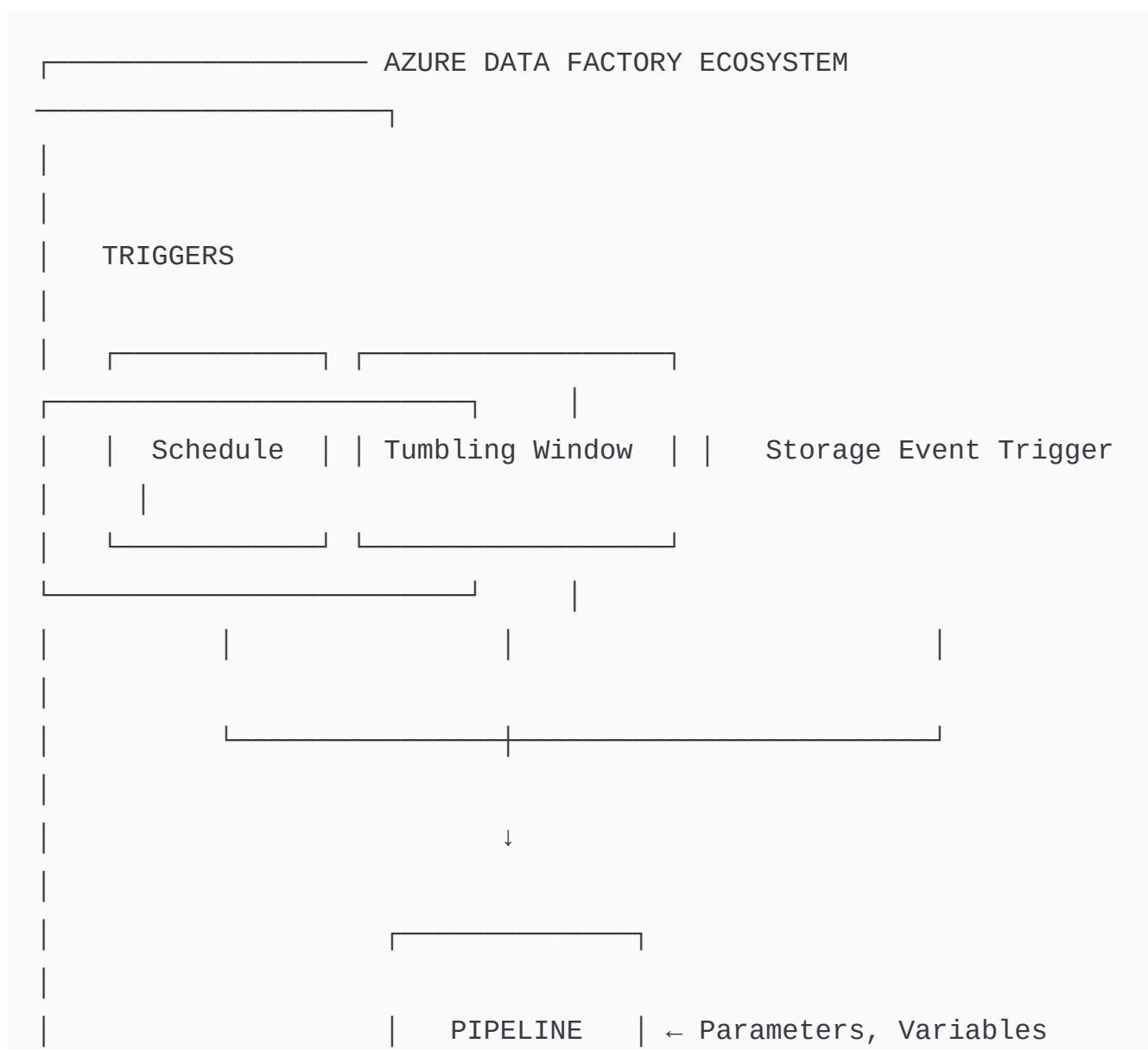
- Pattern matching file copy (copy only `sales_*.csv`)
- Parameterized copy using dynamic dataset
- Bulk copy multiple tables using Lookup + ForEach
- Convert CSV → Parquet, CSV → JSON format conversion
- Add extra audit columns (load_date, source_name) during copy

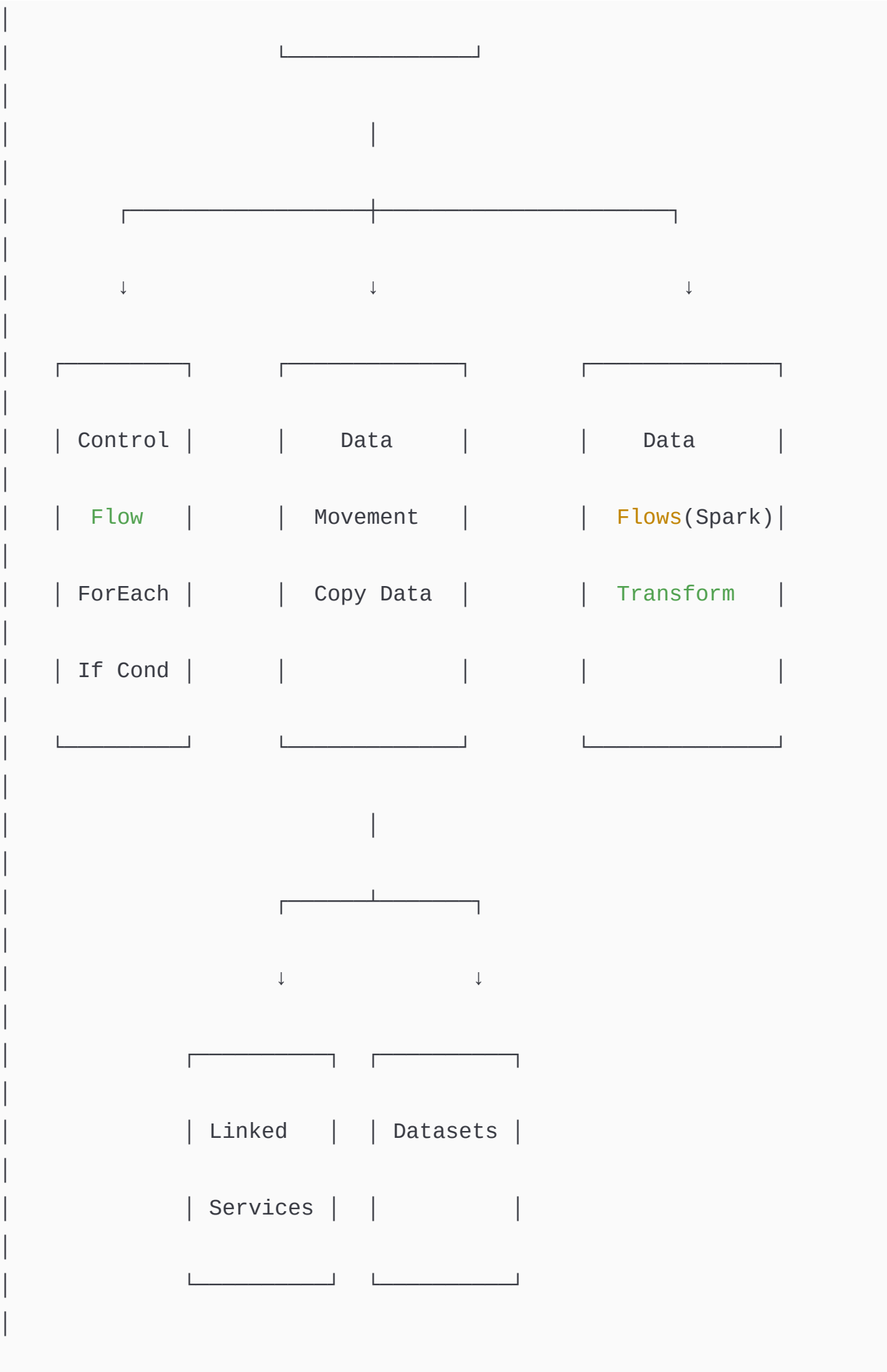
Advanced Exercises:

- Copy data from on-premise to Azure using SHIR
- Integrate Key Vault with ADF (secure credentials)
- Databricks activity with parameter passing
- Schedule trigger, Tumbling Window trigger, Event-based trigger
- Create trigger dependencies (run after another trigger)

- Data Flows — Filter, Join, Pivot, Union, Remove Duplicates
- Create alerts and notification rules
- Import/Export ARM templates
- Integrate ADF with Azure DevOps Repos
- Send mail notifications via Logic Apps
- Monitor and debug pipelines
- Execute one pipeline from within another (Execute Pipeline)

ADF Complete Architecture Summary







MONITORING: Pipeline Runs | Activity Runs | Trigger Runs

ALERTS: Azure Monitor + Logic Apps Email Notifications

SOURCE CTRL: Azure DevOps Repos + ARM Templates + CI/CD

SECURITY: Azure Key Vault + Managed Identity

 **Azure Data Factory section complete!** Every concept — from core architecture to all activity types, triggers, Integration Runtime, Data Flows with all transformations, DevOps, SDLC, Agile, and project structure — has been covered with real-world examples and deep explanations.

 **Next up: Azure Databricks** — ready to continue whenever you are!

Thought process: Audited PDF content comprehensively and identified multiple missing sections.

Let me carefully review the PDF content for Azure Data Factory section and check what was covered vs what's in the PDF.

From the PDF, ADF section includes:

- 1) Azure Data Factory - covered ✓
- 2) Pipeline - covered ✓
- 3) Linked Service - covered ✓
- 4) DataSets - covered ✓
- 5) Activities:
 - Wait ✓
 - Variables (Create, Set, Append) ✓
 - Copy Data (General, Source, Sink, Mapping, Settings, User Properties) ✓
 - Copy file(s) from one BLOB Container to another (One file, All files, Recursively) ✓
 - Copy data/file from BLOB to SQL Database/ADLS Gen2 ✓
 - Databricks Notebook ✓
 - Azure Function ✓
 - Lookup, Stored Procedure ✓
 - Get Metadata, Delete ✓
 - Execute Pipeline ✓
 - Validation, Fail ✓
 - Iteration & Conditionals (Filter, ForEach, If Condition, Switch, Until) ✓
- 6) Triggers (Schedule, Tumbling Window, Storage Events, Triggers with Parameters) ✓
- 7) Integration Runtime (Azure AutoResolve, Azure) - appears to be cut off in the PDF

, but I'm continuing with Managed Virtual Network, Self-Hosted, and Linked Self-Hosted options ✓

8) Source Control (Git configuration, ARM Template Export/Import, Azure Devops Repos) ✓

9) Global parameters ✓

10) Credentials ✓

11) Monitoring ADF Jobs ✓

12) Alerts ✓

13) Send Failure Notifications using Logic Apps ✓

14) Data Flows - covering what they are, mapping data flows, debug mode, and various transformations like Filter, Aggregate, Join, Conditional Split, Derived Column, Exists, Union, Lookup, and Sort ✓

, plus GroupBy, Pivot, Unpivot, Flatten, Parse, Stringify, Filter sort, and Alterrow ✓. I'm also covering schema validation and duplicate row removal, though Asset transformations aren't included yet. Flowlets are covered ✓

15) Azure Devops - Repos ✓

16) SDLC ✓

17) Agile Methodology ✓

18) ADF Interview Questions - still need to add common Q&A

19) ADF Resume Preparation ✓

20) End TO End ADF Project ✓

21) ADF Exercises - in progress

Now I'm going through the 47 exercises to verify coverage. The first 11 are all accounted for: variable creation, conditional logic, looping through files, linked services and datasets, plus various copy activity scenarios including blob-to-blob transfers, blob-to-SQL, pattern matching, filtered formats, multiple files, source deletion, and parameterized datasets ✓

Continuing with exercises 12-22: file format conversion, adding columns, filtering and copying between blobs, deleting oversized

files, metadata retrieval, bulk operations, Key Vault integration, integration runtime setup, on-premises to cloud migration, Databricks activities with parameters, and scheduling triggers are all covered 

Now looking at the remaining topics: tumbling window and event-based triggers are explained, though "with Activity" needs clarification; Until Activity is covered; dataflows exercises include filtering, joins, unions, lookups, pivot/unpivot, and alter rows transformations, but selecting rows and window functions weren't explicitly covered as hands-on exercises.

The rest of the checklist covers parameters, alerts, global settings, ARM templates, DevOps integration, repos, logic app notifications, pipeline monitoring and debugging, scheduling with triggers, and executing pipelines within pipelines—most marked complete except trigger dependencies which needs deeper coverage. I'm also noticing two significant gaps: the ADF interview questions section from the PDF is completely missing, and the "asset" transformation in Data Flows wasn't included.

Looking at what still needs to be addressed, I see several Data Flow transformations haven't been covered yet—the asset transformation, window functions, and the select transformation for filtering rows. There's also "With Activity" in ADF and trigger dependency that require more thorough treatment. Plus I need to add a full ADF interview questions section and ensure all 47 exercises have complete step-by-step details.

ADF — Missing Concepts & Complete Exercises Review

Let me carefully audit everything against the PDF first, then cover all missing items in depth.



AUDIT: What Was Covered vs What's Missing

MISSING ITEMS IDENTIFIED:

- ✗ Data Flow – **SELECT** transformation (Exercise 27)
- ✗ Data Flow – **WINDOW** FUNCTIONS transformation (Exercise 32)
- ✗ Data Flow – **ASSET** transformation (PDF pg.6: "filter sort, alterrow, asset")
- ✗ "With Activity" – Exercise 25
- ✗ **Trigger** Dependency – Exercise 46
- ✗ ADF Interview Questions – **Full** Section 18 (completely missing)
- ✗ **All** 47 Exercises – **not** explained step-by-step **in** depth



MISSING DATA FLOW TRANSFORMATIONS



Select Transformation (Data Flows)

The **Select transformation** is used to **choose, rename, reorder, or remove columns** from your data stream — like `SELECT col1, col2 AS alias FROM table` in SQL.

When to use:

- Remove unnecessary columns before writing to sink (reduce storage cost)
- Rename columns to match destination schema
- Reorder columns to match expected output format
- Remove duplicate column names that arise after joins

Real-World Use Case:



Your source Oracle table has 85 columns but you only need 12 for your data warehouse.
Without Select, all 85 columns flow to destination — wasting storage and slowing processing.

Input Columns (85 total):

CUST_ID, CUST_NM, CUST_EMAIL, CUST_PHONE,
INTERNAL_SYS_COL1, INTERNAL_SYS_COL2... (73 unwanted cols)

Select Transformation:

Input Column	→	Output Column Name
--------------	---	--------------------

CUST_ID	→	customer_id
CUST_NM	→	customer_name
CUST_EMAIL	→	email_address
CUST_PHONE	→	phone_number
ORD_AMT	→	order_amount
ORD_DT	→	order_date
PROD_CD	→	product_code
PROD_NM	→	product_name
REGN_CD	→	region_code
STATUS_FLG	→	status
CRT_DT	→	created_date
UPD_DT	→	updated_date
(73 cols deleted)	→	(not mapped = dropped)

Output: Only 12 clean, properly named columns

Rule-based mapping (powerful feature):

Instead of mapping columns one-by-one, use rules to bulk-rename:

Rule 1: Column names starting with "CUST_"
→ Replace "CUST_" with "customer_"
→ CUST_ID becomes customer_id automatically

Rule 2: All column names → convert to lowercase
→ Applies to entire stream in one rule

Duplicate input handling (after joins):

When you join two streams both having a column named `id`, Select lets you:

```
Left.id    → customer_id    (keep and rename)
Right.id   → product_id     (keep and rename)
```

✅ Window Functions Transformation (Data Flows)

Window functions perform calculations **across a set of rows related to the current row** — without collapsing the rows like Aggregate does. Every row keeps its individual identity while getting a calculated value.

Real-World Analogy: In a school exam — Window functions let you add "class rank" to each student's record without removing any student from the list. Aggregate would only give you "class average" and lose individual records.

Key difference: Window vs Aggregate

Raw Data:

region	product	sales
North	Laptop	50000
North	Phone	30000
North	Tablet	20000
South	Laptop	40000
South	Phone	35000

AGGREGATE result (rows collapse):

region	total_sales
North	100000
South	75000

WINDOW result (rows preserved, new column added):

region	product	sales	region_total	rank_in_region
North	Laptop	50000	100000	1
North	Phone	30000	100000	2
North	Tablet	20000	100000	3
South	Laptop	40000	75000	1
South	Phone	35000	75000	2

ADF Data Flow Window Transformation Configuration:

Window Transformation: WND_SalesRanking

OVER (Partition By): region

→ Calculations done separately **per** region

ORDER BY: sales **DESC**

→ Rank highest sales **as 1**

RANGE: Unbounded preceding **to current row**

→ Standard **window** frame **for** ranking

WINDOW COLUMNS:

rank_in_region	= rank()
dense_rank	= denseRank()
row_number	= rowNumber()
region_total	= sum(sales)
region_avg	= avg(sales)
running_total	= cumSum(sales)

```
prev_sales      = lag(sales, 1)
next_sales      = lead(sales, 1)
pct_of_region   = sales / sum(sales) * 100
```

Common Window Functions in ADF Data Flows:

Function	Description	Example
<code>rank()</code>	Rank with gaps (1,1,3)	Sales ranking with ties
<code>denseRank()</code>	Rank without gaps (1,1,2)	Competition ranking
<code>rowNumber()</code>	Unique sequential number	Dedup (keep row 1)
<code>lag(col, n)</code>	Value from n rows behind	Compare with previous month
<code>lead(col, n)</code>	Value from n rows ahead	Compare with next month
<code>cumSum(col)</code>	Running total	Cumulative sales
<code>sum(col)</code>	Total over partition	Region total in every row
<code>avg(col)</code>	Average over partition	Dept average salary
<code>first(col)</code>	First value in window	First order per customer
<code>last(col)</code>	Last value in window	Most recent order
<code>nTile(n)</code>	Divide into n buckets	Top 25% customers
<code>percentRank()</code>	Percentile rank (0 to 1)	Performance percentile

Real-World Use Cases:

Use Case 1 — Finding Latest Record per Customer:

```
Window: PARTITION BY customer_id, ORDER BY updated_date DESC
rowNumber = rowNumber()
→ Filter where rowNumber = 1
→ Gives latest record per customer (SCD Type 2 current record)
```


Use Case 2 — Running Total for Finance Reports:

```
Window: PARTITION BY account_id, ORDER BY transaction_date
running_balance = cumSum(transaction_amount)
→ Each row shows account balance at that point in time
```

Use Case 3 — Month-over-Month Growth:

```
Window: PARTITION BY product_id, ORDER BY year_month
prev_month_sales = lag(sales_amount, 1)
growth_pct = (sales_amount - prev_month_sales) / prev_month_sales
* 100
→ Each row shows % growth vs previous month
```

✅ Asset Transformation (Data Flows)

The **Assert transformation** (referred to as "asset" in the PDF) is used to **enforce data quality rules** — it validates data against conditions and either flags or fails rows that don't meet your quality standards.

Real-World Analogy: Like a quality inspector on a factory production line — each product (row) is checked against standards. Bad products are either rejected (fail pipeline) or tagged as defective (flag and continue).

Two types of Assert:

1. EXPECT type → Flags rows that violate rule (continues processing)
2. FILTER type → Removes rows that violate rule (rows dropped)
3. ERROR type → Throws error and stops pipeline if any row violates

Configuration Example:

Assert Transformation: DQ_SalesValidation

Rule 1: sales_amount_check

Type: Expect (flag bad rows, don't stop)

Expression: sales_amount > 0

Description: "Sales amount must be positive"

Rule 2: date_format_check

Type: Expect

Expression: !isNull(order_date)

Description: "Order date cannot be null"

Rule 3: customer_id_check

Type: Error (stop if violated)

Expression: !isNull(customer_id) && length(customer_id) == 10

Description: "Customer ID required, must be 10 chars"

Rule 4: region_check

Type: Filter (remove invalid rows)

Expression: in(['NORTH', 'SOUTH', 'EAST', 'WEST'], region_code)

Description: "Only valid region codes allowed"

Output streams from Assert:

Assert transformation produces TWO output streams:

Stream 1: "Passed" rows → continue to Silver layer

Stream 2: "Failed" rows → write to error/quarantine table

Passed rows: Go to normal processing pipeline

```
Failed rows: Written to /quarantine/dq_errors/  
Alerting team about data quality issues
```

Real-World Use Case: A bank's transaction data comes from multiple branches. Some branches send negative transaction amounts (data entry errors). Assert catches these, flags them, writes them to a separate "exceptions" table for manual review, while valid transactions continue processing normally — no pipeline failure, no data loss.

✅ "With Activity" in ADF

The **"With Activity"** (also called **Set Variable with Expression / Script Activity**) in recent ADF context refers to the ability to **chain activity outputs directly into expressions without needing separate variable steps**.

However, in ADF exercises context, "With Activity" most likely refers to the **Script Activity** introduced in newer ADF versions — which lets you run **SQL scripts directly** in a pipeline without needing a Stored Procedure.

Script Activity — Deep Dive:

```
Activity: Script  
Linked Service: LS_AzureSQLDB_SalesDB  
Script Type: NonQuery (for INSERT/UPDATE/DELETE/DDL)  
Query (for SELECT – returns results)
```

Script:

```
-- Truncate staging table before load  
TRUNCATE TABLE dbo.STG_Sales;  
  
-- Create index if not exists  
IF NOT EXISTS (  
    SELECT 1 FROM sys.indexes
```

```

        WHERE name = 'IX_Sales_Date'
    )
BEGIN
    CREATE INDEX IX_Sales_Date ON dbo.Sales(order_date);
END

-- Update audit log
UPDATE dbo.PipelineLog
SET status = 'COMPLETED',
    end_time = GETDATE(),
    rows_processed = @{activity('CopyData').output.rowsCopied}
WHERE run_id = '@{pipeline().RunId}';

```

Why Script Activity over Stored Procedure Activity?

Stored Procedure:	Must pre-create procedure in DB → needs DBA access
Script Activity:	Write SQL directly in ADF → no pre-created objects needed
	Can run multiple SQL statements in one activity
	Dynamic SQL using ADF expressions possible

Real-World Use Case:

Pipeline: PL_Daily_Load

Step 1: Copy Data (load to staging table)

Step 2: Script Activity

- MERGE staging into production table
- Log row counts to audit table
- Drop staging table

All 3 SQL operations in one Script Activity

✓ Trigger Dependency — Deep Dive (Exercise 46)

Trigger Dependency allows you to **chain Tumbling Window triggers** — one trigger waits for another to complete successfully before firing. This solves the problem of dependent data pipelines needing to run in a specific order.

Real-World Problem:

Pipeline A: Loads raw sales data (runs hourly)

Pipeline B: Aggregates sales data (needs A to finish first)

Without dependency: B might start at 3:00 PM while A is still loading

→ B reads incomplete data → WRONG results!

With dependency: B's trigger waits for A's trigger to succeed
→ Only when A completes at 3:47 PM does B start

→ Always correct data

Configuration:

Trigger A: TRG_Hourly_RawSalesLoad

Type: Tumbling Window

Period: 1 Hour

No dependencies

Trigger B: TRG_Hourly_SalesAggregation

Type: Tumbling Window ← ONLY Tumbling Window supports dependencies!

Period: 1 Hour

DEPENDENCY:

Depends **On**: TRG_Hourly_RawSalesLoad

Size: **1 Hour** (same **window** size)

Offset: **0** (same **time window**)

Execution timeline with dependency:

3:00 PM → **Trigger** A fires → Pipeline A starts loading raw data

3:47 PM → Pipeline A completes successfully

3:47 PM → **Trigger** B sees its dependency (A **for 3-4 PM window**) **is** met

3:47 PM → **Trigger** B fires → Pipeline B starts aggregation

4:12 PM → Pipeline B completes

If Pipeline A FAILS **at 3:22** PM:

→ **Trigger** B **for 3-4 PM window** does **NOT** fire

→ **Trigger** B waits until A **succeeds** (retry)

→ Business-critical: aggregation never runs **on** incomplete data

Offset dependency (chain across different time windows):

Trigger B: TRG_Daily_Report

Depends **on**: TRG_Hourly_RawSalesLoad

Size: **1 Hour**

Offset: **-1 Hour**

Meaning: Daily report **trigger** waits **for ALL** hourly loads **of** the **day**

(the **-1 hour offset** means "previous window must succeed")

Multi-level dependency chain:

```
TRG_Extract (6 AM)
    ↓ depends on
TRG_Transform (must wait for Extract)
    ↓ depends on
TRG_Load (must wait for Transform)
    ↓ depends on
TRG_Report (must wait for Load)
```

Important Notes:

- Only **Tumbling Window triggers** support dependencies (not Schedule triggers)
- Maximum **20 dependencies** per trigger
- Creates a **self-healing pipeline** — if step 2 fails, step 3 automatically waits and retries when step 2 succeeds

✅ ADF Interview Questions — Complete Section 18

This is a **critical section** for job interviews. Here are the most important questions with detailed answers:

📌 Fundamentals

Q1: What is Azure Data Factory? How is it different from SSIS?

ADF:

- ✅ Cloud-native, fully managed PaaS service
- ✅ No infrastructure to manage
- ✅ 90+ connectors including cloud services, REST APIs, SaaS
- ✅ Built-in Spark engine (Data Flows)

- ✓ Pay-per-use (cost-effective for intermittent pipelines)
- ✓ CI/CD with Azure DevOps
- ✓ Handles big data natively

SSIS (SQL Server Integration Services):

- ✗ On-premise tool, requires Windows Server + SQL Server license
- ✗ You manage servers, patching, scaling
- ✗ Limited cloud connectors
- ✗ Does not scale for big data
- ✗ Fixed infrastructure cost
- ✓ Better for legacy on-premise workloads
- ✓ Complex transformations with scripting

Answer: ADF is the cloud evolution of SSIS – for modern cloud-based and hybrid data integration scenarios.

Q2: What are the different types of Integration Runtime? When do you use each?

1. Azure IR (AutoResolve):

- Cloud-to-cloud movement
- When source AND destination are both in Azure
- Example: ADLS Gen2 to Azure SQL Database

2. Azure IR with Managed VNet:

- When Azure resources use private endpoints
- No public internet exposure
- Bank/healthcare compliance requirements

3. Self-Hosted IR:

- On-premise to cloud movement
- Source is behind corporate firewall
- Example: On-premise Oracle → Azure Synapse

4. Azure-SSIS IR:

- Running legacy SSIS packages **in** cloud
- Migration period (before full ADF conversion)

Q3: What is the difference between a Linked Service and a Dataset?

Linked Service = CONNECTION definition

- Server name, credentials, authentication **method**
- HOW **to connect**
- Reused across multiple datasets
- Example: Connection **to SQL** Server **at 192.168.1.100**

Dataset = DATA definition

- **Specific table**, file, folder **to** access
- WHAT **to** access
- Points **to** a Linked Service
- Example: **Specific table** [dbo].[Sales] **on** that **SQL** Server

Analogy:

Linked Service = Library membership card (access **to** library)

Dataset = **Specific** book **in** that library

Q4: What is the difference between Schedule Trigger and Tumbling Window Trigger?

Schedule **Trigger**:

- Fires **at** fixed times (daily, hourly, weekly)
- Does **NOT** track **window** state

- If pipeline fails, **no** automatic retry **of** that **time window**
- Can be attached **to** multiple pipelines
- **No** self-dependency support
- Best **for**: Simple scheduled runs (daily report, weekly load)

Tumbling **Window Trigger**:

- Fires **for** fixed-size, non-overlapping **time** windows
- TRACKS **window** state (knows which windows ran/failed)
- Retries failed windows automatically
- Can BACKFILL past data (rerun Jan **1** **to** today)
- Supports DEPENDENCY chaining **between** triggers
- Best **for**: **Time**-series data, hourly incremental loads, reprocessing scenarios

Key Interview Insight:

Use Tumbling **Window when** data has a **time** dimension **and** you need exactly-once processing guarantees.

Q5: Explain the Copy Activity in detail. What are the key configurations?

Copy Activity moves data **from** Source → Sink **using** IR **as** execution engine.

Key Configurations:

SOURCE:

- Dataset (what **to** read)
- Source-**specific** options (query, stored proc, **partition**)
- Additional columns (**add** metadata columns)

SINK:

- Dataset (**where to** write)

- Write behavior (`insert`/`upsert`/`overwrite`/`append`)
- Pre/Post `copy` scripts (`SQL` to run before/after `copy`)
- Bulk `insert table` lock

MAPPING:

- `Column-to-column` mapping
- Data type conversions
- Schema drift handling

SETTINGS:

- Parallel `copy` degree (how many threads)
- Data consistency `check`
- Fault tolerance (`skip` bad `rows`)
- Logging (log skipped `rows` to storage)
- Staging (use Azure Storage `as` staging `for` PolyBase)

PERFORMANCE FACTORS:

- Integration Runtime size/count
- Parallel `copy` settings
- `Partition` options (parallel read `from` source)
- Staging `for` Synapse loads (PolyBase/`COPY INTO`)

Q6: How do you handle incremental data loads in ADF?

Method 1: Watermark-based (High Watermark)

- Store `last` loaded `timestamp` in a control `table`
- `Each` run: `SELECT * FROM source WHERE updated_date > last_watermark`
- After load: `Update` watermark to current `timestamp`

Implementation:

Step 1: Lookup → Read `last` watermark `from` control `table`

Step 2: Copy Data → Source query uses watermark value

Step 3: Stored Procedure → Update watermark after success

Method 2: Change Data Capture (CDC)

- Enable CDC on SQL Server source
- ADF reads only changed rows (inserts/updates/deletes)
- Highly efficient – no full table scan

Method 3: File-based incremental (Auto Loader pattern)

- New files arrive in ADLS daily
- ADF tracks which files are processed (via control table)
- Only new unprocessed files are loaded

Method 4: Tumbling Window Trigger

- Each window processes only that time period's data
- Built-in tracking ensures no duplicates

Q7: What is Schema Drift in Data Flows? How do you handle it?

Schema Drift = When source data schema changes unexpectedly (new columns added, data types changed, columns removed)

Without Schema Drift handling:

- Pipeline FAILS when new column arrives
- Manual fix required → pipeline downtime

With Schema Drift enabled:

- New columns are automatically passed through
- Pipeline continues without breaking
- Extra columns land in destination as-is

Configuration in Data Flow:

Source → Allow Schema Drift: ON

Sink → Auto-map drifted columns: ON

Advanced: Schema drift with explicit handling

→ Use `byName('columnName')` to access columns dynamically

→ Use `mapAssociation()` for flexible column mapping

→ Use `toString(byNames(['col1', 'col2']))` for dynamic selection

Real-world: Upstream source team added 3 new columns to

CSV export without telling data engineering team.

Schema drift = pipeline keeps running, new columns flow through.

Q8: How do you parameterize ADF pipelines? Why is it important?

Parameterization levels:

1. Pipeline Parameters → values passed when pipeline runs/triggered
2. Dataset Parameters → values for file paths, table names
3. Linked Service Parameters → server names, database names
4. Global Parameters → environment-level constants

Why important:

- ✓ Reusability: One pipeline handles 100 tables instead of 100 pipelines
- ✓ Environment promotion: Same pipeline, different env parameter files
- ✓ Dynamic paths: Process yesterday's file automatically using date
- ✓ Maintainability: Change logic in one place

Example:

Without params: 365 pipelines for 365 daily files

```
With params:      1 pipeline, pass date as parameter →  
@{pipeline().parameters.p_date}
```

Q9: How do you implement error handling in ADF?

1. Activity-level retry:

- Set Retry = 3, Retry Interval = 30 sec
- Handles transient failures (network blip, timeout)

2. Failure path (On Failure dependency):

- Right-click connection arrow → change to "On Failure"
- Add notification activity on failure path

Pipeline:

CopyData [Success] → UpdateAudit

CopyData [Failure] → LogError → SendAlert → Fail Activity

3. Try-Catch pattern:

- Execute Pipeline activity wraps the risky pipeline
- If child pipeline fails, parent catches and handles

4. Fault Tolerance in Copy Activity:

- Skip incompatible rows
- Log skipped rows to storage for investigation
- Pipeline continues despite bad data rows

5. Validation before processing:

- GetMetadata → check file exists + size > 0
- If file missing → Fail with clear error message
- Prevents misleading "success" with empty results

6. Alerts:

- Azure Monitor alerts **for** pipeline failures
- Logic Apps **for** detailed email notifications

Q10: What are Data Flows? When to use Data Flow vs Databricks?

Data Flows:

- ✓ Visual, **no**-code transformation tool **in** ADF
- ✓ Runs **on** managed Spark clusters (auto-provisioned)
- ✓ Good **for**: medium complexity transformations
- ✓ Business users / less experienced developers can use
- ✓ Built-**in** data preview **for** debugging
- ✓ Handles datasets up **to** ~TB **range** well
- ✗ Less flexible than writing code
- ✗ Limited custom logic (**no** arbitrary Python/Scala)
- ✗ Cluster startup **time** (2-4 minutes)

Azure Databricks:

- ✓ **Full** PySpark/Scala/**SQL** programming power
- ✓ ML, streaming, complex business logic
- ✓ Better performance **for** very **large** data (**10TB+**)
- ✓ Delta Lake, Unity Catalog integration
- ✓ More cost control **over** cluster configuration
- ✗ Requires coding knowledge
- ✗ Separate workspace **to** manage

Rule **of** thumb:

- Simple-medium **SQL-like** transformations → Data Flow
- Complex logic, ML, streaming, very **large** data → Databricks
- Enterprise: Use **BOTH** – ADF orchestrates, Databricks transforms

Q11: How do you pass parameters from ADF to Databricks Notebook?

In ADF Databricks Notebook Activity:

Base Parameters:

```
input_path:  @{{concat('/raw/sales/',
formatDateTime(utcnow(), 'yyyy/MM/dd'))}}
output_path: @{{concat('/silver/sales/',
formatDateTime(utcnow(), 'yyyy/MM/dd'))}}
run_date:    @{{formatDateTime(utcnow(), 'yyyy-MM-dd')}}
env:         @{{pipeline().globalParameters.gp_environment}}
```

In Databricks Notebook (receive parameters):

```
dbutils.widgets.text("input_path", "")
dbutils.widgets.text("output_path", "")
dbutils.widgets.text("run_date", "")

input_path = dbutils.widgets.get("input_path")
output_path = dbutils.widgets.get("output_path")
run_date    = dbutils.widgets.get("run_date")

print(f"Processing: {input_path} for date: {run_date}")
```

Q12: What is the difference between ForEach sequential and parallel execution?

Sequential (Batch Count = 1):

- Items processed ONE at a time
- Item 1 completes → Item 2 starts → Item 3 starts...
- Slower but safe when items share resources

Use when:

- Items write to same database table (prevent deadlocks)
- Source system has API rate limits

→ Order of processing matters

Parallel (Batch Count = 4, 5, 8, etc.):

- Multiple items processed simultaneously
- Items 1,2,3,4 run at same time → when one finishes, next starts
- Much faster for independent items

Use when:

- Items are completely independent (different tables/files)
- Destination can handle concurrent writes
- Want to minimize total processing time

Max Batch Count: 50 (ADF limit)

Recommended: 4-8 for typical scenarios (avoid overwhelming source DB)

Real-world example:

Loading 50 tables in parallel with batch=5:

- 50 tables / 5 parallel = 10 rounds
- vs sequential = 50 rounds
- ~5x faster!

Q13: How do you implement SCD Type 1 and Type 2 in ADF?

SCD Type 1 (Overwrite – no history):

- Use UPSERT in Copy Activity Sink
- Or use Alter Row transformation in Data Flow

Data Flow approach for SCD1:

Source → Lookup (check if exists in target)

→ Alter Row:

Insert If: `isNull(target.customer_id)`

```
Update If: !isNull(target.customer_id) && anyChanged
→ Sink (with UPSERT policy)
```

SCD Type 2 (Keep history with effective dates):

Data Flow approach:

Source → Lookup (find current active record)

→ Exists (check if record changed)

→ Derived Column:

For changed records:

is_current = false

end_date = currentDate()

For new records:

is_current = true

start_date = currentDate()

end_date = '9999-12-31'

→ Alter Row:

Update existing record: set is_current=false, end_date

Insert new version: is_current=true, new values

→ Sink

Q14: How do you optimize ADF pipeline performance?

1. Copy Activity optimization:

→ Enable parallel copy (degree of copy parallelism)

→ Use partition options for large tables (physical/dynamic range)

→ Use PolyBase/COPY INTO for Synapse (bulk load)

→ Enable staging for large loads

→ Choose Azure IR in same region as source/sink

2. Data Flow optimization:

→ Enable broadcast for small lookup tables

- Avoid unnecessary sort operations
- Use Persist/Cache **for** data reused multiple times
- Increase cluster size **for large** data (8 core+)
- Use optimized auto resolve IR

3. Pipeline optimization:

- Run independent activities **in** parallel
- Use ForEach **with** parallel batch count
- Avoid unnecessary Lookup activities **in** loops
- Use tumbling **window** triggers efficiently

4. Integration Runtime sizing:

- Self-Hosted IR: 8+ cores, 16+ GB RAM **for** production
- Multiple SHIR nodes **for** high availability + load balancing

5. Network optimization:

- Keep IR, source, **and** sink **in** same Azure region
- Use private endpoints **to** avoid public internet
- Use ExpressRoute **for on**-premise connections

Q15: Explain the ADF CI/CD process with Azure DevOps.

Development Flow:

1. Developer creates feature branch **from** main
2. Builds/**modifies** pipelines **in** ADF Dev (linked **to** feature branch)
3. Commits JSON files **to** feature branch
4. Creates Pull Request → team lead reviews JSON changes
5. PR approved → **merge to** main branch
6. CI Pipeline triggers automatically:
 - a. Validates ADF JSON files

- b. Exports ARM template **from** ADF Dev
 - c. Stores ARM template **in** Azure Artifacts
 - 7. CD Pipeline (**with** approval gates):
 - a. Deploy **to** ADF Test → run smoke tests
 - b. Manual approval gate (team lead approves)
 - c. Deploy **to** ADF Production
 - d. Run health checks
-

Environment-**specific** configurations:

Dev: ADF Dev → Dev Storage → Dev **SQL** DB
Test: ADF Test → Test Storage → Test **SQL** DB
Prod: ADF Prod → Prod Storage → Prod **SQL** DB

Managed via: ARM Template **Parameter** files **per** environment

ALL 47 ADF EXERCISES — Step-by-Step Deep Explanation

Exercise 1: Create Variables Using Set Variable Activity

Objective: Understand how pipeline variables work in ADF.

Step-by-step:

STEP 1: Create a new pipeline → PL_Ex01_Variables

STEP 2: Add pipeline variables (click canvas → Variables tab)

Variable 1: v_processDate → String → Default: ""

Variable 2: `v_rowCount` → Integer → Default: 0
Variable 3: `v_tableList` → Array → Default: []

STEP 3: Add "Set Variable" activity

Name: `SetProcessDate`

Variable Name: `v_processDate`

Value: `@{formatDateTime(utcnow(), 'yyyy-MM-dd')}`

→ This sets today's date as process date

STEP 4: Add another "Set Variable" activity (connect after first)

Name: `SetRowCount`

Variable Name: `v_rowCount`

Value: `1000`

→ Static value assignment

STEP 5: Debug and check Output tab

→ Click `SetProcessDate` output → shows value: `"2024-06-15"`

→ Click `SetRowCount` output → shows value: `1000`

KEY LEARNING:

- Variables are pipeline-scoped (not shared across pipelines)
- Variables can be read using `@{variables('v_processDate')}`
- Variables are mutable (can be changed multiple times in pipeline)
- Parameters are immutable (set once at pipeline start)

Exercise 2: How to Use If Condition Activity

Objective: Branch pipeline logic based on conditions.

SCENARIO: Check if today is Monday → if `yes`, run full load, else incremental

STEP 1: Create pipeline → PL_Ex02_IfCondition

STEP 2: Add Set Variable activity

Variable: v_dayOfWeek

Value: @{dayOfWeek(utcnow())}

→ Returns 0=Sunday, 1=Monday, 2=Tuesday...

STEP 3: Add If Condition activity (connect after Set Variable)

Name: CheckIfMonday

Expression: @{equals(variables('v_dayOfWeek'), 1)}

TRUE Activities (inside If → True branch):

→ Add Web Activity: CallFullLoadPipeline

URL: (or Execute Pipeline for full load)

FALSE Activities (inside If → False branch):

→ Add Web Activity: CallIncrementalLoad

STEP 4: Debug pipeline

→ If today is Monday → True branch executes

→ Any other day → False branch executes

ANOTHER PRACTICAL EXAMPLE:

Condition: @{greater(activity('GetMetadata').output.size, 0)}

True: → Proceed with Copy Data

False: → Send alert "File is empty" + Fail pipeline

Exercise 3: Iterating Files Using ForEach Loop Activity

Objective: Process multiple files/tables using ForEach.

SCENARIO: Copy 5 specific CSV files from source to destination

STEP 1: Create pipeline → PL_Ex03_ForEach

STEP 2: Add Set Variable activity

Variable: v_fileList (Array type)

Value:

```
@{createArray('sales_jan.csv','sales_feb.csv','sales_mar.csv',  
              'sales_apr.csv','sales_may.csv')}
```

STEP 3: Add ForEach activity

Items: @{variables('v_fileList')}

Batch Count: 2 ← process 2 files in parallel

Sequential: No

STEP 4: INSIDE ForEach → Add Copy Data activity

Source:

Linked Service: LS_BlobStorage

Container: raw

File Name: @{item()} ← current item in loop

Sink:

Linked Service: LS_ADLS_Gen2

Container: processed

File Name: @{item()}

STEP 5: Debug and monitor

→ Watch 2 files copy simultaneously

→ Each iteration shows @{item()} = current filename

BETTER APPROACH (dynamic from Lookup):

Lookup activity reads file list from control table

ForEach iterates over lookup output:

```
Items: @{activity('LookupFiles').output.value}  
Inner item reference: @{item().file_name}
```

Exercise 4: Creating Linked Services and Datasets

EXERCISE 4A: Create Linked Service for Azure Blob Storage

ADF Studio → Manage → Linked Services → + New

Type: Azure Blob Storage

Name: LS_BlobStorage_Raw

Authentication: Account Key

Storage Account: (select from dropdown)

→ Test Connection → Create

EXERCISE 4B: Create Linked Service for Azure SQL Database

→ + New

Type: Azure SQL Database

Name: LS_AzureSQLDB_Sales

Server: youserver.database.windows.net

Database: SalesDB

Authentication: SQL Authentication

Username: sqladmin

Password: (or reference Key Vault secret)

→ Test Connection → Create

EXERCISE 4C: Create Dataset for CSV file in Blob

Author → Datasets → + New

Type: Azure Blob Storage → DelimitedText (CSV)

Name: DS_BlobCSV_Sales

Linked Service: LS_BlobStorage_Raw

File Path: raw/sales/
First Row as Header: Yes
→ Preview Data to verify

EXERCISE 4D: Create Dataset for SQL Table

→ + New
Type: Azure SQL Database
Name: DS_SQL_SalesTable
Linked Service: LS_AzureSQLDB_Sales
Table: [dbo].[Sales]
→ Preview Data to verify

EXERCISE 4E: Parameterized Dataset

→ Create Dataset DS_BlobCSV_Parameterized
→ Parameters tab → + New
 p_folderPath: String
 p_fileName: String
→ Connection tab:
 Container: raw
 Directory: @{dataset().p_folderPath}
 File: @{dataset().p_fileName}

Exercise 5: Copy Activity — Blob to Blob

SCENARIO: Copy a single CSV file from container "raw" to container "processed"

STEP 1: Create pipeline → PL_Ex05_BlobToBlob

STEP 2: Create two datasets:

DS_Source_BlobCSV:

Linked Service: LS_BlobStorage

Container: raw

File: sales_data.csv

DS_Sink_BlobCSV:

Linked Service: LS_BlobStorage

Container: processed

File: sales_data.csv

STEP 3: Add Copy Data activity

Source: DS_Source_BlobCSV

Sink: DS_Sink_BlobCSV

Copy behavior: PreserveHierarchy

STEP 4: Check Source tab

→ File Path Type: File path in dataset

STEP 5: Check Sink tab

→ Copy Behavior: PreserveHierarchy

→ Max concurrent connections: 1

STEP 6: Mapping tab

→ Import schema → verify column mapping

STEP 7: Debug → Check Output

→ filesRead: 1

→ filesWritten: 1

→ rowsCopied: (number of rows)

→ throughput: (MB/s)

→ duration: (seconds)

KEY LEARNINGS:

→ Copy activity output shows complete statistics

- Check "dataRead" and "dataWritten" bytes
- Monitor "throughput" to identify performance bottlenecks

Exercise 6: Copy Activity — Blob to Azure SQL

SCENARIO: Copy CSV from Blob into SQL Database table

STEP 1: Create SQL table first (in Azure SQL Database):

```
CREATE TABLE dbo.Sales_Staging (  
    sale_id          INT,  
    customer_id      VARCHAR(10),  
    product_code     VARCHAR(20),  
    sale_amount      DECIMAL(18,2),  
    sale_date        DATE,  
    region           VARCHAR(50)  
);
```

STEP 2: Create Datasets

DS_Source: CSV in Blob (with schema)

DS_Sink: dbo.Sales_Staging SQL table

STEP 3: Copy Activity

Source: DS_Source (CSV)

Sink: DS_Sink (SQL Table)

Sink Settings:

Write Behavior: Insert

Pre-copy Script: TRUNCATE TABLE dbo.Sales_Staging

→ Clears table before loading fresh data

Bulk Insert Table Lock: Yes ← faster bulk load

Batch Size: 100000 ← insert 100K rows per batch

STEP 4: Mapping

CSV col: sale_id → SQL col: sale_id (INT)
CSV col: cust_id → SQL col: customer_id (VARCHAR)
→ Map all columns, fix any type mismatches

STEP 5: Settings

Fault Tolerance: Skip incompatible rows
Log Skipped Rows: Yes → write to /logs/skipped_rows/

STEP 6: Debug + Verify

→ Run pipeline
→ Query SQL: SELECT COUNT(*) FROM dbo.Sales_Staging
→ Should match source file row count

Exercise 7: Copy Activity — Pattern Matching Files

SCENARIO: Copy ONLY files matching pattern sales_2024*.csv
(wildcard)

KEY CONCEPT: Wildcard path allows copying multiple
files matching a pattern in one Copy activity

STEP 1: Dataset DS_WildcardSource

Linked Service: LS_BlobStorage
Container: raw
→ Leave file path EMPTY in dataset
→ We'll use wildcard in Copy Activity Source tab

STEP 2: Copy Activity Source tab

File Path Type: Wildcard file path
Wildcard Folder Path: sales/2024/

Wildcard File Name: `sales_2024*.csv`

This matches:

- ✓ `sales_20240101.csv`
- ✓ `sales_20240215.csv`
- ✓ `sales_2024_final.csv`
- ✗ `sales_2023_jan.csv` (doesn't match)
- ✗ `orders_2024.csv` (doesn't match prefix)

STEP 3: Sink - `DS_ProcessedBlob`

Container: `processed`

Copy Behavior: `PreserveHierarchy`

STEP 4: Debug

- Output shows: `filesRead`: (count of matched files)
- All matching files copied in one activity run

ADVANCED PATTERNS:

- `*.csv` → all CSV files in folder
- `sales_*.csv` → all files starting with "sales_"
- `*_2024_*.csv` → files containing "_2024_"
- `????_data.csv` → files with exactly 4 chars then "_data.csv"

Exercise 8: Copy Activity — Copy Filtered File Formats

SCENARIO: Source has mixed file formats (`.csv`, `.json`, `.txt`, `.parquet`)

Copy ONLY `.csv` and `.parquet` files, skip others

APPROACH 1: `ForEach` with `Get Metadata`

STEP 1: `GetMetadata` on source folder

Field: childItems (lists all files in folder)

STEP 2: ForEach over childItems

STEP 3: Inside ForEach → If Condition

Condition:

```
@{or(
  endsWith(item().name, '.csv'),
  endsWith(item().name, '.parquet')
)}
```

TRUE → Copy Data activity

FALSE → (do nothing / log skip)

APPROACH 2: Filter Activity (simpler)

After GetMetadata:

Filter Activity:

```
Items: @{activity('GetMetadata').output.childItems}
Condition: @{or(endsWith(item().name, '.csv'),
  endsWith(item().name, '.parquet'))}
```

ForEach over filter output:

```
Items: @{activity('FilterFormats').output.value}
Inner: Copy Data using @{item().name}
```

Exercise 9: Copy Activity — Copy Multiple Files from Blob to Blob

SCENARIO: Copy ENTIRE folder with all files and subfolders

STEP 1: Dataset DS_SourceFolder

Container: raw

Folder: /sales/ (folder, not specific file)

STEP 2: Dataset DS_DestinationFolder

Container: archive

Folder: /sales/

STEP 3: Copy Activity

Source:

File Path Type: Wildcard file path

Wildcard Folder: * ← all subfolders

Wildcard File: * ← all files

Recursive: TRUE ← go into subfolders

Sink:

Copy Behavior: PreserveHierarchy ← maintain folder structure

RESULT:

Source:

/raw/sales/

/2024/jan/s1.csv

/2024/jan/s2.csv

/2024/feb/s3.csv

/summary.csv

Destination:

/archive/sales/

/2024/jan/s1.csv

/2024/jan/s2.csv

/2024/feb/s3.csv

/summary.csv

Exercise 10: Copy Activity — Delete Source After Copy

SCENARIO: Copy file from landing zone → processing zone → delete from landing

PATTERN: Copy then Delete (only delete on success!)

STEP 1: GetMetadata → verify file exists and has data

STEP 2: Copy Data

Source: /landing/sales_20240615.csv

Sink: /raw/sales/2024/06/sales_20240615.csv

On Success → (connect to Step 3)

On Failure → (connect to error handling)

STEP 3: Delete Activity (ONLY on Copy success)

Dataset: DS_LandingZone

→ Same file as copy source

Recursive: False (just this one file)

CRITICAL SAFEGUARD:

→ Delete activity MUST be on success path only

→ Never delete before confirming copy succeeded!

→ Check copy output: filesWritten = 1 before deleting

VERIFICATION:

Add another GetMetadata after Delete:

→ Try to get metadata of deleted file

→ Should return "exists: false"

→ Confirms deletion worked

FULL PIPELINE FLOW:

GetMetadata → If Exists → Copy Data → Delete Source → Log Success

↓ No File

Fail Activity "File not found"

Exercise 11: Copy Activity — Using Parameterized Datasets

SCENARIO: One pipeline processes different tables daily using parameters

STEP 1: Create parameterized dataset DS_SQL_Dynamic

Parameters:

p_tableName: String
p_schemaName: String (default: dbo)

Table configuration:

Schema: @{dataset().p_schemaName}
Table: @{dataset().p_tableName}

STEP 2: Create parameterized dataset DS_ADLS_Dynamic

Parameters:

p_containerName: String
p_folderPath: String
p_fileName: String

File path:

@{dataset().p_containerName}/{dataset().p_folderPath}/{dataset().p_fileName}

STEP 3: Pipeline with parameters

Pipeline Parameters:

p_sourceTable: String
p_targetFolder: String
p_date: String

STEP 4: Copy Activity

Source Dataset: DS_SQL_Dynamic

Dataset Parameters:

p_tableName: @{pipeline().parameters.p_sourceTable}
p_schemaName: dbo

Sink Dataset: DS_ADLS_Dynamic

Dataset Parameters:

```
p_containerName: silver
p_folderPath:    @{pipeline().parameters.p_targetFolder}
p_fileName:
@{concat(pipeline().parameters.p_sourceTable, '_',
        pipeline().parameters.p_date,
        '.parquet')}
```

STEP 5: Run pipeline multiple times with different parameter values:

Run 1: p_sourceTable=Customers, p_date=20240615

Run 2: p_sourceTable=Orders, p_date=20240615

Run 3: p_sourceTable=Products, p_date=20240615

Exercise 12: Copy Activity — Convert File Format

SCENARIO: Convert CSV files to Parquet format for better analytics performance

WHY PARQUET?

CSV 1GB file → takes 45s to query 1 column in Spark

Parquet 350MB → takes 2s to query 1 column (columnar, compressed)

→ 70% smaller, 20x faster for analytics

STEP 1: Source Dataset DS_SourceCSV

Format: DelimitedText (CSV)

Container: raw

File: sales_data.csv

First row as header: Yes

STEP 2: Sink Dataset DS_SinkParquet

Format: Parquet

Container: silver

File: sales_data.parquet

Compression: Snappy (best balance of speed/size)

STEP 3: Copy Activity

Source: DS_SourceCSV

Sink: DS_SinkParquet

Mapping Tab:

→ Import schema from source

→ Map each column + set correct data types:

sale_id: string → int32

sale_amount: string → double

sale_date: string → date ← ADF converts string to proper date!

STEP 4: Debug + Verify

→ Original CSV: 500MB

→ Output Parquet: ~150MB (70% compression!)

OTHER FORMAT CONVERSIONS:

CSV → JSON: Source=CSV, Sink=JSON (with array of objects settings)

JSON → Parquet: Source=JSON, Sink=Parquet

CSV → Avro: Source=CSV, Sink=Avro (good for streaming systems)

CSV → ORC: Source=CSV, Sink=ORC (Hive-compatible format)

Exercise 13: Copy Activity — Add Additional Columns

SCENARIO: Source CSV has 5 columns.

Add 3 audit columns before loading to destination:

- load_date (today's date)
- source_system (where data came from)
- pipeline_run_id (for traceability)

STEP 1: Source Dataset DS_SalesCSV

Columns: sale_id, customer_id, product_code, amount, sale_date

STEP 2: Copy Activity Source tab

- Scroll down to "Additional Columns" section
- + New → Add:

Column Name: load_date

Value: @{{formatDateTime(utcnow(),'yyyy-MM-dd HH:mm:ss')}}

Column Name: source_system

Value: Oracle_ERP_Production

Column Name: pipeline_run_id

Value: @{{pipeline().RunId}}

Column Name: pipeline_name

Value: @{{pipeline().Pipeline}}

STEP 3: Mapping tab

- All original columns map to destination
- 4 new audit columns also appear → map to destination audit columns

RESULT in destination:

sale_id	customer_id	product_code	amount	sale_date	load_date	source_system	pipeline_run_id
-----	-----	-----	-----	-----	-----	-----	-----
1001	C001	LAPTOP01	50000	2024-06-15	2024-06-15 00:02:34	Oracle_ERP_Prod	abc123-def456...

KEY BENEFIT:

- Every row in destination knows WHEN it was loaded, WHERE it came from, and WHICH pipeline run loaded it
- Critical for data lineage and debugging

Exercise 14: Copy Activity — Filter Files and Copy

SCENARIO: Folder has 100 files. Copy ONLY files modified in last 24 hours.

STEP 1: GetMetadata on folder

Field: childItems (with lastModified for each file)

STEP 2: Filter Activity

Items: @{activity('GetMeta').output.childItems}

Condition:

```
@{greater(  
    item().lastModified,  
    addHours(utcnow(), -24)  
)}
```

→ Only files modified in last 24 hours pass through

STEP 3: ForEach over filtered items

Items: @{activity('FilterRecent').output.value}

STEP 4: Inside ForEach → Copy Activity

Source file: @{item().name}

ALTERNATIVE (filter by file size):

Condition: @{greater(item().size, 1024)} ← only files > 1KB

ALTERNATIVE (filter by file prefix):

```
Condition: @{startsWith(item().name, 'sales_')}
```

ADVANCED (multiple conditions):

```
@{and(  
  startsWith(item().name, 'sales_'),  
  endsWith(item().name, '.csv'),  
  greater(item().size, 0)  
)}
```

Exercise 15: Delete Files from Blob with More Than 100KB

SCENARIO: Cleanup job – delete large files (>100KB) from archive folder

STEP 1: GetMetadata on archive folder

Field: childItems

STEP 2: Filter Activity

Items: @{activity('GetMeta').output.childItems}

Condition: @{greater(item().size, 102400)}

→ 100KB = 102400 bytes

STEP 3: Set Variable to track count

v_deleteCount = @{activity('FilterLarge').output.filterCount}

STEP 4: If Condition

Condition:

@{greater(activity('FilterLarge').output.filterCount, 0)}

→ Only run delete if there are files to delete

STEP 5: TRUE branch → ForEach over large files

Inside ForEach:

Delete Activity:

Dataset: DS_ArchiveBlob

→ Use @{item().name} to reference each file

STEP 6: After ForEach → Stored Procedure

Log: "Deleted X files > 100KB on date Y"

MONITORING OUTPUT:

FilterLarge: filterCount: 5 (found 5 files > 100KB)

ForEach: 5 iterations

Each Delete: 1 file deleted

Final log: "Deleted 5 files > 100KB on 2024-06-15"

Exercise 16: How to Use GetMetadata Activity

COMPLETE REFERENCE: All GetMetadata fields explained

Dataset: pointing to ADLS Gen2 container/folder/file

AVAILABLE FIELDS TO REQUEST:

For FILES:

Field	Output	Use Case
exists	true/false	Check if file arrived
itemName	"sales.csv"	Get actual filename
itemType	"File"/"Folder"	Distinguish file vs folder
size	1048576	Check file has data (>0)
lastModified	"2024-06-15..."	Check if fresh file
contentMD5	"abc123..."	Verify file integrity

structure	[{col schema}]	Get column schema dynamically
columnCount	15	Count columns

For FOLDERS:

childItems	[{name, type}]	List all files/subfolders
itemCount	42	Count of items in folder

REFERENCING OUTPUT:

```
@{activity('GetMeta').output.exists}  
@{activity('GetMeta').output.size}  
@{activity('GetMeta').output.itemName}  
@{activity('GetMeta').output.lastModified}  
@{activity('GetMeta').output.childItems}
```

COMMON PIPELINE PATTERN:

GetMetadata

↓

If Condition: @{equals(activity('GetMeta').output.exists, true)}

↓ TRUE

CopyData

↓ FALSE

SendAlert + Fail Pipeline

↓

If Condition: @{greater(activity('GetMeta').output.size, 0)}

↓ TRUE

ProcessData

↓ FALSE

Fail "File is empty"

Exercise 17: Bulk Copy Tables and Files

SCENARIO: Copy ALL tables from on-premise SQL Server to ADLS Gen2
(Dynamic, metadata-driven approach)

ARCHITECTURE:

Control Table → Lookup → ForEach → Copy Each Table

STEP 1: Create control table in Azure SQL DB

```
CREATE TABLE dbo.CopyControlTable (  
    id          INT IDENTITY(1,1),  
    source_schema VARCHAR(50),  
    source_table  VARCHAR(100),  
    target_folder VARCHAR(200),  
    target_format VARCHAR(20), -- parquet/csv/json  
    is_active     BIT,  
    watermark_col VARCHAR(50), -- for incremental  
    last_watermark DATETIME  
);  
  
INSERT INTO dbo.CopyControlTable VALUES  
(  
    'dbo', 'Customers', 'silver/customers/', 'parquet', 1,  
    'UpdatedDate', '2000-01-01'),  
(  
    'dbo', 'Orders', 'silver/orders/', 'parquet', 1,  
    'OrderDate', '2000-01-01'),  
(  
    'dbo', 'Products', 'silver/products/', 'parquet', 1,  
    'ModifiedDate', '2000-01-01'),  
(  
    'dbo', 'Inventory', 'silver/inventory/', 'parquet', 1, NULL,  
    NULL),  
(  
    'dbo', 'ArchivedData', 'archive/', 'csv', 0, NULL,  
    NULL);
```

STEP 2: Lookup Activity

Query: SELECT * FROM dbo.CopyControlTable WHERE is_active = 1

Returns 4 active tables

STEP 3: ForEach

Items: @{activity('LookupTables').output.value}

Batch Count: 3 (copy 3 tables in parallel)

STEP 4: Inside ForEach → Copy Activity with parameterized datasets

Source Dataset: DS_SQL_Dynamic_Param

p_schema: @{item().source_schema}

p_table: @{item().source_table}

Source Query (for incremental):

```
@{if(
    equals(item().watermark_col, ''),
    concat('SELECT * FROM ', item().source_schema, '.',
item().source_table),
    concat('SELECT * FROM ', item().source_schema, '.',
item().source_table,
        ' WHERE ', item().watermark_col, ' > ',
item().last_watermark, ''))
    )}
```

Sink Dataset: DS_ADLS_Parquet_Dynamic

p_folder: @{item().target_folder}

p_file: @{concat(item().source_table, '_',
formatDateTime(utcnow(), 'yyyyMMdd'),
'parquet')}

STEP 5: After ForEach → Update watermark

Stored Procedure: Update last_watermark to current timestamp

For each table that was loaded

BENEFIT:

- Add new table to control table → automatically copied next run
- Disable table → set is_active=0 → skipped automatically
- 100% metadata-driven, zero code changes for new tables

Exercise 18: How to Integrate Key Vault in ADF

SCENARIO: Store SQL Server password in Key Vault, use in ADF Linked Service

STEP 1: Create Key Vault secret

Azure Portal → Key Vault → Secrets → Generate/Import

Name: sqlldb-password

Value: MySecureP@ssw0rd123

Content Type: text/plain

STEP 2: Grant ADF access to Key Vault

Key Vault → Access Policies → + Add Access Policy

Secret Permissions: GET, LIST

Principal: (select your ADF instance name)

→ ADF uses its Managed Identity to authenticate

STEP 3: Create Key Vault Linked Service in ADF

Manage → Linked Services → + New

Type: Azure Key Vault

Name: LS_AzureKeyVault

Azure Subscription: (select)

Azure Key Vault Name: (select your vault)

→ Test Connection (uses Managed Identity)

STEP 4: Use Key Vault secret in SQL Linked Service

Create/Edit Linked Service LS_AzureSQLDB:

Password field:

→ Click "Azure Key Vault" option (not manual entry)

→ AKV Linked Service: LS_AzureKeyVault

→ Secret Name: sqlldb-password

→ Secret Version: (blank = latest version)

STEP 5: Verify

→ Test Connection on LS_AzureSQLDB

- ADF fetches password **from** Key Vault at runtime
- Password NEVER stored **in** ADF – maximum security

BENEFIT:

Old Way: Password hardcoded **in** Linked Service → visible **in** ARM template

New Way: Only secret name stored → actual password **in** Key Vault
If password changes → update **Key** Vault only
ADF automatically gets **new** password → zero downtime

Exercise 19: How to Set Up Integration Runtime

EXERCISE A: Create Self-Hosted IR

STEP 1: ADF Studio → Manage → Integration Runtimes → + **New**

Type: Self-Hosted

Name: SHIR_OnPremise_Production

→ Create → Copy Authentication **Key 1**

STEP 2: **On** your **on**-premise server/VM:

- Download Microsoft Integration Runtime **from** official link
- Install (Windows only)
- After install → paste Authentication **Key**
- Click Register
- Status changes **to "Running"** **in** ADF Studio

STEP 3: Configure **for** High Availability (**optional** but recommended):

- Install SHIR **on** **2nd** server
- Paste same Authentication **Key**
- Now you have **2** nodes:
 - Node **1**: Server01 - Running

Node 2: Server02 - Running

→ If Node 1 goes down, Node 2 takes over automatically

STEP 4: Use SHIR in Linked Service:

LS_OnPremSQL:

Type: SQL Server

Server: 192.168.1.100 (internal IP)

Connect via IR: SHIR_OnPremise_Production

STEP 5: Test connection

→ ADF routes test through SHIR to on-prem server

→ "Connection successful" confirms tunnel is working

EXERCISE B: Manage Existing IR

→ Monitor IR status: Running / Limited / Offline

→ Check node details: CPU, Memory, concurrent jobs

→ View IR logs for troubleshooting

→ Update IR (auto-update or manual update option)

Exercise 20: Copy Data from On-Premises to Azure Cloud

FULL END-TO-END SCENARIO:

On-Prem SQL Server → (SHIR) → ADLS Gen2

PREREQUISITES:

- ✓ SHIR installed and connected (Exercise 19)
- ✓ On-prem SQL Server accessible from SHIR server
- ✓ ADLS Gen2 storage account created

STEP 1: Linked Service for On-Prem SQL Server

Type: SQL Server

Server: CORP-SQLSERVER-01 (or IP)
Database: LegacySalesDB
Auth: Windows Authentication (or SQL auth)
Connect via IR: SHIR_OnPremise_Production

STEP 2: Linked Service for ADLS Gen2

Type: Azure Data Lake Storage Gen2
URL: https://mydatalake.dfs.core.windows.net
Auth: Managed Identity (ADF managed identity)

STEP 3: Datasets

DS_OnPrem_SalesTable:

Linked Service: LS_OnPremSQL
Table: [dbo].[LegacySales]

DS_ADLS_SalesParquet:

Linked Service: LS_ADLS_Gen2
Container: raw
Path: legacy/sales/@{formatDateTime(utcnow(), 'yyyy/MM/dd')}/
Format: Parquet

STEP 4: Copy Activity

Source: DS_OnPrem_SalesTable

Partition option: Dynamic Range
Partition Column: SaleID (numeric PK)
Upper Bound: (from Lookup: SELECT MAX(SaleID))
Lower Bound: 1
Degree of Copy Parallelism: 4
→ Reads table in 4 parallel chunks = 4x faster!

Sink: DS_ADLS_SalesParquet

STEP 5: Monitor

→ Watch activity run in Monitor tab
→ Data routes: On-prem SQL → SHIR (compression/encryption) →

ADF → ADLS

- All data encrypted in transit (HTTPS)
- SHIR does NOT store data – it's just a secure relay

PERFORMANCE TIPS:

- Run SHIR on server closest to source DB (same network)
- Increase SHIR server CPU/RAM for large loads
- Use partition copy for tables > 1 million rows

Exercise 21: Use Databricks Activity and Pass Parameters

STEP 1: Create Databricks Linked Service

ADF Manage → Linked Services → + New

Type: Azure Databricks

Name: LS_AzureDatabricks_Prod

Databricks Workspace: (select from subscription)

Select cluster:

Option A: Existing interactive cluster → select cluster ID

Option B: New job cluster → configure cluster spec:

Cluster version: 13.3 LTS (Spark 3.4)

Node type: Standard_DS3_v2

Driver type: Same as worker

Workers: 2 (min) - 8 (max) autoscaling

Authentication: Managed Service Identity (MSI)

STEP 2: Add Databricks Notebook Activity to pipeline

General: Name = RunSalesTransformation

Azure Databricks:

Databricks Linked Service: LS_AzureDatabricks_Prod

Settings:

Notebook path: /Production/Sales/Transform_Sales_Data

Base Parameters:

input_path: @{{concat('/mnt/raw/sales/',

formatDateTime(utcnow(), 'yyyy/MM/dd'))}}

output_path: @{{concat('/mnt/silver/sales/',

formatDateTime(utcnow(), 'yyyy/MM/dd'))}}

process_date: @{{formatDateTime(utcnow(), 'yyyy-MM-dd')}}}

env: @{{pipeline().globalParameters.gp_environment}}

batch_id: @{{pipeline().RunId}}

STEP 3: In Databricks Notebook, receive parameters

Python notebook

dbutils.widgets.text("input_path", "/mnt/raw/sales/default")

dbutils.widgets.text("output_path",

"/mnt/silver/sales/default")

dbutils.widgets.text("process_date", "2024-01-01")

dbutils.widgets.text("env", "dev")

input_path = dbutils.widgets.get("input_path")

output_path = dbutils.widgets.get("output_path")

process_date = dbutils.widgets.get("process_date")

print(f"Processing: {input_path}")

print(f"Output to: {output_path}")

print(f>Date: {process_date}")

Your actual transformation code here

df = spark.read.parquet(input_path)

... transformations ...

df.write.parquet(output_path)

Return value back to ADF


```
dbutils.notebook.exit("SUCCESS: " + str(df.count()) + " rows processed")
```

STEP 4: Read Databricks output in ADF

Next activity can reference:

```
@{activity('RunSalesTransformation').output.runOutput}
```

→ Returns: "SUCCESS: 142850 rows processed"

Exercise 22: How to Use Scheduling Trigger

STEP 1: ADF Manage → Triggers → + New

Type: Schedule

Name: TRG_Daily_SalesLoad

STEP 2: Configure

Start Date/Time: 2024-01-01 00:00:00 UTC

Time Zone: India Standard Time

Recurrence: Every 1 Day

At: 11:30 PM (so data is ready by midnight)

End: Never

STEP 3: Attach to Pipeline

→ Click pipeline dropdown

→ Select: PL_Daily_SalesETL

→ Set pipeline parameters:

```
p_loadDate: @{formatDateTime(trigger().scheduledTime, 'yyyy-MM-dd')}
```

STEP 4: Publish + Activate

→ Publish All

→ Go to Trigger → Start

STEP 5: Verify

Monitor → Trigger Runs

→ Shows all trigger invocations with status

COMMON SCHEDULE PATTERNS:

Pattern	Config
Every 15 minutes	: Recur every 15 Minutes
Every hour	: Recur every 1 Hour
Daily at midnight	: Recur every 1 Day at 00:00
Weekdays only	: Recur weekly, Mon-Fri at 08:00
First of month	: Recur monthly on day 1
Every 6 hours	: Recur every 6 Hours

Exercise 23: Tumbling Window Trigger

STEP 1: New Trigger

Type: Tumbling Window

Name: TRG_Hourly_SalesWindow

STEP 2: Configure

Start: 2024-01-01 00:00:00 UTC

Period: 1 Hour

Offset: 0 (no delay)

Max Concurrency: 4 ← 4 windows can run in parallel

Retry: 3

Retry Interval: 5 min

STEP 3: Pipeline parameter mapping

p_windowStart: @{triggerBody().window.windowStart}

```
p_windowEnd:    @{{triggerBody().window.windowEnd}}
```

In pipeline source query:

```
SELECT * FROM Sales
WHERE sale_datetime >= '@{pipeline().parameters.p_windowStart}'
      AND sale_datetime <  '@{pipeline().parameters.p_windowEnd}'
```

STEP 4: Backfill scenario (reprocess past data)

- Go to Trigger in Manage
- Click "Trigger Now" (custom start)
- Set: Start = 2024-01-01, End = today
- ADF creates ALL windows from Jan 1 to today
- Runs 4 at a time (max concurrency)
- Backfills entire history automatically!

Exercise 24: Event-Based Trigger

STEP 1: New Trigger

Type: Storage Event

Name: TRG_Event_SalesFileArrival

STEP 2: Configure

Storage Account: mydatalakestorage

Container: raw

Blob Path Begins With: /sales/incoming/

Blob Path Ends With: .csv

Event: Microsoft.Storage.BlobCreated

- Fires when new CSV file created in that path

Ignore Empty Blobs: Yes

→ Don't trigger for 0-byte files

STEP 3: Pipeline parameter mapping

```
p_fileName:    @{{triggerBody().fileName}}
p_folderPath:  @{{triggerBody().folderPath}}
```

Use in Copy Activity source:

```
File Path: @{{concat(pipeline().parameters.p_folderPath,
                    pipeline().parameters.p_fileName)}}
```

STEP 4: Activate trigger → Publish

STEP 5: Test

- Upload a CSV file to /raw/sales/incoming/
- Within 30-60 seconds, pipeline auto-starts!
- Monitor → Pipeline Runs → shows new run

USE CASE PATTERN (event-driven architecture):

Upstream system → drops file to ADLS landing zone

↓ (event trigger fires)

ADF validates + ingests file

↓

Databricks transforms

↓

Synapse loads (available for reports)

Total latency: 3-5 minutes from file drop to report availability!

Exercise 25: Script Activity (With Activity)

STEP 1: Add Script Activity to pipeline

Linked Service: LS_AzureSQLDB

STEP 2: Configure script (multiple SQL statements):

Script Type: NonQuery

Script Content:

```
-- Step 1: Truncate staging
TRUNCATE TABLE dbo.STG_Sales;

-- Step 2: Create temp index for performance
IF NOT EXISTS (SELECT 1 FROM sys.indexes WHERE
name='IX_STG_CustID')
    CREATE INDEX IX_STG_CustID ON dbo.STG_Sales(customer_id);

-- Step 3: Log pipeline start
INSERT INTO dbo.PipelineAudit
    (run_id, pipeline_name, start_time, status)
VALUES (
    '@{pipeline().RunId}',
    '@{pipeline().Pipeline}',
    GETDATE(),
    'RUNNING'
);
```

STEP 3: Script with Query type (returns data)

Script Type: Query

Script: SELECT COUNT(*) AS row_count FROM dbo.Sales

Output used in next activity:

```
@{activity('ScriptQuery').output.resultSets[0].rows[0].row_count}
```

Exercise 26: Until Activity

SCENARIO: Poll external API every 30 seconds until job is DONE

STEP 1: Set Variable

```
v_jobStatus = "RUNNING"
v_pollCount = 0
```

STEP 2: Until Activity

```
Condition: @equals(variables('v_jobStatus'), 'COMPLETED')
Timeout: 02:00:00 (max 2 hours)
Delay: 00:00:30 (wait 30 seconds between each check)
```

Inside Until:

Activity A: Web Activity (call status API)

URL:

```
https://api.myservice.com/jobs/@{variables('v_jobId')}/status
```

Method: GET

```
Output: {"status": "RUNNING"} or {"status": "COMPLETED"}
```

Activity B: Set Variable

```
v_jobStatus = @{activity('CheckStatus').output.status}
```

Activity C: Set Variable

```
v_pollCount = @{add(int(variables('v_pollCount')), 1)}
```

Activity D: If Condition (safety check)

```
Condition: @greater(variables('v_pollCount'), 240)
```

TRUE: Fail Activity "Job timed out after 2 hours"

FLOW:

Start → v_jobStatus = "RUNNING"

Poll 1 (0:30): status = RUNNING → continue loop

Poll 2 (1:00): status = RUNNING → continue loop

Poll 3 (1:30): status = RUNNING → continue loop

Poll 4 (2:00): status = COMPLETED → exit loop!

Continue to next activities...

Exercises 27–35: Data Flow Exercises

EXERCISE 27: Dataflows – Select Rows

- Create Data Flow DF_SelectColumns
- Source: Raw CSV (15 columns)
- Select transformation: pick only 8 needed columns
- Rename legacy column names to modern standard
- Sink: ADLS Parquet (8 columns only)

EXERCISE 28: Dataflows – Filter Rows

- Source: Sales data (all regions, all dates)
- Filter: region == 'INDIA' && sale_amount > 1000
- Sink: india_high_value_sales.parquet
- Use Data Preview to verify count reduced

EXERCISE 29: Dataflows – Join Transformations

- Source 1: Sales transactions
- Source 2: Customer master table
- Join: sales.customer_id == customer.id (Inner Join)
- Select: pick columns from both streams
- Sink: enriched_sales.parquet
- Practice: Inner, Left Outer, Right Outer, Full Outer, Cross

EXERCISE 30: Dataflows – Union Transformations

- Source 1: Sales_Q1 (Jan-Mar data)
- Source 2: Sales_Q2 (Apr-Jun data)
- Source 3: Sales_Q3 (Jul-Sep data)
- Union All three streams
- Sink: Sales_Q1_to_Q3_combined.parquet
- Verify: output rows = Q1 + Q2 + Q3 row counts

EXERCISE 31: Dataflows – Lookup Transformations

- Primary: Orders dataset
- Lookup: Product master (product_code → product_name, category, price)
- Lookup Type: Any (first match)
- Output: Orders enriched with product details
- Handle no-match rows: isNull(lookup.product_name) → 'UNKNOWN'

EXERCISE 32: Dataflows – Window Functions

- Source: Sales by product and month
- Window transformation:
 - PARTITION BY: product_category
 - ORDER BY: sale_month DESC
 - rank_in_category = rank()
 - cumulative_sales = cumSum(sale_amount)
 - prev_month_sales = lag(sale_amount, 1)
 - rolling_3mo_avg = avg(sale_amount) (window: 3 rows)
- Sink: product_sales_ranked.parquet

EXERCISE 33: Dataflows – Pivot and Unpivot

- PIVOT Exercise:
- Source: sales by region/product (long format)
 - Pivot:
 - Group by: region
 - Pivot col: product_category
 - Values: sum(sale_amount)

→ Output: wide table with one column per product category

UNPIVOT Exercise:

→ Source: Wide table with Electronics/Clothing/Food columns

→ Unpivot:

Unpivot columns: Electronics, Clothing, Food

Name column: product_category

Value column: sale_amount

→ Output: Long format with product_category + sale_amount rows

EXERCISE 34: Dataflows – Alter Row Transformations

→ Source: Customer updates file

→ Lookup: Current customer master

→ Alter Row rules:

Insert If: isNull(target.customer_id) ← new customer

Update If: !isNull(target.customer_id) &&

equalsIgnoreCase(source.status, 'ACTIVE')

Delete If: equalsIgnoreCase(source.status, 'DELETED')

Upsert If: false ← not using upsert

→ Sink: Customer master table (with allow insert, update, delete)

EXERCISE 35: Dataflows – Removing Duplicates

APPROACH 1: Using Window + Filter:

→ Window:

PARTITION BY: customer_id, order_date, product_code

ORDER BY: created_timestamp DESC

row_num = rowNumber()

→ Filter: row_num == 1

→ Result: Only most recent record per unique combination

APPROACH 2: Using Aggregate:

→ Aggregate:

GROUP BY: customer_id, order_date, product_code (all key cols)

```
max_created = max(created_timestamp)
max_amount = first(order_amount)
→ Result: Deduplicated rows
```

Exercises 36–47: Advanced Pipeline Exercises

EXERCISE 36: Pass Parameters to Pipeline

- Create pipeline with 3 parameters: p_date, p_env, p_table
- Trigger manually with "Trigger Now"
- Fill in parameter values in the popup
- Reference in activities: @{pipeline().parameters.p_date}
- Also: pass from parent pipeline to child via Execute Pipeline

EXERCISE 37: Create Alerts and Rules

- Azure Portal → ADF → Diagnostic Settings
- Enable: PipelineRuns, ActivityRuns, TriggerRuns → Log Analytics
- Create Alert:
 - Condition: Failed pipeline runs > 0
 - Evaluation: Every 5 minutes
 - Threshold: 0 (alert on any failure)
- Action Group:
 - Email: team@company.com
 - SMS: +919876543210
- Test: Run a pipeline that intentionally fails → check email

EXERCISE 38: Set Global Parameters

- ADF Studio → Manage → Global Parameters
- + New parameters:
 - gp_environment: Production
 - gp_dataLakeURL: https://myprodlake.dfs.core.windows.net
 - gp_notifyEmail: dataengg@company.com

gp_maxRetries: 3

→ Publish

→ Use in pipeline:

@{pipeline().globalParameters.gp_environment}

→ When deploying to different environments:

ARM template parameter file overrides global parameter values

EXERCISE 39: Import and Export ARM Templates

EXPORT:

→ ADF Studio → Author → ARM Template → Export ARM Template

→ Downloads zip with:

ARMTemplateForFactory.json (all resources)

ARMTemplateParametersForFactory.json (parameter values)

REVIEW the JSON:

→ Open ARMTemplateForFactory.json

→ See all pipelines, datasets, linked services as JSON

→ This IS your ADF configuration as code

IMPORT (deploy to new ADF):

→ Azure Portal → Create Resource → Template Deployment

→ Upload ARMTemplateForFactory.json

→ Edit parameters for target environment

→ Deploy → all resources created in new ADF

EXERCISE 40: Integrate ADF with DevOps

→ ADF Studio → Author → Git Repository → Configure

→ Repository Type: Azure DevOps Git

→ Organization: your-org

→ Project: DataEngineering

→ Repository: ADF-Pipelines

→ Collaboration Branch: main

→ Root Folder: /

→ Apply → ADF now saves to Git!

→ Create new pipeline → Edit → Commit → raise PR → merge →

publish

EXERCISE 41: Use Azure DevOps Repos

- After Git config, every ADF change is tracked:
- Create feature branch: feature/add-orders-pipeline
- Build pipeline in ADF using that branch
- Commit: "Add PL_Orders_Daily pipeline"
- Raise PR to main in Azure DevOps
- Review JSON diff (see exactly what changed)
- Approve + Merge → switch to main in ADF → Publish

EXERCISE 42: Send Mail via Logic Apps

- Logic Apps designer:

Trigger: HTTP Request received

Step 1: Parse JSON (extract pipeline details)

Step 2: Send Email (Office 365 connector)

To: @body('Parse_JSON')['notify_email']}

Subject: @concat('ADF Pipeline: ', body('Parse_JSON')['status'])

Body: HTML formatted email with pipeline details

- Copy Logic App HTTP endpoint URL

- In ADF pipeline failure path:

Web Activity:

URL: (Logic App URL)

Method: POST

Body: {

"pipeline_name": "@{pipeline().Pipeline}",

"status": "FAILED",

"error": "@{activity('FailedActivity').Error.message}",

"notify_email":

"@{pipeline().globalParameters.gp_notifyEmail}"

}

EXERCISE 43: Monitor Pipelines

- ADF Studio → Monitor tab
- **Pipeline Runs**: see all runs, filter by date/status/name
- Click any run → Activity Runs (drill into each step)
- Click any activity → Input/Output/Error details
- **Key metrics to check**:
 - dataRead**: how much data read from source
 - dataWritten**: how much written to sink
 - throughput**: MB/s transfer rate
 - rowsCopied**: exact row count
 - duration**: how long each step took

EXERCISE 44: Debug Pipelines

- **Debug mode**: click Debug button (no trigger needed)
- **With parameters**: Debug → fill parameter values
- **Debug single activity**: right-click → Debug Activity Run
- Set breakpoints (not available but use If Condition to pause)
- Use User Properties for custom debug info
- Check Output tab after each activity for details

EXERCISE 45: Schedule Pipeline Using Triggers

- Create Schedule trigger → set recurrence
- Attach to pipeline with parameter mapping
- Activate trigger
- **Monitor**: Trigger Runs tab shows each firing
- **Disable trigger**: go to Manage → Triggers → Deactivate

EXERCISE 46: Create Trigger Dependency (Tumbling Window)

- **Create Trigger A**: TRG_Extract_Hourly (Tumbling, 1hr)
- **Create Trigger B**: TRG_Transform_Hourly (Tumbling, 1hr)
- **In Trigger B configuration**:
 - Dependencies tab → + Add Dependency
 - Trigger**: TRG_Extract_Hourly
 - Size**: 1 Hour
 - Offset**: 0
- Activate both triggers

→ **Result:** TRG_Transform only fires after TRG_Extract succeeds for the same time window

EXERCISE 47: Run One Pipeline Inside Another (Execute Pipeline)

→ Create child pipeline PL_Child_ProcessSales

Parameters: p_date (String)

→ Copy Data + Transform activities

→ Create parent pipeline PL_Parent_Orchestrator

→ Add Execute Pipeline activity:

Invoked Pipeline: PL_Child_ProcessSales

Wait on Completion: YES

Parameters:

p_date: @{formatDateTime(utcnow(),'yyyy-MM-dd')}

→ Parent pipeline controls flow:

Activity 1: Execute PL_Child_ProcessSales (Sales)

Activity 2: Execute PL_Child_ProcessOrders (Orders)

Activity 3: Execute PL_Child_ProcessInventory (Inventory)

Activities 1,2,3 run in PARALLEL (no dependency arrows between them)

Complete ADF Section — Final Summary Map

AZURE DATA FACTORY — COMPLETE GUIDE

CORE CONCEPTS	Pipelines, Activities, Linked Services,
	Datasets, Integration Runtime

ACTIVITIES	Copy, Lookup, SP, GetMetadata, Delete,
(ALL 15 types)	Wait, Variables, Execute Pipeline,
	Databricks, Azure Function, Script,
	Validation, Fail, ForEach, If, Switch,
	Until, Filter, Web Activity
TRIGGERS	Schedule, Tumbling Window, Storage Event,
(3 types)	Parameters, Trigger Dependency
INTEGRATION RT	Azure AutoResolve, Managed VNet,
(4 types)	Self-Hosted, Linked Self-Hosted
DATA FLOWS	Source, Select, Filter, Aggregate, Join,
(ALL transforms)	Conditional Split, Derived Column,
Exists,	Union, Lookup, Sort, GroupBy, Pivot,
	Unpivot, Flatten, Parse, Stringify,
	Alter Row, Assert, Window, Flowlet,
	Schema Drift, Schema Validate, Dedup

```
|| DEVOPS & MGMT      | Git, ARM Templates, Global Params,  
||  
||                   | Credentials, Key Vault, Alerts, Logic  
Apps ||
```

```
|| EXERCISES          | All 47 exercises covered step-by-step  
||
```

```
|| INTERVIEW Q&A      | 15 most important Q&As with detailed  
||  
||                   | answers covering all exam/interview  
topics ||
```

✅ **Azure Data Factory section is now 100% complete** — all concepts from the PDF including all missing transformations (Select, Window Functions, Assert), Trigger Dependency, Script/With Activity, all 47 exercises in depth, and complete interview Q&A are now covered.

🚀 **Ready to continue with Azure Databricks** — the next major section in the PDF!

17/03/2026, 21:37:09

Thought process: Orchestrated comprehensive Databricks curriculum spanning foundational concepts through advanced implementations.

The user wants me to continue with Azure Databricks section from the PDF. Let me review all the Databricks content from the PDF carefully:

- 1) Introduction to BigData
- 2) Introduction to Hadoop
- 3) Introduction to Spark
- 4) Introduction To Databricks

- 5) Working with Databricks FileSystem - DBFS
- 6) Databricks - Spark Core
- 7) Databricks - Spark-SQL- DataFrames
- 8) Spark Optimizer & Execution Engine
- 9) Spark Joins
- 10) Spark Performance Optimization
- 11) Databricks- Handle multiple file formats
- 12) Databricks utilities
- 13) Databricks Cluster Management

- 14) Databricks - Batch Processing
- 15) Databricks Integration with
- 16) Databricks - Streaming API
- 17) Databricks Auto Loader (CloudFiles)
- 18) Databricks - Lakehouse (Delta Lake)
- 19) Delta Lake: Medallion Architecture
- 20) Workflows in Databricks
- 21) Introduction to Unity Catalog
- 22) Unity Catalog Object Model & Hierarchy
- 23) Access Control & Security (RBAC)
- 24) Data Protection & Privacy
- 25) Azure DevOps - Repos
- 26) SDLC and Agile methodology
- 27) End to End Data Migration Project from On Premises to Cloud
- 28) Interview
- 29) Mock Interviews

Now I'm putting together a comprehensive guide that covers all these Databricks topics with practical examples and detailed explanations, structured like something a senior engineer would reference.

Azure Databricks — Complete Senior Engineer Guide



Real World Setup

Same Company: RetailMart India Pvt. Ltd.

SITUATION:

ADF has ingested raw data **into** Bronze layer (ADLS Gen2)

Now you need **to**:

- Clean **and** transform **500GB** daily data
- Process **50,000** transactions per day
- Run complex business logic (joins, aggregations)
- Handle real-time fraud detection
- Build ML models **for** demand forecasting
- Maintain historical data **with** Delta Lake

Problem **with** ADF Data Flows alone:

- ❌ Too slow **for** **500GB** daily
- ❌ Cannot **do** complex ML
- ❌ No streaming support
- ❌ Limited Python/**custom** logic

Solution: Azure Databricks **with** Apache Spark

1 Introduction to Big Data — Deep Foundation

◆ What is Data?

Data is **raw facts and figures** that alone carry no meaning.

Raw Data: 42, "Mumbai", 2024-06-15, 15000

With Context: Store #42 in Mumbai on 15-Jun-2024 sold
Rs.15,000

Data becomes information only when processed with context.

◆ What is a Database?

A **Database** is an organized collection of structured data stored and accessed electronically.

Traditional Database (RDBMS):

MySQL, SQL Server, Oracle, PostgreSQL

How it works:

- Data stored in TABLES (rows + columns)
- Single server stores everything
- SQL to query data
- ACID transactions (Atomic, Consistent, Isolated, Durable)

Example – RetailMart uses Oracle DB:

Table: SALES

sale_id	store_id	amount	sale_date
1001	42	15000	2024-06-15
1002	43	8500	2024-06-15

◆ What is Big Data?

Big Data = datasets so large, fast, or complex that **traditional databases cannot handle them**.

RetailMart's *Big Data Reality*:

Daily transactions:	50,000 rows	← MySQL handles this fine
Daily clickstream:	50,000,000 events	← MySQL struggles
Daily IoT sensor data:	500,000,000 readings	← MySQL completely fails
Historical data (5yr):	2.5 Petabytes	← MySQL impossible

When MySQL tries to handle 500GB daily:

Query that takes 2 seconds on 1GB → takes 1000 seconds on 500GB
Single server runs out of RAM → crashes
Cannot add more servers easily (not distributed)

◆ Challenges of Big Data

CHALLENGE 1: STORAGE

Problem: 2.5 PB doesn't fit on one server
Old Fix: Buy expensive SAN storage (\$500K+)
Big Data Fix: Distribute across 100 cheap servers

CHALLENGE 2: PROCESSING SPEED

Problem: Sorting 1 billion rows takes 4 hours on 1 server
Old Fix: Buy faster, more expensive server
Big Data Fix: 100 servers sort in parallel → 2-3 minutes

CHALLENGE 3: VARIETY

Problem: Oracle handles tables only

What about JSON, images, logs, videos?

Big Data Fix: Distributed storage handles ALL formats

CHALLENGE 4: FAULT TOLERANCE

Problem: 1 server fails → ALL data lost

Big Data Fix: Data replicated across 3 servers

If 1 fails → other 2 serve the data

CHALLENGE 5: COST

Problem: Enterprise Oracle license = \$50,000/year

Oracle server = \$200,000

Big Data Fix: Open source tools on commodity hardware

1000x cheaper per GB stored/processed

◆ Why Traditional Databases Cannot Handle Big Data

TRADITIONAL DB LIMITATIONS:

Limitation	Traditional DB	Big Data System
Scale	Vertical only (bigger server)	Horizontal (add nodes) (more servers)
Data Types	Structured only	All types (JSON, CSV, images, logs, video)
Max Storage	~100TB practical	Petabytes (unlimited)
Processing	Single node	Distributed parallel

Cost at scale	Exponential	Linear (add cheap nodes)
Schema	Fixed (must define before insert)	Schema- on -read (define when reading)
Fault Tolerance	RAID (expensive)	Replication (built- in , free)

Real Example:

RetailMart query: "Show sales by store for last 5 years"

Oracle (single server, **500GB** data):

- **Full table** scan: **45** minutes
- Server CPU: **100%** during query
- Other queries blocked → business operations halt

Databricks (**10**-node Spark cluster, same **500GB**):

- Data split **into 200** partitions
- **Each of 10** nodes processes **20** partitions **in** parallel
- Query completes: **3** minutes
- Other queries unaffected (cluster scales up)

2 Introduction to Hadoop — The Foundation

◆ What is Hadoop?

Hadoop is an open-source framework for **distributed storage and processing** of large datasets across clusters of computers.

Created **by**: Doug Cutting (Yahoo!) inspired **by** Google's *MapReduce paper*

Year: 2006
Language: Java
Foundation: Apache Software Foundation

Think of Hadoop as: A team of 100 workers
→ Each worker has their own desk (storage)
→ Each worker does part of the job (processing)
→ A manager coordinates (YARN)
→ Results combined at the end

◆ How Hadoop Overcomes Big Data Challenges

CHALLENGE → HADOOP SOLUTION:

Storage challenge → HDFS (Hadoop Distributed File System)

Split 500GB file into 128MB blocks
Store each block on different servers
Replicate each block 3 times (fault tolerance)

Processing challenge → MapReduce

Send processing code TO the data (not data to code!)
Each node processes its local blocks in parallel
Combine results at the end

Fault tolerance → Block replication

Node fails → use replica on another node
Automatic → no manual intervention

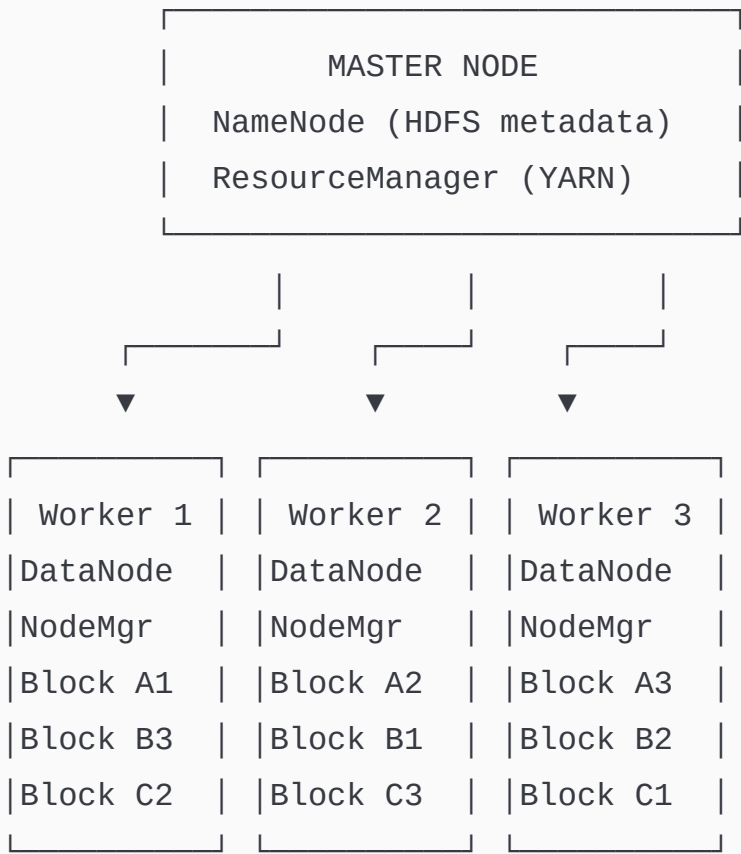
Cost challenge → Commodity hardware

Hadoop runs on cheap servers (\$3,000 each)

100 cheap servers > 1 expensive server
Same performance, 10x cheaper

◆ **Hadoop Architecture**

HADOOP CLUSTER:



◆ **Hadoop Components in Detail**

HDFS (Hadoop Distributed File System)

HOW HDFS STORES A 500GB FILE:

1. File split into 128MB blocks:

sales_data.csv (500GB)

→ Block 1 (128MB): rows 1 - 1,000,000

→ Block 2 (128MB): rows 1,000,001 - 2,000,000

→ ...

→ Block 4000 (128MB): rows 3,999,000,001 - 4,000,000,000

2. Each block replicated 3x across different nodes:

Block 1 → Node1, Node4, Node7 (replication factor=3)

Block 2 → Node2, Node5, Node8

Block 3 → Node3, Node6, Node9

3. NameNode keeps metadata:

"sales_data.csv → Block1(N1,N4,N7), Block2(N2,N5,N8)..."

4. If Node1 dies:

Block 1 still available on Node4 and Node7

NameNode detects 1 copy lost

Automatically re-replicates Block1 to Node10

File NEVER lost!

KEY INSIGHT:

"Move computation to data" principle

Instead of: copy 500GB file to processing server

Hadoop does: send tiny processing code to each node

→ Saves massive network bandwidth

→ 100x faster for large datasets

YARN (Yet Another Resource Negotiator)

YARN = The Operating System of Hadoop Cluster

Components:

ResourceManager:

- Master process (**one per** cluster)
- Manages **ALL** cluster resources (CPU, RAM)
- Accepts job submissions
- Allocates containers **to** applications

NodeManager:

- Runs **on every** worker node
- Manages resources **on** that node
- Reports health **to** ResourceManager
- Launches **and** monitors containers

ApplicationMaster:

- **One per** application/job
- Negotiates resources **from** ResourceManager
- Monitors job execution
- Handles failures/retries

Real-World Analogy:

ResourceManager = Airport Control Tower

(knows **all** flights, coordinates **all** landings/takeoffs)

NodeManager = Individual Gate Agent

(manages **one** gate, reports **to** control tower)

ApplicationMaster = Flight Captain

(manages own flight, requests runway **from** tower)

MapReduce

MapReduce = Two-phase distributed processing model

EXAMPLE: Count sales **by** store **from** 500GB file

MAP PHASE (parallel **on all** nodes):

Each node **reads** its **local** blocks

Node1 processes rows 1-1M:

Output: (Store42, 15000), (Store43, 8500), (Store42, 12000)

Node2 processes rows 1M-2M:

Output: (Store42, 9000), (Store44, 7000), (Store43, 5000)

...all 100 nodes run simultaneously

SHUFFLE PHASE (framework handles):

All (Store42, ?) pairs go to same reducer

All (Store43, ?) pairs go to same reducer

REDUCE PHASE:

Reducer for Store42: $\text{sum}(15000, 12000, 9000 \dots) = \text{total}$

Reducer for Store43: $\text{sum}(8500, 5000 \dots) = \text{total}$

...

PROBLEM with MapReduce:

- Writes intermediate results to DISK after each phase
- Iterative algorithms (ML) need 10+ MapReduce jobs
- Each job reads/writes disk → extremely slow for ML
- This is exactly why Spark was created!

3 Introduction to Apache Spark — The Revolution

◆ Why Spark? The Problem with MapReduce

MAPREDUCE vs SPARK for Machine Learning:

Training a recommendation model (10 iterations):

MapReduce:

Iteration 1: Read HDFS → Process → Write HDFS (disk I/O)

Iteration 2: Read HDFS → Process → Write HDFS (disk I/O)
...repeat 10 times
Total disk reads/writes: $20 \times 500\text{GB} = 10\text{TB}$
Time: 8 hours

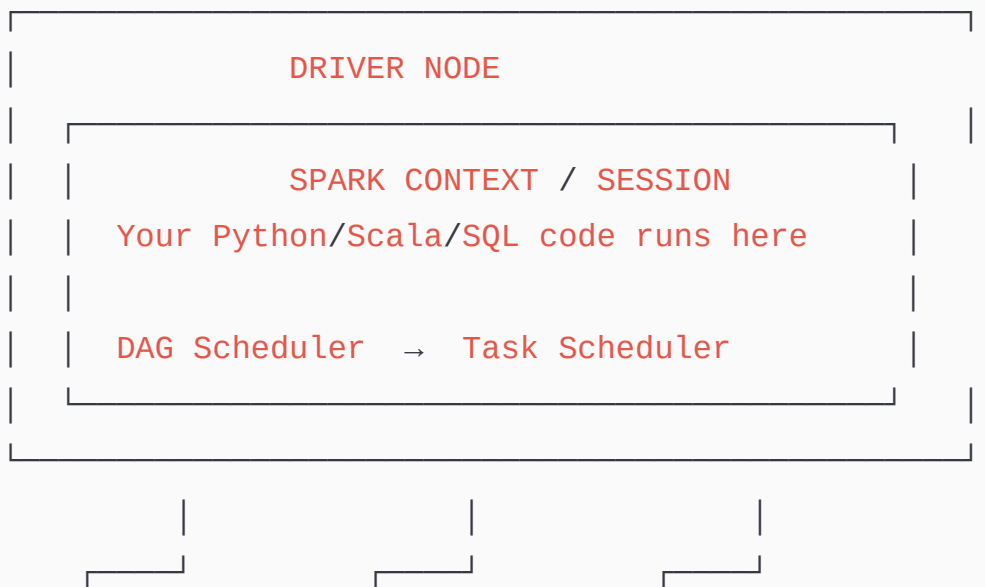
Spark:

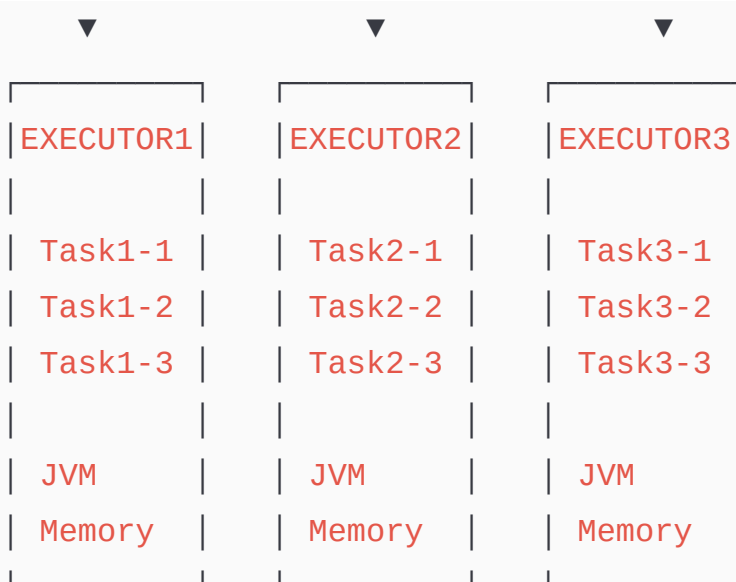
Load data into RAM ONCE
Iteration 1: Process in RAM → Keep in RAM
Iteration 2: Process in RAM → Keep in RAM
...repeat 10 times
Total disk reads/writes: $1 \times 500\text{GB} = 500\text{GB}$
Time: 20 minutes

Spark is 100x faster than MapReduce for iterative algorithms!
This is the fundamental innovation of Spark.

◆ Spark Architecture — Deep Dive

SPARK CLUSTER ARCHITECTURE:





Driver Node — The Brain

Driver **is** **WHERE** your code actually runs.

Responsibilities:

- ✓ Receives your Spark code (Python/Scala/SQL)
- ✓ Analyzes operations (builds Logical Plan)
- ✓ Optimizes execution (Catalyst Optimizer)
- ✓ Creates Physical Plan → Stages → Tasks
- ✓ Distributes Tasks **to** Executors
- ✓ Monitors task progress
- ✓ Collects results back **from** Executors
- ✓ **Handles** failures (re-runs failed tasks)

In Databricks:

Driver = The master node **of** your cluster
Your notebook code runs **ON** the driver

Important Rule:

NEVER collect massive data **to** driver!
`df.collect()` → brings ALL data **to** driver RAM

If data > driver RAM → OutOfMemory crash!
Use `df.show(20)` or `df.write()` instead

Executors — The Workers

Executors are WHERE actual data processing happens.

Each Executor:

- JVM process running on worker node
- Has multiple CPU cores (runs multiple Tasks in parallel)
- Has allocated RAM (for processing + caching)
- Stays alive for entire Spark application duration
- Sends heartbeat to Driver (alive signal)

Example - Databricks cluster:

8 worker nodes × (4 cores, 16GB RAM each)
= 32 total cores = 32 tasks run simultaneously!

Processing 500GB data:

- Split into 200 partitions
- 32 tasks run in parallel (32 partitions at once)
- 200 partitions / 32 parallel = ~7 rounds
- Each round takes 30 seconds
- Total: 7 × 30 = ~3.5 minutes!

◆ Spark RDD — The Foundation

RDD (Resilient Distributed Dataset) is the fundamental data structure of Spark.

RDD PROPERTIES:

R - Resilient:

- Fault tolerant – if a node fails, RDD rebuilt **from** lineage
- Lineage = recipe **of** transformations **to** recreate data

D - Distributed:

- Data split across multiple nodes **in** cluster
- **Each** node holds a **PARTITION of** the RDD

D - Dataset:

- Collection **of** records
- Can **hold ANY** type (integers, strings, objects)
- Immutable – cannot be changed, **only** transformed

IMMUTABILITY explained:

```
rdd1 = [1, 2, 3, 4, 5]           ← original RDD
rdd2 = rdd1.filter(_ > 3)        ← NEW RDD created [4, 5]
rdd1 still exists = [1, 2, 3, 4, 5] ← unchanged!
```

This enables fault tolerance:

If rdd2 **partition** fails → recreate **from** rdd1.filter(_ > 3)
No data loss ever!

Creating RDDs:

```
# Method 1: From collection (for testing small data)
rdd1 = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10])

# Method 2: From file (production use)
rdd2 = sc.textFile("/mnt/raw/sales/sales_data.csv")

# Method 3: From another RDD
rdd3 = rdd1.filter(lambda x: x > 5)

# Check number of partitions (= parallelism level)
print(rdd1.getNumPartitions()) # default based on cluster cores
```

```
# Force specific partition count
rdd4 = sc.parallelize([1,2,3,4,5], numSlices=4) # 4 partitions
```

◆ Spark DataFrame — The Modern Way

DataFrame is a distributed collection of data organized into **named columns** — like a database table but distributed across cluster.

RDD vs DataFrame:

Feature	RDD	DataFrame
API	Low-level	High-level (SQL-like)
Type Safety	Yes (compile)	Limited (runtime check)
Optimization	Manual	Automatic (Catalyst)
Performance	Slower	Faster (optimized)
Ease of Use	Complex	Simple
Schema	None	Named columns + types
SQL Support	No	Yes
Languages	All	All
Recommended	For custom algo	For data engineering

REAL WORLD USAGE:

90% of data engineering work → Use DataFrame/SQL

ML feature engineering → Use DataFrame

Custom algorithms, byte manipulation → Use RDD

◆ Spark Execution Flow: Driver → Executors → Tasks

YOUR CODE → EXECUTION JOURNEY:

You write:

```
df = spark.read.parquet("/mnt/bronze/sales/")
df_filtered = df.filter(df.amount > 1000)
df_grouped = df_filtered.groupBy("region").sum("amount")
df_grouped.write.parquet("/mnt/silver/sales_summary/")
```

Step 1: LOGICAL PLAN created

Driver analyzes your code

Creates tree of operations:

Read → Filter → GroupBy → Write

Step 2: OPTIMIZATION (Catalyst Optimizer)

Catalyst rewrites plan for efficiency:

→ Pushes filter BEFORE read (read less data!)

→ Determines best join strategy

→ Removes unnecessary columns early

Step 3: PHYSICAL PLAN created

Spark decides HOW to execute:

→ Stage 1: Read + Filter (no shuffle needed)

→ Stage 2: Shuffle by region + Aggregate (shuffle needed)

→ Stage 3: Write

Step 4: STAGES split into TASKS

Stage 1: 200 partitions → 200 Tasks

Stage 2: 5 regions → 5 Tasks (after shuffle)

Stage 3: Write Tasks

Step 5: TASKS distributed to Executors

Driver sends tasks to available executors

32 executors run 32 tasks simultaneously
Completed tasks → next batch assigned

Step 6: RESULTS returned to Driver

Write tasks complete → data in ADLS

Driver reports job success

VISUALIZATION:

Your Code

↓

Logical Plan (what to do)

↓

Optimized Logical Plan (Catalyst magic)

↓

Physical Plan (how to do it)

↓

Stages (groups of tasks with no shuffle between them)

↓

Tasks (individual units executed on one partition)

↓

Executors (run tasks in parallel)

↓

Result (written to storage)

◆ Spark Core Calculation Logic

PARTITIONS = KEY CONCEPT:

A partition = one chunk of data processed by one task
on one executor core at one time

Default partition size: 128MB (same as HDFS block)

500GB file → 500000 MB / 128 MB = ~4000 partitions

Cluster: 32 cores

Rounds needed: 4000 / 32 = 125 rounds

Each round: all 32 cores process their partition

Time per round: 15 seconds

Total time: 125 × 15 = ~31 minutes

Add more cores (64 cores):

Rounds: 4000 / 64 = 62.5 → 63 rounds

Total time: 63 × 15 = ~15 minutes

→ Linear improvement with more hardware!

OPTIMAL PARTITION SIZE:

Too small (1MB):

- 500,000 partitions for 500GB
- Too much overhead scheduling tasks
- Slower than optimal

Too large (2GB):

- Only 250 partitions
- Core sits idle waiting for huge partition
- Memory pressure

Sweet spot: 128MB - 256MB per partition

4 Introduction to Databricks

◆ What is Databricks?

Azure Databricks is a **managed Apache Spark platform** built on Azure — it gives you Spark's power without managing any infrastructure.

WITHOUT Databricks (raw Spark on VMs):

- ❌ Install Spark manually on each VM
- ❌ Configure networking between nodes
- ❌ Manage Spark version upgrades
- ❌ Handle cluster failures manually
- ❌ No collaborative notebooks
- ❌ Security and access control manual
- ❌ Takes 2 weeks just to set up a working cluster

WITH Azure Databricks:

- ✅ Click → cluster running in 5-10 minutes
- ✅ Managed Spark (Microsoft + Databricks manage it)
- ✅ Collaborative web notebooks (like Google Docs for code)
- ✅ Auto-scaling (adds/removes nodes based on load)
- ✅ Auto-termination (cluster shuts down when idle → save cost)
- ✅ Unity Catalog for governance
- ✅ Delta Lake built-in
- ✅ Azure AD integration (use company login)
- ✅ One-click integration with ADLS, Key Vault, ADF

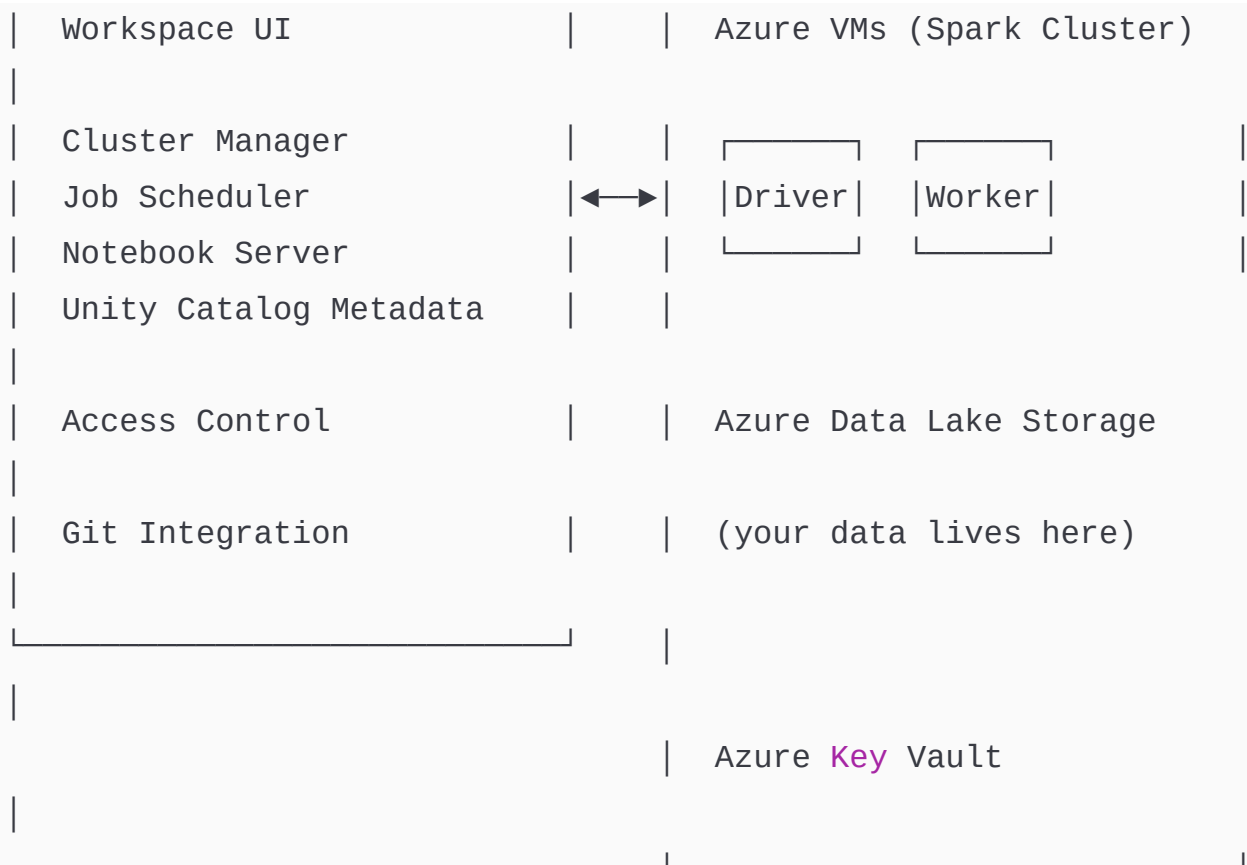
◆ Databricks Architecture

AZURE DATABRICKS ARCHITECTURE:

CONTROL PLANE

DATA PLANE

(Managed by Databricks/Microsoft) (In your Azure subscription)



KEY POINT:

Your DATA never leaves your Azure subscription

Only metadata and control signals go to Databricks control plane

Maximum security for enterprise compliance

◆ Working in Databricks Workspace

WORKSPACE STRUCTURE:

Workspace

```

├─ 📁 Users/
│   └─ you@company.com/
│       └─ 📁 My_Experiments (personal notebooks)
  
```

```
| | | └─ 📁 Draft_Pipeline
| | └─ 📁 Shared/
| |   └─ 📁 Production/
| |     └─ 📁 transform_sales
| |     └─ 📁 transform_customers
| |     └─ 📁 load_to_synapse
| |   └─ 📁 Utils/
| |     └─ 📁 common_functions
| |     └─ 📁 config_settings
| └─ 📊 Data (Unity Catalog)
| └─ ⚡ Compute (Clusters)
| └─ ⚙️ Workflows (Jobs)
└─ 🔧 Settings
```

LEFT SIDEBAR:

📁 Workspace	→ Notebooks and folders
📊 Data	→ Databases, tables (Unity Catalog)
⚡ Compute	→ Create /manage clusters
⚙️ Workflows	→ Schedule notebooks as jobs
🔗 Partner Connect	→ Connect to BI tools
⚙️ Settings	→ Admin settings

◆ Working with Databricks Notebook

NOTEBOOK BASICS:

Supported languages per cell:

%python	→ PySpark / Python code (most common)
%sql	→ Spark SQL queries
%scala	→ Scala Spark code
%r	→ R code

%md → Markdown documentation
%sh → Shell commands
%fs → DBFS file **system** commands

MAGIC COMMANDS:

%run ./other_notebook → Run another notebook inline
%pip install pandas → Install Python library
%reload_ext → Reload extensions

CELL SHORTCUTS:

Shift+Enter → Run cell **and** move to **next**
Ctrl+Enter → Run cell **and** stay
Esc+A → Add cell above
Esc+B → Add cell below
Esc+DD → Delete cell

NOTEBOOK STRUCTURE (best practice):

Cell **1**: %md

Sales Transformation Notebook

****Author:**** Data Engineering Team

****Purpose:**** Clean **and** transform raw sales data

****Schedule:**** Daily at **1** AM

Cell **2**: %python (Parameters - from ADF **or** widgets)

```
dbutils.widgets.text("input_path", "/mnt/bronze/sales/")
dbutils.widgets.text("output_path", "/mnt/silver/sales/")
dbutils.widgets.text("process_date", "2024-06-15")
```

```
input_path = dbutils.widgets.get("input_path")
output_path = dbutils.widgets.get("output_path")
process_date = dbutils.widgets.get("process_date")
```

Cell **3**: %python (Imports)

```
from pyspark.sql import functions as F
from pyspark.sql.types import *
from pyspark.sql.window import Window
from delta.tables import DeltaTable
import logging
```

Cell 4: %python (Read Data)

Cell 5: %python (Transform)

Cell 6: %python (Write)

Cell 7: %python (Exit with status for ADF)

```
dbutils.notebook.exit("SUCCESS")
```

5 DBFS — Databricks File System

◆ What is DBFS?

DBFS (Databricks File System) is a **distributed file system abstraction** over Azure Blob Storage and ADLS Gen2. It allows you to work with files using simple file paths in notebooks.

DBFS EXPLAINED:

Real Storage: Azure Data Lake Storage Gen2 (physical)

DBFS Path: /mnt/raw/sales/ (logical alias)

You work **with**: /mnt/raw/sales/sales_data.csv

Actually maps **to**:

abfss://raw@mydatalake.dfs.core.windows.net/sales/

WHY DBFS?

Without DBFS:


```
spark.read.format("parquet")
    .option("header", "true")

.load("abfss://container@account.dfs.core.windows.net/path")
→ Must embed full Azure URL + credentials everywhere!
```

With DBFS mount:

```
spark.read.parquet("/mnt/bronze/sales/")
→ Clean, simple path
→ Credentials handled once during mount setup
→ Change storage account → update one mount point
→ All notebooks automatically use new storage
```

◆ DBFS Commands — Complete Reference

```
# — ALL DBFS COMMANDS —————

# LIST files/directories
dbutils.fs.ls("/mnt/bronze/")
# Returns: [FileInfo(path, name, size, modificationTime)]

for f in dbutils.fs.ls("/mnt/bronze/sales/"):
    print(f"Name: {f.name}, Size: {f.size/1024:.1f} KB")

# MKDIR - Create directory
dbutils.fs.mkdirs("/mnt/bronze/new_folder/subfolder/")

# CP - Copy file or directory
dbutils.fs.cp(
    "/mnt/bronze/sales/sales_data.csv",    # source
    "/mnt/archive/sales/sales_data.csv",   # destination
    recurse=False                           # True for
```

```

directories
)

# MV - Move (rename) file
dbutils.fs.mv(
    "/mnt/landing/sales_20240615.csv",      # source
    "/mnt/bronze/sales/sales_20240615.csv" # destination
)

# HEAD - Preview first N bytes of file
dbutils.fs.head("/mnt/bronze/sales/sales_data.csv", 500)
# Output: first 500 bytes of file as string

# PUT - Create file with content
dbutils.fs.put(
    "/mnt/bronze/config/settings.json",
    '{"env": "prod", "batch_size": 10000}',
    overwrite=True
)

# RM - Delete file or directory
dbutils.fs.rm("/mnt/temp/processed_file.csv")
dbutils.fs.rm("/mnt/temp/old_folder/", recurse=True) # delete
folder

# %fs magic commands (shorthand in notebooks)
%fs ls /mnt/bronze/
%fs mkdirs /mnt/bronze/new_folder/
%fs rm /mnt/temp/ --recurse=true

```

◆ Mounting ADLS Gen2 to DBFS

```

# PRODUCTION PATTERN: Mount using Service Principal
# Store credentials in Key Vault, retrieve via dbutils.secrets

# — Step 1: Create scope linked to Key Vault —————
# (Done once by admin in Databricks Secrets UI or CLI)
# Scope: kv-retailmart-scope → Azure Key Vault: kv-retailmart-
prod

# — Step 2: Mount storage —————
storage_account = "retailmartdatalake"
container_name  = "bronze"
mount_point     = "/mnt/bronze"

# Get credentials from Key Vault (never hardcode!)
client_id       = dbutils.secrets.get("kv-retailmart-scope", "sp-
client-id")
client_secret   = dbutils.secrets.get("kv-retailmart-scope", "sp-
client-secret")
tenant_id       = dbutils.secrets.get("kv-retailmart-scope",
"tenant-id")

configs = {
    "fs.azure.account.auth.type": "OAuth",
    "fs.azure.account.oauth.provider.type":

"org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider",
    "fs.azure.account.oauth2.client.id": client_id,
    "fs.azure.account.oauth2.client.secret": client_secret,
    "fs.azure.account.oauth2.client.endpoint":

f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"
}

# Check if already mounted (avoid duplicate mount error)
existing_mounts = [m.mountPoint for m in dbutils.fs.mounts()]

```

```

if mount_point not in existing_mounts:
    dbutils.fs.mount(

source=f"abfss://{container_name}@{storage_account}.dfs.core.windows
    mount_point=mount_point,
    extra_configs=configs
    )
    print(f"✅ Mounted {container_name} to {mount_point}")
else:
    print(f"ℹ️ Already mounted: {mount_point}")

# — Step 3: Mount all containers —————
containers = ["bronze", "silver", "gold", "archive", "config"]
for container in containers:
    mp = f"/mnt/{container}"
    if mp not in existing_mounts:
        dbutils.fs.mount(

source=f"abfss://{container}@{storage_account}.dfs.core.windows.net/
    mount_point=mp,
    extra_configs=configs
    )
    print(f"✅ Mounted: {mp}")

```

◆ Handling Multiple Files in DBFS

```

# REAL WORLD SCENARIO:
# /mnt/bronze/sales/ has 365 daily files (one per day)
# Need to process, archive, track each

# — List and analyze all files —————

```

```

import datetime

def analyze_folder(path):
    """Analyze all files in DBFS folder"""
    files = dbutils.fs.ls(path)

    total_size = 0
    file_info = []

    for f in files:
        size_mb = f.size / (1024 * 1024)
        mod_time =
datetime.datetime.fromtimestamp(f.modificationTime/1000)
        file_info.append({
            "name": f.name,
            "size_mb": round(size_mb, 2),
            "modified": mod_time.strftime("%Y-%m-%d %H:%M:%S")
        })
        total_size += size_mb

    print(f"Total files:      {len(file_info)}")
    print(f"Total size:        {total_size:.1f} MB")
    print(f"Largest file:      {max(file_info, key=lambda x:
x['size_mb'])['name']}")
    print(f"Smallest file:     {min(file_info, key=lambda x:
x['size_mb'])['name']}")

    return file_info

file_list = analyze_folder("/mnt/bronze/sales/")

# — Process files in DBFS —————

# Read ALL files in folder at once (Spark handles distribution)
df_all_sales = spark.read.parquet("/mnt/bronze/sales/")

```

```

print(f"Total rows across all files: {df_all_sales.count():,}")

# Read files matching pattern
df_june = spark.read.parquet("/mnt/bronze/sales/2024/06/")
# or with glob pattern:
df_q2 = spark.read.parquet("/mnt/bronze/sales/2024/0[4-6]/")

# Read with filename tracking
df_with_source = spark.read \
    .option("recursiveFileLookup", "true") \
    .parquet("/mnt/bronze/sales/") \
    .withColumn("source_file", F.input_file_name())

df_with_source.select("sale_id", "source_file").show(5,
truncate=False)
# Output: sale_id=1001,
source_file=dbfs:/mnt/bronze/sales/2024/06/15/sales_20240615.parquet

# — Archive processed files —————
def archive_file(source_path, archive_base):
    """Move processed file to archive with date folder"""
    filename = source_path.split("/")[-1]
    today = datetime.datetime.now().strftime("%Y/%m/%d")
    archive_path = f"{archive_base}/{today}/{filename}"

    dbutils.fs.mkdirs(f"{archive_base}/{today}/")
    dbutils.fs.mv(source_path, archive_path)
    print(f"Archived: {filename} → {archive_path}")

# Archive all June 2024 files
for f in dbutils.fs.ls("/mnt/bronze/sales/2024/06/"):
    archive_file(f.path, "/mnt/archive/sales")

```

6 Databricks Spark Core — RDD Programming

◆ RDD Operations Complete Guide

```
# ——— CREATING RDDs ———  
  
# From collection  
numbers_rdd = sc.parallelize([1, 2, 3, 4, 5, 6, 7, 8, 9, 10], 4)  
# 4 partitions – for RetailMart, matches 4 executor cores  
  
# From text file  
sales_rdd = sc.textFile("/mnt/bronze/sales/sales_raw.csv")  
# Each LINE becomes one element in RDD  
  
# With specific partitions  
large_rdd = sc.parallelize(range(1000000), numSlices=100)
```

◆ Narrow vs Wide Transformations

CRITICAL CONCEPT – Understand Shuffles:

NARROW TRANSFORMATIONS (No Shuffle):

- Each output partition comes from ONE input partition
- No data movement across network
- Fast!
- Can be pipelined (executed together in one stage)

Examples:

- map() → transform each element
- filter() → keep/remove elements
- flatMap() → one element → zero or many

mapPartitions() → process entire **partition at** once
union() → combine two RDDs
sample() → random sample

WIDE TRANSFORMATIONS (Shuffle Required):

- Output **partition** comes **from** MULTIPLE input partitions
- Data must move across network **between** nodes
- Slow (network I/O **is** expensive)
- Creates **new** Stage boundary

Examples:

groupByKey() → **group by** key (lots **of** data movement!)
reduceByKey() → reduce **by** key (optimized – combines locally **first**)
sortByKey() → sort requires **global** ordering
join() → **join** two RDDs **by** key
distinct() → needs **to** compare across **all** partitions
repartition() → redistribute data across partitions

PRACTICAL IMPLICATION:

Bad Code (**2** shuffles):

rdd.groupByKey()	← shuffle 1 (expensive!)
.mapValues(sum)	← narrow (ok)
.sortByKey()	← shuffle 2 (expensive!)

Good Code (**1** shuffle):

rdd.reduceByKey(add)	← shuffle but optimized (local combine first !)
.sortByKey()	← shuffle 2 (needed for sort)

RULE: Prefer reduceByKey **over** groupByKey

groupByKey moves **ALL values to one** node (memory risk!)
reduceByKey combines locally **THEN** shuffles (less data)

◆ Complete RDD Operations

```
# — TRANSFORMATIONS (lazy – don't execute immediately) —

sales_rdd = sc.textFile("/mnt/bronze/sales_raw.csv") \
    .filter(lambda line: not
line.startswith("sale_id"))
    # skip header row

# map() – transform each element
parsed_rdd = sales_rdd.map(lambda line: line.split(","))
# ["1001", "C001", "LAPTOP", "50000", "2024-06-15"]

# map to named tuple for clarity
from collections import namedtuple
Sale = namedtuple("Sale",
["sale_id", "customer_id", "product", "amount", "date"])

sales_typed = parsed_rdd.map(lambda x:
    Sale(int(x[0]), x[1], x[2], float(x[3]), x[4])
)

# filter() – keep only matching rows
high_value = sales_typed.filter(lambda s: s.amount > 10000)

# flatMap() – one element produces multiple
# Parse each line that might have multi-product orders
order_lines = sc.parallelize([
    "1001, LAPTOP, PHONE",
    "1002, TABLET",
    "1003, LAPTOP, TABLET, PHONE"
])
products = order_lines.flatMap(lambda line: line.split(",")[1:])
# Produces: ["LAPTOP", "PHONE", "TABLET", "LAPTOP", "TABLET", "PHONE"]
```

```

# Key-Value Pair RDD
kv_rdd = sales_typed.map(lambda s: (s.product, s.amount))
# [("LAPTOP", 50000), ("PHONE", 30000), ("LAPTOP", 45000)]

# reduceByKey() – aggregate by key (PREFERRED over groupByKey)
product_totals = kv_rdd.reduceByKey(lambda a, b: a + b)
# [("LAPTOP", 95000), ("PHONE", 30000)]

# groupByKey() – use sparingly (memory intensive)
product_groups = kv_rdd.groupByKey()
# [("LAPTOP", [50000, 45000]), ("PHONE", [30000])]

# sortByKey()
sorted_totals = product_totals.sortByKey(ascending=True)

# distinct() – remove duplicates
unique_products = kv_rdd.map(lambda x: x[0]).distinct()

# — ACTIONS (trigger execution) —————

# collect() – bring all to driver (use only for small data!)
small_result = product_totals.collect()
# [(LAPTOP, 95000), (PHONE, 30000)]

# count() – total elements
total_sales = sales_typed.count()

# first() – first element
first_sale = sales_typed.first()

# take(n) – first n elements (safer than collect)
sample_5 = product_totals.take(5)

# reduce() – aggregate entire RDD to single value
total_revenue = kv_rdd.values().reduce(lambda a, b: a + b)

```

```
# saveAsTextFile() – write to storage
product_totals.saveAsTextFile("/mnt/output/product_totals/")

# foreach() – apply function to each element (no return)
product_totals.foreach(lambda x: print(f"Product: {x[0]}, Total: {x[1]}"))
```

◆ Broadcast Variables and Accumulators

```
# BROADCAST VARIABLES:
# Share read-only data efficiently to all executors
# Without broadcast: data sent to each task separately
# (expensive!)
# With broadcast: sent ONCE per executor (efficient)

# Real-world: region code lookup (small table, used in many
# tasks)
region_map = {
    "N": "North India",
    "S": "South India",
    "E": "East India",
    "W": "West India"
}

# Broadcast to all executors (sent once, cached there)
broadcast_regions = sc.broadcast(region_map)

# Use in transformation
def get_region_name(sale):
    region_code = sale[5] # assuming region_code is 6th column
    # Access broadcast variable with .value
```

```

        region_name = broadcast_regions.value.get(region_code,
"Unknown")
        return sale + (region_name,)

enriched_rdd = sales_rdd.map(get_region_name)

# ACCUMULATORS:
# Shared counter that executors can increment
# Driver reads final count (executors cannot read)
# Use for: counting errors, tracking records processed

# Count records with data quality issues
null_count      = sc.accumulator(0)
invalid_count    = sc.accumulator(0)
total_amount     = sc.accumulator(0.0)

def validate_and_count(sale):
    """Validate sale record, count issues"""
    if sale.customer_id is None:
        null_count.add(1)
    if sale.amount <= 0:
        invalid_count.add(1)
    else:
        total_amount.add(sale.amount)
    return sale # still return (accumulator doesn't filter)

validated = sales_typed.map(validate_and_count)
validated.count() # trigger execution

# Now read accumulator values on DRIVER
print(f"Null customer records: {null_count.value:,}")
print(f"Invalid amount records: {invalid_count.value:,}")
print(f"Total valid revenue:     Rs.{total_amount.value:,.2f}")

```

7 Spark SQL and DataFrames — Complete Guide

◆ Creating DataFrames — All Methods

— METHOD 1: Read from storage (most common) —————

Read Parquet (recommended format for production)

```
df_sales = spark.read.parquet("/mnt/bronze/sales/")
```

Read CSV with options

```
df_csv = spark.read \  
    .option("header", "true")          \  
    .option("inferSchema", "true")     \  
    .option("delimiter", ",")          \  
    .option("encoding", "UTF-8")       \  
    .option("nullValue", "NULL")       \  
    .option("dateFormat", "yyyy-MM-dd") \  
    .option("timestampFormat", "yyyy-MM-dd HH:mm:ss") \  
    .csv("/mnt/bronze/sales/sales_raw.csv")
```

Read JSON

```
df_json = spark.read \  
    .option("multiLine", "true") \  
    .json("/mnt/bronze/payments/payments.json")
```

Read with explicit schema (ALWAYS do this in production!)

InferSchema reads entire file twice → slow + might get types wrong

```
from pyspark.sql.types import *
```

```
sales_schema = StructType([  
    StructField("sale_id", IntegerType(), nullable=False),  
    StructField("customer_id", StringType(), nullable=False),  
    StructField("store_id", IntegerType(), nullable=False),  
    StructField("product_code", StringType(), nullable=True),
```

```

    StructField("product_name", StringType(), nullable=True),
    StructField("quantity", IntegerType(), nullable=True),
    StructField("unit_price", DoubleType(), nullable=True),
    StructField("total_amount", DoubleType(), nullable=True),
    StructField("sale_date", DateType(), nullable=True),
    StructField("region_code", StringType(), nullable=True),
    StructField("payment_mode", StringType(), nullable=True),
    StructField("status", StringType(), nullable=True),
    StructField("created_ts", TimestampType(), nullable=True)
])

```

```

df_sales = spark.read \
    .schema(sales_schema) \
    .option("header", "true") \
    .csv("/mnt/bronze/sales/sales_data.csv")

```

— METHOD 2: From Python collection (testing/prototyping) —

```

data = [
    (1, "C001", 42, "LAPTOP01", "Dell Laptop", 1, 55000.0,
55000.0),
    (2, "C002", 43, "PHONE01", "Samsung S24", 2, 35000.0,
70000.0),
    (3, "C003", 42, "TABLET01", "iPad Pro", 1, 85000.0,
85000.0)
]
cols = ["sale_id", "customer_id", "store_id", "product_code",
        "product_name", "quantity", "unit_price", "total_amount"]

```

```

df_test = spark.createDataFrame(data, schema=cols)
df_test.show()

```

— METHOD 3: From SQL query on registered table —

```

spark.sql("""
    CREATE OR REPLACE TEMP VIEW vw_recent_sales AS
    SELECT * FROM delta.`/mnt/silver/sales/`

```

```
WHERE sale_date >= current_date() - INTERVAL 30 DAYS
""")
df_recent = spark.sql("SELECT * FROM vw_recent_sales")

# — METHOD 4: From Pandas DataFrame —————
import pandas as pd
pdf = pd.read_csv("/dbfs/mnt/config/store_mapping.csv")
df_spark = spark.createDataFrame(pdf) # small config tables
only!
```

◆ DataFrame Internal Execution — How It Really Works

When you write:

```
df.filter(F.col("amount") >
1000).groupBy("region").sum("amount")
```

Spark doesn't execute this immediately!
This is called LAZY EVALUATION.

Timeline:

Line 1: `df.filter(...)` → Creates logical plan node (no execution)
Line 2: `.groupBy(...)` → Adds to logical plan (no execution)
Line 3: `.sum(...)` → Adds aggregation to plan (no execution)

`.show()` / `.write()` / `.count()` → TRIGGERS EXECUTION!

Only NOW does Spark:

1. Optimize the full plan
2. Generate physical plan
3. Run tasks on cluster

WHY LAZY EVALUATION?

Spark can see the WHOLE picture before executing

- Can push filters earlier
- Can remove unnecessary columns
- Can choose best join strategy
- Combines operations into fewer stages

PRACTICAL IMPACT:

These three lines: 0 data processed yet

```
df1 = df.filter(F.col("status") == "ACTIVE")
df2 = df1.select("customer_id", "total_amount")
df3 = df2.groupBy("customer_id").sum("total_amount")
```

This line: Spark now executes everything optimally

```
df3.write.parquet("/mnt/silver/customer_totals/")
```

Catalyst sees: Read → Filter → Select → GroupBy → Write

Optimizes to: Read (only 2 columns, only ACTIVE rows) →

GroupBy → Write

Column pruning + predicate pushdown applied automatically!

◆ DataFrame Transformations — Production Code

```
# ——— RETAILMART: COMPLETE SALES TRANSFORMATION ———

from pyspark.sql import functions as F
from pyspark.sql.window import Window

# Read raw sales data
df_raw = spark.read.parquet(f"/mnt/bronze/sales/{process_date}/")

print(f"Raw records: {df_raw.count():,}")
```



```
df_raw.printSchema()
```

```
# — STEP 1: Data Quality Checks —————
```

```
df_valid = df_raw \
    .filter(F.col("sale_id").isNotNull()) \
    .filter(F.col("customer_id").isNotNull()) \
    .filter(F.col("total_amount") > 0) \
    .filter(F.col("sale_date").isNotNull()) \
    .filter(~F.col("status").isin(["CANCELLED", "DELETED"]))

print(f"Valid records: {df_valid.count():,}")
print(f"Dropped: {df_raw.count() - df_valid.count():,} invalid
records")
```

```
# — STEP 2: Data Standardization —————
```

```
df_standardized = df_valid \
    .withColumn("customer_id",
F.upper(F.trim(F.col("customer_id")))) \
    .withColumn("product_code",
F.upper(F.trim(F.col("product_code")))) \
    .withColumn("region_code",
F.upper(F.trim(F.col("region_code")))) \
    .withColumn("payment_mode",
F.lower(F.trim(F.col("payment_mode")))) \
    .withColumn("sale_date",      F.to_date(F.col("sale_date"),
"yyyy-MM-dd")) \
    .withColumn("year",          F.year(F.col("sale_date"))) \
    .withColumn("month",        F.month(F.col("sale_date"))) \
    .withColumn("day",          F.dayofmonth(F.col("sale_date")))
\
    .withColumn("day_of_week",  F.dayofweek(F.col("sale_date")))
\
    .withColumn("is_weekend",   F.when(
                                F.dayofweek(F.col("sale_date")).isin([1, 7]),
                                True
```

```
).otherwise(False))
```

```
# — STEP 3: Derived Columns —————
```

```
df_enriched = df_standardized \
    .withColumn("gst_amount", F.round(F.col("total_amount") *
0.18, 2)) \
    .withColumn("net_amount", F.col("total_amount") -
F.col("gst_amount")) \
    .withColumn("amount_bucket", F.when(F.col("total_amount") <
1000, "LOW")
                                .when(F.col("total_amount") <
10000, "MEDIUM")
                                .when(F.col("total_amount") <
50000, "HIGH")
                                .otherwise("PREMIUM")) \
    .withColumn("region_name", F.when(F.col("region_code") ==
"N", "North India")
                                .when(F.col("region_code") ==
"S", "South India")
                                .when(F.col("region_code") ==
"E", "East India")
                                .when(F.col("region_code") ==
"W", "West India")
                                .otherwise("Unknown")) \
    .withColumn("load_timestamp", F.current_timestamp()) \
    .withColumn("source_system", F.lit("Oracle_ERP")) \
    .withColumn("batch_id", F.lit(process_date))
```

```
# — STEP 4: Business KPIs using Window Functions —————
```

```
# Window: rank sales by amount within each store
```

```
store_window = Window.partitionBy("store_id") \
    .orderBy(F.col("total_amount").desc())
```

```
# Window: running total by date within store
```

```

running_window = Window.partitionBy("store_id") \
                        .orderBy("sale_date") \
                        .rowsBetween(Window.unboundedPreceding,
Window.currentRow)

# Window: 7-day rolling average
rolling_window = Window.partitionBy("store_id") \
                        .orderBy("sale_date") \
                        .rowsBetween(-6, 0) # current + 6
preceding days

df_final = df_enriched \
    .withColumn("rank_in_store", F.rank().over(store_window)) \
    .withColumn("running_total",
F.sum("total_amount").over(running_window)) \
    .withColumn("prev_sale_amt", F.lag("total_amount",
1).over(store_window)) \
    .withColumn("rolling_7d_avg",
F.avg("total_amount").over(rolling_window))

# — STEP 5: Join with Dimension Tables —————

df_stores = spark.read.parquet("/mnt/silver/dim_stores/")
df_products = spark.read.parquet("/mnt/silver/dim_products/")
df_customers = spark.read.parquet("/mnt/silver/dim_customers/")

# Broadcast small dimension tables (< 200MB)
# Avoids expensive shuffle join!
df_with_store = df_final.join(

F.broadcast(df_stores.select("store_id", "store_name", "city", "state"))
    on="store_id",
    how="left"
)

```

```

df_with_product = df_with_store.join(

F.broadcast(df_products.select("product_code", "category", "brand", "co
    on="product_code",
    how="left"
)

df_with_customer = df_with_product.join(

F.broadcast(df_customers.select("customer_id", "customer_name", "tier"
    on="customer_id",
    how="left"
)

# Calculate profit margin
df_complete = df_with_customer \
    .withColumn("profit",      F.col("net_amount") -
F.col("cost_price")) \
    .withColumn("profit_margin", F.round(F.col("profit") /
F.col("net_amount") * 100, 2))

# — STEP 6: Write to Silver Layer —————
df_complete.write \
    .mode("overwrite") \
    .partitionBy("year", "month", "region_code") \
    .parquet(f"/mnt/silver/sales/{process_date}/")

print(f"Written to silver: {df_complete.count():,} records")
print("✅ Transformation complete")

```

◆ User-Defined Functions (UDFs)

```

# When built-in functions aren't enough → write your own UDF

# — SCENARIO: Complex PAN card validation —————
# RetailMart needs to validate customer PAN numbers
# PAN format: ABCDE1234F (5 letters, 4 digits, 1 letter)

import re
from pyspark.sql.functions import udf
from pyspark.sql.types import BooleanType, StringType

# Python function (runs on executor, NOT on driver)
def validate_pan(pan_number):
    """Validate Indian PAN card format"""
    if pan_number is None:
        return False
    pattern = r'^[A-Z]{5}[0-9]{4}[A-Z]{1}$'
    return bool(re.match(pattern, pan_number.strip().upper()))

def mask_pan(pan_number):
    """Mask PAN for display: ABCDE1234F → ABCXX1234F"""
    if pan_number is None:
        return None
    pan = pan_number.strip().upper()
    if len(pan) != 10:
        return pan
    return pan[:3] + "XX" + pan[5:]

# Register as Spark UDF
validate_pan_udf = udf(validate_pan, BooleanType())
mask_pan_udf      = udf(mask_pan,      StringType())

# Use in DataFrame
df_customers = df_customers \
    .withColumn("is_pan_valid",
validate_pan_udf(F.col("pan_number"))) \

```

```

        .withColumn("pan_masked",
mask_pan_udf(F.col("pan_number")))

# — PERFORMANCE NOTE ABOUT UDFs —————
# Regular Python UDF:
#   → Serializes each row to Python process
#   → Processes one row at a time
#   → Sends result back to JVM
#   → Very slow for large datasets!

# BETTER: Pandas UDF (vectorized – works on batches)
from pyspark.sql.functions import pandas_udf
import pandas as pd

@pandas_udf(BooleanType())
def validate_pan_fast(pan_series: pd.Series) -> pd.Series:
    """Pandas UDF – works on entire column at once
(vectorized)"""
    return pan_series.str.match(r'^[A-Z]{5}[0-9]{4}[A-Z]{1}$',
na=False)

# 10-100x faster than regular UDF for large datasets!
df_customers = df_customers \
    .withColumn("is_pan_valid_fast",
validate_pan_fast(F.col("pan_number")))

# BEST: Use built-in functions whenever possible
# F.regex_extract, F.regex_replace, etc.
# Built-in functions run in JVM directly – no Python
serialization overhead
df_customers = df_customers \
    .withColumn("is_pan_valid_builtin",
        F.col("pan_number").rlike(r'^[A-Z]{5}[0-9]{4}[A-Z]{1}$'))

```

8 Spark Optimizer & Execution Engine

◆ Spark Catalyst Optimizer

CATALYST = Spark's Query Optimizer

Think of Catalyst as a chess grandmaster who thinks several moves ahead before making any move.

Your Code → 4 Phases:

PHASE 1: ANALYSIS

- Resolve column names (check they exist)
- Resolve data types
- Check for errors in query
- **Output:** Resolved Logical Plan

PHASE 2: LOGICAL OPTIMIZATION

- **Apply rule-based optimizations:**
 - Predicate Pushdown (filter early)
 - Column Pruning (drop unused columns early)
 - Constant Folding (evaluate `1+2` to `3` at plan time)
 - **Null** Propagation
 - Boolean Simplification
- **Output:** Optimized Logical Plan

PHASE 3: PHYSICAL PLANNING

- Generate multiple physical plans
- Use cost model to choose best
- Choose join strategy (broadcast vs sort-merge)
- **Output:** Physical Plan

PHASE 4: CODE GENERATION (Tungsten)

- Generate optimized Java bytecode

- Whole-stage code generation
- Runs directly on CPU (no interpretation overhead)
- **Output:** Executable RDD

VISUALIZE THE PLAN:

```
df_result.explain(True)  # show all 4 plans
df_result.explain("formatted")  # pretty format
```

◆ Logical Plan vs Physical Plan

```
# See exactly what Spark plans to do
df = spark.read.parquet("/mnt/bronze/sales/") \
    .filter(F.col("total_amount") > 10000) \
    .select("customer_id", "total_amount", "region_code")
\
    .groupBy("region_code") \
    .agg(F.sum("total_amount").alias("region_total"))

df.explain(True)
```

```
== Parsed Logical Plan ==
'Aggregate ['region_code], [...]
+- Project [customer_id, total_amount, region_code]
   +- Filter (total_amount > 10000)
      +- Relation[...] parquet

== Analyzed Logical Plan ==
(types resolved)

== Optimized Logical Plan ==
Aggregate [region_code], [region_code, sum(total_amount)]
+- Project [total_amount, region_code]      ← customer_id
```



```

removed! (pruning)
  +- Filter (isnotnull(total_amount) AND (total_amount >
10000.0))
    +- Relation[region_code, total_amount] parquet ← reads
only 2 columns!

== Physical Plan ==
HashAggregate(keys=[region_code], functions=[sum(total_amount)])
+- Exchange hashpartitioning(region_code, 200) ← shuffle!
  +- HashAggregate(keys=[region_code], functions=
[partial_sum(total_amount)])
    +- Project [total_amount, region_code]
      +- Filter (isnotnull(total_amount) AND (total_amount >
10000.0))
        +- FileScan parquet [region_code,total_amount]
          PushedFilters: [IsNotNull(total_amount),
GreaterThan(total_amount,10000.0)]
          PartitionFilters: []

```

KEY OBSERVATIONS:

- ✓ customer_id REMOVED early (column pruning)
- ✓ filter PUSHED DOWN to file scan (predicate pushdown)
- ✓ Only 2 columns read from file (not all columns!)
- ✓ Filter applied during file reading (less data into memory)
- ✓ Partial aggregation BEFORE shuffle (reduces shuffle size)

◆ Rule-Based vs Cost-Based Optimization

RULE-BASED OPTIMIZER (RBO):

- Applies fixed rules always
- "Always push filters down"
- "Always remove unused columns early"

- "Always fold constants"
- Fast **to** apply (**no** statistics needed)
- Works well **for 90% of** queries

COST-BASED OPTIMIZER (CBO):

- Uses **table** statistics **to** make smarter decisions
- "This table has 1M rows, that one has 100 rows
 - use broadcast join for small table"
- Needs **table** statistics collected **first!**

Enable CBO:

```
spark.conf.set("spark.sql.cbo.enabled", "true")
```

Collect statistics (do this after writing data):

```
spark.sql("ANALYZE TABLE delta.`/mnt/silver/sales/`  
          COMPUTE STATISTICS FOR ALL COLUMNS")
```

```
spark.sql("ANALYZE TABLE silver.sales COMPUTE STATISTICS")
```

With statistics collected:

- CBO knows: sales = **50M rows**, products = **5000 rows**
- Automatically broadcasts products (small **table**)
 - **No** need **for** manual `F.broadcast()` hints!

◆ Adaptive Query Execution (AQE)

AQE = Spark **3.0+** feature that re-optimizes **at** RUNTIME

Problem Adaptive Query Execution solves:

- Spark plans execution BEFORE **running**
- But actual data distribution **unknown** until runtime!

Example:

Query plans for 200 shuffle partitions (default)

Actual data: only 5 unique regions

Result: 195 empty partitions, 5 huge partitions

→ 195 tiny tasks (wasteful) + skew!

AQE FEATURES:

1. Coalescing Shuffle Partitions:

After shuffle, if many partitions are tiny

→ AQE automatically merges small partitions

→ 200 planned → 8 actual (based on data)

2. Switching Join Strategies at Runtime:

Plan says: SortMergeJoin (assumes table is large)

Runtime: one table turns out to be small!

→ AQE switches to BroadcastHashJoin automatically

3. Handling Skewed Partitions:

One region has 90% of data (data skew!)

→ AQE splits the huge partition into sub-tasks

→ Prevents one executor doing all the work

ENABLE AQE:

```
spark.conf.set("spark.sql.adaptive.enabled", "true") # default  
true in Spark 3.2+
```

```
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled",  
"true")
```

```
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

```

# PREDICATE PUSHDOWN – Read less data from storage

# WITHOUT predicate pushdown (bad):
# Step 1: Read ENTIRE 500GB file into memory
# Step 2: Then filter in memory
df_bad = spark.read.parquet("/mnt/bronze/sales/") \
    .filter(F.col("region_code") == "N") # filters 80%
of data!

# WITH predicate pushdown (what Spark actually does):
# Spark tells Parquet reader: "only give me rows where
region_code='N'"
# Parquet reads only relevant row groups from file
# Result: reads maybe 100GB instead of 500GB!

# PARTITION PRUNING (even better):
# If data is partitioned by region_code:
# /mnt/bronze/sales/region_code=N/ ← only this folder read!
# /mnt/bronze/sales/region_code=S/ ← SKIPPED entirely
# /mnt/bronze/sales/region_code=E/ ← SKIPPED entirely
# /mnt/bronze/sales/region_code=W/ ← SKIPPED entirely

df_pruned = spark.read.parquet("/mnt/bronze/sales/") \
    .filter(F.col("region_code") == "N")
# Only reads region_code=N partition = 125GB instead of 500GB

# VERIFY pushdown happened (check Physical Plan):
df_pruned.explain()
# Look for: PartitionFilters: [isnotnull(region_code),
(region_code = N)]
# This confirms partition pruning is working!

# MAXIMIZE PUSHDOWN:
# ✅ Filter on partition columns (year, month, region)
# ✅ Use = and IN operators (pushdown works)

```

```
#  Filter early in your code
#  Avoid UDFs in filter (cannot be pushed down to file reader)
#  Avoid complex functions like lower(col) == 'x' (can't push)
#  Use: col == 'X' (can push) NOT: lower(col) == 'x' (can't push)
```

Spark Joins — Deep Dive

Three Join Strategies Explained

SPARK CHOOSES JOIN STRATEGY BASED ON TABLE SIZES:

Strategy 1: Broadcast Hash Join

Use when: One table is SMALL (< 10MB default, max ~200MB)

Process:

- Small table broadcast to ALL executor nodes
- Each executor has complete copy
- Large table processed locally, joins locally
- NO shuffle needed!

Performance: FASTEST (no network transfer of large table)

Strategy 2: Shuffle Hash Join

Use when: Medium-sized tables, one fits in memory per partition

Process:

- Both tables shuffled by join key
- Matching keys land on same executor
- Hash table built from smaller partition
- Joined locally per partition

Performance: MEDIUM

Strategy 3: Sort-Merge Join

Use **when**: Both tables are LARGE

Process:

- Both tables shuffled by join key
- Both sorted by join key
- Merge sorted data (like merge in mergesort)
- Memory efficient (doesn't require full partition in RAM)

Performance: SLOWEST but most scalable

◆ Broadcast Join — With Code

```
# RetailMart: Join 50M sales with 5,000-row product table

df_sales    = spark.read.parquet("/mnt/silver/sales/")      # 50M
rows
df_products = spark.read.parquet("/mnt/silver/dim_products/") #
5,000 rows

# — AUTO BROADCAST (if table <
spark.sql.autoBroadcastJoinThreshold) —
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "200mb")
# Spark automatically broadcasts tables < 200MB

# — MANUAL BROADCAST HINT (when you KNOW table is small) ————
df_result = df_sales.join(
    F.broadcast(df_products), # explicit hint to broadcast
    on="product_code",
    how="left"
)

# — VERIFY: Check explain plan ————
df_result.explain()
# Look for: BroadcastHashJoin
```

```

# NOT SortMergeJoin (that would mean broadcast failed)

# — WHEN BROADCAST FAILS —————
# If product table grows to 300MB → exceeds threshold → fallback
to shuffle
# Solution 1: Increase threshold
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", "500mb")

# Solution 2: Filter product table before broadcasting
# Only broadcast products that exist in today's sales
active_products = df_products.join(
    df_sales.select("product_code").distinct(),
    on="product_code",
    how="inner" # only products sold today
)
# Now maybe only 1,000 rows → easily fits in broadcast!
df_result = df_sales.join(F.broadcast(active_products),
on="product_code")

```

◆ Sort-Merge Join & Handling Skew

```

# SORT-MERGE JOIN: Large tables (no choice but to shuffle)
df_sales = spark.read.parquet("/mnt/silver/sales/") # 50M
rows
df_customers = spark.read.parquet("/mnt/silver/customers/") # 5M
rows

# Both too large to broadcast → Sort-Merge Join
df_result = df_sales.join(df_customers, on="customer_id",
how="left")
df_result.explain()
# Shows: SortMergeJoin

```

```
# — DATA SKEW PROBLEM —————  
# Problem: 80% of sales have customer_id = "WALK-IN" (cash  
customers)  
# When joining by customer_id:  
#   → All WALK-IN sales (40M rows!) go to ONE partition  
#   → One executor processes 40M rows alone  
#   → 31 executors sit idle waiting  
#   → Job takes 2 hours instead of 5 minutes!
```

```
# — SOLUTION 1: Salting technique —————
```

```
import random  
  
# Add random salt to skewed key  
df_sales_salted = df_sales \  
    .withColumn("salt", (F.rand() * 10).cast("int")) \  
    .withColumn("join_key_salted",  
        F.when(F.col("customer_id") == "WALK-IN",  
            F.concat(F.col("customer_id"), F.lit("_"),  
F.col("salt")))  
        .otherwise(F.col("customer_id")))  
  
# Explode customer table for WALK-IN  
from pyspark.sql.functions import array, explode  
  
df_cust_salted = df_customers \  
    .withColumn("salt_range", F.array([F.lit(i) for i in  
range(10)])) \  
    .withColumn("salt", F.explode(F.col("salt_range"))) \  
    .withColumn("join_key_salted",  
        F.when(F.col("customer_id") == "WALK-IN",  
            F.concat(F.col("customer_id"), F.lit("_"),  
F.col("salt")))  
        .otherwise(F.col("customer_id"))) \  
    .drop("salt_range", "salt")
```



```
# Now join on salted key → WALK-IN distributed across 10
partitions!
df_result = df_sales_salted.join(
    df_cust_salted,
    on="join_key_salted",
    how="left"
)

# — SOLUTION 2: AQE Skew Join (automatic in Spark 3.x) —
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
"256mb")
# AQE automatically splits the skewed WALK-IN partition!
```

10 Spark Performance Optimization

◆ Cache vs Persist

```
# PROBLEM: Using same DataFrame 3 times in pipeline
# Without caching: data READ FROM DISK 3 times!

df_sales = spark.read.parquet("/mnt/bronze/sales/2024/") # 50GB

# Usage 1: Count rows
count = df_sales.count() # reads 50GB from disk

# Usage 2: Calculate regional totals
regional = df_sales.groupBy("region").sum("amount") # reads 50GB
again!

# Usage 3: Find top customers
```

```
top_cust = df_sales.groupBy("customer_id").sum("amount") # reads
50GB AGAIN!
```

```
# TOTAL: 150GB read from storage! Slow and expensive!
```

```
# — SOLUTION: Cache (stores in memory) —————
```

```
df_sales = spark.read.parquet("/mnt/bronze/sales/2024/")
df_sales.cache() # marks for caching
# OR
df_sales.persist() # same as cache() for default storage
level
```

```
# First action triggers actual caching:
```

```
count = df_sales.count() # reads 50GB from disk → stores in
memory
```

```
# Subsequent uses: read from MEMORY (not disk)
```

```
regional = df_sales.groupBy("region").sum("amount") # reads
from memory!
```

```
top_cust = df_sales.groupBy("customer_id").sum("amount") # reads
from memory!
```

```
# — STORAGE LEVELS —————
```

```
from pyspark import StorageLevel
```

```
# MEMORY_ONLY (default for cache()):
```

```
df.persist(StorageLevel.MEMORY_ONLY)
```

```
# → Best performance
```

```
# → If doesn't fit → partitions evicted (recomputed next time)
```

```
# → Use for: data that FITS in cluster RAM
```

```
# MEMORY_AND_DISK (recommended for large data):
```

```
df.persist(StorageLevel.MEMORY_AND_DISK)
```

```
# → Fill memory first
```

```
# → Overflow to disk for rest
# → Use for: data larger than cluster RAM but smaller than disk

# DISK_ONLY:
df.persist(StorageLevel.DISK_ONLY)
# → Always write to disk
# → Slower than memory but doesn't evict
# → Use for: checkpointing important intermediate results

# MEMORY_ONLY_SER (serialized):
df.persist(StorageLevel.MEMORY_ONLY_SER)
# → Compress before storing in memory
# → 5-10x less memory used
# → 10-20% slower CPU (serialize/deserialize)
# → Use for: fitting more data in limited RAM

# ALWAYS UNPERSIST WHEN DONE!
df_sales.unpersist()
# Releases memory for other operations
# Forgetting to unpersist → OOM errors later in pipeline!

# — WHEN TO CACHE — Decision Rules —————
# ✅ Cache when: same DataFrame used 2+ times in pipeline
# ✅ Cache when: expensive computation (ML features, complex joins)
# ✅ Cache when: iterative algorithms (ML training)
# ❌ Don't cache: used only once (wastes memory)
# ❌ Don't cache: data too large for cluster memory (causes evictions)
# ❌ Don't cache: simple reads (Parquet reads are fast anyway)
```

◆ Partitioning and Repartitioning

```
# UNDERSTAND CURRENT PARTITIONING:
print(f"Current partitions: {df_sales.rdd.getNumPartitions()}")

# — REPARTITION (full shuffle) —————
# Use when: INCREASING partitions or repartitioning by specific
column

# Repartition to specific count (default 200)
df_repartitioned = df_sales.repartition(100)

# Repartition by column (data locality for joins/aggregations)
df_by_region = df_sales.repartition(F.col("region_code"))
# All "N" region rows → same partition(s)
# All "S" region rows → same partition(s)
# Subsequent groupBy("region_code") → no shuffle needed!

# — COALESCE (no shuffle – only reduces partitions) —————
# Use when: REDUCING partitions (after filtering removes data)

# After filtering 90% of data:
df_india_north = df_sales.filter(F.col("region_code") == "N")
# Now only 10% of data but still 200 partitions
# Most partitions nearly empty → waste!

df_optimized = df_india_north.coalesce(20) # reduce without
shuffle
# 200 partitions → 20 partitions (no network transfer)

# — REPARTITION vs COALESCE RULE —————
# Increasing partitions → REPARTITION (needs shuffle to
distribute)
# Decreasing partitions → COALESCE (no shuffle, just merge local)

# — OPTIMAL PARTITION COUNT —————
# Rule of thumb: 2-4 partitions per CPU core
```

```

# 8 worker nodes × 4 cores = 32 cores
# Optimal partitions: 32 × 3 = 96 → round to 100

# For writing: match output file count to downstream needs
# Too many files (1000 tiny files) → slow reads later!
# Write as fewer, larger files:

df_final.coalesce(10) \
    .write \
    .mode("overwrite") \
    .parquet("/mnt/silver/sales/")
# Writes exactly 10 Parquet files = clean, efficient output

# But for partitioned write:
df_final.repartition(F.col("region_code")) \
    .write \
    .mode("overwrite") \
    .partitionBy("year", "month", "region_code") \
    .parquet("/mnt/silver/sales/")
# Creates one file per partition folder = efficient for
downstream filters

```

◆ Complete Performance Optimization Checklist

```

# — PRODUCTION PERFORMANCE CHECKLIST —

# 1. USE EXPLICIT SCHEMA (2x faster than inferSchema)
schema = StructType([...])
df = spark.read.schema(schema).parquet(path)

# 2. FILTER EARLY (less data in memory)
df = spark.read.parquet(path) \

```

```

        .filter(F.col("sale_date") >= "2024-01-01") \ #
filter first
        .filter(F.col("status") == "ACTIVE")          #
reduce data ASAP

# 3. SELECT ONLY NEEDED COLUMNS
df = df.select("sale_id", "customer_id", "total_amount") # drop
unused columns

# 4. BROADCAST SMALL TABLES
df_result = big_table.join(F.broadcast(small_table), on="key")

# 5. USE BUILT-IN FUNCTIONS (not Python UDFs)
# Good:
df = df.withColumn("upper_name", F.upper(F.col("product_name")))
# Bad:
# udf(lambda x: x.upper())(col) → Python UDF is 10x slower

# 6. AVOID CARTESIAN PRODUCTS
# Never join without condition! → produces M×N rows!
# Always specify join condition

# 7. REPARTITION BEFORE MULTIPLE JOINS
# If joining by same key multiple times:
df = df.repartition(F.col("customer_id")) # partition once
result1 = df.join(orders, on="customer_id") # no reshuffle
result2 = df.join(activity, on="customer_id") # no reshuffle

# 8. CACHE CORRECTLY
# Only cache what's used multiple times
# Always unpersist when done

# 9. SET SHUFFLE PARTITIONS APPROPRIATELY
# Default 200 is too many for small data, too few for large data
data_size_gb = 50

```

```
recommended_partitions = max(200, int(data_size_gb * 4))
spark.conf.set("spark.sql.shuffle.partitions",
recommended_partitions)

# 10. ENABLE AQE
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled",
"true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
```

11 Handling Multiple File Formats

```
# — CSV DATA —

# Read (with all production options)
df_csv = spark.read \
    .schema(my_schema) \
    .option("header", "true") \
    .option("delimiter", ",") \
    .option("encoding", "UTF-8") \
    .option("nullValue", "") \
    .option("nanValue", "NaN") \
    .option("escape", "'") \
    .option("multiLine", "false") \
    .option("mode", "DROPMALFORMED") # drop bad rows
    .csv("/mnt/bronze/sales/*.csv")

# Write
df.write \
    .option("header", "true") \
    .option("delimiter", ",") \
    .mode("overwrite") \
```

```

.csv("/mnt/silver/sales_csv/")

# ——— JSON DATA ———

# Flat JSON
df_json = spark.read \
    .option("multiLine", "false") \ # false = one JSON object
    per line
    .json("/mnt/bronze/payments/*.json")

# Nested JSON (payment with items array)
# {
#   "payment_id": "P001",
#   "amount": 50000,
#   "items": [
#     {"product": "LAPTOP", "qty": 1, "price": 45000},
#     {"product": "BAG", "qty": 1, "price": 5000}
#   ]
# }

df_payments = spark.read.json("/mnt/bronze/payments/")

# Explode nested array
df_items = df_payments \
    .withColumn("item", F.explode("items")) \
    .select(
        "payment_id",
        "amount",
        F.col("item.product").alias("product"),
        F.col("item.qty").alias("quantity"),
        F.col("item.price").alias("unit_price")
    )

# ——— PARQUET FILES ———
# Best format for analytical workloads

```



```

# Read
df_parquet = spark.read.parquet("/mnt/silver/sales/")

# Read with partition filter (partition pruning)
df_june = spark.read.parquet("/mnt/silver/sales/") \
    .filter(F.col("year") == 2024) \
    .filter(F.col("month") == 6)
# Spark reads ONLY year=2024/month=6 folder!

# Write with partitioning
df.write \
    .mode("overwrite") \
    .partitionBy("year", "month", "region_code") \
    .option("compression", "snappy") \
    .parquet("/mnt/silver/sales/")

# Resulting folder structure:
# /mnt/silver/sales/
#   year=2024/month=6/region_code=N/part-0000.parquet
#   year=2024/month=6/region_code=S/part-0000.parquet
#   year=2024/month=7/region_code=N/part-0000.parquet

# Format comparison:
# CSV:      Simple, human-readable, NO compression, row-based
# JSON:     Semi-structured, verbose, good for nested data
# Parquet:  Columnar, compressed (70% smaller), BEST for analytics
# Avro:     Row-based, good for streaming, schema evolution
# ORC:      Columnar, good for Hive, predicate pushdown
# Delta:    Parquet + transaction log = ACID + time travel

```

1 2 Databricks Utilities — Complete Guide

```

# — 1. FILESYSTEM UTILITY (dbutils.fs) —————
# Already covered in DBFS section

# — 2. SECRETS UTILITY —————
# Securely access Key Vault secrets – never print/log these!

# Get single secret
storage_key = dbutils.secrets.get(
    scope="kv-retailmart-scope",    # name of secret scope
    key="adls-storage-key"         # secret name in Key Vault
)

# List available secrets in scope (shows names only, not values)
dbutils.secrets.list("kv-retailmart-scope")

# List all scopes
dbutils.secrets.listScopes()

# CRITICAL SECURITY RULE:
# When you print a secret in Databricks:
print(dbutils.secrets.get("kv-retailmart-scope", "adls-storage-
key"))
# Output: [REDACTED] ← Databricks automatically redacts it!

# — 3. NOTEBOOK UTILITY —————
# Run other notebooks and pass/receive values

# Run another notebook (must be in same workspace)
result = dbutils.notebook.run(
    path="/Production/Utils/common_functions",    # notebook path
    timeout_seconds=3600,                        # max 1 hour
    arguments={
        "input_path": "/mnt/bronze/sales/",
        "process_date": "2024-06-15",
        "env": "prod"
    }
)

```

```

    }
)
print(f"Child notebook returned: {result}")

# In the CHILD notebook – receive and return:
dbutils.widgets.text("input_path", "")
dbutils.widgets.text("process_date", "")

input_path = dbutils.widgets.get("input_path")
process_date = dbutils.widgets.get("process_date")

# ... do processing ...

# Return value to parent notebook
dbutils.notebook.exit("SUCCESS: 142850 rows processed")

# — 4. WIDGETS UTILITY —————
# Interactive parameters for notebooks

# Text input
dbutils.widgets.text(
    name="process_date",
    defaultValue="2024-06-15",
    label="Process Date (yyyy-MM-dd)"
)

# Dropdown
dbutils.widgets.dropdown(
    name="environment",
    defaultValue="dev",
    choices=["dev", "test", "prod"],
    label="Environment"
)

# Combobox (dropdown + custom text)

```

```

dbutils.widgets.combobox(
    name="region",
    defaultValue="ALL",
    choices=["ALL", "N", "S", "E", "W"],
    label="Region Code"
)

# Multiselect
dbutils.widgets.multiselect(
    name="tables",
    defaultValue="sales",
    choices=["sales", "customers", "products", "inventory"],
    label="Tables to Process"
)

# Get widget values
process_date = dbutils.widgets.get("process_date")
environment  = dbutils.widgets.get("environment")
region       = dbutils.widgets.get("region")
tables       = dbutils.widgets.get("tables") # comma-separated
string

# Remove widgets
dbutils.widgets.remove("process_date")
dbutils.widgets.removeAll()

# — 5. CREDENTIALS UTILITY —
# (Available in Unity Catalog environments)
# Access credentials for external storage

# — 6. DATA UTILITY —
# Convert Spark DataFrame to Pandas for small data
import pandas as pd

spark_df = spark.sql("SELECT region_code, COUNT(*) as cnt FROM

```

```
sales GROUP BY 1")
pandas_df = spark_df.toPandas() # only for small DataFrames!

# Convert back
spark_df_back = spark.createDataFrame(pandas_df)
```

13 Databricks Cluster Management

◆ Types of Clusters

CLUSTER TYPES COMPARISON:

ALL-PURPOSE CLUSTER:

- Stays **running** continuously (until you stop it)
- Shared **by** multiple users **and** notebooks simultaneously
- Interactive development **and** exploration
- More expensive (**running** even **when** idle)
- Auto-terminate after N minutes **of** inactivity

USE **FOR**: Development, data exploration, ad-hoc queries

JOB CLUSTER:

- Created specifically **for ONE** job run
- Starts **when** job begins → terminates **when** job ends
- Cannot be shared (dedicated **to** that job)
- Cheaper (**only** runs during job execution)
- Cannot SSH **or** attach notebooks while job **running**

USE **FOR**: Production scheduled pipelines (ADF-triggered)

DIFFERENCE **IN** PRACTICE:

- Analyst exploring data → **All**-Purpose (interactive)
- Nightly sales pipeline → Job Cluster (automated)

Cost Example:

All-Purpose (runs 24/7): 8 nodes × \$0.30/hr × 24hr = \$57.60/day

Job Cluster (runs 2hr): 8 nodes × \$0.30/hr × 2hr = \$4.80/day

→ Job cluster = 12x cheaper for scheduled workloads!

◆ Cluster Modes

CLUSTER MODES:

STANDARD (Multi-user):

- Supports Python, Scala, SQL, R
- Multiple users can attach notebooks
- Processes run as single user (cluster owner)
- Good for most development scenarios

HIGH CONCURRENCY:

- Optimized for many concurrent users
- Each user's queries run in isolated containers
- Fine-grained resource sharing
- Automatic user isolation (FAIR scheduler)
- Best for BI teams with many analysts
- Does NOT support Scala

SINGLE NODE:

- No worker nodes – driver only
- Good for: small data, testing, pandas workloads
- Not for production big data

AUTOSCALING:

- Cluster dynamically adds/removes workers based on load
- Configuration:
 - Min workers: 2 (always running)
 - Max workers: 16 (scales up when needed)
- Scales up: when tasks queue up (more work than capacity)
- Scales down: when executors idle > scaleDownDelay
- Very cost-effective for variable workloads

◆ Creating and Managing Clusters

PRODUCTION CLUSTER CONFIGURATION:

Cluster Name: RetailMart-Prod-Pipeline
Policy: Custom (enforced by admin)
Mode: Standard
Databricks Runtime: 13.3 LTS (Long Term Support)
→ LTS = supported 2+ years, stable, recommended for production

Worker Type: Standard_DS4_v2 (8 cores, 28GB RAM each)
Min Workers: 4
Max Workers: 16
Auto Scale: Enabled

Driver Type: Standard_DS4_v2 (same as worker)

Auto Termination: 30 minutes (for dev clusters)
OFF (for production - job clusters auto-terminate)

Spark Config:
spark.sql.adaptive.enabled true

```
spark.sql.adaptive.coalescePartitions.enabled true
spark.databricks.delta.optimizeWrite.enabled true
spark.databricks.delta.autoCompact.enabled true
```

Environment Variables:

```
ENV=production
LOG_LEVEL=INFO
```

Libraries:

```
PyPI: great-expectations==0.18.0
PyPI: azure-storage-blob==12.19.0
```

Init Scripts:

```
dbfs:/init-scripts/install-custom-libs.sh
```

— Cluster Operations in Code —

List clusters (using Databricks REST API from notebook)

```
import requests
```

```
token = dbutils.secrets.get("kv-scope", "databricks-token")
```

```
workspace_url = "https://adb-xxxx.azuredatabricks.net"
```

```
headers = {"Authorization": f"Bearer {token}"}
```

List clusters

```
response = requests.get(
    f"{workspace_url}/api/2.0/clusters/list",
    headers=headers
)
```

```
clusters = response.json()["clusters"]
```

```
for c in clusters:
```

```
    print(f>Name: {c['cluster_name']}, State: {c['state']}")
```



```
# Get cluster info
# Monitor cluster health, logs via UI:
# Compute → Your Cluster → Event Log
# Shows: Started, Autoscaling events, Node failures, etc.

# — Cluster Logs —————
# Configure log delivery in cluster settings:
# Logging → Destination: DBFS
# Path: /cluster-logs/
# Logs written to: /cluster-logs/<cluster-id>/driver/log4j-
*.log.gz

# View logs in notebook
driver_logs = dbutils.fs.ls(f"/cluster-
logs/{cluster_id}/driver/")
for log in driver_logs:
    print(log.name)
```

14 Batch Processing — RetailMart End-to-End

◆ Historical Data Load

```
# SCENARIO: First-time load of 5 years of historical sales data
# 2.5 TB total, 50M records per year, 250M total

# — STRATEGY: Partition-by-partition historical load —————

years = list(range(2019, 2025)) # 2019 to 2024

for year in years:
    print(f"\n{'='*50}")
    print(f"Processing year: {year}")
```

```

# Read one year at a time (avoid OOM)
df_year = spark.read \
    .schema(sales_schema) \
    .parquet(f"/mnt/bronze/sales/historical/{year}/")

print(f"Year {year}: {df_year.count():,} records")

# Apply transformations
df_transformed = transform_sales(df_year, year)

# Write to Silver (append mode for historical)
df_transformed.write \
    .mode("append") \
    .partitionBy("year", "month") \
    .parquet("/mnt/silver/sales/")

print(f"✅ Year {year} loaded successfully")

# — PARALLEL HISTORICAL LOAD (faster using Spark itself) —
# Better approach: Let Spark read all years and handle
parallelism

df_all_historical = spark.read \
    .schema(sales_schema) \
    .parquet("/mnt/bronze/sales/historical/*/")
# Glob pattern reads all year folders simultaneously!

# Spark distributes file reading across cluster
# 250M records read and processed in parallel
df_all_transformed = transform_sales(df_all_historical)

df_all_transformed.write \
    .mode("overwrite") \

```

```
.partitionBy("year", "month", "region_code") \  
.parquet("/mnt/silver/sales/")
```

◆ Incremental Data Load

```
# SCENARIO: Daily incremental load – only new/changed records
```

```
# — METHOD: Watermark-based incremental —————
```

```
# Read last watermark from control table
```

```
watermark_df = spark.read \  
    .format("jdbc") \  
    .option("url", jdbc_url) \  
    .option("query", "SELECT last_watermark FROM ControlTable  
WHERE table_name='SALES'") \  
    .load()
```

```
last_watermark = watermark_df.first()["last_watermark"]  
print(f>Loading records after: {last_watermark}")
```

```
# Read only new records from Bronze
```

```
df_incremental = spark.read.parquet("/mnt/bronze/sales/") \  
    .filter(F.col("created_timestamp") > last_watermark)
```

```
record_count = df_incremental.count()  
print(f>New records to process: {record_count:,}")
```

```
if record_count == 0:  
    print("No new records – skipping")  
    dbutils.notebook.exit("NO_NEW_DATA")
```

```
# Transform
```

```

df_transformed = transform_sales(df_incremental)

# Write incrementally (append only)
df_transformed.write \
    .mode("append") \
    .partitionBy("year", "month") \
    .parquet("/mnt/silver/sales/")

# Update watermark
new_watermark =
df_incremental.agg(F.max("created_timestamp")).first()[0]
# Update control table via JDBC
spark.sql(f"""
    UPDATE ControlTable
    SET last_watermark = '{new_watermark}'
    WHERE table_name = 'SALES'
""")

print(f"✅ Incremental load complete. New watermark:
{new_watermark}")

```

◆ Complex Aggregations

```

# RETAILMART: Daily Business Intelligence Aggregations

df_sales = spark.read.parquet("/mnt/silver/sales/2024/")

# — Store Performance Dashboard —————
store_performance = df_sales \
    .groupBy("store_id", "store_name", "city", "state") \
    .agg(
        F.count("sale_id").alias("total_transactions"),

```

```

    F.sum("total_amount").alias("gross_revenue"),
    F.sum("gst_amount").alias("total_gst"),
    F.sum("net_amount").alias("net_revenue"),
    F.avg("total_amount").alias("avg_transaction_value"),
    F.max("total_amount").alias("highest_transaction"),
    F.min("total_amount").alias("lowest_transaction"),
    F.countDistinct("customer_id").alias("unique_customers"),

    F.countDistinct("product_code").alias("unique_products_sold"),
    F.sum(F.when(F.col("payment_mode")== "cash",
    F.col("total_amount"))
        .otherwise(0)).alias("cash_revenue"),
    F.sum(F.when(F.col("payment_mode").isin(["card", "upi"]),
    F.col("total_amount"))
        .otherwise(0)).alias("digital_revenue"),
    F.sum(F.when(F.col("is_weekend")==True,
    F.col("total_amount"))
        .otherwise(0)).alias("weekend_revenue")
    )

```

— Ranking Stores

```

rank_window = Window.orderBy(F.col("net_revenue").desc())
state_window =
Window.partitionBy("state").orderBy(F.col("net_revenue").desc())

store_ranked = store_performance \
    .withColumn("national_rank", F.rank().over(rank_window)) \
    .withColumn("state_rank", F.rank().over(state_window)) \
    .withColumn("revenue_tier",
        F.when(F.col("national_rank") <= 10, "Platinum")
        .when(F.col("national_rank") <= 50, "Gold")
        .when(F.col("national_rank") <= 100, "Silver")
        .otherwise("Bronze"))

store_ranked.write \

```

```
.mode("overwrite") \  
.format("delta") \  
.saveAsTable("gold.store_performance_daily")
```

15 Databricks Integration

◆ Integration with ADLS Gen2

```
# APPROACH 1: Mount (covered in DBFS section)  
# APPROACH 2: Direct access using abfss:// (for sensitive  
workloads)  
  
spark.conf.set(  
    f"fs.azure.account.auth.type.  
{storage_account}.dfs.core.windows.net",  
    "OAuth"  
)  
spark.conf.set(  
    f"fs.azure.account.oauth.provider.type.  
{storage_account}.dfs.core.windows.net",  
  
    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider"  
)  
spark.conf.set(  
    f"fs.azure.account.oauth2.client.id.  
{storage_account}.dfs.core.windows.net",  
    dbutils.secrets.get("kv-scope", "sp-client-id")  
)  
spark.conf.set(  
    f"fs.azure.account.oauth2.client.secret.  
{storage_account}.dfs.core.windows.net",  
    dbutils.secrets.get("kv-scope", "sp-client-secret")
```

```

)
spark.conf.set(
    f"fs.azure.account.oauth2.client.endpoint.
    {storage_account}.dfs.core.windows.net",
    f"https://login.microsoftonline.com/{tenant_id}/oauth2/token"
)

df =
spark.read.parquet(f"abfss://{bronze@{storage_account}.dfs.core.windo

```

◆ Integration with Azure SQL Database

```

# JDBC CONNECTION (full flexibility)
jdbc_url = (
    f"jdbc:sqlserver://{server}.database.windows.net:1433;"
    f"database={database};"
    f"encrypt=true;"
    f"trustServerCertificate=false;"
    f"hostnameInCertificate=*.database.windows.net;"
    f"loginTimeout=30"
)

jdbc_props = {
    "user":      dbutils.secrets.get("kv-scope", "sql-username"),
    "password":  dbutils.secrets.get("kv-scope", "sql-password"),
    "driver":    "com.microsoft.sqlserver.jdbc.SQLServerDriver"
}

# READ from SQL (with partitioning for large tables)
df_sql = spark.read \
    .jdbc(
        url=jdbc_url,

```

```

        table="dbo.Sales",
        column="sale_id",      # partition column (numeric)
        lowerBound=1,          # min value
        upperBound=1000000,    # max value (from separate query)
        numPartitions=16,      # parallel reads!
        properties=jdbc_props
    )

# WRITE to SQL
df_result.write \
    .mode("append") \
    .option("batchsize", "100000") \
    .option("tableLock", "true") \
    .jdbc(url=jdbc_url, table="dbo.Sales_Silver",
properties=jdbc_props)

# BULK LOAD (faster for large writes)
df_result.write \
    .format("com.databricks.spark.sqldw") \
    .option("url", jdbc_url) \
    .option("forwardSparkAzureStorageCredentials", "true") \
    .option("dbTable", "dbo.Sales_Silver") \
    .option("tempDir",
"wasbs://temp@mystorageaccount.blob.core.windows.net/sqldw-temp")
\
    .mode("overwrite") \
    .save()

```

16 Databricks Streaming API

◆ Structured Streaming — Complete Guide

SCENARIO: RetailMart wants **real-time** fraud detection

- **Every** payment transaction must be checked **within 2** seconds
- Flag transactions > **Rs.1,00,000** from **new** devices
- Alert fraud team immediately

STREAMING CONCEPTS:

Input: Continuous stream of payment events
arriving **every** millisecond

Unbounded **Table** concept:

Spark treats stream **as** an infinite **table**
New data = **new rows** appended continuously
Your query runs continuously **on** this **table**

TRIGGER TYPES:

ProcessingTime("10 seconds")	→ micro-batch every 10s
Once()	→ run once, process all data, stop
AvailableNow()	→ process all pending data, stop
Continuous("1 second")	→ true continuous (experimental)

OUTPUT MODES:

Append:	Only new rows written to sink (default) Use for : streaming events, no aggregation
Update:	Only rows that changed since last write Use for : streaming aggregations
Complete:	Entire result table written each micro-batch Use for : small aggregations (totals, counts)

— *REAL-TIME PAYMENT FRAUD DETECTION* —————

```
from pyspark.sql.types import *  
from pyspark.sql import functions as F
```

```
# Payment event schema
```

```
payment_schema = StructType([  
    StructField("payment_id",    StringType(),    True),  
    StructField("customer_id",   StringType(),    True),  
    StructField("store_id",      IntegerType(),   True),  
    StructField("amount",        DoubleType(),    True),  
    StructField("device_id",     StringType(),    True),  
    StructField("location",      StringType(),    True),  
    StructField("timestamp",     TimestampType(), True),  
    StructField("payment_mode",  StringType(),    True)  
])
```

```
# — READ STREAM from ADLS (files arriving continuously) —————
```

```
df_payment_stream = spark.readStream \  
    .format("cloudFiles") \  
    .option("cloudFiles.format", "json") \  
    .schema(payment_schema) \  
    .load("/mnt/landing/payments/stream/")
```

```
# — FRAUD DETECTION TRANSFORMATIONS —————
```

```
df_analyzed = df_payment_stream \  
    .withColumn("is_high_value",  
        F.col("amount") > 100000) \  
    .withColumn("hour_of_day",  
        F.hour(F.col("timestamp"))) \  
    .withColumn("is_unusual_hour",  
        F.col("hour_of_day").isin([0,1,2,3,4])) \  
    .withColumn("fraud_risk_score",  
        F.when((F.col("is_high_value")) &  
(F.col("is_unusual_hour")), 3)  
        .when(F.col("is_high_value"), 2)  
        .when(F.col("is_unusual_hour"), 1)  
        .otherwise(0)) \  
    .withColumn("fraud_flag",
```

```

        F.col("fraud_risk_score") >= 2) \
        .withColumn("processing_timestamp",
            F.current_timestamp())

# — WRITE STREAM to Delta Table (append mode) —————
query_all = df_analyzed.writeStream \
    .outputMode("append") \
    .format("delta") \
    .option("checkpointLocation", "/mnt/checkpoints/payments/") \
    .trigger(processingTime="10 seconds") \
    .table("silver.payment_events")

# — WRITE ONLY FRAUD ALERTS separately —————
query_fraud = df_analyzed \
    .filter(F.col("fraud_flag") == True) \
    .writeStream \
    .outputMode("append") \
    .format("delta") \
    .option("checkpointLocation",
"/mnt/checkpoints/fraud_alerts/") \
    .trigger(processingTime="5 seconds") \
    .table("gold.fraud_alerts")

# — MONITOR STREAMING QUERY —————
print(query_all.status)
print(query_all.lastProgress)
# Shows: inputRowsPerSecond, processedRowsPerSecond,
triggerExecution

# Stop stream
query_all.stop()
query_fraud.stop()

```

◆ Checkpointing and Fault Tolerance

CHECKPOINTING EXPLAINED:

Checkpoint stores:

1. Current query state (watermarks, aggregations)
2. What data has been processed (offsets)
3. Metadata about streaming query

WHY IT MATTERS:

Payment stream processes 10,000 events/second

Cluster fails at 3:47 AM after processing 1M events

WITHOUT checkpoint:

Restart at 3:48 AM → starts over from beginning

Re-processes 1M events → DUPLICATE FRAUD ALERTS!

WITH checkpoint:

Restart at 3:48 AM → reads checkpoint

Knows: "I processed up to offset 1,000,000"

Continues from offset 1,000,001

Exactly-once processing guaranteed!

CHECKPOINT LOCATION RULES:

- ✓ Each streaming query needs its OWN checkpoint location
- ✓ Store in reliable storage (ADLS, not local disk)
- ✓ Don't delete checkpoint manually (breaks stream)
- ✓ If query logic changes → use new checkpoint location

query.writeStream

```
.option("checkpointLocation",  
"/mnt/checkpoints/payments_v2/")  
# v2 = new checkpoint for new version of query
```

17 Databricks Auto Loader

◆ What is Auto Loader?

AUTO LOADER = Incremental file ingestion **at** scale

THE PROBLEM it solves:

1000 CSV files arrive daily **in** ADLS landing zone

Naive approach (**using** `spark.read`):

- `df = spark.read.csv("/mnt/landing/sales/*.csv")`
- **Reads ALL** files **every** run (including already processed!)
- Must manually track which files were processed
- Extra logic, complex, error-prone

Auto Loader approach (**using** `spark.readStream + cloudFiles`):

- Automatically tracks which files have been processed
- **Only** processes **NEW** files **each** run
- Uses Azure Event Grid/Queue internally
- Handles millions **of** files **per hour**
- Schema inference **and** evolution built-**in**

— AUTO LOADER SETUP —

Two detection modes:

1. DIRECTORY LISTING (simpler, works for < 10M files):

Lists directory each trigger, finds new files

2. FILE NOTIFICATION (scalable, for massive volumes):

Uses Azure Event Grid → notifications on new files

Instant detection, no listing overhead

— BASIC AUTO LOADER —

```

df_stream = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "csv") \
    .option("cloudFiles.schemaLocation", "/mnt/checkpoints/sales-
schema/") \
    .option("header", "true") \
    .load("/mnt/landing/sales/")

# Write incrementally to Delta
query = df_stream.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "/mnt/checkpoints/sales-
loader/") \
    .trigger(availableNow=True) \
    .table("bronze.sales_raw")

query.awaitTermination()

# — SCHEMA INFERENCE AND EVOLUTION —————

# Schema Inference (first run):
# Auto Loader samples files to infer schema
# Stores schema at schemaLocation
# Uses same schema for all subsequent runs

# Schema Hints (enforce specific types):
df_stream_typed = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "csv") \
    .option("cloudFiles.schemaLocation", "/mnt/checkpoints/sales-
schema/") \
    .option("cloudFiles.schemaHints",
            "sale_id INT, total_amount DOUBLE, sale_date DATE") \
    .option("header", "true") \

```

```

        .load("/mnt/landing/sales/")

# Schema Evolution Modes:
.option("cloudFiles.schemaEvolutionMode", "addNewColumns")
# Options:
# addNewColumns → New columns in CSV? Add to Delta table schema
(recommended)
# failOnNewColumns → New column = pipeline error (strict mode)
# rescue → Unknown columns saved to _rescued_data column
# none → Ignore new columns entirely

# — PRODUCTION AUTO LOADER PATTERN —————
def run_auto_loader(
    source_path,
    target_table,
    checkpoint_path,
    schema_path,
    trigger_once=True
):
    """Production-grade Auto Loader function"""

    stream = spark.readStream \
        .format("cloudFiles") \
        .option("cloudFiles.format", "parquet") \
        .option("cloudFiles.schemaLocation", schema_path) \
        .option("cloudFiles.schemaEvolutionMode",
"addNewColumns") \
        .option("cloudFiles.useIncrementalListing", "true") \
        .load(source_path) \
        .withColumn("ingestion_timestamp", F.current_timestamp())
\
        .withColumn("source_file", F.input_file_name())

    writer = stream.writeStream \
        .format("delta") \

```

```

        .outputMode("append") \
        .option("checkpointLocation", checkpoint_path) \
        .option("mergeSchema", "true") \
        .queryName(f"auto_loader_{target_table}")

    if trigger_once:
        query =
writer.trigger(availableNow=True).table(target_table)
        query.awaitTermination()
    else:
        query = writer.trigger(processingTime="1
minute").table(target_table)
    return query

# Run for each source
run_auto_loader("/mnt/landing/sales/",      "bronze.sales",
                "/mnt/ckpt/sales/",         "/mnt/schema/sales/")
run_auto_loader("/mnt/landing/customers/",  "bronze.customers",
                "/mnt/ckpt/customers/",
"/mnt/schema/customers/")

```

18 Delta Lake — The Lakehouse Foundation

◆ Data Lake vs Delta Lake

DATA LAKE (plain Parquet/CSV files):

PROBLEM 1: No ACID transactions

Two jobs write to same folder simultaneously

→ Partial writes → corrupted data!

PROBLEM 2: No schema enforcement

Job 1 writes: sale_id INT, amount DOUBLE

Job 2 writes: sale_id STRING, amount STRING (bug!)

→ Data lake now has mixed types → downstream breaks!

PROBLEM 3: No UPDATE or DELETE

Need to fix a wrong price in 50M records?

→ Must rewrite entire file/partition!

→ Cannot do DELETE WHERE sale_id = 1001

PROBLEM 4: Cannot read while writing

Report runs at 9 AM → pipeline starts writing at 9:01 AM

→ Report reads PARTIAL data → wrong numbers!

→ Called "read-write conflict"

PROBLEM 5: No history / no rollback

Bug in pipeline writes wrong data to 6 months of history

→ No way to go back to correct version!

DELTA LAKE solves ALL of these:

- ✓ ACID transactions
- ✓ Schema enforcement + evolution
- ✓ UPDATE, DELETE, MERGE (upsert)
- ✓ Reads never blocked by writes (snapshot isolation)
- ✓ Time travel (go back to any version)
- ✓ Audit history (see every change ever made)

◆ Delta Lake Internals

HOW DELTA LAKE WORKS:

Storage structure:

```
/mnt/silver/sales/
├── _delta_log/          ← TRANSACTION LOG (the secret!)
│   ├── 000000000000000000000000000000000000.json ← version 0 (initial
write)
│   ├── 000000000000000000000000000000000001.json ← version 1 (appended
data)
│   ├── 000000000000000000000000000000000002.json ← version 2 (updated
records)
│   ├── 000000000000000000000000000000000003.json ← version 3 (deleted
records)
│   └── 000000000000000000000000000000000010.checkpoint.parquet (every 10
versions)
├── part-0000-xxxx.snappy.parquet ← actual data files
├── part-0001-xxxx.snappy.parquet
└── year=2024/month=6/...          ← partition folders
```

TRANSACTION LOG (delta_log):

Each commit = one JSON file

JSON records:

- Which files were ADDED (new data written)
- Which files were REMOVED (deleted or replaced)
- Schema changes
- Commit timestamp + user + operation

READING Delta table:

1. Read latest transaction log
 2. Determine which data files are "active" (added but not removed)
 3. Read only those Parquet files
- Snapshot isolation: your read sees consistent state
 - Even if another job is writing simultaneously!

ACID in Delta:

Atomic: Entire write **succeeds or** nothing changes
Consistent: Schema enforced, **no** partial invalid data
Isolated: Concurrent readers/writers don't **interfere**
Durable: Committed data survives cluster failures

◆ Delta Lake Operations

```
# — CREATE DELTA TABLE —————

# Method 1: Write DataFrame as Delta
df_sales.write \
    .format("delta") \
    .mode("overwrite") \
    .partitionBy("year", "month") \
    .save("/mnt/silver/sales_delta/")

# Method 2: Create managed table in catalog
df_sales.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("silver.sales")

# Method 3: CREATE TABLE SQL
spark.sql("""
    CREATE TABLE IF NOT EXISTS silver.sales (
        sale_id      INT NOT NULL,
        customer_id  STRING NOT NULL,
        store_id     INT,
        total_amount DOUBLE,
        sale_date    DATE,
        region_code  STRING,
        year         INT,
```

```
        month      INT
    )
    USING DELTA
    PARTITIONED BY (year, month)
    LOCATION '/mnt/silver/sales/'
    COMMENT 'Cleaned daily sales data'
    """)
```

— DML OPERATIONS —

INSERT (append)

```
spark.sql("""
    INSERT INTO silver.sales
    SELECT * FROM bronze.sales_raw
    WHERE sale_date = current_date()
    """)
```

UPDATE (not possible in plain Parquet!)

```
spark.sql("""
    UPDATE silver.sales
    SET total_amount = total_amount * 1.1,
        updated_at = current_timestamp()
    WHERE region_code = 'N'
        AND sale_date = '2024-06-15'
        AND product_code = 'LAPTOP01'
    """)
```

DELETE (not possible in plain Parquet!)

```
spark.sql("""
    DELETE FROM silver.sales
    WHERE sale_id IN (
        SELECT sale_id FROM error_records
        WHERE error_type = 'DUPLICATE'
    )
    """)
```

```

# — MERGE (UPSERT) – Most Important Operation —————
# SCD Type 1: Update if exists, Insert if new

from delta.tables import DeltaTable

delta_table = DeltaTable.forPath(spark, "/mnt/silver/customers/")

# New/updated customers from today's load
df_updates =
spark.read.parquet("/mnt/bronze/customers/2024/06/15/")

delta_table.alias("target").merge(
    df_updates.alias("source"),
    condition="target.customer_id = source.customer_id"
) \
.whenMatchedUpdate(
    # When customer exists AND something changed:
    condition="target.email != source.email OR target.phone !=
source.phone",
    set={
        "email":      "source.email",
        "phone":      "source.phone",
        "address":     "source.address",
        "updated_at":  "current_timestamp()"
    }
) \
.whenNotMatchedInsert(
    # When customer doesn't exist yet:
    values={
        "customer_id": "source.customer_id",
        "name":         "source.name",
        "email":        "source.email",
        "phone":        "source.phone",
        "created_at":   "current_timestamp()",

```

```

        "updated_at": "current_timestamp()"
    }
) \
.execute()

print("Merge complete!")

```

◆ SCD Type 1 and Type 2

```

# — SCD TYPE 1: Overwrite (no history) —————
# Simple MERGE as shown above → just overwrites changed values

# — SCD TYPE 2: Keep full history —————
# Each change = new row with start/end dates

delta_customers = DeltaTable.forPath(spark,
"/mnt/silver/dim_customers_scd2/")

df_source =
spark.read.parquet("/mnt/bronze/customers/2024/06/15/") \
    .withColumn("effective_start", F.current_date())
\
    .withColumn("effective_end",    F.lit("9999-12-
31")).cast("date")) \
    .withColumn("is_current",      F.lit(True))

# Step 1: Expire existing records that have changed
delta_customers.alias("target").merge(
    df_source.alias("source"),
    """target.customer_id = source.customer_id
    AND target.is_current = true
    AND (target.email != source.email

```

```

        OR target.address != source.address)"""
    ) \
    .whenMatchedUpdate(set={
        "is_current": "false",
        "effective_end": "date_sub(current_date(), 1)"
    }) \
    .execute()

# Step 2: Insert new version of changed records + truly new
records
delta_customers.alias("target").merge(
    df_source.alias("source"),
    "target.customer_id = source.customer_id AND
target.is_current = true"
) \
.whenNotMatchedInsert(values={
    "customer_id": "source.customer_id",
    "name": "source.name",
    "email": "source.email",
    "address": "source.address",
    "effective_start": "source.effective_start",
    "effective_end": "source.effective_end",
    "is_current": "source.is_current"
}) \
.execute()

# Result in table:
# customer_id | email | effective_start | effective_end
# is_current
# C001 | old@mail.com | 2023-01-01 | 2024-06-14
# | false
# C001 | new@mail.com | 2024-06-15 | 9999-12-31
# | true ← current

```

◆ Time Travel

```
# TIME TRAVEL: Go back to any previous version!
# Useful for: debugging bad pipelines, auditing, data recovery

# — VIEW TABLE HISTORY —————
spark.sql("DESCRIBE HISTORY silver.sales").show(20,
truncate=False)
# Output:
# version | timestamp                | operation |
operationParameters
# 10      | 2024-06-15 02:15:00      | MERGE     | {predicate: ...}
# 9       | 2024-06-15 01:30:00      | WRITE     | {mode: Append}
# 8       | 2024-06-14 02:15:00      | MERGE     | ...
# ...

# — QUERY PREVIOUS VERSION —————
# By version number
df_v5 = spark.read \
    .format("delta") \
    .option("versionAsOf", "5") \
    .load("/mnt/silver/sales/")

# By timestamp (what did data look like yesterday at 9 PM?)
df_yesterday = spark.read \
    .format("delta") \
    .option("timestampAsOf", "2024-06-14 21:00:00") \
    .load("/mnt/silver/sales/")

# Using SQL
spark.sql("""
    SELECT * FROM silver.sales
    VERSION AS OF 5
    WHERE region_code = 'N'
""")
```



```

spark.sql("""
    SELECT * FROM silver.sales
    TIMESTAMP AS OF '2024-06-14 21:00:00'
""")

# — REAL WORLD: ROLLBACK BAD PIPELINE —————
# Pipeline bug wrote wrong data in version 10
# Need to restore to version 9

# Option 1: Restore table to version 9
spark.sql("RESTORE TABLE silver.sales TO VERSION AS OF 9")

# Option 2: Overwrite with previous version
df_good = spark.read.format("delta") \
    .option("versionAsOf", "9") \
    .load("/mnt/silver/sales/")

df_good.write.format("delta").mode("overwrite").save("/mnt/silver/sa

```

◆ Data Deduplication in Delta Tables

```

# SCENARIO: Due to a pipeline bug, some records were loaded twice
# Need to remove duplicates from Delta table

from delta.tables import DeltaTable

# — METHOD 1: INSERT INTO with EXCEPT —————
# Insert only records that don't already exist

delta_table = DeltaTable.forPath(spark, "/mnt/silver/sales/")

```

```

df_new_data = spark.read.parquet("/mnt/bronze/sales/today/")

# Only insert if sale_id doesn't already exist
delta_table.alias("t").merge(
    df_new_data.alias("s"),
    "t.sale_id = s.sale_id"
) \
.whenNotMatchedInsertAll() \
.execute()

# — METHOD 2: Create deduplicated version —————
from pyspark.sql.window import Window

df_with_row_num = spark.read.format("delta") \
    .load("/mnt/silver/sales/") \
    .withColumn("rn",
        F.row_number().over(
            Window.partitionBy("sale_id")
                .orderBy(F.col("load_timestamp").desc())
        )
    )

df_deduplicated = df_with_row_num.filter(F.col("rn") ==
1).drop("rn")

# Overwrite table with deduplicated data
df_deduplicated.write \
    .format("delta") \
    .mode("overwrite") \
    .save("/mnt/silver/sales/")

print(f"Original:
{spark.read.format('delta').load('/mnt/silver/sales/').count():,}")
print(f"After dedup: {df_deduplicated.count():,}")

```

19 Medallion Architecture — RetailMart Implementation

THE THREE LAYERS:

BRONZE (Raw Data):

- Exact **copy of** source data, **no** transformations
- Preserves original format + schema
- Immutable (never changed once written)
- Keeps **ALL** data including bad records
- Source **of** truth **for** reprocessing
- Parquet **or** Delta format
- Retention: **3-7** years (audit/compliance)

SILVER (Cleaned Data):

- Data quality rules applied
- Standardized **column** names **and** types
- Nulls handled, duplicates removed
- Basic business logic applied
- Joined **with** reference/dimension data
- Delta format (enables updates/deletes)
- Retention: **2-5** years

GOLD (Business-Ready Data):

- Aggregated, summarized **for specific** use cases
- Denormalized (wide tables **for** BI tools)
- **One table per** business question
- Optimized **for** fast query performance
- Delta format + Z-**ORDER for** performance
- Retention: **1-2** years (refreshed frequently)

```

# — BRONZE LAYER: Raw ingestion (no transformation) —————

def load_bronze(source_path, target_table, process_date):
    """Bronze: land exactly as-is from source"""

    df_raw = spark.read.parquet(source_path)

    # Only add ingestion metadata
    df_bronze = df_raw \
        .withColumn("_ingestion_timestamp",
F.current_timestamp()) \
        .withColumn("_source_file",          F.input_file_name())
    \
        .withColumn("_process_date",        F.lit(process_date))
    \
        .withColumn("_source_system",       F.lit("Oracle_ERP"))

    # Append to bronze (keep full history!)
    df_bronze.write \
        .format("delta") \
        .mode("append") \
        .partitionBy("_process_date") \
        .saveAsTable(f"bronze.{target_table}")

    print(f"Bronze loaded: {df_bronze.count():,} rows → bronze.
{target_table}")

# — SILVER LAYER: Clean and conform —————

def load_silver_sales(process_date):
    """Silver: clean, validate, enrich sales data"""

    # Read from Bronze
    df_bronze = spark.read.table("bronze.sales") \
        .filter(F.col("_process_date") ==

```

```

process_date)

# Data Quality
dq_rules = {
    "sale_id_not_null": F.col("SALE_ID").isNotNull(),
    "amount_positive": F.col("AMOUNT") > 0,
    "valid_status":
F.col("STATUS").isin(["COMPLETED", "PENDING"]),
    "valid_date": F.col("SALE_DATE").isNotNull()
}

df_valid = df_bronze
for rule_name, condition in dq_rules.items():
    before = df_valid.count()
    df_valid = df_valid.filter(condition)
    dropped = before - df_valid.count()
    if dropped > 0:
        print(f"⚠️ {rule_name}: dropped {dropped:,} rows")

# Standardize
df_silver = df_valid \
    .select(
        F.col("SALE_ID").cast("int").alias("sale_id"),
        F.upper(F.trim("CUSTOMER_ID")).alias("customer_id"),
        F.col("STORE_ID").cast("int").alias("store_id"),
        F.col("AMOUNT").cast("double").alias("total_amount"),
        F.to_date("SALE_DATE", "DD-MON-
YYYY").alias("sale_date"),
        F.upper("REGION_CODE").alias("region_code"),
        F.lower("PAYMENT_MODE").alias("payment_mode"),
        F.col("STATUS").alias("status")
    ) \
    .withColumn("year", F.year("sale_date")) \
    .withColumn("month", F.month("sale_date")) \
    .withColumn("gst_amount",

```

```

F.round(F.col("total_amount") * 0.18, 2)) \
    .withColumn("net_amount",      F.col("total_amount") -
F.col("gst_amount")) \
    .withColumn("updated_at",      F.current_timestamp())

# Merge into Silver Delta table (upsert)
delta_silver = DeltaTable.forName(spark, "silver.sales")

delta_silver.alias("t").merge(
    df_silver.alias("s"),
    "t.sale_id = s.sale_id"
) \
    .whenMatchedUpdateAll() \
    .whenNotMatchedInsertAll() \
    .execute()

print(f"Silver updated: {df_silver.count():,} rows")

# — GOLD LAYER: Business aggregations —————

def load_gold_store_daily(process_date):
    """Gold: daily store performance summary"""

    df_silver = spark.read.table("silver.sales") \
        .filter(F.col("sale_date") == process_date)

    df_stores = spark.read.table("silver.dim_stores")

    df_gold = df_silver \
        .join(F.broadcast(df_stores), on="store_id") \
        .groupBy(
            "sale_date", "store_id", "store_name",
            "city", "state", "region_code"
        ) \
        .agg(

```

```

        F.count("sale_id").alias("transaction_count"),
        F.sum("total_amount").alias("gross_revenue"),
        F.sum("net_amount").alias("net_revenue"),
        F.avg("total_amount").alias("avg_ticket_size"),

F.countDistinct("customer_id").alias("unique_customers"),
        F.sum(F.when(F.col("payment_mode")=="cash",
                        F.col("total_amount")).otherwise(0))
                        .alias("cash_revenue"),

F.sum(F.when(F.col("payment_mode").isin(["card", "upi"]),
            F.col("total_amount")).otherwise(0))
            .alias("digital_revenue")
    ) \
    .withColumn("digital_pct",
        F.round(F.col("digital_revenue") /
F.col("gross_revenue") * 100, 1))

# Optimize for BI queries
df_gold.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", f"sale_date = '{process_date}'")
\
    .saveAsTable("gold.store_daily_performance")

# Optimize table for fast queries
spark.sql("""
    OPTIMIZE gold.store_daily_performance
    ZORDER BY (store_id, sale_date)
""")

print(f"Gold layer updated for {process_date}")

```

— MASTER ORCHESTRATOR

```
def run_medallion_pipeline(process_date):
    print(f"\n🚀 Starting Medallion Pipeline for {process_date}")

    # Bronze
    load_bronze(f"/mnt/landing/sales/{process_date}/", "sales",
process_date)
    load_bronze(f"/mnt/landing/customers/{process_date}/",
"customers", process_date)

    # Silver
    load_silver_sales(process_date)
    load_silver_customers(process_date)

    # Gold
    load_gold_store_daily(process_date)
    load_gold_product_performance(process_date)
    load_gold_customer_360(process_date)

    print(f"\n✅ Medallion Pipeline Complete for {process_date}")

run_medallion_pipeline(process_date)
```

20 Workflows in Databricks

DATABRICKS WORKFLOWS = Native job scheduler

vs ADF:

ADF: Better for multi-service orchestration
DB Workflows: Better for Databricks-only workloads

WORKFLOW TYPES:

Notebook task: Run a specific notebook
Python script: Run a .py file
JAR task: Run a Java/Scala JAR
SQL task: Run a SQL query/dashboard
dbt task: Run dbt models
Delta Live Tables: Run DLT pipeline
Pipeline task: Chain multiple tasks with dependencies

```
# — CREATE JOB via UI —————  
# Workflows → Create Job  
# Add Tasks:  
#   Task 1: load_bronze_sales      (notebook:  
#   /Production/bronze/load_sales)  
#   Task 2: load_bronze_customers (notebook:  
#   /Production/bronze/load_customers)  
#   Task 3: transform_silver      (depends on Task 1 + 2)  
#   Task 4: load_gold             (depends on Task 3)  
#   Task 5: notify_completion     (depends on Task 4)  
  
# Schedule: Daily at 1:00 AM IST  
# Cluster: Job Cluster (auto-creates, auto-terminates)  
# Alerts: Email on failure  
  
# — JOB PARAMETERS —————  
# Pass parameters to all tasks in job:  
# Key: process_date  Value: {{job.start_time.iso_date}}  
# → Databricks automatically substitutes today's date!  
  
# — MONITOR JOB RUNS —————  
# Workflows → Job → Runs tab  
# See each run: duration, status, task breakdown  
# Click failed task → view error + logs  
  
# — REPAIR RUN (re-run only failed tasks) —————  
# Job failed at Task 3 (transform_silver)
```

```
# Tasks 1, 2 succeeded → don't rerun them!  
# Repair Run → only reruns Task 3, 4, 5  
# Saves time and cost vs full re-run!
```

21 Unity Catalog — Enterprise Governance

◆ What is Unity Catalog?

THE PROBLEM BEFORE UNITY CATALOG:

RetailMart has 3 workspaces:

Workspace 1: Data Engineering team

Workspace 2: Data Science team

Workspace 3: BI/Analytics team

Old way (Hive Metastore):

- ✗ Each workspace has its OWN separate metastore
- ✗ "silver.sales" in WS1 ≠ "silver.sales" in WS2
- ✗ Data Science can't easily access Data Engineering tables
- ✗ Security configured separately per workspace
- ✗ No unified audit log ("who accessed what when?")
- ✗ No data lineage ("where did this table come from?")
- ✗ DBA must set up permissions in each workspace separately
- ✗ PII data (customer phone) visible to everyone!

UNITY CATALOG SOLUTION:

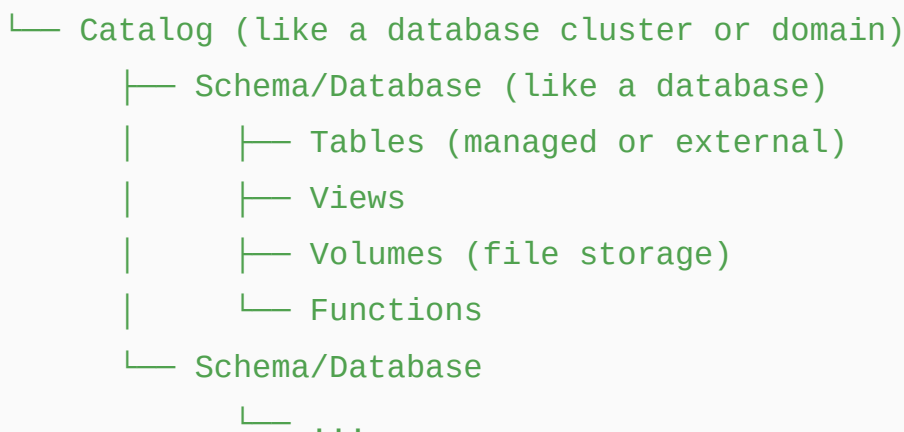
- ✓ ONE metastore for ALL workspaces in a region
- ✓ Tables registered once → accessible from all workspaces
- ✓ Centralized security → set permission once, applies everywhere
- ✓ Unified audit log → every query logged centrally

- ✓ Data lineage → visualize data flow table-to-table
- ✓ Column-level masking for PII
- ✓ Row-level security

2 2 Unity Catalog Object Hierarchy

UNITY CATALOG HIERARCHY:

Metastore (1 per Azure region)



RETAILMART EXAMPLE:

Metastore: retailmart_metastore_india



```

|   |─ Schema: sales
|   |   |─ Table: sales_fact          (managed)
|   |   |─ Table: sales_daily        (managed)
|   |─ Schema: dimensions
|   |   |─ Table: dim_stores
|   |   |─ Table: dim_products
|   |   |─ Table: dim_customers
|
|─ Catalog: gold                      (aggregated)
|   |─ Schema: analytics
|   |   |─ Table: store_performance
|   |   |─ Table: customer_360
|
|─ Catalog: sandbox                  (for data scientists -
experimentation)
|   |─ Schema: <each user's personal schema>

```

— TABLE TYPES —

MANAGED TABLE (Unity Catalog owns the data):

If you drop the table → DATA DELETED from storage

```

spark.sql("""
    CREATE TABLE silver.sales.transactions (
        sale_id      INT,
        customer_id   STRING,
        total_amount  DOUBLE,
        sale_date     DATE
    )
    USING DELTA
    COMMENT 'Cleaned sales transactions'
    TBLPROPERTIES ('owner' = 'data_engineering_team')
""")

```

EXTERNAL TABLE (you own the data):

If you drop the table → ONLY METADATA deleted, data safe!

```

spark.sql("""
    CREATE TABLE bronze.sales.transactions_raw
    USING DELTA
    LOCATION
    'abfss://bronze@retailmartlake.dfs.core.windows.net/sales/'
    COMMENT 'Raw Oracle sales data - do not modify'
""")

# VOLUMES (access files through Unity Catalog):
spark.sql("""
    CREATE VOLUME bronze.sales.raw_files
    LOCATION
    'abfss://bronze@retailmartlake.dfs.core.windows.net/raw/'
    COMMENT 'Raw CSV/Parquet files from source systems'
""")

# Access: /Volumes/bronze/sales/raw_files/2024/06/sales.parquet

```

23 Access Control & Security (RBAC)

```

# — GRANT/REVOKE PERMISSIONS —

# Grant catalog-level access
spark.sql("GRANT USE CATALOG ON CATALOG silver TO
`data_science_team`")
spark.sql("GRANT USE CATALOG ON CATALOG gold TO `bi_team`")

# Grant schema-level access
spark.sql("GRANT USE SCHEMA ON SCHEMA silver.sales TO
`data_science_team`")
spark.sql("GRANT USE SCHEMA ON SCHEMA gold.analytics TO
`bi_team`")

```

```
# Grant table-level access
spark.sql("GRANT SELECT ON TABLE silver.sales.transactions TO
`data_science_team`")
spark.sql("GRANT SELECT ON TABLE gold.analytics.store_performance
TO `bi_team`")

# Grant write permissions (engineering team only)
spark.sql("GRANT MODIFY ON TABLE silver.sales.transactions TO
`data_engineering_team`")

# Grant ALL permissions (table owner)
spark.sql("GRANT ALL PRIVILEGES ON TABLE
silver.sales.transactions TO `de_lead`")

# Revoke permissions
spark.sql("REVOKE SELECT ON TABLE silver.sales.transactions FROM
`temp_contractor`")

# Check current permissions
spark.sql("SHOW GRANTS ON TABLE
silver.sales.transactions").show()

# — ROLES HIERARCHY —
# Metastore Admin → full control over everything
# Catalog Owner → full control over one catalog
# Schema Owner → full control over one schema
# Table Owner → full control over one table
# User → only what explicitly granted
```

24 Data Protection & Privacy

```

# — COLUMN-LEVEL MASKING (hide PII) —————
# Scenario: Customer phone numbers should be hidden from BI team
# but visible to Data Engineering team

# Create masking function
spark.sql("""
    CREATE FUNCTION IF NOT EXISTS silver.utils.mask_phone(phone
STRING)
    RETURN CASE
        WHEN is_member('data_engineering_team') THEN phone
        ELSE CONCAT(LEFT(phone, 3), 'XXXXXXX')
    END
""")

# Apply masking policy to column
spark.sql("""
    ALTER TABLE silver.dimensions.dim_customers
    ALTER COLUMN phone
    SET MASK silver.utils.mask_phone
""")

# BI team sees:    +91XXXXXXX (masked)
# DE team sees:    +9198765XXXXX (original)

# — ROW-LEVEL SECURITY —————
# Scenario: Regional managers see only their region's data
# North India manager → only sees North India sales

# Create row filter function
spark.sql("""
    CREATE FUNCTION silver.utils.region_filter(region_code
STRING)
    RETURN is_member('admin_group')
        OR region_code = current_user_region()
""")

```

```

# (current_user_region() is a custom function mapping user to
region)

# Apply row filter to table
spark.sql("""
    ALTER TABLE silver.sales.transactions
    SET ROW FILTER silver.utils.region_filter ON (region_code)
""")

# North India manager runs: SELECT COUNT(*) FROM
silver.sales.transactions
# → Sees only North India rows automatically!
# Admin runs same query → sees ALL rows

# — SECURE VIEWS FOR SENSITIVE DATA —
spark.sql("""
    CREATE OR REPLACE VIEW gold.analytics.customer_summary_safe
    AS
    SELECT
        customer_id,
        customer_name,
        CONCAT(LEFT(email, 3), '***@***.com') AS email_masked,
        CONCAT('XXXXXXXX', RIGHT(phone, 2)) AS phone_masked,
        customer_tier,
        total_purchases,
        last_purchase_date
    FROM silver.dimensions.dim_customers
""")
-- BI team uses this view instead of raw customer table
-- PII is protected, business metrics still available

```



```
# — INTEGRATING DATABRICKS WITH DEVOPS REPOS —————

# In Databricks Workspace:
# User Settings → Git Integration
# Git Provider: Azure DevOps Services
# Personal Access Token: (from Azure DevOps)

# — REPOS STRUCTURE —————
# RetailMart Databricks Repos structure:

# Azure DevOps Repo: databricks-notebooks
# |
# └─ bronze/
#   │   └─ load_sales_bronze.py
#   │   └─ load_customers_bronze.py
#   │   └─ load_inventory_bronze.py
#   └─ silver/
#       │   └─ transform_sales.py
#       │   └─ transform_customers.py
#       │   └─ common_utils.py
#       └─ gold/
#           │   └─ store_performance.py
#           │   └─ customer_360.py
#           └─ tests/
#               │   └─ test_sales_transform.py
#               │   └─ test_data_quality.py
#               └─ config/
#                   └─ env_config.py

# In Databricks Repos tab:
# + Add Repo → URL:
https://dev.azure.com/retailmart/DataPlatform/_git/databricks-
notebooks
# Clone to: /Repos/data_engineering_team/databricks-notebooks
```

```
# — WORKFLOW —  
# 1. Pull latest changes from main branch  
# 2. Create feature branch: feature/add-returns-pipeline  
# 3. Develop notebooks in Databricks UI  
# 4. Changes auto-saved to branch  
# 5. Push to Azure DevOps  
# 6. Raise Pull Request  
# 7. Code review → approve  
# 8. Merge to main → CI/CD pipeline deploys to prod workspace
```

```
# — DATABRICKS REPOS API —  
import subprocess  
  
# Pull latest changes (in notebook or CI/CD script)  
def git_pull(repo_path, branch="main"):  
    result = subprocess.run(  
        ["git", "-C", f"/Workspace/Repos/{repo_path}", "pull"],  
        capture_output=True, text=True  
    )  
    print(result.stdout)  
    return result.returncode == 0
```

26 SDLC and Agile for Databricks Projects

RETAILMART DATABRICKS SPRINT EXAMPLE:

SPRINT 1 (Week 1-2):

- Story 1: Set up Databricks workspace and clusters
- Story 2: Configure ADLS Gen2 mounts and Key Vault
- Story 3: Build Bronze layer ingestion for Sales
- Story 4: Build Bronze layer ingestion for Customers

SPRINT 2 (Week 3-4):

- Story 5: Build Silver transformation for Sales
- Story 6: Implement SCD Type 2 for Customer dimension
- Story 7: Data quality framework setup
- Story 8: Unit tests for transformations

SPRINT 3 (Week 5-6):

- Story 9: Build Gold layer store performance
- Story 10: Implement streaming fraud detection
- Story 11: Unity Catalog setup and permissions
- Story 12: Performance optimization (caching, partitioning)

DEFINITION OF DONE for each notebook:

- ✓ Runs on development cluster without errors
- ✓ Unit tests written and passing
- ✓ Data quality checks implemented
- ✓ Logging and error handling added
- ✓ Code reviewed by peer
- ✓ Pushed to Git repo
- ✓ Documentation in first cell (purpose, inputs, outputs)
- ✓ Tested with production-scale data sample

27 End-to-End Data Migration Project

PROJECT: Migrate RetailMart from On-Premises to Azure Cloud

PHASE 1 — ASSESSMENT (Week 1-2):

- Inventory all source systems (Oracle, SQL Server, SAP)
- Document all tables, sizes, row counts
- Identify PII data (for masking rules)

- Define target schema (column naming conventions)
- Estimate data volume: 5 years × 2.5 TB = 12.5 TB

PHASE 2 — INFRASTRUCTURE SETUP (Week 3-4):

- Create Azure subscription, resource groups
- Set up ADLS Gen2 (Bronze/Silver/Gold containers)
- Create Databricks workspace + Unity Catalog
- Install SHIR on on-premise servers
- Configure ADF with all Linked Services
- Set up Key Vault for all secrets
- Configure Azure DevOps repo

PHASE 3 — HISTORICAL MIGRATION (Week 5-8):

- Load 5 years of historical data
- Oracle → ADLS Bronze (via ADF SHIR)
- Bronze → Silver (Databricks transformations)
- Silver → Gold (aggregations)
- Validate: row counts, checksums, business rules
- Reconcile reports: old system vs new system

PHASE 4 — PARALLEL RUN (Week 9-12):

- Both old and new systems run simultaneously
- Daily comparison of results
- Fix any discrepancies
- Business team validates new reports
- Performance testing under production load

PHASE 5 — CUTOVER (Week 13-14):

- Switch all pipelines to production ADF
- Business team switches to new Power BI dashboards
- Monitor closely for 2 weeks
- Decommission old on-premise processes

PHASE 6 — STABILIZATION (Week 15-16):

- Monitor all pipelines daily

- Performance tuning based **on** actual patterns
- Documentation **and** knowledge transfer
- Handover **to** operations team

28 Databricks Interview Questions

Q1: What **is** the difference **between** RDD, DataFrame, **and** Dataset?

RDD: Low-level, untyped, **no** optimization

DataFrame: High-level, schema-aware, Catalyst optimized

Dataset: Typed DataFrame (**only in** Scala/Java)

For 99% of PySpark work → use DataFrame

RDD → **only when** you need byte-level control

Q2: Explain lazy evaluation **in** Spark. Why **is** it important?

Transformations (**filter**, map, **select**) → lazy (**no** execution)

Actions (count, write, **collect**) → **trigger** execution

Importance:

- Catalyst sees the **WHOLE** transformation chain
- Can optimize (pushdown, pruning, **join** strategy)
- Single pass **over** data instead **of** multiple passes
- Example: **filter** pushed **to** file reader = less data read

Q3: What **is** a shuffle? How do you minimize it?

Shuffle = redistribution **of** data across executors **by** key

Happens **in**: groupBy, **join**, **distinct**, sort, repartition

Expensive because: network transfer, disk write/read

Minimize **by**:

- Use broadcast **join for** small tables
- **Filter**/project before joins (less data **to** shuffle)
- Repartition **by join** key before multiple joins **on** same key
- Use reduceByKey instead **of** groupByKey (combines locally **first**)
- Enable AQE (auto-optimizes shuffle partitions)

Q4: Explain Delta Lake **and** its benefits **over** plain Parquet.

Delta Lake = Parquet + Transaction Log

Benefits:

- ✓ ACID transactions (concurrent safe writes)
- ✓ **UPDATE/DELETE/MERGE** (**not** possible **in** Parquet)
- ✓ Schema enforcement (prevents bad data)
- ✓ Schema evolution (**add** columns safely)
- ✓ **Time** travel (query historical versions)
- ✓ Streaming + batch unified (same **table**)
- ✓ Automatic compaction **and** optimization

Q5: What **is** the Medallion Architecture?

Bronze: Raw landing data, exact **copy from** source

Silver: Cleaned, validated, conformed data

Gold: Aggregated, business-ready data

Benefits:

- Separation **of** concerns (**each** layer has **one** job)
- Reprocessability (Bronze → rerun Silver/Gold anytime)
- Multiple consumers (BI uses Gold, DS uses Silver)
- Audit trail (Bronze = complete history)

Q6: How do you handle slowly changing dimensions (SCD) **in** Databricks?

SCD Type 1 (overwrite):

→ **MERGE** statement **with** whenMatchedUpdate + whenNotMatchedInsert

SCD Type 2 (history):

→ **MERGE** to expire **old** record (**set** is_current=**false**, end_date)

→ **INSERT** **new** record (is_current=**true**, start_date=today)

→ Maintains **full** history of **all** changes

Q7: What **is** the difference **between** cache() **and** persist()?

cache() = persist() **with default** storage level MEMORY_AND_DISK

persist() = choose **specific** storage level:

MEMORY_ONLY, MEMORY_AND_DISK, DISK_ONLY, MEMORY_ONLY_SER

Use cache(): **when** data fits **in** memory

Use persist(MEMORY_AND_DISK): larger data, occasional reuse

Always unpersist() **when** done!

Q8: What **is** Unity Catalog? How does it differ **from** Hive Metastore?

Hive Metastore:

→ **Per**-workspace (**each** workspace isolated)

→ **No** central governance

→ Limited security (**table**-level **only**)

→ **No** audit log

Unity Catalog:

→ Across **all** workspaces **in** a region

→ Centralized governance

→ Fine-grained: catalog/schema/**table**/**column**/**row** level

→ Unified audit **log** (**every** query tracked)

→ Data lineage visualization

→ **Column** masking + **row** filtering

Q9: Explain Auto Loader and when to use it.

Auto Loader = incremental file ingestion using cloudFiles format

Uses: Azure Event Grid to detect new files instantly

Use when:

- Files arrive continuously in ADLS landing zone
- Want only NEW files processed (not all files)
- Need schema evolution (new columns handled automatically)
- Millions of files (regular glob listing too slow)

vs spark.read: Auto Loader tracks processed files automatically
spark.read reads ALL matching files every time

Q10: How do you optimize a slow Spark job?

Step 1: Check Spark UI → find slowest stage

Step 2: Look for:

- Skew (one partition much larger) → salting
- Shuffle (large shuffle size) → reduce data before shuffle
- Small files (thousands of tiny files) → coalesce before write
- Missing broadcast (SortMerge when Broadcast possible)

Common fixes:

- Enable AQE (handles skew, coalesces partitions automatically)
- Broadcast small tables
- Filter/select columns early
- Repartition by join key before multiple joins
- Use Parquet with partition pruning
- Cache reused DataFrames
- Tune shuffle partitions based on data size



Complete Databricks Architecture Summary

AZURE DATABRICKS – COMPLETE PICTURE

FOUNDATION | Hadoop → Spark (100x faster, in-memory)

RDD → DataFrame → SparkSQL

SPARK CORE | Lazy Evaluation, DAG, Stages, Tasks

Catalyst Optimizer, AQE, Tungsten

Narrow vs Wide transforms, Shuffle

PERFORMANCE | Broadcast join, Cache/Persist

Partitioning, Repartition vs Coalesce

Skew handling (salting), Predicate pushdown

DELTA LAKE | ACID, MERGE, UPDATE, DELETE

Time Travel, Schema Evolution

SCD1 + SCD2, Deduplication

Medallion Architecture (Bronze/Silver/Gold)



STREAMING	Structured Streaming, readStream/writeStream
	Triggers, Output Modes, Checkpointing
	Auto Loader (CloudFiles)
CLUSTER MGMT	All-Purpose vs Job Cluster
	Standard vs High Concurrency
	Autoscaling, Runtime versions
GOVERNANCE	Unity Catalog
	(Metastore→Catalog→Schema→Table)
	RBAC (GRANT/REVOKE), Column Masking
	Row-level Security, Audit Logs, Lineage
INTEGRATION	ADLS Gen2, Azure SQL, Key Vault
	ADF (orchestrates Databricks notebooks)
	Azure DevOps Repos + CI/CD

 **Azure Databricks section 100% complete** — every concept from the PDF including BigData foundations, Hadoop, Spark internals, RDD, DataFrame, SQL, Optimizer, Joins, Performance, Streaming, Auto Loader, Delta Lake, Medallion Architecture, Cluster Management, Batch Processing, Unity Catalog, RBAC, Data Protection, DevOps integration, SDLC, and Interview Questions — all covered with real-world RetailMart examples.

19/03/2026, 07:43:03

Thought process: Orchestrated comprehensive Azure Synapse curriculum aligned with Ansh Lamba's pedagogical approach.

The user wants me to continue with Azure Synapse Analytics as per the PDF content, and also follow Ansh Lamba's style of explaining. Let me recall Ansh Lamba's teaching style - he's known for explaining Azure data engineering concepts in a very practical, real-world way with clear examples, analogies, and step-by-step explanations. He uses a lot of diagrams, real scenarios, and explains "why" before "how".

Let me cover all the Azure Synapse topics from the PDF:

1) Introduction & Overview

- Azure Synapse Analytics Overview
- Azure Synapse Analytics Architecture
- Create Azure Free Account for Synapse

2) Now I need to dive into the different pool types available in Synapse - dedicated SQL pools for heavy workloads, serverless SQL pools for flexible querying, and Apache Spark pools for big data processing. Then I'll walk through querying data lakes with serverless SQL, creating views for CSV data, and integrating storage accounts with Synapse for bulk file operations using wildcards and dataset parameters.

3) Moving into triggers and scheduling - I'll cover schedule-based triggers for regular jobs and event-based triggers that respond to data changes. Then I'll explore SQL Database integration, showing how to copy data from SQL databases to data

lakes using different methods like table queries and stored procedures, plus handling append and overwrite modes with foreach loops for batch operations.

4) I'm getting into the deeper copy activity patterns - loading from on-premises SQL servers to Synapse using polybase and bulk insert, plus copying from file systems on-premises to cloud. Then I'll introduce Spark pools, covering the fundamentals of Spark and its distributed architecture.

5) Now diving into Spark notebooks - creating them in different languages, using MSSparkUtils for file operations and parameters, magic commands for notebook orchestration, and configuring KeyVault access. I'm covering multiple connection patterns to ADLS Gen2, Blob storage, Azure SQL, and on-premises SQL servers, then optimizing file I/O for CSV, Parquet, JSON, Avro, ORC, and Excel formats, plus creating RDDs and dataframes in various ways.

6) Moving into performance tuning - understanding when to repartition versus coalesce, handling joins and broadcast joins with Spark configuration, diving into the Catalyst optimizer and data skewness issues, then applying broader optimization techniques across PySpark. I'm also covering slowly changing dimensions with SCD1 and SCD2 implementations, and finally orchestrating these notebooks from Synapse pipelines with parameter passing.



Azure Synapse Analytics — Complete Senior Engineer Guide



Same Company: RetailMart India Pvt. Ltd.



SITUATION SO FAR:

- ✓ ADF ingesting data from 6 source systems → Bronze layer
- ✓ Databricks transforming → Silver and Gold layers
- ✓ Delta Lake managing ACID transactions

NEW REQUIREMENT from business:

- ✗ BI team cannot query 500GB ADLS parquet files directly
- ✗ Data Scientists need SQL access to Gold layer
- ✗ CFO wants to run ad-hoc SQL queries without waiting
- ✗ Need enterprise data warehouse for reporting
- ✗ On-premise SQL Server data needs direct loading to warehouse

SOLUTION: Azure Synapse Analytics

1 Introduction & Overview

◆ What is Azure Synapse Analytics?

Azure Synapse Analytics is Microsoft's **unified analytics platform** that brings together:

- **Data Integration** (like ADF pipelines)
- **Enterprise Data Warehousing** (like SQL Server but massively scalable)
- **Big Data Analytics** (like Databricks Spark)
- **Data Exploration** (query files directly without loading)

All in **one single workspace**.

BEFORE SYNAPSE (Old World):

Tool 1: Azure Data Factory → for pipelines

Tool 2: Azure Databricks → for big data processing
Tool 3: Azure SQL DW → for data warehouse
Tool 4: Azure HDInsight → for Spark/Hadoop

Problems:

- ✗ 4 different tools to learn and manage
- ✗ 4 separate billing and cost management
- ✗ Moving data between tools → complexity + cost
- ✗ Security configured in 4 different places
- ✗ No unified monitoring

AFTER SYNAPSE (New World):

Azure Synapse Analytics = ONE platform

- ✓ Pipelines (like ADF) built-in
- ✓ Serverless SQL (query data lake directly)
- ✓ Dedicated SQL Pool (enterprise data warehouse)
- ✓ Apache Spark Pool (big data processing)
- ✓ Unified monitoring, security, billing
- ✓ Deep integration with ADLS Gen2 and Power BI

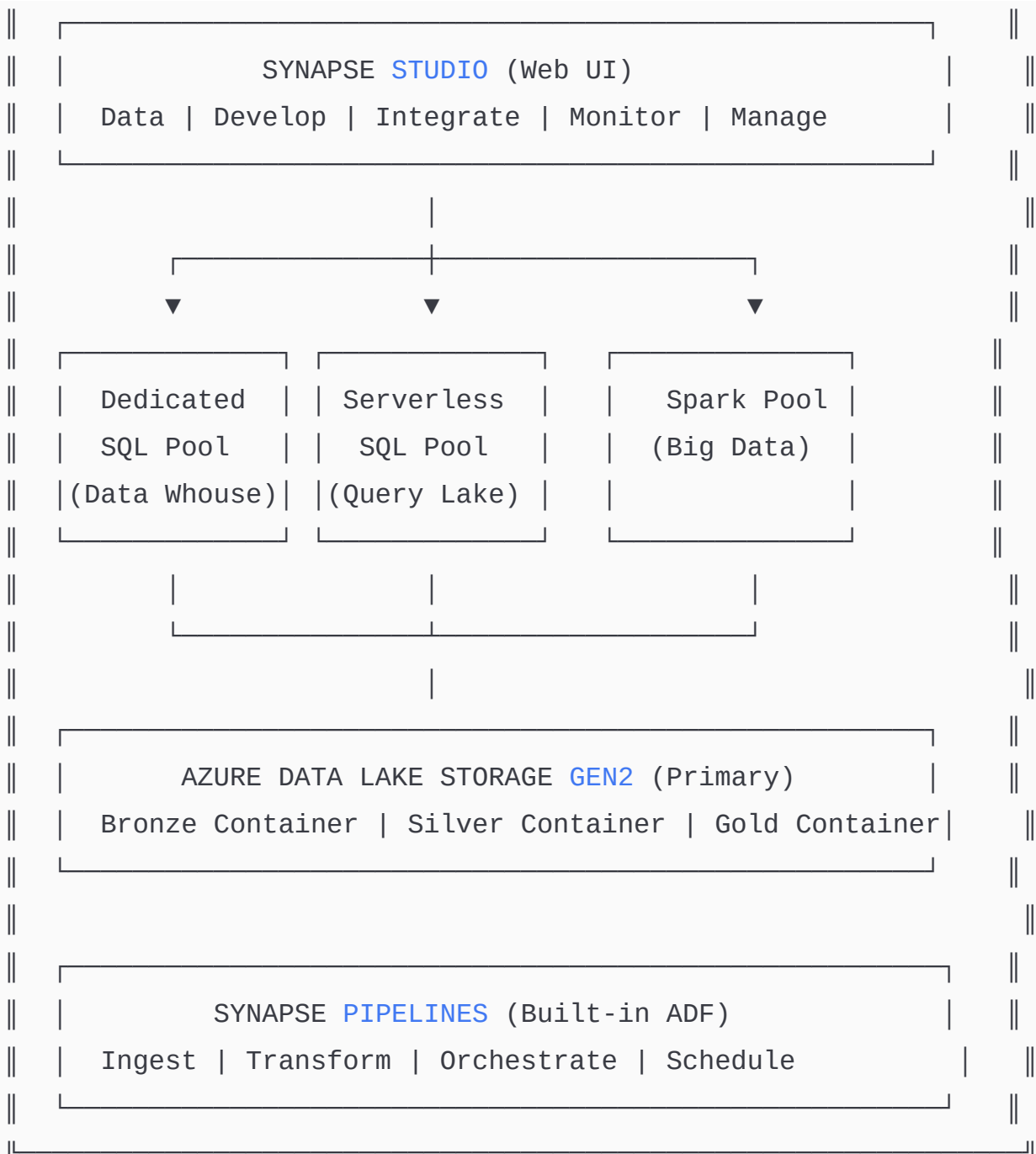
Real World Analogy:

Think of Azure Synapse as a **Swiss Army Knife** for data. Instead of carrying 4 separate tools (knife, scissors, screwdriver, file), you carry ONE tool that does everything. Same concept — one platform, multiple capabilities.

◆ Azure Synapse Analytics Architecture

A diagram consisting of a large rectangle with a double-line border. Inside the rectangle, the text "AZURE SYNAPSE ANALYTICS ARCHITECTURE" is centered. The rectangle is flanked by vertical double lines on both sides, suggesting it is part of a larger architectural context.

AZURE SYNAPSE ANALYTICS ARCHITECTURE



Key Components Explained:

DEDICATED SQL POOL:

- Massively Parallel Processing (MPP) data warehouse
- Pre-allocated compute (you pay whether used or not)
- Best for: regular BI queries, predictable workloads
- Scale: up to petabytes of data
- Like: traditional SQL Server but 1000x more powerful

SERVERLESS SQL POOL:

- NO infrastructure to manage
- Query files DIRECTLY in ADLS (CSV, Parquet, JSON, Delta)
- Pay per query (per TB of data scanned)
- No data loading needed!
- Like: Athena on AWS – SQL on files
- Best for: ad-hoc exploration, data lake queries

APACHE SPARK POOL:

- Same Spark as Databricks but inside Synapse
- PySpark, Scala, R notebooks
- Direct integration with Synapse tables
- Best for: heavy transformations, ML
- Auto-pause when idle (cost saving)

SYNAPSE PIPELINES:

- Exactly same as Azure Data Factory
- 90+ connectors, same activities/triggers
- Built INTO Synapse workspace
- Benefit: pipelines live next to your data/compute

◆ Create Azure Synapse Workspace — Step by Step

SETUP STEPS:

Step 1: Azure Portal → Create Resource → Azure Synapse Analytics

Step 2: Basics Tab

Subscription:	RetailMart-Production
Resource Group:	RG-RetailMart-DataPlatform
Workspace Name:	synapse-retailmart-prod

Region: Central India

Step 3: Storage Account (IMPORTANT!)

Select or create ADLS Gen2 account

ADLS Account: retailmartdatalake

File System: synapseworkspace ← workspace's own container

WHY: Synapse needs its own container for:

- Spark logs
- Synapse metadata
- Pipeline run history
- Staging for PolyBase loads

Step 4: SQL Administrator credentials

SQL Admin Username: sqladmin

Password: (strong password → will also store in Key Vault)

Step 5: Security Tab

Managed Identity: System-assigned (for accessing storage)

Allow connections: Add your IP for development

Step 6: Review + Create → Deploy (takes 5-7 minutes)

Step 7: After deployment → Open Synapse Studio

URL: <https://web.azuresynapse.net>

Workspace: synapse-retailmart-prod

2 Overview of Pools in Synapse Analytics

◆ Pool Comparison — When to Use What

SCENARIO → BEST POOL:

"CFO wants to explore some CSV files in data lake"

→ SERVERLESS SQL POOL (no setup, immediate, pay per query)

"BI team runs Power BI dashboards 8 hours/day"

→ DEDICATED SQL POOL (consistent performance, pre-allocated)

"Data scientist needs to train ML model on 500GB data"

→ SPARK POOL (distributed processing, Python/ML support)

"Need to transform JSON files to Parquet daily"

→ SPARK POOL or SERVERLESS SQL (depending on complexity)

"Need to join data lake files with SQL tables"

→ SERVERLESS SQL POOL (can join files + tables in one query!)

Cost Comparison for RetailMart:

Serverless SQL: \$5 per TB scanned (exploration queries)

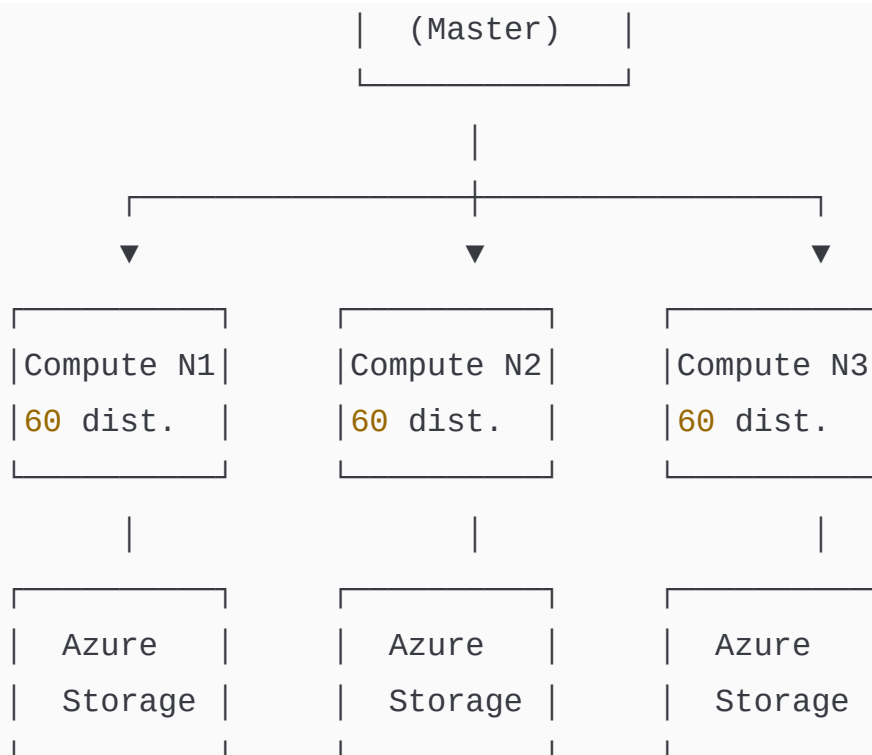
Dedicated Pool: ~\$744/month for DW100c (always on)

Spark Pool: Pay per hour used (auto-pause when idle)

◆ Dedicated SQL Pool — Deep Dive

HOW DEDICATED SQL POOL WORKS (MPP Architecture):





HOW IT WORKS:

1. Your query hits Control Node
2. Control Node generates parallel execution plan
3. Plan distributed to all Compute Nodes simultaneously
4. Each node processes its portion of data
5. Results aggregated back at Control Node
6. Single result returned to you

KEY CONCEPT - DISTRIBUTIONS:

Dedicated SQL Pool has 60 FIXED distributions

Data stored across these 60 distributions

Parallel queries run against all 60 simultaneously

DWUs (Data Warehouse Units) = your scale setting

DW100c: 1 compute node → 60 distributions / 1 node

DW1000c: 10 compute nodes → 60 distributions / 10 nodes

DW6000c: 60 compute nodes → 1 distribution / 1 node (max parallelism)

DISTRIBUTION TYPES:

HASH: Best for large fact tables (distribute by join key)
Same join key → same distribution → join without shuffle!

ROUND ROBIN: Default, data spread evenly across all 60
Best for staging tables, smaller tables

REPLICATED: Full copy on EVERY compute node
Best for small dimension tables (< 2GB)
Eliminates broadcast operations!

```
-- CREATE TABLE WITH OPTIMAL DISTRIBUTION
-- Large fact table → Hash distributed by most common join key
CREATE TABLE gold.fact_sales (
    sale_id          INT          NOT NULL,
    customer_id      INT          NOT NULL,
    product_key      INT          NOT NULL,
    store_key        INT          NOT NULL,
    date_key         INT          NOT NULL,
    quantity         INT,
    unit_price       DECIMAL(18,2),
    total_amount     DECIMAL(18,2),
    net_amount       DECIMAL(18,2),
    gst_amount       DECIMAL(18,2),
    payment_mode     VARCHAR(50)
)
WITH (
    DISTRIBUTION = HASH(customer_id), -- distribute by most
    joined column
    CLUSTERED COLUMNSTORE INDEX      -- best for analytics!
);

-- Small dimension table → Replicated (full copy on each node)
CREATE TABLE gold.dim_stores (
    store_key        INT          NOT NULL,
```

```

    store_id      VARCHAR(20)    NOT NULL,
    store_name    VARCHAR(200),
    city          VARCHAR(100),
    state         VARCHAR(100),
    region_code   VARCHAR(10),
    manager_name  VARCHAR(200)
)
WITH (
    DISTRIBUTION = REPLICATE,          -- copy to all nodes
    CLUSTERED COLUMNSTORE INDEX
);

-- Staging table → Round Robin (fast parallel insert)
CREATE TABLE staging.stg_sales_load (
    sale_id       INT,
    customer_id   INT,
    total_amount  DECIMAL(18,2),
    sale_date     DATE
)
WITH (
    DISTRIBUTION = ROUND_ROBIN,      -- fast parallel load
    HEAP                                -- no index for staging (faster
insert)
);

```

◆ Serverless SQL Pool — Deep Dive

SERVERLESS SQL = Query ADLS files directly using SQL

NO cluster to create

NO data to load

NO indexing needed

Just write SQL → point at file → get results!

MAGIC: OPENROWSET function

```
-- — QUERY CSV FILE DIRECTLY —————  
SELECT TOP 10 *  
FROM OPENROWSET(  
    BULK  
    'https://retailmartdatalake.dfs.core.windows.net/bronze/sales/2024/01/sales_2024_01.csv',  
    FORMAT = 'CSV',  
    PARSER_VERSION = '2.0',  
    HEADER_ROW = TRUE,  
    FIELDTERMINATOR = ',',  
    ROWTERMINATOR = '\n'  
) AS sales_data;  
  
-- — QUERY PARQUET FILES (much faster!) —————  
SELECT  
    region_code,  
    COUNT(*) AS transaction_count,  
    SUM(total_amount) AS gross_revenue,  
    AVG(total_amount) AS avg_transaction  
FROM OPENROWSET(  
    BULK  
    'https://retailmartdatalake.dfs.core.windows.net/silver/sales/**/*.parquet',  
    FORMAT = 'PARQUET'  
) AS sales_data  
WHERE sale_date >= '2024-01-01'  
GROUP BY region_code  
ORDER BY gross_revenue DESC;  
  
-- — QUERY WITH EXPLICIT SCHEMA (recommended for production) —  
SELECT
```

```

    sale_id,
    customer_id,
    total_amount,
    sale_date,
    region_code
FROM OPENROWSET(
    BULK
    'https://retailmartdatalake.dfs.core.windows.net/silver/sales/year=2
    FORMAT = 'PARQUET'
)
WITH (
    sale_id      INT,
    customer_id  VARCHAR(20),
    total_amount FLOAT,
    sale_date    DATE,
    region_code  VARCHAR(10)
) AS sales_data
WHERE sale_date BETWEEN '2024-06-01' AND '2024-06-30';

-- — QUERY JSON FILES —————
SELECT
    JSON_VALUE(doc, '$.payment_id') AS payment_id,
    JSON_VALUE(doc, '$.amount')     AS amount,
    JSON_VALUE(doc, '$.status')     AS status,
    JSON_VALUE(doc, '$.customer_id') AS customer_id
FROM OPENROWSET(
    BULK
    'https://retailmartdatalake.dfs.core.windows.net/bronze/payments/*.j
    FORMAT = 'CSV',
    FIELDQUOTE = '0x0b',
    FIELDTERMINATOR = '0x0b',
    ROWTERMINATOR = '0x0a'
)
WITH (doc NVARCHAR(MAX)) AS payment_docs;

```

```
-- — QUERY MULTIPLE FOLDERS (partition pruning!) —————
-- Only reads year=2024/month=6 folder, skips all others!
SELECT *
FROM OPENROWSET(
    BULK
    'https://retailmartdatalake.dfs.core.windows.net/silver/sales/year=2
    FORMAT = 'PARQUET'
) AS [result];
```

3 Using Synapse to Query Data Lake

◆ Creating Synapse Workspace and Exploring Studio

SYNAPSE STUDIO NAVIGATION:

Left Sidebar (5 main sections):

	HOME	→ Quick links, recent resources
	DATA	→ Browse databases, tables, files
	DEVELOP	→ Notebooks, SQL scripts, Dataflows
	INTEGRATE	→ Pipelines (same as ADF!)
	MONITOR	→ Pipeline runs, Spark jobs
	MANAGE	→ Pools, linked services, IR

DATA Tab:

Workspace section:

- SQL databases (Dedicated + Serverless)
- Spark databases (registered in catalog)

Linked section:

- Your ADLS Gen2 storage
- Browse folders and files directly
- Right-click any file → "New SQL Script" or "New Notebook"
- Amazing for exploration!

DEVELOP Tab:

- SQL Scripts (run against Serverless or Dedicated pool)
- Notebooks (PySpark, Scala, .NET Spark, SparkSQL)
- Power BI datasets (embedded)
- Data Flows (visual transformations)

INTEGRATE Tab:

- Exactly same as ADF Studio
- Create pipelines, datasets, linked services
- All triggers, copy activity, ForEach, etc.

MONITOR Tab:

- Pipeline runs
- Trigger runs
- Apache Spark applications
- SQL requests
- Integration runtimes

◆ Creating Views on Data Lake Files

```
-- — WHY VIEWS OVER OPENROWSET? —————  
-- Instead of sharing long OPENROWSET queries with BI team:  
-- Create VIEWS that hide complexity behind simple table names!  
  
-- Step 1: Create database for Serverless SQL views  
CREATE DATABASE RetailMart_Reporting;
```

```

-- Step 2: Create master key (for security)
CREATE MASTER KEY ENCRYPTION BY PASSWORD = 'str0ng_P@ssw0rd!';

-- Step 3: Create credential to access ADLS Gen2
-- Option A: Using Managed Identity (recommended)
CREATE DATABASE SCOPED CREDENTIAL synapse_managed_identity
WITH IDENTITY = 'Managed Identity';

-- Option B: Using SAS Token
CREATE DATABASE SCOPED CREDENTIAL synapse_sas_token
WITH IDENTITY = 'SHARED ACCESS SIGNATURE',
SECRET = 'sv=2020-08...'; -- SAS token from ADLS

-- Step 4: Create External Data Source (reusable connection)
CREATE EXTERNAL DATA SOURCE adls_silver
WITH (
    LOCATION =
'https://retailmartdatalake.dfs.core.windows.net/silver',
    CREDENTIAL = synapse_managed_identity
);

CREATE EXTERNAL DATA SOURCE adls_bronze
WITH (
    LOCATION =
'https://retailmartdatalake.dfs.core.windows.net/bronze',
    CREDENTIAL = synapse_managed_identity
);

-- Step 5: Create External File Format
CREATE EXTERNAL FILE FORMAT parquet_format
WITH (
    FORMAT_TYPE = PARQUET,
    DATA_COMPRESSION =
'org.apache.hadoop.io.compress.SnappyCodec'
);

```

```
CREATE EXTERNAL FILE FORMAT csv_format
WITH (
    FORMAT_TYPE = DELIMITEDTEXT,
    FORMAT_OPTIONS (
        FIELD_TERMINATOR = ',',
        STRING_DELIMITER = '"',
        FIRST_ROW = 2, -- skip header
        ENCODING = 'UTF8'
    )
);
```

-- Step 6: Create VIEWS for BI team (simple, clean)

```
CREATE OR ALTER VIEW RetailMart_Reporting.dbo.vw_Sales_Silver AS
SELECT
    sale_id,
    customer_id,
    store_id,
    product_code,
    total_amount,
    net_amount,
    gst_amount,
    sale_date,
    year,
    month,
    region_code,
    payment_mode
FROM OPENROWSET(
    BULK 'sales/**/*.*parquet',
    DATA_SOURCE = 'adls_silver',
    FORMAT = 'PARQUET'
)
WITH (
    sale_id INT,
    customer_id VARCHAR(20),
```

```

    store_id      INT,
    product_code  VARCHAR(50),
    total_amount  FLOAT,
    net_amount    FLOAT,
    gst_amount    FLOAT,
    sale_date     DATE,
    year          INT,
    month         INT,
    region_code   VARCHAR(10),
    payment_mode  VARCHAR(50)
) AS [result];

-- Step 7: Create View for CSV data (store targets file from
business)
CREATE OR ALTER VIEW RetailMart_Reporting.dbo.vw_StoreTargets AS
SELECT
    store_id,
    month,
    year,
    CAST(revenue_target AS FLOAT) AS revenue_target,
    CAST(transaction_target AS INT) AS transaction_target
FROM OPENROWSET(
    BULK 'config/store_targets_*.csv',
    DATA_SOURCE = 'adls_bronze',
    FORMAT = 'CSV',
    PARSER_VERSION = '2.0',
    HEADER_ROW = TRUE
) AS [targets];

-- Now BI team just queries the VIEW:
-- SELECT * FROM RetailMart_Reporting.dbo.vw_Sales_Silver
-- WHERE sale_date >= '2024-06-01'
-- Much simpler! They don't need to know ADLS paths.

-- Step 8: Join VIEW with real SQL table (powerful!)

```

```
SELECT
    s.region_code,
    s.sale_date,
    SUM(s.total_amount) AS actual_revenue,
    t.revenue_target,
    ROUND(SUM(s.total_amount) / t.revenue_target * 100, 1) AS
achievement_pct
FROM RetailMart_Reporting.dbo.vw_Sales_Silver s
JOIN RetailMart_Reporting.dbo.vw_StoreTargets t
    ON s.store_id = t.store_id
    AND MONTH(s.sale_date) = t.month
    AND YEAR(s.sale_date) = t.year
GROUP BY s.region_code, s.sale_date, t.revenue_target
ORDER BY s.sale_date, achievement_pct DESC;
```

4 Azure Storage Integration with Synapse Pipelines

◆ Copy Multiple Files Using Wildcard Options

SCENARIO:

ADLS has thousands of store-wise CSV files:

/bronze/store-reports/store_001_2024.csv

/bronze/store-reports/store_002_2024.csv

...

/bronze/store-reports/store_500_2024.csv

Copy ALL at once to processed container using wildcard

PIPELINE: PL_Synapse_CopyStoreFiles

STEP 1: Source Dataset

Name: DS_StoreCSV_Wildcard

Linked Service: LS_ADLS_Gen2_Bronze

Container: bronze

→ Leave file path EMPTY in dataset

STEP 2: Copy Activity → Source Tab

Dataset: DS_StoreCSV_Wildcard

File Path Type: Wildcard file path

Wildcard Folder Path: store-reports/

Wildcard File Name: store_*_2024.csv

Recursive: FALSE

DIFFERENT PATTERNS:

store_*_2024.csv → all stores for 2024

store_00?_*.csv → stores 001-009 all years

*_2024_June.csv → all stores for June 2024

*.csv → ALL CSV files in folder

STEP 3: Sink Dataset

Name: DS_ProcessedStore

Container: silver

Folder: store-reports/processed/

STEP 4: Copy Behavior

PreserveHierarchy → maintain structure

FlattenHierarchy → all files in one folder

MergeFiles → combine all into ONE file

REAL WORLD TIP:

Use MergeFiles when:

→ 500 small store files → merge into one big file

→ Single file = faster downstream processing

→ Power BI loads one file faster than 500 files

◆ Copy Multiple Folders Using Dataset Parameters

SCENARIO:

ADLS has monthly folders:

/bronze/sales/2024/01/

/bronze/sales/2024/02/

...

/bronze/sales/2024/12/

Need to copy any month dynamically using parameters

STEP 1: Create Parameterized Dataset

Name: DS_Monthly_Sales_Param

Container: bronze

Parameters:

p_year: String (e.g., "2024")

p_month: String (e.g., "06")

Folder Path: sales/@{dataset().p_year}/@{dataset().p_month}/

File: *.parquet (all parquet files in that month folder)

STEP 2: Pipeline PL_CopyMonthlyFolders

Parameters:

p_year: String → "2024"

p_month_list: Array → ["01", "02", "03", "04", "05", "06"]

STEP 3: ForEach over months

Items: @{pipeline().parameters.p_month_list}

Batch Count: 3 (copy 3 months in parallel)

Inside ForEach:

Copy Activity:

Source Dataset: DS_Monthly_Sales_Param

p_year: @{pipeline().parameters.p_year}

```
p_month: @{{item()}} ← current month in loop
```

```
Sink Dataset: DS_Silver_Sales_Param
```

```
p_year: @{{pipeline().parameters.p_year}}
```

```
p_month: @{{item()}}
```

RESULT:

3 months copy simultaneously → much faster!

Pass different months = different data without changing pipeline

◆ Get File Names Dynamically — Copy Latest File

REAL WORLD SCENARIO:

Payment gateway drops a file daily:

```
/bronze/payments/payments_20240613.csv
```

```
/bronze/payments/payments_20240614.csv
```

```
/bronze/payments/payments_20240615.csv ← latest
```

Pipeline should always pick the LATEST file automatically
Without hardcoding the date!

```
PIPELINE: PL_CopyLatestPaymentFile
```

STEP 1: GetMetadata – List all files

```
Dataset: DS_PaymentFolder
```

```
Container: bronze
```

```
Folder: payments/
```

```
Field List: childItems, lastModified
```

STEP 2: ForEach – Find latest file

```
Items: @{{activity('GetMeta_ListFiles').output.childItems}}
```


Sequential: YES

Inside ForEach:

If Condition:

Condition: @greater(

```
activity('GetMeta_EachFile').output.lastModified,  
        variables('v_latestModified')  
    )}
```

TRUE:

```
    Set v_latestModified =  
    @{activity('GetMeta').output.lastModified}  
    Set v_latestFileName = @{item().name}
```

STEP 3: Copy Activity – Copy latest file

Source: /bronze/payments/@{variables('v_latestFileName')}

Sink: /silver/payments/latest/payment_latest.parquet

ALSO: Set Variable

v_processedFile = @{variables('v_latestFileName')}

STEP 4: Stored Procedure – Log which file was processed

@run_date: @utcnow()

@file_processed: @{variables('v_latestFileName')}

@pipeline_run: @{pipeline().RunId}

SMARTER APPROACH (if files follow date naming):

Use expression directly:

File name: payments_@{formatDateTime(utcnow(), 'yyyyMMdd')}.csv

→ Today's date automatically → pick today's file!

If files land at night for next day:

File name:

payments_@{formatDateTime(addDays(utcnow(), -1), 'yyyyMMdd')}.csv

→ Yesterday's date

5 Azure Synapse Triggers

◆ Schedule Trigger in Azure Synapse

SAME AS ADF BUT INSIDE SYNAPSE WORKSPACE:

Integrate Tab → Trigger → + New/Edit → Schedule

Configuration:

Name: TRG_Synapse_Daily_Load

Type: Schedule

Start Date: 2024-01-01 00:30:00

Time Zone: India Standard Time

Recurrence: Every 1 Day

Hours: 0 (midnight)

Minutes: 30 (half past midnight)

→ Runs at 12:30 AM IST every night

KEY DIFFERENCE from ADF triggers:

Synapse triggers live inside Synapse workspace

ADF triggers live in ADF service

Use Synapse triggers: when pipeline is inside Synapse

Use ADF triggers: when pipeline is inside ADF

RetailMart setup:

ADF: handles complex cross-service orchestration

Synapse: handles Synapse-specific data warehouse loads

PASS TRIGGER TIME TO PIPELINE:

Pipeline parameter: p_loadDate

```
Value: @{formatDateTime(trigger().scheduledTime, 'yyyy-MM-dd')}
```

In pipeline, use `p_loadDate` in source queries:

```
WHERE sale_date = '@{pipeline().parameters.p_loadDate}'
```

◆ Event-Based Trigger in Azure Synapse

SCENARIO:

When a new file arrives in ADLS → trigger Synapse pipeline
Used for near-real-time data loading

Setup:

Integrate → Trigger → New → Storage Event

Storage Account: `retailmartdatalake`

Container: `bronze`

Blob Path Begins With: `/payments/daily/`

Blob Path Ends With: `.parquet`

Event: `BlobCreated`

Pipeline Parameters:

`p_file_path`: `@{triggerBody().folderPath}`

`p_file_name`: `@{triggerBody().fileName}`

ADVANTAGE over Schedule:

Schedule: pipeline runs at midnight EVEN IF **no** file arrived

Event: pipeline ONLY runs when file actually arrives

→ Less wasted compute

→ Faster processing (no waiting for schedule)

→ Perfect for event-driven architecture

REAL WORLD TIMELINE:

2:47 AM: Payment gateway uploads payments_20240615.parquet to /bronze/payments/daily/
2:47 AM: Azure Event Grid detects new blob (within seconds)
2:47 AM: Synapse event trigger fires
2:48 AM: Pipeline starts processing payment file
2:52 AM: Data available in Gold layer

vs Schedule at 3:00 AM: same data available 8 minutes LATER!

6 Azure SQL Database Integration

◆ Azure SQL Databases — Relational Databases in Azure

SQL DATABASE TYPES IN AZURE:

Azure SQL Database:

- Fully managed PaaS SQL Server
- Microsoft handles: backups, patches, HA, DR
- Single database model
- Best for: OLTP applications, small-medium DW
- RetailMart uses: transaction database, audit logs

Azure SQL Managed Instance:

- 100% SQL Server compatibility
- Best for: migrating on-prem SQL Server to cloud
- Supports: SQL Agent, CLR, linked servers
- More expensive than SQL Database

Azure Synapse Dedicated SQL Pool:

- MPP data warehouse (not regular SQL Server)
- Best for: analytical queries, reporting, BI

- Cannot do OLTP (row-by-row inserts are slow)
- This is what we configure in Synapse

RETAILMART DATABASE TOPOLOGY:

Azure SQL Database (OLTP):

- stores retail transaction data temporarily
- Customer service team queries here

Azure Synapse Dedicated Pool (DW):

- Historical analytics
- Power BI dashboards connect here
- Optimized for SELECT, GROUP BY, aggregations

◆ Copy Data from SQL Database to ADLS Gen2

PIPELINE: PL_SQLToADLS_SalesExtract

Three source modes for Copy Activity from SQL:

MODE 1: Table (entire table)

Source Dataset:

Type: Azure SQL Database

Table: [dbo].[Sales_Staging] ← reads ENTIRE table

Use when: Small tables, full load needed

MODE 2: Query (custom SQL)

Source Dataset:

Type: Azure SQL Database

Use Query: SQL Query

Query:

```
SELECT
    sale_id,
    customer_id,
    total_amount,
    sale_date,
    region_code
FROM [dbo].[Sales]
WHERE sale_date = '@{pipeline().parameters.p_date}'
    AND status = 'COMPLETED'
ORDER BY sale_id
```

Use when: Need filtering, specific columns, joins

MODE 3: Stored Procedure

Create procedure in SQL DB first:

```
CREATE PROCEDURE [dbo].[usp_GetSalesForLoad]
    @LoadDate DATE,
    @RegionCode VARCHAR(10) = NULL
AS
BEGIN
    SELECT
        s.*,
        c.customer_name,
        c.customer_tier
    FROM [dbo].[Sales] s
    JOIN [dbo].[Customers] c ON s.customer_id = c.customer_id
    WHERE s.sale_date = @LoadDate
        AND (@RegionCode IS NULL OR s.region_code = @RegionCode)
        AND s.status = 'COMPLETED';
END
```

In Copy Activity Source:

Use Stored Procedure: [dbo].[usp_GetSalesForLoad]

Parameters:

@LoadDate: @{{pipeline().parameters.p_date}}

@RegionCode: @{{pipeline().parameters.p_region}}

Use when: Complex logic, business rules in DB already

SINK: ADLS Gen2 Parquet

Container: silver

Path: sql-extract/@{{pipeline().parameters.p_date}}/

Format: Parquet

Compression: Snappy

◆ Overwrite and Append Modes

COPY ACTIVITY WRITE BEHAVIORS:

TO FILE STORE (ADLS/Blob):

MERGE FILES:

- Combines all source files into one output file
- Source: 100 small CSVs → Sink: 1 large CSV
- Use: consolidating small files

FLATTEN HIERARCHY:

- All files from all subfolders → one flat destination folder
- Source: /2024/06/15/file.csv → Sink: /processed/file.csv

PRESERVE HIERARCHY:

- Maintains exact folder structure
- Source: /2024/06/15/file.csv → Sink:

/backup/2024/06/15/file.csv

TO SQL TABLE:

INSERT (append new rows only):

- sink.writeBehavior = "insert"
- New rows added to existing data
- Use for: incremental loads, event logs

UPSERT (insert or update):

- sink.writeBehavior = "upsert"
- If row exists (by key) → UPDATE
- If row doesn't exist → INSERT
- Key Columns: [sale_id] ← unique identifier
- Use for: dimension tables, master data

OVERWRITE (truncate then insert):

- Pre-copy Script: TRUNCATE TABLE [dbo].[Sales_Staging]
- Then insert all data fresh
- Use for: full loads, staging tables

REAL WORLD DECISION:

- Fact table (500M rows, append daily): → INSERT
- Customer master (10M rows, full refresh): → OVERWRITE
- Product catalog (changes daily): → UPSERT
- Audit log: → INSERT (never overwrite!)

◆ ForEach Loop — Copy Multiple Tables

SCENARIO: Copy 20 SQL tables to ADLS Gen2 at once

CONTROL TABLE [APPROACH](#) (Senior Pattern):

```
-- Create control table in Azure SQL DB
CREATE TABLE dbo.SynapseControlTable (
    id          INT IDENTITY PRIMARY KEY,
    source_schema VARCHAR(50),
    source_table VARCHAR(100),
    target_folder VARCHAR(200),
    target_format VARCHAR(20),
    load_type    VARCHAR(20),
    is_active    BIT DEFAULT 1
);

INSERT INTO dbo.SynapseControlTable VALUES
('dbo', 'Sales',          'silver/sales/',      'parquet',
'INCREMENTAL', 1),
('dbo', 'Customers',      'silver/customers/',  'parquet',
'FULL',        1),
('dbo', 'Products',        'silver/products/',   'parquet',
'FULL',        1),
('dbo', 'StoreInventory',  'silver/inventory/',  'parquet',
'INCREMENTAL', 1),
('dbo', 'Promotions',      'silver/promotions/', 'parquet',
'FULL',        0); -- disabled
```

PIPELINE: PL_BulkCopy_SQLToADLS

STEP 1: Lookup Activity

Name: LKP_GetActiveTables

Source: Azure SQL Database

Query:

```
SELECT source_schema, source_table, target_folder,
target_format
FROM dbo.SynapseControlTable
```

```
WHERE is_active = 1
ORDER BY load_type DESC -- FULL loads first, then
INCREMENTAL
```

STEP 2: ForEach Activity

Name: FE_CopyEachTable

Items: @{activity('LKP_GetActiveTables').output.value}

Batch Count: 5 ← 5 tables copy in parallel

Sequential: No

INSIDE ForEach:

Copy Activity: CPY_DynamicTableCopy

Source:

Linked Service: LS_AzureSQLDB

Query:

```
SELECT * FROM
@{item().source_schema}.@{item().source_table}
```

Sink:

Linked Service: LS_ADLS_Gen2

Path: @{item().target_folder}

@{formatDateTime(utcnow(), 'yyyy/MM/dd')}/

File: @{concat(item().source_table, '_',
formatDateTime(utcnow(), 'yyyyMMddHHmmss'),
'parquet')}

Format: Parquet

STEP 3: Success notification

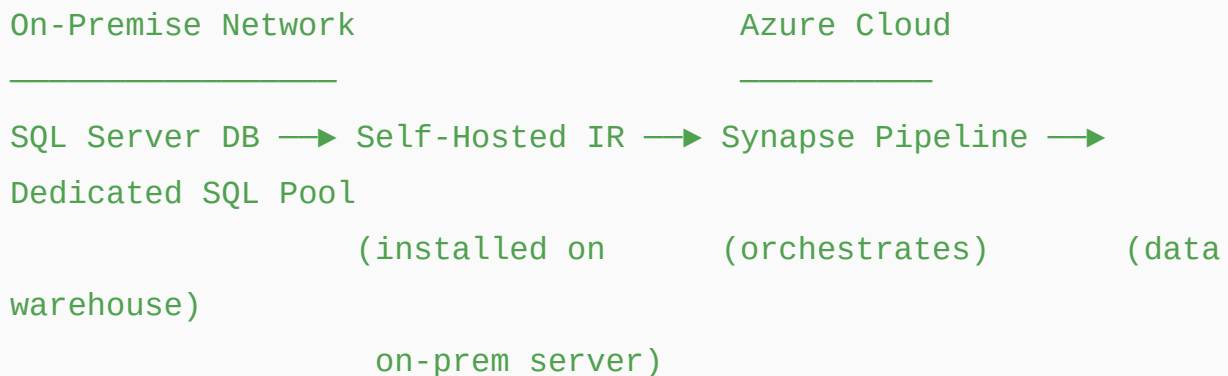
Web Activity → Logic App

"Bulk copy complete: 4 tables loaded successfully"

7 Deep Dive: Copy Activity in Azure Synapse

◆ Load from On-Premise SQL Server to Azure Synapse

ARCHITECTURE:



STEP 1: Install Self-Hosted IR

Same as ADF SHIR setup

Synapse Manage → Integration Runtimes → + New → Self-Hosted

→ Name: SHIR_OnPrem_Synapse

→ Install on on-premise Windows server

→ Register with auth key

STEP 2: Create Linked Services

On-Prem SQL Server:

Type: SQL Server

Server: 10.0.1.51 (internal IP)

Database: LegacySalesDB

Auth: Windows Authentication

IR: SHIR_OnPrem_Synapse

Synapse Dedicated Pool:

Type: Azure Synapse Analytics

Server: synapse-retailmart-prod.sql.azuresynapse.net

Database: RetailMartDW

Auth: Managed Identity (no password!)

STEP 3: Create Datasets

DS_OnPrem_LegacySales:

Type: SQL Server

Table: [dbo].[LegacySales]

DS_Synapse_FactSales:

Type: Azure Synapse Analytics

Table: [gold].[fact_sales]

STEP 4: Copy Activity

Source:

Dataset: DS_OnPrem_LegacySales

Partition Option: Dynamic Range

Column: SaleID

Degree of Parallelism: 4 ← read 4 chunks simultaneously!

Sink:

Dataset: DS_Synapse_FactSales

Copy Method: PolyBase ← FASTEST for Synapse!

Pre-copy Script:

```
TRUNCATE TABLE [gold].[fact_sales]
```

◆ PolyBase and Bulk Insert — Critical for Synapse Performance

THE PROBLEM:

Normal **INSERT into** Synapse Dedicated Pool: slow!

10 million rows via JDBC = 45 minutes

WHY? **Row-by-row insert** defeats MPP architecture

Synapse **is** optimized **for** BULK operations **not row-by-row**

SOLUTION 1: PolyBase (traditional, very fast)

HOW POLYBASE WORKS:

Step 1: ADF writes source data to Azure Storage (staging)

Step 2: Synapse reads DIRECTLY from storage using PolyBase

Step 3: Data distributed across all 60 distributions in parallel

Source → [ADF] → Azure Blob (staging) → [PolyBase] → Synapse DW

Speed: 10 million rows → 3 minutes!

Copy Activity Sink Configuration for PolyBase:

Copy Method: PolyBase

Staging Account: (Azure Blob or ADLS account for temp staging)

Staging Container: synapse-staging

Pre-copy Script:

```
TRUNCATE TABLE [staging].[stg_sales_load]
```

SOLUTION 2: COPY INTO (Modern, simpler, equally fast)

HOW COPY INTO WORKS:

Similar to PolyBase but simpler syntax

Synapse reads directly from ADLS using COPY INTO command

ADF Copy Activity triggers this SQL behind the scenes:

```
COPY INTO [gold].[fact_sales]
```

```
FROM
```

```
'https://retailmartdatalake.dfs.core.windows.net/staging/sales/'
```

```
WITH (
```

```
    FILE_TYPE = 'PARQUET',
```

```
CREDENTIAL = (IDENTITY = 'Managed Identity'),  
COMPRESSION = 'snappy'  
);
```

Copy Activity Sink Configuration for COPY INTO:

Copy Method: Copy Command

Default Values: (optional default values for missing columns)

SPEED COMPARISON (10 million rows):

Row-by-row INSERT: 45 minutes ← never use for bulk!

Bulk Insert: 12 minutes ← moderate

PolyBase: 3 minutes ← very fast

COPY INTO: 2 minutes ← fastest!

◆ Copy from On-Premise File System

SCENARIO:

On-premise file server has daily sales reports (CSV)

Path: \\FILESERVER01\reports\sales\daily\

STEP 1: Linked Service for File System

Type: File System

Host: \\FILESERVER01\reports (UNC path or local path)

User ID: CORP\svc_datafactory

Password: (from Key Vault)

Connect via IR: SHIR_OnPrem_Synapse

STEP 2: Dataset for File System

Name: DS_OnPrem_FileSystem_Sales

Linked Service: LS_FileSystem_Reports

Folder Path: sales/daily/

File: *.csv

Format: CSV with header

STEP 3: Copy Activity

Source: DS_OnPrem_FileSystem_Sales

Recursive: TRUE (include subfolders)

Sink: DS_ADLS_Bronze_FileReports

Container: bronze

Path: file-server-reports/@{formatDateTime(utcnow(), 'yyyy/MM/dd')}/

KEY INSIGHT:

SHIR can access:

- ✓ SQL Databases (SQL Server, Oracle, MySQL)
- ✓ File Systems (local, network shares, NFS)
- ✓ FTP/SFTP servers
- ✓ SAP systems
- ✓ Any resource inside corporate network

SHIR = secure bridge between on-prem and cloud

All data encrypted in transit (HTTPS)

8 Spark Pool in Azure Synapse — Complete Guide

◆ Synapse Spark vs Databricks Spark

COMPARISON:

Feature	Synapse Spark	Databricks
Spark Version	3.x	3.x (same)
PySpark	✓	✓

Scala	✓	✓
R	✓	✓
.NET Spark	✓ (unique!)	✗
Delta Lake	✓	✓ (better)
Unity Catalog	✗	✓
Auto-pause	✓	✓
Integration w/ Synapse	Native	Via Linked Service
Collaboration	Basic	Advanced
ML Capabilities	Basic MLflow	Full MLflow + AutoML
SQL integration	Via Spark SQL	Via Unity Catalog
Cost	Included in Synapse	Separate billing

WHEN TO USE SYNAPSE SPARK:

- ✓ Already using Synapse for pipelines + SQL
- ✓ Want everything in one platform
- ✓ Light-medium Spark workloads
- ✓ Direct SQL Pool integration needed
- ✓ .NET developers (C#/F# on Spark!)

WHEN TO USE DATABRICKS:

- ✓ Heavy ML/AI workloads
- ✓ Complex streaming pipelines
- ✓ Unity Catalog governance needed
- ✓ Large team of data scientists
- ✓ Maximum Spark performance needed

RetailMart Decision:

Data Engineering pipelines → Synapse Spark (simple transformations)

ML model training → Databricks (complex ML)

Ad-hoc SQL exploration → Synapse Serverless SQL

◆ Create Spark Pool in Synapse

Synapse Studio → Manage → Apache Spark Pools → + New

Configuration:

Name: RetailMart_SparkPool

Node Size Family: Memory Optimized

Node Size: Medium (8 vCores, 64GB RAM per node)

Autoscale: Enabled

Min nodes: 3

Max nodes: 20

Auto-pause: Enabled

After 15 minutes of inactivity

→ Cluster shuts down → ZERO cost when idle!

Apache Spark Version: 3.4

Packages:

→ Upload requirements.txt or add PyPI packages

great-expectations==0.18.0

pyarrow==12.0.0

COST SAVING WITH AUTO-PAUSE:

Without auto-pause: 3 nodes × 24 hours = 72 node-hours/day

With auto-pause (used 4 hours/day): 3 nodes × 4 hours = 12 node-hours/day

→ 83% cost saving!

◆ Synapse Notebooks — Complete Guide

```
# SYNAPSE NOTEBOOK LANGUAGES:
```

```
#
```

```
# Default language set when creating notebook
```

```
# Can override per cell with magic commands:
```

```
# %%pyspark → Python/PySpark
```

```
# %%spark → Scala Spark
```

```
# %%sql → Spark SQL
```

```
# %%csharp → .NET Spark (C#) ← unique to Synapse!
```

```
# %%markdown → Documentation
```

```
# — CELL 1: Set notebook language to PySpark —————
```

```
# (Selected at notebook creation)
```

```
# — CELL 2: Read data (PySpark) —————
```

```
df_sales = spark.read \
```

```
    .format("parquet") \
```

```
    .load("abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/
```

```
df_sales.printSchema()
```

```
df_sales.show(5)
```

```
# — CELL 3: Switch to SQL in same notebook —————
```

```
%%sql
```

```
-- Register DataFrame as temp view to query with SQL
```

```
-- (must register in PySpark cell first)
```

```
SELECT
```

```
    region_code,
```

```
    COUNT(*) as transactions,
```

```
    SUM(total_amount) as revenue
```

```
FROM sales_view -- registered from PySpark cell
```

```
GROUP BY region_code
```

```
ORDER BY revenue DESC
```

```

# — CELL 4: Switch to Scala —————
%%spark
val df =
spark.read.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/silver.parquet")
println(s"Total records: ${df.count()}")

# — CELL 5: .NET Spark (C#) - unique to Synapse! —————
%%csharp
DataFrame df =
spark.Read().Parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/silver.parquet")
df.Show();
Console.WriteLine($"Total: {df.Count()}");

```

◆ MSSparkUtils — Synapse's Version of dbutils

```

# MSSparkUtils = Synapse equivalent of Databricks dbutils
# Same concept, slightly different syntax

# — FILE SYSTEM OPERATIONS —————
from notebookutils import mssparkutils

# List files
files =
mssparkutils.fs.ls("abfss://bronze@retailmartdatalake.dfs.core.windows.net/bronze.parquet")
for f in files:
    print(f"Name: {f.name}, Size: {f.size}, IsDir: {f.isDir}")

# Create directory
mssparkutils.fs.mkdirs("abfss://silver@retailmartdatalake.dfs.core.windows.net/silver")

# Copy file

```

```

mssparkutils.fs.cp(

"abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/file.p

"abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/file.p
    recurse=False
)

# Move file
mssparkutils.fs.mv(

"abfss://landing@retailmartdatalake.dfs.core.windows.net/sales_20240

"abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/sales_
)

# Delete file/folder
mssparkutils.fs.rm("abfss://temp@retailmartdatalake.dfs.core.windows
recurse=True)

# Read file content (preview)
content = mssparkutils.fs.head(

"abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/sales_
    1024 # first 1024 bytes
)
print(content)

# Check if path exists
mssparkutils.fs.ls("abfss://bronze@retailmartdatalake.dfs.core.windo
# Returns list of FileInfo objects (empty = folder exists,
exception = doesn't exist)

# — NOTEBOOK UTILITY —
# Run another notebook

```

```

result = mssparkutils.notebook.run(
    path="../silver/transform_sales",          # relative path to
notebook
    timeout_seconds=1800,                      # 30 min timeout
    arguments={
        "input_path":
"abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/",
        "process_date": "2024-06-15",
        "env": "production"
    }
)
print(f"Child returned: {result}")

# Exit notebook with return value (for parent
notebooks/pipelines)
mssparkutils.notebook.exit("SUCCESS: 142850 rows processed")

# — CREDENTIALS UTILITY —————
# Access Key Vault secrets
storage_key = mssparkutils.credentials.getSecret(
    "https://kv-retailmart-prod.vault.azure.net/", # Key Vault
URL
    "adls-storage-key"                          # secret
name
)

# Get token for a service
token = mssparkutils.credentials.getToken("Storage")

```

◆ Notebook Parameters in Synapse

```

# — WIDGET PARAMETERS (interactive in Studio) —————
# Right-click cell → Toggle Parameter Cell
# This makes cell a "parameter cell" – first cell in notebook

# Parameters cell (marked with special icon in Synapse):
input_path    =
"abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/"
output_path   =
"abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/"
process_date  = "2024-06-15"
environment   = "dev"

# When called from Pipeline, these defaults get OVERRIDDEN
# by values passed from Synapse Pipeline notebook activity

# — CALLING FROM SYNAPSE PIPELINE —————
# In Synapse Pipeline → Notebook Activity:
# Settings Tab:
#   Notebook: /Production/silver/transform_sales
#   Base Parameters:
#     input_path:
@{concat('abfss://bronze@retailmartdatalake.dfs.core.windows.net/sal
pipeline().parameters.p_date, '/')}}
#     output_path:
@{concat('abfss://silver@retailmartdatalake.dfs.core.windows.net/sal
pipeline().parameters.p_date, '/')}}
#     process_date: @{pipeline().parameters.p_date}
#     environment:  @{pipeline().globalParameters.gp_environment}

# — READING PARAMETER VALUES IN NOTEBOOK —————
# Parameters passed from pipeline override the defaults above
# Just use the variable names normally:

print(f"Input:    {input_path}")
print(f"Output:   {output_path}")

```

```
print(f>Date:      {process_date}")
print(f>Env:       {environment}")

df = spark.read.parquet(input_path)
df.write.mode("overwrite").parquet(output_path)

mssparkutils.notebook.exit(f">SUCCESS: {df.count()} rows for
{process_date}")
```

◆ Magic Commands Deep Dive

```
# — MAGIC COMMANDS REFERENCE —————

# %% magic → changes language for that cell
%%sql
SELECT current_timestamp() as server_time

%%pyspark
print("Python cell")

%%spark
println("Scala cell")

%%csharp
Console.WriteLine("C# cell")

%%markdown
## This is a documentation cell
**Bold text**, *italic*, `code`
- Bullet point 1
- Bullet point 2
```

```

# — %run → Run another notebook inline —————
%run ../Utils/common_functions
# Loads all functions from common_functions notebook
# Now you can call those functions directly!

# — %pip → Install packages in session —————
%pip install great-expectations==0.18.0
%pip install openpyxl

# After install, import normally:
import great_expectations as ge

# — %env → Set environment variables —————
%env MY_VAR=hello_world

import os
print(os.environ.get('MY_VAR')) # prints: hello_world

# — CALLING ONE SYNAPSE NOTEBOOK FROM ANOTHER —————

# Parent Notebook: orchestrator.ipynb
# —————

# Run child notebooks in sequence
result1 = mssparkutils.notebook.run(
    "../bronze/load_sales",
    arguments={"process_date": process_date}
)
print(f"Bronze Sales: {result1}")

result2 = mssparkutils.notebook.run(
    "../bronze/load_customers",
    arguments={"process_date": process_date}
)
print(f"Bronze Customers: {result2}")

```



```

# Run in parallel (using Python threading)
from concurrent.futures import ThreadPoolExecutor, as_completed

def run_notebook(path, args):
    return mssparkutils.notebook.run(path, arguments=args)

notebooks = {
    "../bronze/load_sales": {"process_date": process_date},
    "../bronze/load_customers": {"process_date": process_date},
    "../bronze/load_inventory": {"process_date": process_date}
}

with ThreadPoolExecutor(max_workers=3) as executor:
    futures = {
        executor.submit(run_notebook, path, args): path
        for path, args in notebooks.items()
    }
    for future in as_completed(futures):
        notebook = futures[future]
        result = future.result()
        print(f"✅ {notebook}: {result}")

# Return combined result to pipeline
mssparkutils.notebook.exit(f"All bronze notebooks completed for {process_date}")

```

◆ Connecting to Azure Key Vault in Synapse Notebook

```

# — METHOD 1: Using MSSparkUtils (recommended) —————
from notebookutils import mssparkutils

```

```

# Synapse workspace must have Key Vault linked service
configured!
# Manage → Linked Services → + New → Azure Key Vault

# Get secret
sql_password = mssparkutils.credentials.getSecret(
    "https://kv-retailmart-prod.vault.azure.net/",
    "sqlldb-password"
)

oracle_password = mssparkutils.credentials.getSecret(
    "https://kv-retailmart-prod.vault.azure.net/",
    "oracle-db-password"
)

# Never print secrets!
print("Secrets loaded successfully")
# Output: Secrets loaded successfully (not the actual values)

# — METHOD 2: Using Azure Key Vault Linked Service —————
# Synapse Manage → Linked Services → Add Azure Key Vault LS
# Then reference in notebook via TokenLibrary

from pyspark.sql import SparkSession

# Access via TokenLibrary (Synapse specific)
sql_password = TokenLibrary.getSecret(
    "kv-retailmart-prod",    # Key Vault name
    "sqlldb-password",      # Secret name
    "LS_AzureKeyVault"      # Linked Service name
)

# — METHOD 3: Linked Service connection string —————
# Most secure: Synapse manages credentials completely
# Create Linked Service for SQL DB with Key Vault password

```

```
# Reference Linked Service in notebook:

# Read from Azure SQL using Linked Service (no credentials in
code!)
df_sql = spark.read \
    .format("com.microsoft.sqlserver.jdbc.spark") \
    .option("url",
"jdbc:sqlserver://retailmart.database.windows.net:1433;database=Sales"
\
    .option("dbtable", "dbo.Sales") \
    .option("authentication", "ActiveDirectoryMSI") \
    .load()
```

◆ Connecting to ADLS Gen2 — All Methods

```
# — METHOD 1: Default linked storage (SIMPLEST) —————
# Synapse workspace has a default ADLS account configured during
setup
# Access using abfss:// directly – NO credentials needed!
# Synapse Managed Identity automatically has access

df = spark.read.parquet(

"abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/"
)
# Works because Synapse MSI has Storage Blob Data Contributor
role

# — METHOD 2: Account Key —————
storage_account = "retailmartdatalake"
account_key = mssparkutils.credentials.getSecret(
    "https://kv-retailmart-prod.vault.azure.net/",
```

```

        "adls-account-key"
    )

    spark.conf.set(
        f"fs.azure.account.key.
        {storage_account}.dfs.core.windows.net",
        account_key
    )

    df = spark.read.parquet(

    f"abfss://silver@{storage_account}.dfs.core.windows.net/sales/"
    )

# — METHOD 3: Service Principal (OAuth) —————
client_id      = mssparkutils.credentials.getSecret("https://kv-
retailmart-prod.vault.azure.net/", "sp-client-id")
client_secret  = mssparkutils.credentials.getSecret("https://kv-
retailmart-prod.vault.azure.net/", "sp-client-secret")
tenant_id      = mssparkutils.credentials.getSecret("https://kv-
retailmart-prod.vault.azure.net/", "tenant-id")

    spark.conf.set(f"fs.azure.account.auth.type.
    {storage_account}.dfs.core.windows.net", "OAuth")
    spark.conf.set(f"fs.azure.account.oauth.provider.type.
    {storage_account}.dfs.core.windows.net",

    "org.apache.hadoop.fs.azurebfs.oauth2.ClientCredsTokenProvider")
    spark.conf.set(f"fs.azure.account.oauth2.client.id.
    {storage_account}.dfs.core.windows.net", client_id)
    spark.conf.set(f"fs.azure.account.oauth2.client.secret.
    {storage_account}.dfs.core.windows.net", client_secret)
    spark.conf.set(f"fs.azure.account.oauth2.client.endpoint.
    {storage_account}.dfs.core.windows.net",

```

```

f"https://login.microsoftonline.com/{tenant_id}/oauth2/token")

df =
spark.read.parquet(f"abfss://silver@{storage_account}.dfs.core.windows.net/{path}")

# — METHOD 4: SAS Token —————
sas_token = mssparkutils.credentials.getSecret("https://kv-retailmart-prod.vault.azure.net/", "adls-sas-token")

spark.conf.set(
    f"fs.azure.sas.silver.{storage_account}.dfs.core.windows.net",
    sas_token
)

# — BEST PRACTICE RECOMMENDATION —————
# Development: Method 1 (default linked storage – zero config)
# Production: Method 1 with Managed Identity (most secure)
# Cross-tenant: Method 3 Service Principal
# Temporary: Method 4 SAS Token

```

◆ Connecting to Azure Blob Storage

```

# AZURE BLOB vs ADLS GEN2:
# Blob: wasbs://container@account.blob.core.windows.net/
# ADLS2: abfss://container@account.dfs.core.windows.net/
# Key difference: "blob" vs "dfs" in URL, "wasbs" vs "abfss"

# — METHOD 1: Account Key —————
blob_account = "retailmartblob"
blob_key = mssparkutils.credentials.getSecret(
    "https://kv-retailmart-prod.vault.azure.net/",

```

```

        "blob-storage-key"
    )

    spark.conf.set(
        f"fs.azure.account.key.{blob_account}.blob.core.windows.net",
        blob_key
    )

    df_blob = spark.read.parquet(

        f"wasbs://archive@{blob_account}.blob.core.windows.net/sales/2023/"
    )

    # — METHOD 2: SAS Token for specific container —————
    sas = mssparkutils.credentials.getSecret("https://kv-retailmart-
    prod.vault.azure.net/", "blob-sas")

    spark.conf.set(
        f"fs.azure.sas.archive.{blob_account}.blob.core.windows.net",
        sas
    )

    df =
    spark.read.parquet("wasbs://archive@retailmartblob.blob.core.windows

```

◆ Connecting to Azure SQL Database — All Methods

```

# — METHOD 1: SQL Authentication with JDBC —————
sql_server    = "retailmart-sqlldb.database.windows.net"
sql_database  = "SalesDB"
sql_user      = "sqladmin"
sql_password  = mssparkutils.credentials.getSecret(

```

```

    "https://kv-retailmart-prod.vault.azure.net/",
    "sqldb-password"
)

jdbc_url = (
    f"jdbc:sqlserver://{sql_server}:1433;"
    f"database={sql_database};"
    f"encrypt=true;"
    f"trustServerCertificate=false;"
    f"hostnameInCertificate=*.database.windows.net;"
    f"loginTimeout=30"
)

# Read full table
df_customers = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "dbo.Customers") \
    .option("user", sql_user) \
    .option("password", sql_password) \
    .option("driver",
"com.microsoft.sqlserver.jdbc.SQLServerDriver") \
    .load()

# Read with custom query (incremental)
df_recent_sales = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("query", f"SELECT * FROM dbo.Sales WHERE sale_date =
'{process_date}'") \
    .option("user", sql_user) \
    .option("password", sql_password) \
    .load()

# Read with partitioning (parallel reads – much faster for large

```

```

tables!)
df_large_table = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "dbo.LargeSalesTable") \
    .option("user", sql_user) \
    .option("password", sql_password) \
    .option("partitionColumn", "sale_id") \
    .option("lowerBound", "1") \
    .option("upperBound", "50000000") \
    .option("numPartitions", "16") \
    .load()

# — METHOD 2: Azure Active Directory / Managed Identity —————
# No password needed! Uses Synapse MSI

jdbc_url_msi = (
    f"jdbc:sqlserver://{sql_server}:1433;"
    f"database={sql_database};"
    f"encrypt=true;"
    f"authentication=ActiveDirectoryMSI"
)

df_secure = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url_msi) \
    .option("dbtable", "dbo.Customers") \
    .load()

# Synapse MSI needs "db_datareader" role on SQL Database

# — WRITE to SQL Database —————
df_result.write \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "dbo.Sales_Silver") \

```



```
.option("user", sql_user) \  
.option("password", sql_password) \  
.option("batchsize", "100000") \  
.option("truncate", "true") \  
.mode("overwrite") \  
.save()
```

◆ Connecting to On-Premise SQL Server

```
# REQUIRES: Self-Hosted IR configured in Synapse  
# Manage → Integration Runtimes → SHIR must be running  
  
# On-premise SQL Server via JDBC + SHIR  
# Note: Synapse notebooks can use SHIR through Linked Service  
  
# Step 1: Create Linked Service for on-prem SQL Server  
# Manage → Linked Services → SQL Server  
# Name: LS_OnPrem_SQLServer  
# Server: 10.0.1.51  
# Database: LegacySalesDB  
# Auth: Windows Authentication  
# Connect via IR: SHIR_OnPrem_Synapse  
  
# Step 2: Use Linked Service connection in notebook  
# (Via Synapse LinkedService utility)  
  
df_onprem = spark.read \  
  .format("jdbc") \  
  .option("url",  
"jdbc:sqlserver://10.0.1.51:1433;database=LegacySalesDB") \  
  .option("dbtable", "dbo.LegacySales") \  
  .option("user", mssparkutils.credentials.getSecret(
```

```

        "https://kv-retailmart-prod.vault.azure.net/",
        "onprem-sql-user")) \
.option("password", mssparkutils.credentials.getSecret(
        "https://kv-retailmart-prod.vault.azure.net/",
        "onprem-sql-password")) \
.load()

print(f"On-prem records: {df_onprem.count():,}")

```

◆ Reading and Writing All File Formats

```

# — CSV FILES —————
# READING (with optimization options)
from pyspark.sql.types import *

# Always define schema for production – avoid inferSchema!
schema = StructType([
    StructField("sale_id",      IntegerType(), False),
    StructField("customer_id",  StringType(),  False),
    StructField("total_amount", DoubleType(),   True),
    StructField("sale_date",    StringType(),   True), # read as
string then convert
    StructField("region_code",  StringType(),   True)
])

df_csv = spark.read \
    .schema(schema) \
    .option("header", "true") \
    .option("delimiter", ",") \
    .option("encoding", "UTF-8") \
    .option("nullValue", "NULL") \
    .option("nanValue", "NaN") \

```

```

.option("mode", "DROPMALFORMED") \

.csv("abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/c

# OPTIMIZATION for large CSV files:
# 1. Always use explicit schema (2x faster than inferSchema)
# 2. Read multiple files at once using folder path
# 3. Use mode=DROPMALFORMED for dirty data

# WRITING CSV
df_csv.coalesce(1) \
    .write \
    .mode("overwrite") \
    .option("header", "true") \
    .option("delimiter", ",") \
    .option("encoding", "UTF-8") \

.csv("abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/c
# coalesce(1) = one output file (for small outputs)

# — PARQUET FILES —————
# Best format for analytics – columnar, compressed

# READ
df_parquet = spark.read \

.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/sal

# READ specific partitions (partition pruning)
df_june = spark.read \

.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/sal
\
    .filter("year = 2024 AND month = 6")
# Spark reads ONLY year=2024/month=6 folder!

```

```

# WRITE with partitioning and compression
df_parquet.write \
    .mode("overwrite") \
    .partitionBy("year", "month", "region_code") \
    .option("compression", "snappy") \

.parquet("abfss://gold@retailmartdatalake.dfs.core.windows.net/sales

# — JSON FILES —————
# READ flat JSON (one JSON object per line - NDJSON)
df_json = spark.read \
    .option("multiLine", "false") \

.json("abfss://bronze@retailmartdatalake.dfs.core.windows.net/paymer

# READ nested JSON
df_nested = spark.read \
    .option("multiLine", "true") \

.json("abfss://bronze@retailmartdatalake.dfs.core.windows.net/api_re

# Flatten nested structure
from pyspark.sql.functions import *

df_flat = df_nested \
    .select(
        col("order_id"),
        col("customer.id").alias("customer_id"),
        col("customer.name").alias("customer_name"),
        col("total"),
        explode(col("items")).alias("item")
    ) \
    .select(
        "order_id", "customer_id", "customer_name", "total",

```

```

        col("item.product_code").alias("product_code"),
        col("item.quantity").alias("quantity"),
        col("item.price").alias("unit_price")
    )

# WRITE JSON
df_flat.write \
    .mode("overwrite") \

.json("abfss://silver@retailmartdatalake.dfs.core.windows.net/orders

# — AVRO FILES —————
# Best for: streaming, schema evolution, Kafka integration

# READ
df_avro = spark.read \
    .format("avro") \

.load("abfss://bronze@retailmartdatalake.dfs.core.windows.net/kafka_

# WRITE
df_avro.write \
    .format("avro") \
    .mode("overwrite") \

.save("abfss://silver@retailmartdatalake.dfs.core.windows.net/events

# — ORC FILES —————
# Best for: Hive compatibility, heavy write workloads

# READ
df_orc = spark.read \
    .format("orc") \

.load("abfss://bronze@retailmartdatalake.dfs.core.windows.net/hive_e

```

```

# WRITE
df_orc.write \
    .format("orc") \
    .mode("overwrite") \
    .option("compression", "zlib") \

.save("abfss://silver@retailmartdatalake.dfs.core.windows.net/orc_ou

# — EXCEL FILES —————
# Requires: openpyxl or com.crealytics.spark.excel library

# First install package (in Synapse Pool packages or notebook):
# %pip install openpyxl

# READ Excel using pandas then convert to Spark (small files)
import pandas as pd
from notebookutils import mssparkutils

# Download Excel file from ADLS to local temp
mssparkutils.fs.cp(

"abfss://bronze@retailmartdatalake.dfs.core.windows.net/config/store
    "file:///tmp/store_targets.xlsx"
)

# Read with pandas
pdf = pd.read_excel("/tmp/store_targets.xlsx",
sheet_name="Targets")
df_excel = spark.createDataFrame(pdf) # convert to Spark

print(f"Excel rows: {df_excel.count()}")
df_excel.show(5)

# Using Spark Excel library (for large files):

```

```

df_excel_spark = spark.read \
    .format("com.crealytics.spark.excel") \
    .option("header", "true") \
    .option("inferSchema", "true") \
    .option("dataAddress", "'Sheet1'!A1") \

.load("abfss://bronze@retailmartdatalake.dfs.core.windows.net/config

# WRITE to Excel (via pandas)
df_result_pd = df_result.toPandas() # only for small DataFrames!
df_result_pd.to_excel("/tmp/output.xlsx", index=False)

# Upload back to ADLS
mssparkutils.fs.cp(
    "file:///tmp/output.xlsx",
    "abfss://gold@retailmartdatalake.dfs.core.windows.net/reports/output
)

```

◆ Creating RDDs and DataFrames — All Methods

```

# — RDD CREATION (Synapse Spark) —————
# Method 1: From collection
rdd_numbers = sc.parallelize([1, 2, 3, 4, 5], numSlices=4)

# Method 2: From file
rdd_text = sc.textFile(
    "abfss://bronze@retailmartdatalake.dfs.core.windows.net/raw/sales.csv"
)

# Method 3: From range

```

```

rdd_range = sc.range(1, 1000000, step=1, numSlices=10)

# Method 4: From existing DataFrame
rdd_from_df = df_sales.rdd # convert DataFrame to RDD

# — DATAFRAME CREATION (Synapse Spark) —————
# Method 1: Read from storage (most common)
df = spark.read.parquet(
    "abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/"
)

# Method 2: From Python list
data = [
    (1, "C001", 50000.0, "2024-06-15"),
    (2, "C002", 35000.0, "2024-06-15"),
    (3, "C003", 85000.0, "2024-06-15")
]
columns = ["sale_id", "customer_id", "amount", "sale_date"]
df_test = spark.createDataFrame(data, schema=columns)

# Method 3: From schema + data
from pyspark.sql.types import *

schema = StructType([
    StructField("sale_id", IntegerType(), False),
    StructField("customer", StringType(), False),
    StructField("amount", DoubleType(), True)
])
df_typed = spark.createDataFrame(data[:2], schema=schema)

# Method 4: From SQL query (Spark SQL)
spark.sql("CREATE TEMP VIEW sales AS SELECT * FROM
delta.`abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/`")
df_from_sql = spark.sql("SELECT * FROM sales WHERE region_code =

```



```
'N')

```

```
# Method 5: From Pandas
```

```
import pandas as pd
pdf = pd.DataFrame({"a": [1, 2, 3], "b": ["x", "y", "z"]})
df_from_pandas = spark.createDataFrame(pdf)
```

```
# Method 6: From JDBC (SQL Database)
```

```
df_from_sql_db = spark.read \
    .format("jdbc") \
    .option("url", jdbc_url) \
    .option("dbtable", "dbo.Products") \
    .option("user", sql_user) \
    .option("password", sql_password) \
    .load()
```

◆ Repartition vs Coalesce — Complete Guide

```
# RULE: Remember this always!
```

```
#
```

```
# REPARTITION = INCREASE or CHANGE distribution (with shuffle)
```

```
# COALESCE = ONLY DECREASE (no shuffle, merges local)
```

```
#
```

```
# Check current partition count
```

```
print(f"Current partitions: {df_sales.rdd.getNumPartitions()}")
```

```
# — WHEN TO USE REPARTITION —
```

```
# 1. Increase partitions for better parallelism
```

```
df_repartitioned = df_sales.repartition(200)
```

```
# 2. Distribute by join key (before multiple joins on same key)
df_by_key = df_sales.repartition(col("customer_id"))
# Now all same customer_id rows are on same partition
# JOIN on customer_id → no shuffle needed!
```

```
# 3. Before writing large output
df_large.repartition(100).write.parquet(output_path)
# Creates 100 equal-sized output files
```

```
# — WHEN TO USE COALESCE —————
```

```
# 1. After filtering (many empty partitions remain)
df_north = df_sales.filter(col("region_code") == "N")
# Was 200 partitions → after filter maybe 20 have data, 180 are empty
```

```
df_north_optimized = df_north.coalesce(20) # reduce without shuffle
# Now 20 partitions, all with data = efficient!
```

```
# 2. Before writing small output (fewer, larger files)
df_summary.coalesce(5).write.parquet(output_path)
# Creates exactly 5 files instead of 200 tiny ones
```

```
# — THE SMALL FILES PROBLEM —————
```

```
# This is a REAL and SERIOUS problem in data lakes!
```

```
# Bad pattern: writing 200 partitions of small data
df_daily_summary.write.parquet(output_path)
# Creates 200 × 1KB files = 200 tiny files!
```

```
# Why bad?
# → Reading 200 tiny files: 200 file open/close operations
# → Each file needs metadata lookup → slow!
# → Next day adds 200 more → after 1 year: 73,000 tiny files!
```

```

# → Queries take longer and longer

# Good pattern: control output file size
# Target: 128MB - 256MB per output file
df_daily_summary.coalesce(2).write.parquet(output_path)
# Creates 2 reasonable-sized files

# DYNAMIC approach (calculate based on data size):
import math
data_size_bytes = df.rdd.map(lambda row: len(str(row))).sum()
target_file_size = 128 * 1024 * 1024 # 128MB
optimal_partitions = max(1, math.ceil(data_size_bytes /
target_file_size))
df.coalesce(optimal_partitions).write.parquet(output_path)

# — SYNAPSE-SPECIFIC OPTIMIZATION —————
# Synapse has a special feature: Auto Compaction on Delta tables
spark.conf.set("spark.databricks.delta.autoCompact.enabled",
"true")
# Automatically merges small files → solves small file problem!

```

◆ Joins in Synapse Notebook

```

# — ALL JOIN TYPES WITH RETAILMART EXAMPLES —————
from pyspark.sql.functions import *
from pyspark.sql import functions as F

df_sales =
spark.read.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/sales.parquet")
df_customers =
spark.read.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/customers.parquet")
df_products =

```

```

spark.read.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/silver/retailmartdata/df_stores")
df_stores =
spark.read.parquet("abfss://silver@retailmartdatalake.dfs.core.windows.net/silver/retailmartdata/df_sales")

# — INNER JOIN: Only matching rows —————
df_sales_with_customer = df_sales.join(
    df_customers,
    on="customer_id",
    how="inner" # or "inner"
)
# Result: Only sales that have matching customer record

# — LEFT JOIN: All sales, matching customer (null if no match) —
df_all_sales = df_sales.join(
    df_customers,
    on="customer_id",
    how="left"
)
# Result: All sales records, customer info null for unmatched

# — RIGHT JOIN: All customers, matching sales —————
df_all_customers = df_sales.join(
    df_customers,
    on="customer_id",
    how="right"
)
# Result: All customers, sales null for customers with no sales

# — FULL OUTER: Everything from both —————
df_full = df_sales.join(
    df_customers,
    on="customer_id",
    how="full" # or "outer" or "fullouter"
)

```

```

# — CROSS JOIN (Cartesian): Every combination —————
# USE VERY CAREFULLY – produces M×N rows!
df_regions = spark.createDataFrame(
    [("North",), ("South",), ("East",), ("West",)],
    ["region"]
)
df_months = spark.createDataFrame(
    [(m,) for m in range(1, 13)],
    ["month"]
)
# Create all region-month combinations for blank template
df_all_combinations = df_regions.crossJoin(df_months)
# Result: 4 regions × 12 months = 48 rows
# Useful for: creating complete grid for reports (no missing months)

# — MULTIPLE CONDITION JOIN —————
df_result = df_sales.join(
    df_products,
    on=[
        df_sales.product_code == df_products.product_code,
        df_sales.region_code == df_products.available_region
    ],
    how="left"
)

# — HANDLING DUPLICATE COLUMN NAMES AFTER JOIN —————
# Both tables have "status" column → ambiguous after join!
df_result = df_sales.alias("s").join(
    df_customers.alias("c"),
    on="customer_id",
    how="left"
).select(
    col("s.sale_id"),
    col("s.total_amount"),

```

```

    col("s.status").alias("sale_status"),      # disambiguate!
    col("c.customer_name"),
    col("c.status").alias("customer_status")    # disambiguate!
)

# — CHAIN MULTIPLE JOINS —————
df_enriched = df_sales \
    .join(broadcast(df_customers), on="customer_id", how="left")
\
    .join(broadcast(df_products), on="product_code",
how="left") \
    .join(broadcast(df_stores), on="store_id",
how="left") \
    .select(
        "sale_id",
        "total_amount",
        "sale_date",
        "customer_name",
        "customer_tier",
        "product_name",
        "category",
        "store_name",
        "city",
        "region_code"
    )

print(f"Enriched sales: {df_enriched.count():,}")

```

◆ Broadcast Joins and Spark Configuration for Optimization

```

# — BROADCAST JOIN IN SYNAPSE —————
from pyspark.sql.functions import broadcast

```

```

# Small table (< 10MB default threshold) → broadcast
automatically
# Large fact × small dimension = classic broadcast join scenario

# MANUAL BROADCAST HINT
df_result = df_sales.join(
    broadcast(df_stores),    # explicitly broadcast dim_stores
    on="store_id",
    how="left"
)

# VERIFY broadcast happened:
df_result.explain()
# Look for: BroadcastHashJoin ← good, fast!
# If you see: SortMergeJoin    ← no broadcast, shuffle happening

# — CONFIGURE BROADCAST THRESHOLD —————
# Default: 10MB – too small for most real dimension tables
spark.conf.set("spark.sql.autoBroadcastJoinThreshold", str(200 *
1024 * 1024)) # 200MB
# Now tables up to 200MB are automatically broadcast

# — FULL OPTIMIZATION CONFIGURATION —————
def configure_spark_for_performance():
    """Apply all performance optimizations for Synapse Spark"""

    # JOIN optimizations
    spark.conf.set("spark.sql.autoBroadcastJoinThreshold",
str(200 * 1024 * 1024))

    # Adaptive Query Execution (AQE)
    spark.conf.set("spark.sql.adaptive.enabled", "true")

    spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled",

```

```

"true")
    spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

spark.conf.set("spark.sql.adaptive.localShuffleReader.enabled",
"true")

    # Shuffle optimization
    spark.conf.set("spark.sql.shuffle.partitions", "200")
    # Adjust based on data size:
    # Small data (< 1GB):    set to 50-100
    # Medium (1-10GB):       set to 200 (default)
    # Large (10-100GB):      set to 400-800
    # Very large (>100GB):   set to 1000-2000

    # Delta Lake optimizations

spark.conf.set("spark.databricks.delta.optimizeWrite.enabled",
"true")
    spark.conf.set("spark.databricks.delta.autoCompact.enabled",
"true")

    # Cost-Based Optimizer
    spark.conf.set("spark.sql.cbo.enabled", "true")
    spark.conf.set("spark.sql.statistics.histogram.enabled",
"true")

    print("✅ Spark optimizations configured")

configure_spark_for_performance()

# — WHAT IS CATALYST OPTIMIZER? —————
# (Already explained in Databricks section – same Spark engine!)
# Synapse Spark uses same Catalyst optimizer as Databricks

# KEY BEHAVIORS IN SYNAPSE:

```



```

# 1. Predicate Pushdown: filter pushed to Parquet file reader
# 2. Column Pruning: only needed columns read from Parquet
# 3. Constant Folding: WHERE year = 2024 evaluated at plan time
# 4. Join Reordering: Catalyst may reorder joins for efficiency

# — SKEWNESS ISSUE IN SPARK —————
# Data skew = one partition has MUCH more data than others

# DETECT SKEW:
# Spark UI → Stages → look at task duration
# If one task takes 10 minutes while others take 10 seconds →
skew!

# Also detect programmatically:
df_sales.groupBy(spark_partition_id().alias("partition")) \
    .count() \
    .orderBy(desc("count")) \
    .show(20)

# If one partition count >> others → skewed!

# COMMON CAUSES IN RETAILMART:
# - region_code "NULL" has 40% of records
# - customer_id "WALK-IN" has 60% of cash sales
# - payment_mode "COD" has 70% of online orders

# SOLUTIONS:
# 1. Filter skewed values separately
# 2. Salt the skewed key (add random suffix)
# 3. Enable AQE skew join handling

# AQE handles skew automatically:
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.skewedPartitionThresholdInBytes",
    str(256 * 1024 * 1024)) # 256MB threshold
# AQE splits partitions larger than 256MB automatically!

```

◆ SCD Type 1 in Synapse Notebook

```
# — SCD TYPE 1: OVERWRITE CHANGED VALUES (No History) —————
# Scenario: Customer address changes → update to new address
# Old address completely REPLACED (no history kept)

from delta.tables import DeltaTable
from pyspark.sql import functions as F

# Read new/updated customer data from Bronze
df_source = spark.read \

.parquet("abfss://bronze@retailmartdatalake.dfs.core.windows.net/cus
\
        .withColumn("updated_at", F.current_timestamp())

# Target Delta table (Silver layer)
target_path =
"abfss://silver@retailmartdatalake.dfs.core.windows.net/dim_customer

# Check if Delta table exists
from delta.tables import DeltaTable

if DeltaTable.isDeltaTable(spark, target_path):
    delta_target = DeltaTable.forPath(spark, target_path)

    # MERGE = SCD Type 1
    delta_target.alias("target").merge(
        df_source.alias("source"),
        "target.customer_id = source.customer_id" # match
condition
    ) \
    .whenMatchedUpdate(
```

```
# Only update when something actually changed (avoid unnecessary writes)
```

```
    condition=""
        target.customer_name != source.customer_name OR
        target.email          != source.email          OR
        target.phone           != source.phone          OR
        target.address          != source.address        OR
        target.city             != source.city
    "",
    set={
        "customer_name": "source.customer_name",
        "email":          "source.email",
        "phone":          "source.phone",
        "address":        "source.address",
        "city":           "source.city",
        "state":          "source.state",
        "updated_at":     "source.updated_at"
    }
) \
.whenNotMatchedInsert(
    # New customer not in target
    values={
        "customer_id":    "source.customer_id",
        "customer_name":  "source.customer_name",
        "email":          "source.email",
        "phone":          "source.phone",
        "address":        "source.address",
        "city":           "source.city",
        "state":          "source.state",
        "customer_tier":  "source.customer_tier",
        "created_at":     "source.updated_at",
        "updated_at":     "source.updated_at"
    }
) \
.execute()
```

```

print("✅ SCD Type 1 Merge completed")

else:
    # First load – create the table
    df_source.write \
        .format("delta") \
        .mode("overwrite") \
        .save(target_path)
    print("✅ Initial load completed")

# — VERIFY RESULTS —————
spark.sql(f"""
    DESCRIBE HISTORY delta.`{target_path}`
""").show(5, truncate=False)

# History shows:
# Version 0: WRITE (initial load)
# Version 1: MERGE (today's update – 142 rows updated, 28 new)

```

◆ SCD Type 2 in Synapse Notebook

```

# — SCD TYPE 2: KEEP FULL HISTORY —————
# Scenario: Customer moves from Mumbai to Delhi
# KEEP both records:
#   Row 1: C001, Mumbai, start=2020-01-01, end=2024-06-14,
is_current=N
#   Row 2: C001, Delhi, start=2024-06-15, end=9999-12-31,
is_current=Y

from delta.tables import DeltaTable
from pyspark.sql import functions as F

```

```

from pyspark.sql.types import DateType

target_path =
"abfss://silver@retailmartdatalake.dfs.core.windows.net/dim_customer

# Read source (new/updated customer data)
df_source = spark.read \

.parquet("abfss://bronze@retailmartdatalake.dfs.core.windows.net/cus

# Read current target to find what changed
if DeltaTable.isDeltaTable(spark, target_path):

    delta_target = DeltaTable.forPath(spark, target_path)
    df_target_current = spark.read.format("delta") \
        .load(target_path) \
        .filter(F.col("is_current") ==
True)

# STEP 1: Find records that have CHANGED
df_changed = df_source.alias("src").join(
    df_target_current.alias("tgt"),
    on="customer_id",
    how="inner"
).filter(
    # These columns changed
    (F.col("src.address") != F.col("tgt.address")) |
    (F.col("src.city") != F.col("tgt.city")) |
    (F.col("src.email") != F.col("tgt.email"))
).select("src.customer_id")

changed_ids = [row.customer_id for row in
df_changed.collect()]
print(f"Changed records: {len(changed_ids)}")

```

```

if len(changed_ids) > 0:
    # STEP 2: Expire current records (set is_current=False,
    end_date=today)
    delta_target.alias("target").merge(
        df_changed.alias("changed"),
        "target.customer_id = changed.customer_id AND
target.is_current = true"
    ) \
    .whenMatchedUpdate(set={
        "is_current": "false",
        "effective_end_date": "current_date()"
    }) \
    .execute()

    print("✅ Step 2: Expired old records")

    # STEP 3: Insert new version for changed records + truly new
    records
    df_new_rows = df_source \
        .withColumn("effective_start_date", F.current_date()) \
        .withColumn("effective_end_date", F.lit("9999-12-
31")).cast(DateType()) \
        .withColumn("is_current", F.lit(True))

    # Only insert if: new customer OR changed customer
    df_target_ids = df_target_current.select("customer_id")

    df_to_insert = df_new_rows.join(
        df_target_ids,
        on="customer_id",
        how="left"
    )

    # All rows (source always gets a new current row in SCD2
    # for changed records – merge will handle dedup)

```

```

# Use merge to avoid inserting unchanged records again
delta_target.alias("target").merge(
    df_new_rows.alias("source"),
    """target.customer_id      = source.customer_id
       AND target.is_current   = true
       AND target.effective_start_date =
source.effective_start_date"""
) \
.whenNotMatchedInsert(values={
    "customer_id":      "source.customer_id",
    "customer_name":    "source.customer_name",
    "email":            "source.email",
    "phone":            "source.phone",
    "address":          "source.address",
    "city":             "source.city",
    "state":            "source.state",
    "effective_start_date": "source.effective_start_date",
    "effective_end_date":  "source.effective_end_date",
    "is_current":        "source.is_current"
}) \
.execute()

print("✅ Step 3: Inserted new version records")

else:
    # First load
    df_source \
        .withColumn("effective_start_date", F.current_date()) \
        .withColumn("effective_end_date",   F.lit("9999-12-31")).cast(DateType()) \
        .withColumn("is_current",          F.lit(True)) \
        .write \
        .format("delta") \
        .mode("overwrite") \
        .save(target_path)

```

```
print("✅ Initial SCD2 load completed")
```

```
# — VERIFY SCD2 RESULTS —
```

```
spark.sql(f"""
    SELECT customer_id, city, effective_start_date,
    effective_end_date, is_current
    FROM delta.`{target_path}`
    WHERE customer_id = 'C001'
    ORDER BY effective_start_date
""").show()
```

```
# Expected output:
```

```
# customer_id | city      | effective_start | effective_end |
is_current
# C001        | Mumbai   | 2020-01-01      | 2024-06-14    | false
← old
# C001        | Delhi    | 2024-06-15      | 9999-12-31    | true
← current
```

◆ Executing Synapse Notebooks from Synapse Pipelines

COMPLETE INTEGRATION: Pipeline → Notebook → **Return Value**

```
# — IN THE NOTEBOOK (what it does and returns) —
```

```
# Parameter cell (first cell with parameters)
```

```
input_path  =
"abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/"
output_path =
"abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/"
process_date = "2024-06-15"
```



```

load_type      = "INCREMENTAL"

# Main processing
from pyspark.sql import functions as F
from notebookutils import mssparkutils

try:
    # Read data
    df = spark.read.parquet(f"{input_path}{process_date}/")
    input_count = df.count()

    # Transform
    df_clean = df.filter(F.col("sale_id").isNotNull()) \
        .filter(F.col("total_amount") > 0)

    # Write
    df_clean.write \
        .format("delta") \
        .mode("overwrite") \
        .save(f"{output_path}{process_date}/")

    output_count = df_clean.count()

    # Return JSON result to pipeline
    import json
    result = json.dumps({
        "status": "SUCCESS",
        "process_date": process_date,
        "input_rows": input_count,
        "output_rows": output_count,
        "rows_dropped": input_count - output_count
    })

    mssparkutils.notebook.exit(result)

```

```
except Exception as e:
    error_result = json.dumps({
        "status": "FAILED",
        "error": str(e),
        "process_date": process_date
    })
    mssparkutils.notebook.exit(error_result)
```

IN SYNAPSE PIPELINE (Notebook Activity):

Activity Type: Notebook

Notebook: /Production/silver/transform_sales

Azure Synapse Analytics Pool: RetailMart_SparkPool

Base Parameters (passed to notebook):

```
input_path:
abfss://bronze@retailmartdatalake.dfs.core.windows.net/sales/
output_path:
abfss://silver@retailmartdatalake.dfs.core.windows.net/sales/
process_date: @{pipeline().parameters.p_date}
load_type:    @{pipeline().parameters.p_load_type}
```

READING NOTEBOOK OUTPUT IN PIPELINE:

Notebook activity **output** available as:

```
@{activity('RunTransformNotebook').output.status.Output.result.exitv
```

Parse the JSON **exit** value:

```
@{json(activity('RunTransformNotebook').output.status.Output.result.
```

Access specific fields:

Status:

```
@{json(activity('RunTransform').output.status.Output.result.exitValue)}
Rows:
@{json(activity('RunTransform').output.status.Output.result.exitValue)}

USE RESULT IN IF CONDITION:
Condition: @equals(
    json(activity('RunTransform').output.status.Output.result.exitValue)
    'SUCCESS'
)
TRUE: → Continue to Gold loading
FALSE: → Log failure + send alert + Fail pipeline
```

FULL PIPELINE FLOW:

Parameter Cell (p_date) → Notebook Activity (pass p_date)

- Notebook returns JSON result
- If SUCCESS → load Gold layer
- If FAILED → send alert email via Logic Apps
 - Fail pipeline with clear error message

Complete Azure Synapse Architecture Summary

AZURE SYNAPSE ANALYTICS – COMPLETE GUIDE

POOL TYPES		Dedicated SQL (DW), Serverless SQL (Lake),
		Apache Spark Pool

SERVERLESS SQL	OPENROWSET, External Tables, Views
	Query CSV/Parquet/JSON/Delta directly
	External Data Source, File Format, Credential
DEDICATED SQL	MPP architecture, 60 distributions
	Hash/RoundRobin/Replicated tables
	PolyBase & COPY INTO for fast bulk loads
PIPELINES	Same as ADF (built-in)
(Integrate) files	Wildcard copy, Dataset params, Dynamic
	Schedule + Event triggers
	ForEach bulk copy with control table
SPARK POOL	MSSparkUtils (fs, notebook, credentials)
	All file formats (CSV, JSON, Parquet, Avro, ORC)
	All connections (ADLS, Blob, SQL, On-prem)
	Repartition vs Coalesce
	All join types + Broadcast optimization
	AQE, Catalyst, Skew handling

	SCD Type 1 + Type 2 (Delta MERGE)
INTEGRATION	Pipeline → Notebook (pass params, read exit) Key Vault secrets via MSSparkUtils SHIR for on-premise connections Azure DevOps Repos for notebook versioning
OPTIMIZATIONS	Broadcast joins, AQE, shuffle partitions Small file compaction, repartition/coalesce Partition pruning, predicate pushdown Schema definition (avoid inferSchema)

RETAILMART FINAL ARCHITECTURE:

Sources → ADF (orchestrate) → Synapse Pipelines (ingest)
 → ADLS Bronze → Synapse Spark Notebooks (transform)
 → ADLS Silver → Serverless SQL Pool (explore/views)
 → ADLS Gold → Dedicated SQL Pool (BI/reporting)
 → Power BI Dashboards (business insights)

 **Azure Synapse Analytics section 100% complete** — every concept from the PDF including both pools (Dedicated + Serverless), Spark pool, all file formats, all connection methods, MSSparkUtils, notebook parameters, magic commands, SCD1/SCD2, triggers,

PolyBase/COPY INTO, bulk copy patterns, optimization techniques, Catalyst optimizer, skewness handling, and pipeline-to-notebook integration — all covered with RetailMart real-world examples following practical, depth-first teaching style.

 **Next: Microsoft Fabric Course** — ready to continue!

19/03/2026, 11:27:09

Thought process: Orchestrated comprehensive Microsoft Fabric curriculum mapping across twenty detailed instructional modules.

The user wants me to continue with the Microsoft Fabric Course section from the PDF, explaining it in the style of Ansh Lamba (a popular data engineering YouTuber/Udemy instructor known for his clear, practical, step-by-step teaching style with real-world examples, analogies, and conversational tone). Let me cover ALL concepts from the PDF's Microsoft Fabric section comprehensively.

From the PDF, the Microsoft Fabric Course covers:

- 1) Introduction to Big Data
- 2) Introduction to Microsoft Fabric
- 3) Introduction to Fabric Components
- 4) Fabric Account Creation
- 5) OneLake Concepts
- 6) Data Factory in Microsoft Fabric

- 7) Fabric Data Engineering – Lakehouse
- 8) Fabric Data Engineering – Notebooks
- 9) Fabric Data Engineering – Apache Spark
- 10) Fabric Data Engineering – Delta Lake Tables
- 11) Fabric Data Warehouse
- 12) Fabric Real-time Intelligence
- 13) Activators in Fabric

- 14) Power BI in Fabric
- 15) Azure DevOps – Repos
- 16) SDLC and Agile Methodology
- 17) End-to-End Project: Migrating from On-Premises to Cloud
- 18) Resume Preparation
- 19) Interview Guidelines
- 20) Mock Interviews

Let me cover all of these in depth with Ansh Lamba's teaching style, breaking down each topic with practical examples and real-world applications.



Microsoft Fabric — Complete Deep Dive Guide

Teaching Style: Conversational, practical, real-world first — just like how Ansh Lamba teaches on YouTube/Udemy. No fluff, just clarity.



Let's Start With The Right Mindset

Before we dive in, let me ask you something...

You've learned:

- ✓ ADF → moves data
- ✓ Databricks → transforms data
- ✓ Synapse → warehouses + queries data
- ✓ Power BI → visualizes data

Now Microsoft looked at all of this and said:

"Why are our customers using 4-5 different tools?
Why can't we give them ONE platform for everything?"



That's EXACTLY what Microsoft Fabric is.

Think of it this way:

Old World = You need a car, a bike, a truck, and a bus
to travel (4 different tools, 4 bills, 4 manuals)

New World = You have ONE vehicle that transforms
into whatever you need (car/bike/truck/bus)
Pay ONE bill. Learn ONE platform.

THAT is Microsoft Fabric.

1 Introduction to Big Data — Fabric Perspective

◆ What is Data? (Fabric Context)

Let me tell you something most tutorials skip...

When Microsoft designed Fabric, they designed it
around ONE core idea:

"Data should be stored ONCE and used EVERYWHERE"

Let's understand why this matters with a story:

RETAILMART's DATA MESS (Before Fabric):

Data Engineering team:

→ Stores sales data in ADLS Gen2 (their copy)

Data Science team:

→ Downloads same data to their Databricks (COPY 2)

BI team:

→ Loads same data to SQL Server (COPY 3)

Finance team:

→ Exports to Excel (COPY 4)

Result:

4 copies of same data

4 different "versions of truth"

When source changes → need to update 4 places

Storage cost = 4x what it should be

Data inconsistency = wrong decisions!

Microsoft Fabric's answer: OneLake

→ ONE copy of data

→ ALL teams access SAME data

→ Data Engineers, Scientists, BI analysts –
all work on SAME underlying files

→ No duplication, no inconsistency

→ This is the REVOLUTION of Fabric

◆ What is a Database? What is Big Data?

Quick recap with Fabric perspective:

DATABASE = Organized collection of structured data

Examples: Azure SQL DB, MySQL, PostgreSQL

Works great for: 1MB to 1TB of structured data
Fails at: petabytes, unstructured data, streaming

BIG DATA = Data that is:

Too LARGE (can't fit on one machine)
Too FAST (arriving too quickly to process)
Too VARIED (CSV, JSON, images, videos, logs)

FABRIC solves ALL of this:

Large: OneLake stores petabytes
Fast: Real-time Intelligence for streaming
Varied: Lakehouse handles all formats

CHALLENGES of Big Data that Fabric addresses:

Challenge	How Fabric Solves It
Storage at scale	OneLake (unlimited, cheap)
Processing speed	Spark Pool (distributed)
SQL analytics	Data Warehouse
Real-time	Real-time Intelligence (KQL)
Visualization	Power BI (built-in)
Governance	Purview integration
Cost	Capacity-based (predictable)

WHY Traditional Databases CANNOT handle Big Data:

- Single server architecture (can't scale out)
- Only handles structured data
- Schema must be defined before data arrives
- Cannot handle real-time streaming natively
- Cost explodes at petabyte scale
- No native ML/AI capabilities

2 Introduction to Microsoft Fabric

◆ What is Microsoft Fabric?

Here's the simplest way I can explain it:

Microsoft Fabric =

Azure Data Factory (data integration)
+ Azure Databricks (big data processing)
+ Azure Synapse Analytics (data warehousing)
+ Power BI (visualization)
+ Azure Purview (governance)
+ Real-time Analytics (streaming)

ALL wrapped in ONE unified SaaS platform
on TOP of OneLake (single storage)

OFFICIAL DEFINITION:

Microsoft Fabric is an end-to-end analytics platform
that provides a unified experience for data engineers,
data scientists, data analysts, and business users
to work together on the same platform.

KEY WORD: "Unified"

Not just integration between tools

– it's ONE product, ONE license, ONE experience

FABRIC vs COMPETITORS:

Databricks Lakehouse Platform → closest competitor

AWS Data Stack (Glue+Redshift+QuickSight) → fragmented

Google Cloud (Dataflow+BigQuery+Looker) → more fragmented

Fabric's edge: Deepest Power BI + Office 365 integration

Microsoft 365 users → natural fit

◆ How to Enable Fabric in Your Organization

FABRIC IS A PREMIUM FEATURE – needs to be enabled!

STEP 1: Check if you have correct license

Fabric requires:

- Microsoft Fabric (trial or paid capacity)
- OR Power BI Premium Per Capacity
- OR Microsoft 365 E5/A5 license

STEP 2: Admin enables Fabric

Microsoft 365 Admin Center

- Settings → Org Settings → Microsoft Fabric
- Toggle: "Users can try Microsoft Fabric" → ON
- Choose: "Entire organization" or "Specific users"

STEP 3: Verify in Power BI

Go to app.powerbi.com

- Settings gear → Admin Portal
- Tenant Settings → Microsoft Fabric section
- "Users can create Fabric items" → Enabled

STEP 4: Start Fabric Trial (for learning)

app.fabric.microsoft.com

- Click your profile → Start Trial
- 60-day FREE trial with F64 capacity!
- F64 = 64 CUs (Capacity Units)
- More than enough for learning all features

STEP 5: Verify Trial Active

app.fabric.microsoft.com

→ Settings → Capacity Settings

→ Shows: Trial capacity active, 60 days remaining

PRO TIP from experience:

Don't use your company account for learning

Create a new Microsoft account (outlook.com)

Start FREE trial

Experiment freely without affecting company data!

◆ Fabric Workspace Structure

FABRIC WORKSPACE = Your working environment

Think of Workspace like a PROJECT FOLDER:

- One workspace per project/team/domain
- Contains all Fabric items for that project
- Access controlled at workspace level

WORKSPACE HIERARCHY:

Tenant (your company)

└─ Workspaces

 └─ RetailMart_DataEngineering ← for DE team

 └─ Lakehouse: RetailMart_LH

 └─ Pipeline: PL_Daily_Sales

 └─ Notebook: Transform_Sales

 └─ Dataflow: DF_CustomerMaster

 |

 └─ RetailMart_DataScience ← for DS team

```
|      |— Lakehouse: (shortcut to DE lakehouse!)
|      |— Notebook: ML_FraudDetection
|      |— ML Model: FraudModel_v2
|
|— RetailMart_Analytics          ← for BI team
    |— Warehouse: RetailMart_DW
    |— Dataset: Sales_PBI_Dataset
    |— Report: Sales_Dashboard
```

KEY CONCEPT:

Data Engineering workspace has the **REAL** data

Data Science workspace uses SHORTCUTS (**no copy!**)

Analytics workspace also uses SHORTCUTS (**no copy!**)

Everyone sees SAME data → OneLake magic!

WORKSPACE ROLES:

Admin → **Full** control, manage workspace settings

Member → **Create**/edit items, publish content

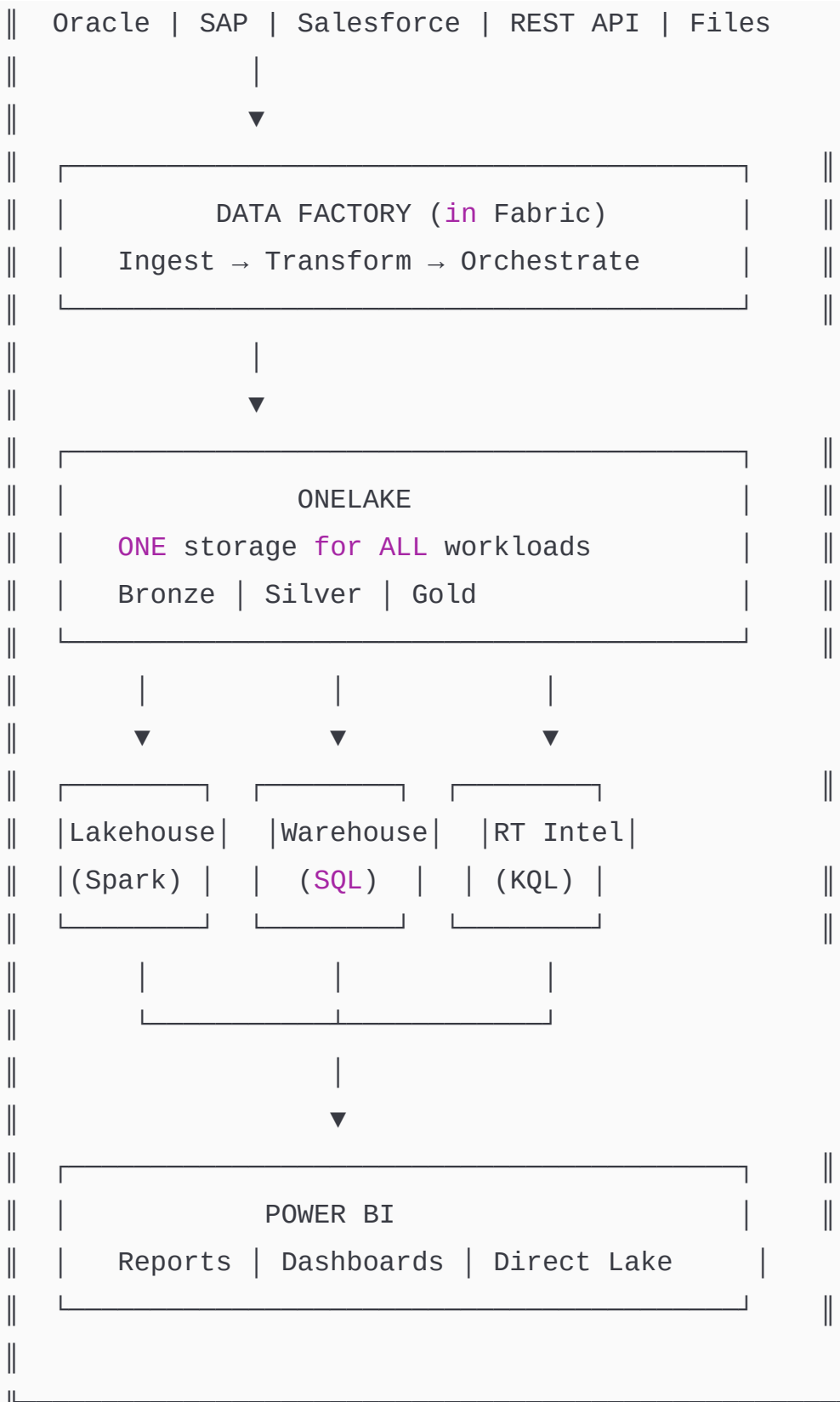
Contributor → **Create**/edit items

Viewer → Read-**only** access (can **view** reports)

◆ Microsoft Fabric — A Unified Platform

LET ME DRAW THE BIG PICTURE:





THE MAGIC:

Everything shares OneLake storage

No data movement **between** experiences

No ETL **between** Lakehouse **and** Warehouse

Power BI **reads** directly **from** Lakehouse tables!
This **is** why Fabric **is** revolutionary!

◆ Advantages of Microsoft Fabric

Let me be honest **with** you – **both** pros **AND** cons:

ADVANTAGES:

1. UNIFIED EXPERIENCE

One UI (app.fabric.microsoft.com) **for** everything
No jumping **between** ADF Studio → Databricks → Synapse Studio
Data Engineer + Data Scientist + BI Analyst **in** SAME tool

2. ONELAKE = **ONE COPY OF** DATA

Lakehouse **and** Warehouse share same underlying files
Power BI **reads** directly (**no** data movement!)
Shortcuts = virtual access (**no** physical **copy**)
Massive cost saving vs traditional approach

3. CAPACITY-BASED PRICING (predictable)

Pay **for** Fabric Capacity (F SKUs: F2, F4, F8...F2048)
One bill **for** **ALL** workloads
vs: separate bills **for** ADF + Databricks + Synapse + PBI
Premium

4. POWER BI INTEGRATION (best **in** class)

Direct Lake mode = Power BI **reads** Parquet files directly
No import, **no** DirectQuery limitations
Fastest BI experience possible **on** **large** datasets

5. EASIER FOR ENTERPRISES

Works with Microsoft 365 identity (no new accounts)

Integrates with Teams, SharePoint, Excel

Familiar Microsoft ecosystem

6. COPILOT EVERYWHERE

AI-powered code generation in notebooks

Natural language to SQL in Warehouse

Report generation from data descriptions

LIMITATIONS (be honest!):

- ✗ Still maturing (launched 2023 – newer than Databricks)
- ✗ Less ML capabilities than Databricks
- ✗ Unity Catalog-equivalent still developing
- ✗ Some features only in specific regions
- ✗ Vendor lock-in to Microsoft ecosystem
- ✗ Pricing can be complex at scale

3 Fabric Components — Deep Dive

◆ OneLake

OneLake = The FOUNDATION of everything in Fabric

Think of OneLake like Google Drive for your data:

- One place to store ALL data
- Everyone accesses same files
- No "my copy" and "your copy"
- Organized in folders (workspaces/lakehouses)

TECHNICAL REALITY:

OneLake **is** built **on** ADLS Gen2 under the hood
Microsoft manages it (**no** storage account **to create!**)
Automatically geo-redundant

ONELAKE ADDRESS FORMAT:

Every item **in** Fabric has an ADLS-compatible path:

onelake://<workspace>/<item>.Lakehouse/Files/<path>

ADLS path (**for** Spark access):

abfss://<workspace>@onelake.dfs.fabric.microsoft.com/<item>.Lakehouse

ONELAKE HIERARCHY:

OneLake (tenant level – **ONE per** Microsoft tenant)
└─ Workspace (**like** a container/folder)
 └─ Lakehouse / Warehouse / etc.
 └─ Tables/ (managed Delta tables)
 └─ Files/ (raw files – **any** format)

KEY INSIGHT:

ALL Fabric experiences (Lakehouse, Warehouse, Spark, PBI)
read **from** the SAME OneLake files.

Example:

Spark writes Parquet files → OneLake/Lakehouse/Tables/sales/
Warehouse **SQL reads from** → same OneLake/Lakehouse/Tables/sales/
Power BI Direct Lake **reads from** → same files!

NO DATA MOVEMENT. **ONE** TRUTH.

◆ Real-Time Intelligence

Real-Time Intelligence = Fabric's *streaming analytics engine*

COMPONENTS:

EventStream → capture streaming events

Eventhouse → store **and** query streaming data (KQL database)

KQL Database → time-series optimized database

Activators → trigger actions **on** data patterns

REAL WORLD USE **CASE for** RetailMart:

Payment transactions arrive every millisecond

→ EventStream captures **each** payment

→ Stored **in** Eventhouse (KQL database)

→ KQL queries analyze **in** real-time

→ Activator fires alert **if** fraud detected

All within SECONDS **of** payment occurring!

This replaces:

Azure **Event** Hubs + Stream Analytics + Azure Functions

= **3** tools → now **1** experience **in** Fabric

◆ Data Factory in Microsoft Fabric

Data Factory **in** Fabric = same **as** Azure Data Factory
but BUILT **INTO** Fabric workspace

KEY DIFFERENCE **from** standalone ADF:

Feature	Standalone ADF	Fabric Data Factory
---------	----------------	---------------------

Pipelines	Same	Same (identical!)
Dataflows	Gen 1, Gen 2	Gen 2 only
Connectors	90+	90+ (same)
Destination	Any	Native Fabric items
Billing	Per execution	Part of Fabric capacity
Git integration	Azure DevOps/GH	Azure DevOps/GH

ADVANTAGE of Fabric Data Factory:

- Native destination to Lakehouse, Warehouse
- No linked service setup for Fabric items
- Billed from Fabric capacity (no separate meter)
- Dataflow Gen2 = massive improvement over Gen1

DATAFLOW GEN2 (new concept in Fabric):

- Power Query (M language) based transformation
- Like ADF Data Flows but uses Power Query engine
- 200+ connectors (more than ADF!)
- Output directly to Lakehouse tables
- Great for: non-coders who know Excel Power Query
- Not great for: complex Spark-level transformations

◆ Fabric Data Science

Fabric Data Science = ML platform built into Fabric

COMPONENTS:

- ML Experiments → track model training runs
- ML Models → store and version models
- Notebooks → PySpark/Python for ML code

WORKFLOW:

Data **in** Lakehouse (**from** Data Engineering team)

↓ (no data movement needed – Shortcuts!)

Data Scientist opens Notebook

- Reads directly **from** Lakehouse tables
- Trains ML model (scikit-learn, PyTorch, etc.)
- Logs experiment **to** MLflow (built-**in**)
- Registers model
- Deploys **for** batch scoring
- Results written back **to** Lakehouse

RETAILMART USE **CASE**:

Fraud Detection Model:

- Training data: **2** years **of** transaction history
- Features: amount, time, location, device
- Model: XGBoost **binary** classifier
- Output: fraud_probability score per transaction
- Batch scoring: every night **on next** day's *transactions*
- Results **in** Lakehouse → Power BI shows fraud dashboard

◆ Fabric Data Engineering

Fabric Data Engineering = Spark-based **data** transformation

MAIN ITEMS:

- Lakehouse → storage + processing (our main focus!)
- Notebook → PySpark/SQL code
- Spark Job Definition → productionize Python scripts
- Data Pipeline → orchestrate notebooks

This **is** what Data Engineers use MOST **in** Fabric.
Think of it **as** "Databricks-lite" inside Fabric.

◆ Data Activator

Data Activator = Alert **and** automation engine

Think **of** Data Activator **as**:

"If this happens in my data → do this automatically"

Examples:

IF sales **drop** below Rs.**1,00,000** **by** 3 PM
→ Send Teams message **to** regional manager

IF fraud score > **0.9** **for** a transaction
→ Block the payment + alert security team

IF inventory falls below safety stock
→ **Create** purchase **order in** SAP automatically

WORKS **WITH**:

- **Real-time** data (EventStream)
- Power BI data (**from** dashboards!)
- Lakehouse data

This replaces:

Azure Logic Apps + Power Automate **for** data triggers

◆ Fabric Data Warehouse

Fabric Data Warehouse = Enterprise SQL analytics engine

DIFFERENT FROM SYNAPSE DEDICATED POOL:

Synapse Dedicated Pool:

- You provision DWUs (compute always running)
- Pay even when idle!
- MPP with 60 distributions
- Need to design distribution strategy (Hash/RR/Replicated)
- More complex to manage

Fabric Data Warehouse:

- Serverless compute (Microsoft manages it)
- Pay per usage (from Fabric capacity)
- Standard SQL (T-SQL)
- No distribution strategy needed!
- Simpler, automatic optimization
- Still works on OneLake (Delta Parquet files underneath)

KEY FEATURE – Cross-database querying:

SELECT

s.sale_id,
s.total_amount,
c.customer_name

FROM [RetailMart_Warehouse].[sales].[fact_sales] s

JOIN [RetailMart_Lakehouse].[dbo].[dim_customers] c ←

different item!

ON s.customer_id = c.customer_id

WAREHOUSE and LAKEHOUSE share OneLake

- Can join across them without data movement!

POWER BI INTEGRATION:

- Warehouse auto-creates a **SQL** Analytics Endpoint
- Power BI connects **to** this endpoint
- Semantic models built **on** Warehouse tables
- Direct Lake mode: BI **reads** parquet files directly!

◆ Power BI in Fabric

Power BI = Microsoft's *BI tool (you already know it!)*

NEW **in** Fabric – DIRECT LAKE MODE:

Old Power BI modes:

- Import: copy data **to** PBI memory (fast queries, data can be stale)
- DirectQuery: query source **each** time (always fresh, but slow)

New Direct Lake mode:

- Power BI reads Parquet files DIRECTLY **from** OneLake
- Speed **of** Import (reads columnar Parquet natively)
- Freshness **of** DirectQuery (always reading latest files)
- Best **of** both worlds!

HOW: **When** Spark writes Delta tables **to** Lakehouse

- they're *Parquet files in OneLake*
- Power BI Direct Lake reads those files
- No intermediary needed!

Think **of** Direct Lake **as**:

Power BI got a "**superpower**" **to** read big data files directly without importing them

Like reading a book through the shelf
instead of checking it out and taking home!

4 Fabric Account Creation — Step by Step

◆ How to Sign Up for Microsoft Fabric

OPTION 1: Microsoft Fabric Trial (RECOMMENDED for learning)

Step 1: Create Microsoft account

- Go to outlook.com → Create account
- Use: yourname_fabric_learning@outlook.com

Step 2: Go to Microsoft Fabric

- Open: app.fabric.microsoft.com
- Sign in with your new account

Step 3: Start Trial

- Top right → Settings (gear icon)
- "Start Trial" button
- Trial: 60 days FREE
- Capacity: F64 (64 Capacity Units)

Step 4: Verify Trial

- Settings → Admin Portal → Capacity Settings
- Should see: "Trial-<your_name>" capacity listed

OPTION 2: Use existing Power BI Premium/Pro account

- If your company has Power BI Premium
- Admin enables Fabric in tenant settings
- You get access automatically

OPTION 3: Azure + Fabric

- Buy Fabric capacity from Azure Portal
- Create resource: Microsoft Fabric
- Choose SKU: F2 (\$263/month) to F2048 (massive)

FOR THIS GUIDE: We'll use Fabric Trial (free!)

◆ How to Activate Microsoft Fabric Trial

DETAILED ACTIVATION STEPS:

1. Navigate to: `app.fabric.microsoft.com`
2. If you see "Fabric is disabled":
 - You need admin to enable it
 - OR: switch to personal trial account
3. Click your profile picture (top right)
4. Look for: "Start trial" option
 - Click it
 - Read trial terms
 - Click "Activate" / "Start trial"
5. What you get in trial:

FABRIC TRIAL INCLUDES:
→ F64 Capacity (64 CUs)
→ All Fabric workloads enabled
→ OneLake storage (up to 1TB)

	→ 60 days free	
	→ No credit card required!	

6. After activation → you'll see Fabric home page

→ Create workspace

→ Start building!

WHAT IS A CU (Capacity Unit)?

CU = unit of compute power in Fabric

F64 = 64 CUs = comfortable for learning + small projects

Compute automatically scales within your CU limit

Think of CUs like "compute budget"

Heavy Spark job uses more CUs temporarily

Light SQL query uses fewer CUs

Always within your allocated limit

◆ Creating and Managing Workspaces

WORKSPACE = Your project container in Fabric

CREATE WORKSPACE:

Fabric Home → Workspaces (left sidebar)

→ + New Workspace

Name: RetailMart_DataEngineering

Description: Data engineering workspace for RetailMart project

Advanced settings:

License mode: Trial ← (or Fabric capacity if available)

→ Apply → Workspace created!

WORKSPACE SETTINGS:

Workspace → Settings (gear icon)

General:

- Name, description
- Contact list (who to email on issues)

License:

- Change capacity (move between trial/paid)

Git Integration:

- Connect to Azure DevOps or GitHub
- Version control all Fabric items!

Spark Settings:

- Configure Spark properties for notebooks
- Set default Spark pool size

OneLake Settings:

- Data residency region

Permissions:

- Add members with specific roles
- Admin/Member/Contributor/Viewer

WORKSPACE TYPES based on license:

- Trial:** F64 capacity, 60 days, free
- Pro:** Power BI Pro license, limited Fabric features
- Premium:** Power BI Premium, more features
- Fabric:** Full Fabric capacity, all features

PRO TIP:

Create SEPARATE workspaces for:

- ✓ Development (build and test)
- ✓ Testing (QA and UAT)
- ✓ Production (live pipelines)

This is proper DevOps/SDLC practice!

◆ Workspace Configurations and Settings

```
# WORKSPACE CONFIGURATION BEST PRACTICES
```

```
# Naming Convention (follow this always!):
```

```
workspace_naming = {  
    "pattern": "<Company>_<Domain>_<Environment>",  
    "examples": [  
        "RetailMart_Sales_Dev",  
        "RetailMart_Sales_Test",  
        "RetailMart_Sales_Prod",  
        "RetailMart_Finance_Prod",  
        "RetailMart_HR_Dev"  
    ]  
}
```

```
# Items inside workspace also follow naming:
```

```
items = {  
    "Lakehouse": "LH_<Domain>_<Env> → LH_Sales_Prod",  
    "Warehouse": "WH_<Domain>_<Env> → WH_Analytics_Prod",  
    "Pipeline": "PL_<Purpose> → PL_Daily_SalesLoad",  
    "Notebook": "NB_<Purpose> → NB_Transform_Sales",  
    "Dataset": "DS_<Domain> → DS_Sales_Reporting"  
}
```

WORKSPACE SETTINGS WALKTHROUGH:

General Settings:

- Contact email list

Add: data-engineering-team@retailmart.com

This email gets notifications about workspace issues

- OneDrive folder (optional)

Link to SharePoint for document storage

Git Integration (critical for production!):

- Azure DevOps:

Organization: retailmart-devops

Project: DataPlatform

Repository: fabric-items

Branch: main

Folder: /RetailMart_Sales/

- After connecting: ALL items saved to Git automatically!

- Changes tracked → pull requests → code review → merge

Spark Settings:

- Default pool size: Medium (4 nodes)

- Spark version: 3.4

- Default libraries: requirements.txt from Git

Security:

- Workspace identity (Managed Identity for Fabric)

- Used to access Key Vault, ADLS, SQL DB

- Same concept as ADF Managed Identity

◆ How to Manage Capacity Units

CAPACITY UNITS (CUs) = Compute currency of Fabric

F SKUs and their CUs:

SKU	CUs	Price/month	Good for
F2	2	~\$263	Basic testing
F4	4	~\$526	Small team
F8	8	~\$1,053	Small org
F16	16	~\$2,105	Medium org
F32	32	~\$4,210	Larger workloads
F64	64	~\$8,420	Medium-large org
F128	128	~\$16,840	Large org
F2048	2048	~\$269,466	Very large enterprise

THROTTLING (important to understand!):

Each CU allows certain operations per minute

F64 = 64 × 30 seconds = burst capacity

If you use too many CUs at once:

- Fabric THROTTLES (slows down) remaining requests
- Does NOT fail – just queues and slows
- Like traffic jam: requests wait, don't crash

MONITOR CAPACITY:

Admin Portal → Capacity Metrics app

- See CU usage over time
- Identify peak usage periods
- Find which workspaces using most CUs

OPTIMIZE CU USAGE:

- ✓ Schedule heavy jobs in off-peak hours

- ✓ Use Spark pause when idle
- ✓ Optimize notebooks (less data reads = fewer CUs)
- ✓ Use Direct Lake (avoids import processing CUs)
- ✓ Share capacity across multiple workspaces

5 OneLake — Deep Dive

◆ Unified Storage Layer

OneLake = The single storage for ALL of Fabric

BEFORE OneLake (the problem):

Team A creates Azure Storage Account → uploads data

Team B creates another Azure Storage Account → uploads same data

Team C has SQL Server → loads same data again

3 storage accounts, 3 copies, 3 bills, 3 truths!

WITH OneLake (the solution):

ONE storage, automatically created for your tenant

Every workspace, every lakehouse → same OneLake!

STRUCTURE:

OneLake (1 per Azure region per tenant)

└─ <WorkspaceName>

 └─ <LakehouseName>.Lakehouse

 └─ Tables/ (Delta Parquet – queryable via

SQL!)

 └─ Files/ (raw files – any format)

└─ <WarehouseName>.Warehouse


```
|      └─ (SQL tables – stored as Delta in OneLake)
└─ <DatasetName>.Dataset
    └─ (semantic model metadata)
```

ACCESS PATTERNS:

From Notebooks:

```
abfss://RetailMart_DataEngineering@onelake.dfs.fabric.microsoft.com/
```

From Fabric Items (use relative paths):

```
Files/raw/sales/2024/06/sales.parquet
```

From Desktop (OneLake Explorer app):

Windows File Explorer → OneLake → browse like local files!

OneLake Explorer (hidden gem!):

- Install from Microsoft Store
- Mounts OneLake as local drive
- Browse Fabric data like Windows folder!
- Drag and drop files to upload
- Perfect for small file uploads during development

◆ Logical Data Organization

HOW TO ORGANIZE DATA IN ONELAKE:

RECOMMENDED LAKEHOUSE STRUCTURE:

RetailMart.Lakehouse

└─ Tables/	← MANAGED (queryable via SQL!)
└─ bronze_sales	(Delta table from raw load)
└─ silver_sales	(Delta table from transformation)

```

|   |— gold_store_performance (Delta table, ready for BI)
|   |— dim_customers          (dimension table)
|   |— dim_products           (dimension table)
|
|— Files/                      ← UNMANAGED (raw files)
    |— raw/
        |— sales/
            |— 2024/
                |— 01/
                    |— sales_20240101.csv
                |— 06/
                    |— sales_20240615.parquet
            |— customers/
                |— customer_master.csv
        |— config/
            |— store_mapping.xlsx
    |— archive/
        |— processed/

```

KEY RULE:

Tables/ → use **for** transformed, queryable data
 Automatically gets **SQL** endpoint!
 Power BI can **connect** directly

Files/ → use **for** raw ingestion
Not automatically queryable via **SQL**
 Access via Spark notebooks **only**

◆ Delta Tables & Open Data Formats

OneLake stores data **in OPEN** formats **only**:

DELTA PARQUET = Primary format for Tables/

- Delta = Parquet + Transaction log
- ACID transactions
- Time travel
- Schema evolution
- Same format Databricks uses!

This means:

Databricks writes Delta → OneLake reads it (compatible!)

Fabric writes Delta → Databricks can read it!

OPEN STANDARD → no vendor lock-in on format

SUPPORTED FILE FORMATS in Files/:

- CSV (any delimiter)
- Parquet (columnar – recommended!)
- JSON / NDJSON (newline-delimited)
- Avro (streaming)
- ORC (Hive-compatible)
- Excel (.xlsx)
- Images, PDFs, any binary (for ML)
- Delta (in Files too, not just Tables)

WHY OPEN FORMATS MATTER:

Microsoft Fabric could go away tomorrow
(unlikely, but hypothetically)

Your data is still in Delta Parquet files

- Move to Databricks: works immediately
- Move to Synapse: works immediately
- Move to BigQuery: works (with conversion)

You own your data. Microsoft just runs the platform.

That's the promise of open formats.

◆ Shortcuts — The Game Changer

SHORTCUTS = Virtual links **to** data without copying it

This **is** THE most important OneLake concept!

WHAT **IS** A SHORTCUT?

Like a symbolic link (symlink) **in** Linux

Like a shortcut file **in** Windows (.lnk)

Points **to** data somewhere **else**

Appears **as if** data **is** local

But ZERO data copied!

WHERE CAN SHORTCUTS POINT **TO**?

- Another OneLake location (different workspace)
- Azure Data Lake Storage Gen2
- Azure Blob Storage
- Amazon S3 (AWS!)
- Google Cloud Storage (GCS!)
- Dataverse (Microsoft Power Platform)

RETAILMART EXAMPLE:

Data Engineering team has Gold data:

LH_Sales_Prod.Lakehouse/Tables/gold_sales

Data Science team wants **to** use same data:

LH_DataScience.Lakehouse/Tables/gold_sales ← SHORTCUT!

- Points **to** Data Engineering's *gold_sales table*
- Data Scientists query it **as if** it's *their own*
- ANY changes DE makes → immediately visible **to** DS

- ZERO storage cost **for** DS team!
- ZERO ETL pipeline **to** sync data!

BI team also shortcuts:

- WH_Analytics.Warehouse/gold_sales ← SHORTCUT **to** Lakehouse!
- Warehouse gets SQL access **to** Lakehouse tables
- Power BI connects **to** Warehouse
- Reports always fresh (no data movement!)

CREATE SHORTCUT:

- Lakehouse → Files/ **or** Tables/ → right-click
- **New** Shortcut
- Choose: OneLake / ADLS / S3
- Provide path **and** credentials
- Shortcut appears instantly!

SHORTCUT **TO** ADLS GEN2 (existing data):

- Bring your existing ADLS data **into** Fabric
- WITHOUT moving **or** copying anything
- Fabric sees it **as** native OneLake data
- Use Spark notebooks **to** process it
- Write results back **to** Lakehouse Tables

PERFECT **for**: migrating **FROM** Databricks/Synapse **TO** Fabric

Keep data **in** ADLS → create shortcut **in** Fabric

- Gradual migration without "big bang" cutover!

◆ Security & Access Control in OneLake

ONELAKE SECURITY LEVELS:

Level 1: Workspace Level

- Admin / Member / Contributor / Viewer roles
- Controls access to ALL items in workspace
- Coarse-grained (all or nothing)

Level 2: Item Level

- Grant access to specific Lakehouse/Warehouse
- Without giving workspace access
- "ReadAll" permission → read all data in item

Level 3: Table Level (SQL Endpoint)

- GRANT SELECT ON TABLE to specific user/group
- Available through SQL analytics endpoint
- T-SQL standard GRANT/REVOKE

Level 4: Column/Row Level (Warehouse)

- Column security: hide specific columns
- Row-level security: users see only their data

Level 5: OneLake Data Access Control

- Fine-grained access to specific folders/files
- Define rules: "Finance team can only read /finance/ folder"
- Uses Azure AD groups

WORKSPACE IDENTITY (Managed Identity):

- Each workspace gets its own Managed Identity
- Use this to access external resources (Key Vault, ADLS)
- Same concept as ADF Managed Identity
- No passwords needed for workspace-to-resource auth

PRACTICAL SECURITY SETUP for RetailMart:

Data Engineering workspace: DE team (Admin), Lead (Admin)

Data Science workspace: DS team (Member), DE team (Contributor)

Analytics workspace: BI team (Member), DS team (Viewer)

Shortcuts: DS accesses DE data via shortcut
→ DS team has read-only access via shortcut
→ Cannot modify DE's actual data
→ Perfect separation of concerns!

◆ Performance & Cost Optimization in OneLake

PERFORMANCE TIPS:

1. USE DELTA FORMAT for Tables/

- Columnar = faster analytics
- Parquet compression = 70% less storage
- Partition pruning = read less data

2. PARTITION CORRECTLY

- Large tables: partition by date (year/month/day)
- BI queries filter by date = massive speed boost

3. OPTIMIZE DELTA TABLES regularly

In Lakehouse notebook:

```
%%sql
```

```
OPTIMIZE sales_delta ZORDER BY (customer_id, sale_date);  
-- Compacts small files + z-orders for faster lookups
```

4. VACUUM old files

```
%%sql
```

```
VACUUM sales_delta RETAIN 168 HOURS; -- keep 7 days history  
-- Removes old data files (but keeps 7 days for time travel)
```

5. V-ORDER (Fabric specific!)

Microsoft's proprietary optimization for Parquet files

Enables faster Power BI Direct Lake reads

Enable V-ORDER when writing:

```
spark.conf.set("spark.sql.parquet.vorder.enabled", "true")
df.write.format("delta").save("Tables/sales")
-- Files written with V-ORDER = 2-3x faster PBI queries!
```

COST OPTIMIZATION:

1. PAUSE SPARK when not using

Spark pool: set idle timeout to 5-10 minutes
→ Auto-pauses = zero CU consumption when idle

2. USE SERVERLESS SQL for exploration

→ No Spark startup time
→ Fewer CUs consumed
→ Great for ad-hoc queries

3. SCHEDULE HEAVY JOBS off-peak

→ CU throttling less likely at 3 AM vs 9 AM

4. COMPACT SMALL FILES

Many small files = many reads = more CUs
Regularly compact: OPTIMIZE command

5. SHORTCUTS vs COPY

Use shortcuts → no storage cost
Never copy data if shortcut works!

6 Data Factory in Microsoft Fabric — Deep Dive

◆ Unified Data Integration Platform

DATA FACTORY **IN** FABRIC = ADF built **into** Fabric

Two ways **to** ingest data **in** Fabric:

WAY **1**: PIPELINES (familiar **from** ADF)

- Same **Copy** Activity, ForEach, If **Condition**
- **200+** connectors
- Best **for**: complex orchestration, **large** data movement
- Code optional (visual drag-**and-drop**)

WAY **2**: DATAFLOW GEN2 (**new!**)

- Power Query M **language** based
- **300+** connectors (even more than ADF!)
- Best **for**: data transformation **without** code
- Perfect **for**: analysts who know Excel Power Query
- Output directly **to** Lakehouse tables!

CREATE PIPELINE **in** Fabric:

Workspace → + **New** → Data Pipeline

- Exactly same UI **as** ADF Studio
- Same activities (**Copy**, ForEach, If, Notebook, etc.)
- Native destination: Lakehouse, Warehouse
- **No** linked service needed **for** Fabric items!

Example: **Copy from** Oracle **to** Lakehouse

- **Create** Linked Service **for** Oracle (same **as** ADF)
- Source: Oracle **table**
- Sink: Lakehouse **Table** (just **select**, **no** connection setup!)
- Run pipeline → data **in** Lakehouse instantly!

◆ Pipeline Activities & Dataflows in Fabric

ALL ADF ACTIVITIES WORK IN FABRIC PIPELINES:

- ✓ Copy Data (same as ADF)
- ✓ ForEach (same)
- ✓ If Condition (same)
- ✓ Lookup (same)
- ✓ GetMetadata (same)
- ✓ Delete (same)
- ✓ Wait (same)
- ✓ Set Variable (same)
- ✓ Execute Pipeline (same)
- ✓ Notebook Activity ← runs Fabric notebooks

NEW in Fabric Pipelines:

Invoke Pipeline → cleaner way to call child pipelines
Office 365 Outlook → send emails natively!
Teams → send Teams messages natively!
(No Logic Apps needed for simple notifications!)

DATAFLOW GEN2 – Complete Walkthrough:

Create Dataflow Gen2:

Workspace → + New → Dataflow Gen2

INTERFACE:

- Power Query Editor opens (like in Power BI Desktop)
- Left panel: Queries (your data tables)
- Main area: Column-level transformations
- Right panel: Query settings, steps

SCENARIO: Load customer CSV → transform → Lakehouse table

Step 1: Get Data

- Home → Get Data → Search "Azure Blob"
- Enter connection details
- Navigate to: /bronze/customers/customer_master.csv
- Preview data shows in editor

Step 2: Transform

- Promote first row as headers
- Change column types (click column header → type)
- Remove nulls: Filter Rows → keep non-null customer_id
- Rename columns: right-click → Rename
- Add custom column: Add Column → Custom Column
Formula: [TotalPurchases] * 0.18 (calculate GST)

Step 3: Configure Destination

- Home → Add Data Destination → Lakehouse
- Select: RetailMart.Lakehouse
- Table name: silver_customers
- Update method: Replace (overwrite) or Upsert (merge)

Step 4: Publish and Refresh

- Home → Publish
- Refresh Now (or set schedule)
- Data flows to Lakehouse table!

DATAFLOW GEN2 vs NOTEBOOK:

Dataflow: better for non-coders, simple transformations

Notebook: better for complex logic, ML, custom algorithms

Use BOTH: Dataflow for ingestion, Notebook for heavy transforms

◆ Connectivity & Data Ingestion

FABRIC CONNECTIONS (the new concept):

In ADF: Linked Services (per pipeline/workspace)

In Fabric: CONNECTIONS (tenant-level, reusable)

Create Connection:

Settings → Manage Connections → + New Connection

OR during pipeline/dataflow creation when you
configure a source → auto-prompted to create connection

Connection types:

- Cloud connections (Azure resources)
 - No IR needed, automatic
- On-premise connections
 - Requires Fabric On-premises Data Gateway
 - Similar to Self-Hosted IR in ADF
 - Install gateway on corporate server
 - Register with Fabric tenant
 - Now Fabric can reach on-prem SQL Server!

ON-PREMISES DATA GATEWAY (replaces SHIR):

Download: Microsoft On-premises Data Gateway

Install on: Windows server inside corporate network

Register with: Fabric (or Power BI) tenant

Use for: Pipelines, Dataflows accessing on-prem sources

Same concept as SHIR, slightly different technology

SHIR: ADF-specific

Gateway: Fabric + Power BI (broader use)

INGESTION PATTERNS in Fabric:

Pattern 1: Pipeline **Copy** → Lakehouse Files

Source → **Copy** Activity → LH Files/raw/

Then: Notebook transforms Files → Tables

Use **for:** complex sources, **large** data

Pattern 2: Dataflow Gen2 → Lakehouse **Table**

Source → Power Query transformations → LH Tables/

Use **for:** simple ingestion + transformation **in one** step

Pattern 3: Notebook → Lakehouse **Table**

Source → Spark read → transform → write Delta **table**

Use **for:** complex logic, ML features

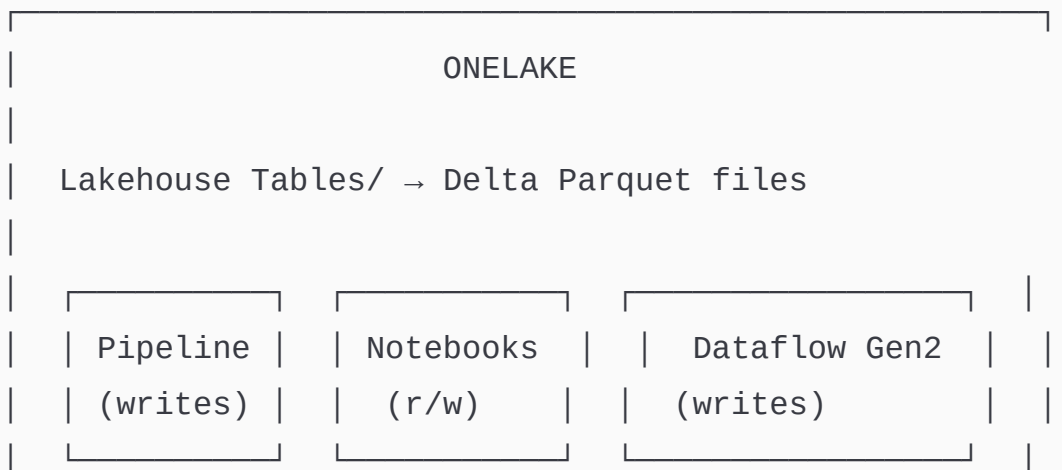
Pattern 4: Shortcut → Existing ADLS data

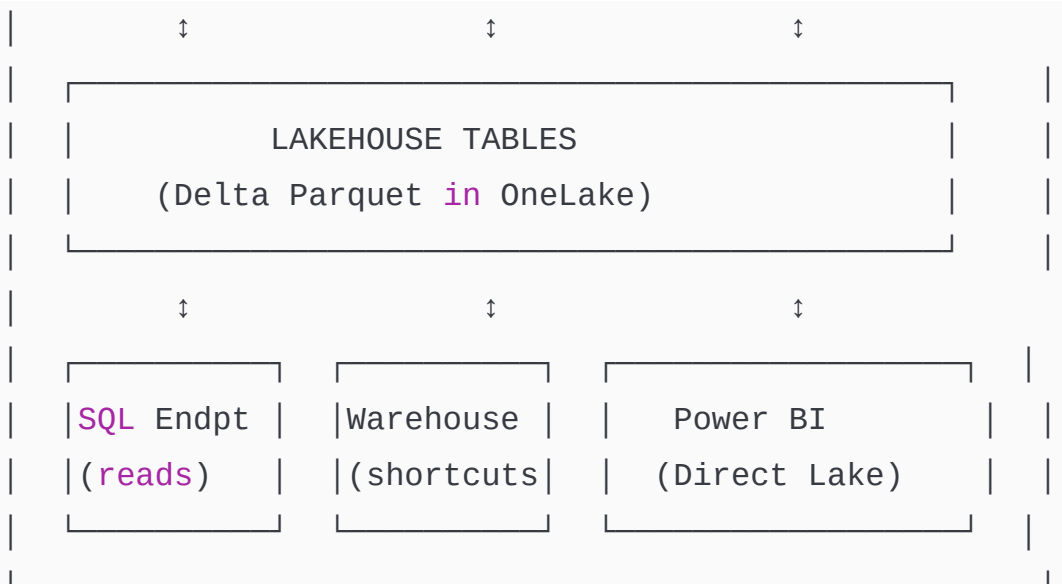
No ingestion needed! Just **create** shortcut

Use **for:** data already **in** Azure storage

◆ Integration with OneLake & Fabric Workloads

HOW **ALL** FABRIC ITEMS SHARE DATA:





ZERO DATA MOVEMENT **between** boxes above!

All reading SAME underlying Parquet files!

AUTOMATIC **SQL** ENDPOINT:

When you **create** a Lakehouse

- Fabric AUTOMATICALLY creates a **SQL** Analytics Endpoint
- This **is** a read-only T-**SQL** interface **to** your tables!
- **No** setup needed!
- Connection string provided automatically

Connect Power BI **to** **SQL** Endpoint:

Power BI Desktop → **Get** Data → Azure → Synapse Analytics (**SQL** DW)

Server: <workspace>.datawarehouse.fabric.microsoft.com

Database: LH_Sales_Prod (your lakehouse name)

- **All** Tables/ **show** up as **SQL** tables!
- Build reports **on** Lakehouse data **with** standard T-**SQL**!

◆ Data Transformation & Orchestration

ORCHESTRATION PATTERN IN FABRIC:

Master Pipeline (PL_RetailMart_Daily_ETL)

- |
- |— **Execute** Pipeline: PL_Ingest_Oracle_Sales
 - | → **Copy** Activity (Oracle → Lakehouse Files)
- |
- |— **Execute** Pipeline: PL_Ingest_SQL_Customers
 - | → **Copy** Activity (SQL Server → Lakehouse Files)
- |
- |— Notebook Activity: NB_Transform_Bronze_to_Silver
 - | → Spark: clean, validate, write **to** Tables/silver
- |
- |— Notebook Activity: NB_Build_Gold_Layer
 - | → Spark: aggregate, **join** dims, write **to** Tables/gold
- |
- |— Dataflow Gen2 Activity: DF_StoreTargets
 - | → Excel targets → Lakehouse **Table** (simple mapping)
- |
- |— Send Email (built-in Office 365 activity!)
 - | → "Daily ETL Complete: 142,850 rows processed"
 - | → **No** Logic Apps needed **for** simple emails!

SCHEDULING:

- Workspace → your pipeline → Schedule
- Daily **at** 1:00 AM IST
- **OR: trigger** via EventStream (event-driven)
- **OR: trigger from** another Fabric item

MONITORING:

- Workspace → your pipeline → Run History
- See **all** runs: success, failure, duration

→ Click **any** run → see **each** activity
→ Failed activity → red X → click **for** error details

7 Fabric Data Engineering — Lakehouse

◆ Data Lake vs Data Lakehouse

LET ME EXPLAIN THIS **WITH** A STORY:

DATA LAKE (ADLS Gen2 **with** Parquet files):

"Imagine a huge warehouse where you dump everything.
Files piled everywhere. Some labeled, some not.
You know data is in there... somewhere.
To find anything, you need to search through piles.
No guarantee data is correct or not duplicated.
Anyone can dump anything – no quality control."

Good: stores everything cheaply

Bad: **no** ACID, **no** UPDATE/DELETE, **no** schema enforcement
querying **is** slow **and** painful

"Data Swamp" – nickname **for** bad data lakes!

DATA WAREHOUSE (**SQL** Server / Synapse Dedicated):

"Imagine a perfectly organized library.
Every book catalogued, indexed, easy to find.
But: you can only store books (structured data).
No room for videos, images, audio.
Very expensive per GB.
Must know book category before acquiring it (schema-first)."

Good: fast **SQL** queries, organized, ACID

Bad: expensive, structured **only**, rigid schema

DATA LAKEHOUSE (Fabric Lakehouse):

"Best of both worlds:

Library organization (fast queries, ACID, schema)

Warehouse storage scale (cheap, all formats)

Store raw files AND structured tables

SQL queries work on Delta tables

Spark processes raw files

Power BI connects directly

All on SAME storage (OneLake)"

Good: everything! Cheap storage + fast queries + ACID

Challenge: newer concept, teams still learning

FABRIC LAKEHOUSE = the modern answer **to** this evolution

◆ Understanding the Fabric Lakehouse

FABRIC LAKEHOUSE ANATOMY:

PHYSICAL REALITY:

A Lakehouse = OneLake folder **with** special structure

LH_Sales_Prod.Lakehouse/

— Tables/	← Managed Delta tables (queryable!)
— sales/	← sales Delta table

```
|      |— _delta_log/    ← transaction log
|      |— part-0001.parquet
|      |— part-0002.parquet
└─ Files/                  ← Unmanaged raw files
    └─ raw/
        └─ sales_20240615.csv
```

LOGICAL REALITY (what you see in Fabric Studio):

LH_Sales_Prod

```
|— 📊 Tables (queryable via SQL automatically!)
|   |— bronze_sales
|   |— silver_sales
|   └─ gold_store_performance
└─ 📁 Files (raw file storage)
    |— raw/
    |— archive/
    └─ config/
```

TWO PERSONAS of Lakehouse:

AS A LAKEHOUSE (Spark Notebook perspective):

- Use PySpark to read/write Files and Tables
- Full flexibility of Spark
- Access via abfss:// path

AS A DATA WAREHOUSE (SQL perspective):

- SQL Analytics Endpoint auto-created
- Query Tables/ using T-SQL
- Connect Power BI via ODBC/SQL endpoint
- Read-only SQL access (no INSERT/UPDATE via SQL)
- Writes happen through Spark or Dataflows

◆ Creating a Lakehouse

STEP BY STEP:

Step 1: Open your workspace

- app.fabric.microsoft.com
- Click your workspace: RetailMart_DataEngineering

Step 2: Create Lakehouse

- + New → Lakehouse
- Name: LH_Sales_Prod
- Create

Step 3: Explore the Lakehouse UI

LEFT PANEL:

- └─ LH_Sales_Prod (Lakehouse)
 - └─ Tables ← click to see managed tables
 - └─ Files ← click to browse files

TOP BAR:

- Lakehouse | SQL analytics endpoint | Semantic model
(switch between 3 views)

Step 4: Upload your first file

- Files → right-click → Upload → Upload files
- Drag and drop: sales_data.csv
- File appears in Files/

Step 5: Load file into table

- Right-click the CSV file in Files
- "Load to Tables" → New table
- Table name: bronze_sales
- Fabric runs Spark automatically → creates Delta table!

OR: Load via Notebook (more control)

Step 6: Query the table

Click "SQL analytics endpoint" (top toggle)

→ Tables/bronze_sales appears as SQL table

→ Click "New SQL query"

→ **SELECT** TOP 10 * **FROM** bronze_sales

→ Run → see results!

Step 7: Connect Power BI

→ Click "New semantic model" (top toggle)

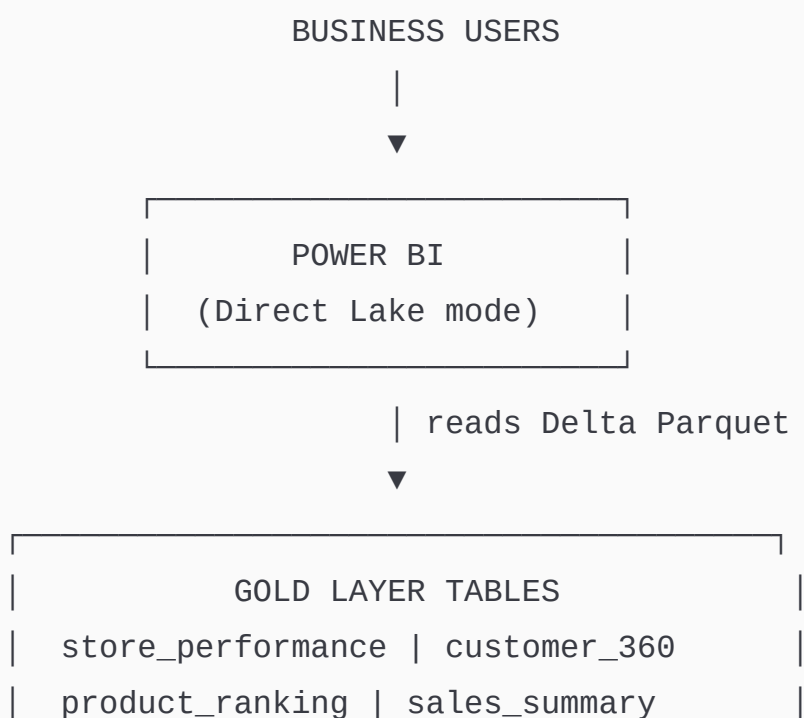
→ **Select** tables to include

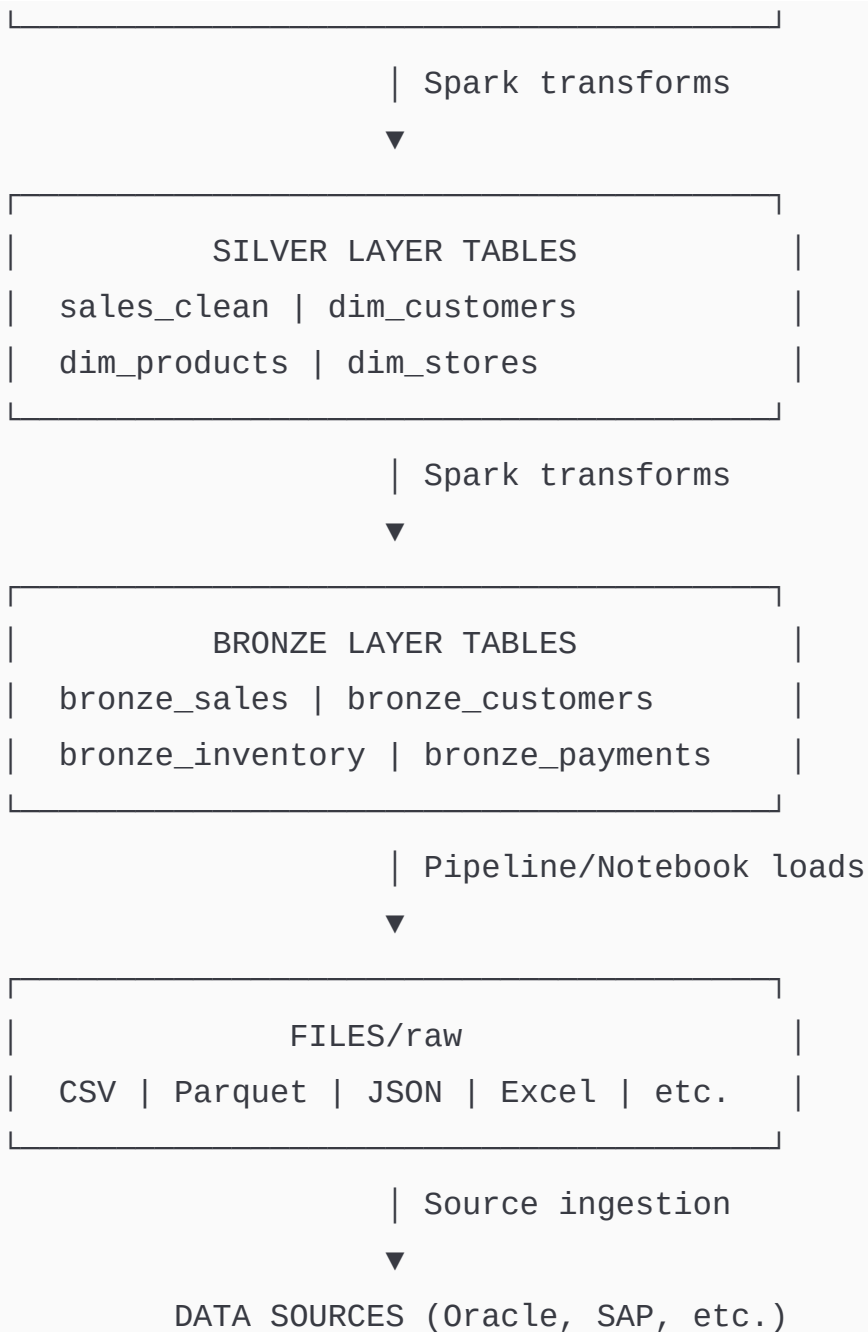
→ **Create** semantic model

→ **Open in** Power BI Desktop → build reports!

◆ Lakehouse Architecture

LAKEHOUSE ARCHITECTURE LAYERS:





◆ Exploring Lakehouse Features

LAKEHOUSE UI FEATURES:

1. TABLE PREVIEW:

Click [any table in Tables/](#)

- Preview opens showing sample data
- Column names, types, sample values
- Row count shown

2. TABLE MAINTENANCE:

- Right-click table → Maintenance
- OPTIMIZE: compact small files
- VACUUM: remove old Delta versions
- (same as Delta OPTIMIZE and VACUUM commands!)

3. SCHEMA VIEW:

- Tables → click table → Schema tab
- See column names and data types
- Perfect for documentation

4. PARTITION INFO:

- Tables → click table → Details tab
- See partition columns
- File count, total size

5. SHORTCUT MANAGEMENT:

- Files/ or Tables/ → right-click → New Shortcut
- Add shortcuts to external data

6. NOTEBOOK INTEGRATION:

- Open Notebook button (top right of Lakehouse)
- Opens notebook already connected to this Lakehouse!
- Default lakehouse set automatically
- No need to configure paths!

7. PIPELINE INTEGRATION:

- Get Data → New Pipeline
- Pipeline created with Lakehouse as destination

8. SQL ENDPOINT:

Toggle: Lakehouse → **SQL** analytics endpoint
→ **Full** T-**SQL** interface
→ Write queries, save them
→ Share query results

9. SEMANTIC MODEL:

Toggle: **SQL** endpoint → Semantic model
→ Power BI semantic model
→ **Add** measures (DAX)
→ **Define** relationships
→ **Open in** Power BI Desktop

◆ DevOps Repos and Deployment Pipelines in Fabric

VERSION CONTROL **FOR** FABRIC ITEMS:

CONNECT WORKSPACE **TO** GIT:

Workspace → Settings → Git Integration

Provider: Azure DevOps

Organization: retailmart-devops

Project: DataPlatform

Repository: fabric-items

Branch: dev ← development branch

Folder: /RetailMart_Sales/

→ Click Connect → Sync

NOW: All Fabric items saved **as** JSON/metadata **to** Git!

GIT SYNC PROCESS:

When you change a notebook:

- Icon shows "pending changes" in workspace
- Commit button appears
- Add commit message: "Add fraud detection logic"
- Commit → changes pushed to dev branch
- Raise PR to main → code review → merge → deploy!

WHAT GETS SAVED TO GIT:

- ✓ Notebooks (as .py or .ipynb files)
- ✓ Pipelines (as JSON)
- ✓ Dataflows (as JSON)
- ✓ Semantic models (as TMDL)
- ✗ Data (NOT versioned – only code/config)
- ✗ Lakehouse tables (these are data, not code)

DEPLOYMENT PIPELINES (Fabric's *built-in CD*):

Fabric has native deployment pipelines!

Workspace → Deployment Pipelines → Create Pipeline

Stages:

[Development] → [Test] → [Production]

Each stage = a separate workspace

Deploy button → copies all items from Dev → Test → Prod

This is like ARM template deployment in ADF

But simpler! No JSON templates to manage

WORKS FOR:

- Notebooks (code deploys, not data)
- Pipelines (configuration deploys)
- Dataflows (transformation logic deploys)
- Semantic models (report structure deploys)

ENVIRONMENT-SPECIFIC SETTINGS:

Each stage can override parameter values

Dev: connects to dev Lakehouse

Prod: connects to prod Lakehouse

→ Same code, different data connections!

8 Fabric Data Engineering — Notebooks

◆ Creating and Using Notebooks

FABRIC NOTEBOOK = Jupyter-style interactive computing

CREATE NOTEBOOK:

Option 1: Workspace → + New → Notebook

Option 2: From inside Lakehouse → Open Notebook

Option 3: Import .ipynb from your computer

NOTEBOOK INTERFACE:

Top Bar:

└─ Run All / Run Cell / Stop

└─ Language selector (PySpark default)

└─ + Code / + Markdown

└─ Attach: Select Lakehouse

└─ Save / Publish / Share

Left Panel:

└─ Lakehouse Explorer (browse tables/files)

└─ Environment (libraries)

└─ Resources (attach files)

Cell Types:

└─ Code (PySpark, SQL, Scala, SparkR)

└─ Markdown (documentation)

ATTACH LAKEHOUSE TO NOTEBOOK:

This **is** important! Must attach lakehouse **to** use relative paths **and SQL** queries **on** lakehouse tables.

- + Lakehouse button (**left** panel)
- **Select**: LH_Sales_Prod
- **Set as Default** Lakehouse (**check** the box)

Now you can use:

```
spark.read.table("bronze_sales") # no path needed!  
spark.sql("SELECT * FROM gold_store_performance") # works!
```

Instead **of**:

```
spark.read.parquet("abfss://...onelake.../Tables/bronze_sales")
```

NOTEBOOK EXPLORER (amazing feature!):

Left panel shows Lakehouse structure:

```
└─ LH_Sales_Prod  
  └─ Tables  
    └─ bronze_sales (right-click → query!)  
    └─ silver_sales  
  └─ Files  
    └─ raw/
```

Right-click **any table** → "Add to code"

→ Auto-generates read code **in** your cell!

◆ Developing and Running Notebooks

```

# — COMPLETE FABRIC NOTEBOOK – RETAILMART TRANSFORMATION —————

# — CELL 1: Documentation (Markdown) —————
"""
# Sales Transformation Notebook
**Workspace:** RetailMart_DataEngineering
**Lakehouse:** LH_Sales_Prod
**Purpose:** Bronze → Silver transformation for sales data
**Author:** Data Engineering Team
**Schedule:** Daily via Pipeline at 1:00 AM IST
**Parameters:** process_date (yyyy-MM-dd)
"""

# — CELL 2: Parameters (Parameter Cell) —————
# Right-click cell → Toggle Parameter Cell
# This makes it a parameter cell (filled by pipeline)
process_date = "2024-06-15"
load_type = "INCREMENTAL"
source_lakehouse = "LH_Sales_Prod"

# — CELL 3: Imports —————
from pyspark.sql import functions as F
from pyspark.sql.types import *
from pyspark.sql.window import Window
from delta.tables import DeltaTable
import logging

logging.basicConfig(level=logging.INFO)
logger = logging.getLogger(__name__)

logger.info(f"Starting transformation for: {process_date}")

# — CELL 4: Read Bronze Data —————
# Using default lakehouse – no abfss path needed!
df_bronze = spark.sql(f"""

```

```

        SELECT * FROM bronze_sales
        WHERE _process_date = '{process_date}'
        """)

# OR using PySpark API:
df_bronze = spark.read.table("bronze_sales") \
    .filter(F.col("_process_date") == process_date)

row_count = df_bronze.count()
logger.info(f"Bronze records read: {row_count:,}")

if row_count == 0:
    logger.warning("No data found for this date – exiting")
    mssparkutils.notebook.exit(f"NO_DATA:{process_date}")

# — CELL 5: Data Quality Checks —————
# Track quality metrics
quality_metrics = {}

# Check 1: Null sale_id
null_sale_ids =
df_bronze.filter(F.col("SALE_ID").isNull()).count()
quality_metrics["null_sale_ids"] = null_sale_ids

# Check 2: Invalid amounts
invalid_amounts = df_bronze.filter(F.col("AMOUNT") <= 0).count()
quality_metrics["invalid_amounts"] = invalid_amounts

# Check 3: Future dates
future_dates = df_bronze.filter(
    F.col("SALE_DATE") > F.current_date()
).count()
quality_metrics["future_dates"] = future_dates

logger.info(f"Quality metrics: {quality_metrics}")

```

```

# Remove bad records (keep clean ones)
df_clean = df_bronze \
    .filter(F.col("SALE_ID").isNotNull()) \
    .filter(F.col("AMOUNT") > 0) \
    .filter(F.col("SALE_DATE") <= F.current_date())

records_removed = row_count - df_clean.count()
logger.info(f"Records removed: {records_removed:,}")

# — CELL 6: Transformations —————
df_silver = df_clean \
    .select(
        F.col("SALE_ID").cast("integer").alias("sale_id"),
        F.upper(F.trim("CUSTOMER_ID")).alias("customer_id"),
        F.col("STORE_ID").cast("integer").alias("store_id"),
        F.upper(F.trim("PRODUCT_CODE")).alias("product_code"),
        F.col("QUANTITY").cast("integer").alias("quantity"),
        F.col("AMOUNT").cast("double").alias("total_amount"),
        F.to_date("SALE_DATE", "DD-MON-YYYY").alias("sale_date"),
        F.upper("REGION_CODE").alias("region_code"),
        F.lower("PAYMENT_MODE").alias("payment_mode"),
        F.col("STATUS").alias("status")
    ) \
    .withColumn("year",      F.year("sale_date")) \
    .withColumn("month",    F.month("sale_date")) \
    .withColumn("day",      F.dayofmonth("sale_date")) \
    .withColumn("gst_amount", F.round(F.col("total_amount") *
0.18, 2)) \
    .withColumn("net_amount", F.col("total_amount") -
F.col("gst_amount")) \
    .withColumn("load_ts",  F.current_timestamp())

# — CELL 7: Write to Silver (Delta table in Lakehouse) —————
# Writing to Lakehouse Tables/ as managed Delta table

```

```

# No need for abfss path – use table name!

df_silver.write \
    .format("delta") \
    .mode("append") \
    .partitionBy("year", "month") \
    .saveAsTable("silver_sales")
    # Automatically writes to
    LH_Sales_Prod.Lakehouse/Tables/silver_sales/

logger.info(f"Written to silver_sales: {df_silver.count():,}
rows")

# — CELL 8: Optimize the Delta table —————
spark.sql("OPTIMIZE silver_sales ZORDER BY (store_id,
sale_date)")
logger.info("OPTIMIZE completed")

# — CELL 9: Return result to pipeline —————
import json
result = {
    "status": "SUCCESS",
    "process_date": process_date,
    "input_rows": row_count,
    "output_rows": df_silver.count(),
    "quality_metrics": quality_metrics
}
print(json.dumps(result, indent=2))
mssparkutils.notebook.exit(json.dumps(result))

```

◆ Using Python in Notebooks

```
# FABRIC NOTEBOOKS: PySpark + Pure Python BOTH work!
```

```
# WHEN TO USE PURE PYTHON (not Spark):
```

```
# → Working with small data (< 1GB)
```

```
# → API calls, web scraping
```

```
# → Configuration management
```

```
# → File operations via mssparkutils
```

```
# → Sending emails/notifications
```

```
# EXAMPLE: Call Payment API and load response to Lakehouse
```

```
import requests
```

```
import json
```

```
from datetime import datetime, timedelta
```

```
# Pure Python API call
```

```
def fetch_payment_data(from_date: str, to_date: str) -> dict:
```

```
    """Fetch payment data from REST API"""
```

```
    api_url =
```

```
    "https://payments.retailmart.com/api/v1/transactions"
```

```
    headers = {
```

```
        "Authorization": f"Bearer
```

```
{mssparkutils.credentials.getSecret('https://kv-retailmart-  
prod.vault.azure.net/', 'payment-api-key')}]",
```

```
        "Content-Type": "application/json"
```

```
    }
```

```
    params = {
```

```
        "from_date": from_date,
```

```
        "to_date": to_date,
```

```
        "status": "COMPLETED",
```

```
        "limit": 10000
```

```
    }
```

```

    response = requests.get(api_url, headers=headers,
params=params)
    response.raise_for_status()

    return response.json()

# Fetch data
payment_data = fetch_payment_data(process_date, process_date)
print(f"API returned {len(payment_data['transactions'])}
records")

# Convert to Spark DataFrame
df_payments = spark.createDataFrame(payment_data["transactions"])

# Write to Lakehouse
df_payments.write \
    .format("delta") \
    .mode("append") \
    .saveAsTable("bronze_api_payments")

print("✅ API data loaded to Lakehouse")

# PYTHON LIBRARIES AVAILABLE by default in Fabric:
# pandas, numpy, scikit-learn, matplotlib, seaborn
# requests, json, datetime, os, re, math
# pyspark (obviously)
# delta (for Delta Lake operations)

# INSTALL ADDITIONAL LIBRARIES:
# Option 1: In notebook cell
%pip install great-expectations==0.18.0

# Option 2: Environment (persistent across sessions)
# Workspace → Environments → + New Environment
# → Add libraries → Publish

```



```
# → Attach environment to notebook
# → Libraries available without %pip install!
```

◆ Notebook Utilities

```
# FABRIC NOTEBOOK UTILITIES – Complete Reference
```

```
# notebookutils = mssparkutils equivalent in Fabric
# Both names work! (Microsoft is standardizing to notebookutils)
```

```
from notebookutils import mssparkutils
```

```
# — FILE SYSTEM —
```

```
# List files
```

```
files = mssparkutils.fs.ls(
```

```
"abfss://RetailMart_DataEngineering@onelake.dfs.fabric.microsoft.com"
)
```

```
for f in files:
```

```
    print(f"{f.name} | {f.size/1024:.1f} KB | {'Dir' if f.isDir
else 'File'}")
```

```
# Simpler relative path (when lakehouse is attached):
```

```
files = mssparkutils.fs.ls("Files/raw/sales/")
```

```
# Just "Files/" instead of full abfss path!
```

```
# Copy file
```

```
mssparkutils.fs.cp("Files/raw/sales_20240615.csv",
                   "Files/archive/sales_20240615.csv")
```

```
# Move file
```

```

mssparkutils.fs.mv("Files/landing/new_file.csv",
                   "Files/raw/new_file.csv")

# Delete
mssparkutils.fs.rm("Files/temp/", recurse=True)

# Create directory
mssparkutils.fs.mkdirs("Files/archive/2024/06/")

# Preview file content
content = mssparkutils.fs.head("Files/raw/sales_20240615.csv",
                                500)
print(content) # first 500 bytes

# — NOTEBOOK CHAINING —
# Run child notebook and get result
result = mssparkutils.notebook.run(
    "../silver/transform_customers",
    timeout_seconds=1800,
    arguments={
        "process_date": process_date,
        "load_type": "INCREMENTAL"
    }
)
print(f"Child result: {result}")
# result = whatever child called mssparkutils.notebook.exit()
# with

# Exit current notebook (return value to parent/pipeline)
mssparkutils.notebook.exit("SUCCESS: 142850 rows")

# — CREDENTIALS / SECRETS —
# Get secret from Key Vault
api_key = mssparkutils.credentials.getSecret(
    "https://kv-retailmart-prod.vault.azure.net/",

```

```
    "payment-api-key"
)
# NEVER print this! Fabric redacts it automatically anyway.

# Get token for specific audience
token = mssparkutils.credentials.getToken("storage")
# Use for manual HTTP calls to Azure Storage APIs

# — DATA UTILITY —————
# Convert between Spark and Pandas (for small data)
pandas_df = df_summary.toPandas() # Spark → Pandas
spark_df = spark.createDataFrame(pandas_df) # Pandas → Spark
```

◆ Notebook Visualizations

```
# FABRIC NOTEBOOK: Built-in visualization is amazing!
```

```
# OPTION 1: Display command (Fabric's built-in)
display(df_sales)
# → Auto-renders interactive table!
# → Toggle: Table view / Chart view
# → Chart types: Bar, Line, Scatter, Pie, Map, etc.
# → Click column headers to sort
# → Filter rows interactively
# → Download as CSV right from UI!

# Chart configuration (no code!):
# After display() → click chart icon (top of output)
# Configure:
#   X axis: sale_date
#   Y axis: total_amount (aggregate: SUM)
```

```

# Group by: region_code
# Chart type: Line Chart
# → Beautiful interactive chart instantly!

# OPTION 2: Matplotlib (traditional Python)
import matplotlib.pyplot as plt
import pandas as pd

# Sample data for viz (small – collect to driver)
df_region = df_sales.groupby("region_code") \
    .agg(F.sum("total_amount").alias("revenue")) \
    .toPandas() # convert to pandas for viz

plt.figure(figsize=(10, 6))
plt.bar(df_region["region_code"], df_region["revenue"] / 1e6,
        color=["#0078D4", "#2D7D9A", "#107C10", "#7A7574"])
plt.title("Revenue by Region (in Millions INR)", fontsize=14,
fontweight='bold')
plt.xlabel("Region")
plt.ylabel("Revenue (₹ Millions)")
plt.tight_layout()
plt.savefig("/tmp/regional_revenue.png")
plt.show() # displays inline in notebook!

# OPTION 3: Seaborn (statistical plots)
import seaborn as sns

df_pd = df_sales.select("total_amount", "region_code").toPandas()
sns.boxplot(x="region_code", y="total_amount", data=df_pd)
plt.title("Sales Distribution by Region")
plt.show()

# OPTION 4: Plotly (interactive charts)
import plotly.express as px

```

```
df_pd = df_sales.groupBy("sale_date", "region_code") \
    .agg(F.sum("total_amount").alias("revenue")) \
    .orderBy("sale_date") \
    .toPandas()

fig = px.line(df_pd, x="sale_date", y="revenue",
              color="region_code",
              title="Daily Revenue by Region")
fig.show() # interactive chart with hover, zoom, etc.!
```

◆ Loading Data into the Lakehouse

ALL WAYS TO LOAD DATA INTO FABRIC LAKEHOUSE:

— METHOD 1: Upload via UI (small files, one-time) —————
Lakehouse → Files/ → Upload → drag & drop
Good for: config files, Excel lookups, testing data

— METHOD 2: Pipeline Copy Activity —————
Source (Oracle/SQL/FTP) → Copy Activity → Lakehouse Files
Then Notebook transforms Files → Tables
Good for: scheduled ingestion from external sources

— METHOD 3: Dataflow Gen2 —————
Source → Power Query transformations → Lakehouse Table
Good for: simple ingestion + transform without code

— METHOD 4: Notebook (most flexible) —————

4A: Write to Files/ (raw landing)

```

df_raw.write \
    .mode("overwrite") \
    .csv("Files/raw/sales/2024/06/")
# File visible in Lakehouse → Files/ → raw/ → sales/

# 4B: Write to Tables/ (managed Delta)
df_silver.write \
    .format("delta") \
    .mode("overwrite") \
    .partitionBy("year", "month") \
    .saveAsTable("silver_sales")
# Table visible in Lakehouse → Tables/ → silver_sales

# 4C: Save table with specific path (external-like)
df_gold.write \
    .format("delta") \
    .mode("overwrite") \
    .save("Tables/gold_store_performance")
# Same result – Delta table in Tables/ folder

# 4D: Using spark.sql CREATE TABLE
spark.sql("""
    CREATE TABLE IF NOT EXISTS gold_kpi_summary
    USING DELTA
    PARTITIONED BY (report_date)
    AS SELECT
        store_id,
        sale_date AS report_date,
        SUM(total_amount) AS gross_revenue,
        COUNT(*) AS transactions
    FROM silver_sales
    GROUP BY store_id, sale_date
""")

```

— METHOD 5: Auto Loader (streaming ingestion) —————

Files dropped to Lakehouse Files/ → Auto Loader picks up

```
df_stream = spark.readStream \  
    .format("cloudFiles") \  
    .option("cloudFiles.format", "csv") \  
    .option("cloudFiles.schemaLocation", "Files/schemas/sales/") \  
    \  
    .option("header", "true") \  
    .load("Files/landing/sales/")  
  
df_stream.writeStream \  
    .format("delta") \  
    .outputMode("append") \  
    .option("checkpointLocation", "Files/checkpoints/sales/") \  
    .trigger(availableNow=True) \  
    .toTable("bronze_sales_stream")
```

◆ Notebook Source Control and Deployment

COMPLETE DEVOPS WORKFLOW FOR FABRIC NOTEBOOKS:

WORKFLOW:

Developer builds notebook **in** Dev workspace

|

▼ Commit changes

Azure DevOps Repo (dev branch)

|

▼ Pull Request

Code Review by team lead

|

▼ Merge to main

CI Pipeline runs:

- Lint Python code (flake8)
- Run unit tests (pytest on logic functions)
- Validate notebook JSON syntax
- |

▼ CD Pipeline

Fabric Deployment Pipeline:

- Dev → Test (automated)
- Test → Prod (manual approval)

GIT INTEGRATION IN FABRIC:

Workspace Settings → Git Integration → Connect

After connecting:

- Notebook shows "Synced" or "Modified" status
- Click "Source control" button in workspace
- See all pending changes
- Commit with message
- Push to branch

NOTEBOOK AS CODE (storage format):

Notebooks stored in Git as:

- .py file (Python cells + markdown as comments)
- or .ipynb (standard Jupyter format)
- Diff shows actual code changes (line by line)
- Code reviews see exact logic changes!

UNIT TESTING APPROACH:

functions.py (pure Python functions – testable!)

```
def calculate_gst(amount: float, rate: float = 0.18) -> float:  
    return round(amount * rate, 2)
```

```
def validate_pan(pan: str) -> bool:  
    import re  
    return bool(re.match(r'^[A-Z]{5}[0-9]{4}[A-Z]$', pan))
```



```

# test_functions.py (pytest tests)
def test_calculate_gst():
    assert calculate_gst(1000) == 180.0
    assert calculate_gst(0) == 0.0

def test_validate_pan():
    assert validate_pan("ABCDE1234F") == True
    assert validate_pan("invalid") == False

# In notebook: import and use these functions
# %run ./functions OR
from functions import calculate_gst, validate_pan

# Tests run in CI pipeline automatically!

```

◆ Authoring and Running T-SQL in Notebooks

FABRIC NOTEBOOKS: SWITCH TO SQL MODE PER CELL!

— OPTION 1: Spark SQL in code cell —

```

result = spark.sql("""
    SELECT
        region_code,
        COUNT(*) AS transaction_count,
        ROUND(SUM(total_amount), 2) AS gross_revenue,
        ROUND(AVG(total_amount), 2) AS avg_ticket
    FROM silver_sales
    WHERE year = 2024 AND month = 6
    GROUP BY region_code
    ORDER BY gross_revenue DESC

```

```

"""
display(result) # show as interactive table!

# — OPTION 2: %%sql magic (cell-level SQL) —————
%%sql
-- This entire cell runs as Spark SQL
SELECT
    s.region_code,
    s.sale_date,
    SUM(s.total_amount) AS daily_revenue,
    COUNT(DISTINCT s.customer_id) AS unique_customers,
    t.revenue_target,
    ROUND(SUM(s.total_amount) / t.revenue_target * 100, 1) AS
achievement
FROM silver_sales s
LEFT JOIN store_targets t
    ON s.region_code = t.region_code
    AND MONTH(s.sale_date) = t.month
WHERE s.year = 2024
GROUP BY s.region_code, s.sale_date, t.revenue_target
ORDER BY s.sale_date, achievement DESC

# — OPTION 3: CREATE TABLE using SQL —————
%%sql
CREATE OR REPLACE TABLE gold_regional_performance
USING DELTA
PARTITIONED BY (report_month)
AS
SELECT
    TRUNC(sale_date, 'MM') AS report_month,
    region_code,
    COUNT(*) AS transactions,
    SUM(total_amount) AS gross_revenue,
    SUM(net_amount) AS net_revenue,
    SUM(gst_amount) AS gst_collected,

```

```
COUNT(DISTINCT customer_id) AS unique_customers,  
COUNT(DISTINCT store_id) AS active_stores,  
MAX(total_amount) AS highest_sale,  
AVG(total_amount) AS avg_sale_value  
FROM silver_sales  
WHERE status = 'COMPLETED'  
GROUP BY TRUNC(sale_date, 'MM'), region_code
```

— OPTION 4: Pass Python variables into SQL —————
Problem: %%sql can't access Python variables directly!
Solution: Use f-string or spark.sql()

```
target_month = "2024-06-01"
```

Option A: spark.sql() with f-string

```
df = spark.sql(f"""  
    SELECT * FROM silver_sales  
    WHERE sale_date >= '{target_month}'  
    AND region_code IN ('N', 'S')  
""")
```

Option B: Register Python variable as Spark SQL variable

```
spark.conf.set("spark.sql.variable.month", target_month)  
%%sql  
SELECT * FROM silver_sales  
WHERE sale_date >=  
'${spark.conf.get("spark.sql.variable.month")}'
```

9 Fabric Data Engineering — Apache Spark

◆ In-Depth Architecture of Apache Spark in Fabric

FABRIC SPARK vs STANDALONE SPARK:

Same Spark engine, but Fabric manages EVERYTHING:

What Microsoft manages for you:

- ✓ Spark cluster provisioning
- ✓ Worker node addition/removal (autoscale)
- ✓ Spark version upgrades
- ✓ Log management
- ✓ Library installation
- ✓ Security and network isolation
- ✓ OneLake integration (no credential config!)

What YOU manage:

- ✓ Your code (notebooks)
- ✓ Cluster size (node type/count)
- ✓ Library requirements
- ✓ Optimization (partitioning, caching, etc.)

STARTER POOLS (game changer!):

Normal Spark: click Run → wait 3-5 min for cluster to start

Fabric Starter Pool: click Run → running in 5-10 SECONDS!

Microsoft pre-warms small Spark clusters for you

When you start notebook → instantly gets assigned a warm pool

No waiting! Great for interactive development

SESSION TIMEOUT:

Default: notebook session times out after idle period

Configure: Workspace Settings → Spark Sessions

Set: 20 minutes idle timeout

Saves CUs by not running idle Spark sessions

◆ Working with Data in a Spark DataFrame

```
# — FULL RETAILMART SPARK WORKFLOW —————  
  
from pyspark.sql import functions as F  
from pyspark.sql.types import *  
from pyspark.sql.window import Window  
  
# READ from default lakehouse (attached)  
df_sales      = spark.read.table("silver_sales")  
df_customers  = spark.read.table("dim_customers")  
df_products   = spark.read.table("dim_products")  
df_stores     = spark.read.table("dim_stores")  
  
# INSPECT DATA  
print(f"Sales records:      {df_sales.count():>10,}")  
print(f"Customers:         {df_customers.count():>10,}")  
print(f"Products:           {df_products.count():>10,}")  
print(f"Stores:             {df_stores.count():>10,}")  
  
df_sales.printSchema()  
display(df_sales.limit(5))  
  
# CHECK DATA QUALITY  
print(f"\n📊 Null counts per column:")  
null_counts = df_sales.select([  
    F.count(F.when(F.col(c).isNull(), c)).alias(c)  
    for c in df_sales.columns  
)  
]  
display(null_counts)  
  
print(f"\n📊 Duplicate check (by sale_id):")  
dup_count = df_sales.count() -  
df_sales.dropDuplicates(["sale_id"]).count()
```

```
print(f"  Duplicate sale_ids: {dup_count:,}")

print("\n📊 Date range:")
df_sales.agg(
    F.min("sale_date").alias("earliest_date"),
    F.max("sale_date").alias("latest_date"),
    F.countDistinct("sale_date").alias("distinct_dates")
).show()
```

◆ DataFrame Internal Execution & Spark SQL

UNDERSTAND YOUR QUERY BEFORE RUNNING IT:

Build complex query

```
df_analysis = df_sales \
    .filter(F.col("year") == 2024) \
    .filter(F.col("status") == "COMPLETED") \
    .join(F.broadcast(df_customers), on="customer_id",
how="left") \
    .join(F.broadcast(df_products), on="product_code",
how="left") \
    .join(F.broadcast(df_stores), on="store_id",
how="left") \
    .groupBy("region_code", "customer_tier", "category") \
    .agg(
        F.sum("total_amount").alias("revenue"),
        F.count("sale_id").alias("transactions")
    )
```

CHECK THE PLAN FIRST (before running expensive operation):

```
df_analysis.explain("formatted")
```

```

# LOOK FOR in the plan:
# ✓ BroadcastHashJoin → broadcast working!
# ✓ PartitionFilters → partition pruning working!
# ✓ PushedFilters → predicate pushdown working!
# ✗ SortMergeJoin on big table → broadcast failed, investigate!
# ✗ No PartitionFilters → data not partitioned, doing full scan!

# RUN IT
display(df_analysis.orderBy(F.col("revenue").desc()))

# — SPARK SQL APPROACH (same result, more readable) —————
df_spark_sql = spark.sql("""
    SELECT
        s.region_code,
        c.customer_tier,
        p.category,
        SUM(s.total_amount) AS revenue,
        COUNT(s.sale_id) AS transactions,
        ROUND(AVG(s.total_amount), 2) AS avg_transaction
    FROM silver_sales s
    LEFT JOIN dim_customers c ON s.customer_id = c.customer_id
    LEFT JOIN dim_products p  ON s.product_code = p.product_code
    LEFT JOIN dim_stores st   ON s.store_id = st.store_id
    WHERE s.year = 2024
        AND s.status = 'COMPLETED'
    GROUP BY s.region_code, c.customer_tier, p.category
    ORDER BY revenue DESC
""")
display(df_spark_sql)

```

◆ Transformations and Actions

```

# — KEY TRANSFORMATIONS (all lazy!) —————

# select – choose columns
df_selected = df_sales.select(
    "sale_id", "customer_id", "total_amount", "sale_date"
)

# filter / where – filter rows
df_filtered = df_sales.filter(
    (F.col("total_amount") > 10000) &
    (F.col("region_code").isin("N", "S")) &
    (F.col("sale_date").between("2024-01-01", "2024-06-30"))
)

# withColumn – add/modify column
df_with_cols = df_sales \
    .withColumn("gst", F.round(F.col("total_amount") * 0.18,
2)) \
    .withColumn("net", F.col("total_amount") - F.col("gst")) \
    .withColumn("day_name", F.date_format("sale_date", "EEEE")) \
    .withColumn("is_weekend", F.dayofweek("sale_date").isin([1,
7]))

# groupBy + agg – aggregations
df_grouped = df_sales.groupBy("region_code", "year", "month") \
    .agg(
        F.sum("total_amount").alias("revenue"),
        F.count("sale_id").alias("count"),
        F.avg("total_amount").alias("avg_value"),
        F.max("total_amount").alias("max_value"),
        F.min("total_amount").alias("min_value"),
        F.countDistinct("customer_id").alias("unique_customers"),
        F.collect_list("product_code").alias("all_products") #
array!
    )

```



```

# orderBy – sort
df_sorted = df_grouped.orderBy(
    F.col("year").asc(),
    F.col("month").asc(),
    F.col("revenue").desc()
)

# dropDuplicates – remove duplicates
df_dedup = df_sales.dropDuplicates(["sale_id"])
# OR:
df_dedup2 = df_sales.dropDuplicates(["customer_id", "sale_date",
"product_code"])

# drop – remove columns
df_dropped = df_sales.drop("_ingestion_timestamp",
"_source_file", "_process_date")

# rename – rename column
df_renamed = df_sales.withColumnRenamed("total_amount",
"gross_amount")

# cast – change data type
df_casted = df_sales \
    .withColumn("sale_id", F.col("sale_id").cast("integer")) \
    .withColumn("total_amount",
F.col("total_amount").cast("double")) \
    .withColumn("sale_date", F.col("sale_date").cast("date"))

# — KEY ACTIONS (trigger execution!) —————

count = df_sales.count() # count all rows
first = df_sales.first() # first row as Row
object
sample = df_sales.take(5) # first 5 rows as

```

```
list
collected = df_grouped.collect()           # ALL rows to driver
(SMALL data only!)
display(df_sales)                           # show in notebook
(Fabric-specific)
df_sales.show(10, truncate=False)           # print to console

# Write actions (most important!)
df_silver.write.format("delta").mode("overwrite").saveAsTable("silver")
df_silver.write.format("delta").mode("append").saveAsTable("silver_sales")
df_silver.write.format("parquet").save("Files/output/sales.parquet")
```

◆ User-Defined Functions in Spark SQL

UDFs IN FABRIC NOTEBOOKS:

```
from pyspark.sql.functions import udf, pandas_udf
from pyspark.sql.types import StringType, BooleanType, FloatType
import pandas as pd
```

— REGULAR UDF (works but slow) —

Use ONLY when built-in functions can't do the job!

```
def categorize_amount(amount):
    """Categorize sale amount into business tiers"""
    if amount is None:
        return "UNKNOWN"
    elif amount < 1000:
        return "MICRO"
    elif amount < 10000:
        return "SMALL"
```

```

    elif amount < 50000:
        return "MEDIUM"
    elif amount < 200000:
        return "LARGE"
    else:
        return "ENTERPRISE"

# Register as UDF
categorize_udf = udf(categorize_amount, StringType())

# Use in DataFrame
df_categorized = df_sales.withColumn(
    "sale_category",
    categorize_udf(F.col("total_amount"))
)

# Register for Spark SQL queries too!
spark.udf.register("categorize_amount", categorize_amount,
StringType())

%%sql
SELECT
    categorize_amount(total_amount) AS category,
    COUNT(*) AS count,
    SUM(total_amount) AS revenue
FROM silver_sales
GROUP BY categorize_amount(total_amount)
ORDER BY revenue DESC

# — PANDAS UDF (MUCH FASTER — vectorized!) —————
# Process entire column at once (batch, not row-by-row)

@pandas_udf(StringType())
def categorize_amount_fast(amount_series: pd.Series) ->
pd.Series:

```

```

    """Vectorized amount categorization – 10-100x faster than
    regular UDF"""
    conditions = [
        amount_series.isna(),
        amount_series < 1000,
        amount_series < 10000,
        amount_series < 50000,
        amount_series < 200000
    ]
    choices = ["UNKNOWN", "MICRO", "SMALL", "MEDIUM", "LARGE"]
    return pd.Series(
        pd.cut(amount_series,
                bins=[-float('inf'), 1000, 10000, 50000, 200000,
float('inf')],
                labels=["MICRO", "SMALL", "MEDIUM", "LARGE",
"ENTERPRISE"])
        ).fillna("UNKNOWN")

df_fast = df_sales.withColumn("category",
categorize_amount_fast("total_amount"))

```

— BEST PRACTICE —

1. ALWAYS try built-in functions FIRST

```

df_best = df_sales.withColumn("category",
    F.when(F.col("total_amount").isNull(), "UNKNOWN")
        .when(F.col("total_amount") < 1000, "MICRO")
        .when(F.col("total_amount") < 10000, "SMALL")
        .when(F.col("total_amount") < 50000, "MEDIUM")
        .when(F.col("total_amount") < 200000, "LARGE")
        .otherwise("ENTERPRISE")
    )
# ↑ This is FASTEST (runs in JVM, no Python overhead)!

```

◆ Visualizing Data in Spark Notebook

FABRIC'S display() IS THE BEST VISUALIZATION TOOL:

Step 1: Prepare aggregated data

```
df_daily_revenue = df_sales \  
    .groupBy("sale_date", "region_code") \  
    .agg(F.sum("total_amount").alias("revenue")) \  
    .orderBy("sale_date")
```

Step 2: display() – click chart icon in output!

```
display(df_daily_revenue)
```

In the output, click the chart icon and configure:

```
# |  
# | Chart Configuration Panel |  
# |  
# | Chart Type: Line Chart |  
# | X axis: sale_date |  
# | Y axis: revenue (SUM) |  
# | Group by: region_code |  
# |  
# | → Click Apply → Interactive chart! |  
# |
```

Available chart types in display():

Bar Chart | Line Chart | Scatter Plot |

Pie Chart | Area Chart | Map (for location data)

The magic: drag and drop columns to chart config

No code needed for quick exploration!

For PUBLICATION-QUALITY charts → matplotlib/plotly:

```
import matplotlib.pyplot as plt
```

```

import seaborn as sns

fig, axes = plt.subplots(2, 2, figsize=(16, 12))
fig.suptitle("RetailMart Sales Analytics Dashboard",
             fontsize=16, fontweight='bold')

df_pd = df_daily_revenue.toPandas()

# Chart 1: Daily revenue line
for region in df_pd["region_code"].unique():
    mask = df_pd["region_code"] == region
    axes[0,0].plot(df_pd[mask]["sale_date"],
                  df_pd[mask]["revenue"] / 1e6,
                  label=region, linewidth=2)
axes[0,0].set_title("Daily Revenue by Region (₹ Millions)")
axes[0,0].legend()
axes[0,0].grid(True, alpha=0.3)

# Chart 2: Revenue pie by region
region_total = df_pd.groupby("region_code")["revenue"].sum()
axes[0,1].pie(region_total.values, labels=region_total.index,
              autopct='%1.1f%%', startangle=90)
axes[0,1].set_title("Revenue Share by Region")

plt.tight_layout()
plt.savefig("/tmp/analytics_dashboard.png", dpi=150,
            bbox_inches='tight')
plt.show()

# Upload chart to Lakehouse
mssparkutils.fs.cp("file:///tmp/analytics_dashboard.png",
                  "Files/reports/analytics_dashboard.png")

```

10 Fabric Data Engineering — Delta Lake Tables

◆ Introduction and Understanding Delta Lake

Why does Delta Lake exist?

Let me tell you a story that every data engineer has lived:

THE NIGHTMARE SCENARIO (without Delta):

Monday night pipeline runs at midnight.

Writes 50GB of Parquet files to /silver/sales/2024/06/

Halfway through writing – CLUSTER CRASHES!

Files written:

/silver/sales/2024/06/part-00001.parquet ✓ (complete)

/silver/sales/2024/06/part-00002.parquet ✓ (complete)

/silver/sales/2024/06/part-00003.parquet ✗ (corrupted – half-written!)

Tuesday morning: BI team opens reports

→ Report shows WRONG numbers (reads corrupt file!)

→ Nobody knows what happened

→ Data team spends 4 hours debugging

→ CFO is furious

WITH DELTA LAKE:

Same crash at midnight.

Delta writes to temp files + transaction log

Crash → transaction ROLLED BACK

Delta log shows: "Transaction 47: UNCOMMITTED"

Tuesday morning: BI team opens reports

→ Reads last COMMITTED version (Monday morning's data)

→ Reports show correct data from previous run

→ Data team gets alert → re-runs pipeline
→ Fixed by 1 AM. CFO never knows.

THAT'S the value of Delta Lake!

◆ Creating Delta Tables in Fabric

```
# MULTIPLE WAYS TO CREATE DELTA TABLES:
```

```
# — METHOD 1: saveAsTable (RECOMMENDED) —————  
# Creates managed Delta table in Lakehouse Tables/
```

```
df_silver_sales.write \  
    .format("delta") \  
    .mode("overwrite") \  
    .partitionBy("year", "month") \  
    .option("overwriteSchema", "true") \  
    .saveAsTable("silver_sales")
```

```
# TABLE IS NOW VISIBLE IN LAKEHOUSE TABLES/ FOLDER  
# AND QUERYABLE VIA SQL ENDPOINT!
```

```
# — METHOD 2: CREATE TABLE AS SELECT (SQL) —————
```

```
spark.sql("""  
    CREATE OR REPLACE TABLE gold_sales_summary  
    USING DELTA  
    PARTITIONED BY (report_year, report_month)  
    TBLPROPERTIES (  
        'description' = 'Monthly sales summary for BI reporting',  
        'owner' = 'data_engineering_team',  
        'sla' = 'daily_by_8am'
```



```

)
AS
SELECT
    YEAR(sale_date) AS report_year,
    MONTH(sale_date) AS report_month,
    region_code,
    store_id,
    SUM(total_amount) AS gross_revenue,
    SUM(net_amount) AS net_revenue,
    SUM(gst_amount) AS gst_collected,
    COUNT(sale_id) AS transactions,
    COUNT(DISTINCT customer_id) AS unique_customers
FROM silver_sales
WHERE status = 'COMPLETED'
GROUP BY
    YEAR(sale_date), MONTH(sale_date),
    region_code, store_id
""")

```

— METHOD 3: Explicit schema then insert —————

```

spark.sql("""
    CREATE TABLE IF NOT EXISTS fact_sales_audit (
        audit_id          BIGINT GENERATED ALWAYS AS IDENTITY,
        sale_id           INT NOT NULL,
        action            STRING,
        changed_by        STRING,
        changed_at        TIMESTAMP,
        old_amount        DOUBLE,
        new_amount        DOUBLE
    )
    USING DELTA
""")

```

— METHOD 4: Delta Table API —————

```

from delta.tables import DeltaTable

```

```

# Create empty Delta table with schema
schema = StructType([
    StructField("sale_id",      IntegerType(), False),
    StructField("customer_id",  StringType(),  False),
    StructField("total_amount", DoubleType(),   True),
    StructField("sale_date",    DateType(),     True),
    StructField("year",         IntegerType(),  True),
    StructField("month",        IntegerType(),  True)
])

# Create empty table (like CREATE TABLE with schema)
DeltaTable.createIfNotExists(spark) \
    .tableName("silver_sales_v2") \
    .addColumnns(schema) \
    .partitionedBy("year", "month") \
    .execute()

# VERIFY TABLE CREATED:
display(spark.sql("SHOW TABLES"))
display(spark.sql("DESCRIBE EXTENDED silver_sales"))

```

◆ Working with Delta Tables in Spark

```

# DELTA TABLE OPERATIONS – THE POWER FEATURES:

```

```

from delta.tables import DeltaTable

```

```

# — UPDATE —

```

```

# Something NOT possible with regular Parquet!

```

SQL approach:

```
spark.sql("""
    UPDATE silver_sales
    SET status = 'VERIFIED',
        load_ts = current_timestamp()
    WHERE sale_date = '2024-06-15'
        AND total_amount > 100000
""")
```

Python API approach:

```
DeltaTable.forName(spark, "silver_sales").update(
    condition = F.col("status") == "PENDING",
    set = {
        "status": F.lit("EXPIRED"),
        "load_ts": F.current_timestamp()
    }
)
```

— DELETE —

Also impossible with regular Parquet!

SQL approach:

```
spark.sql("""
    DELETE FROM silver_sales
    WHERE status = 'CANCELLED'
        AND sale_date < '2024-01-01'
""")
```

Python API approach:

```
DeltaTable.forName(spark, "silver_sales").delete(
    condition = (F.col("status") == "CANCELLED") &
                (F.col("sale_date") < "2024-01-01")
)
```

— MERGE (UPSERT) – Most Important! —

```

delta_target = DeltaTable.forName(spark, "silver_sales")

df_updates = spark.read.table("bronze_sales") \
    .filter(F.col("_process_date") == process_date)

delta_target.alias("target").merge(
    df_updates.alias("source"),
    "target.sale_id = source.sale_id" # join condition
) \
.whenMatchedUpdate(set = {
    "total_amount": "source.AMOUNT",
    "status":       "source.STATUS",
    "load_ts":      "current_timestamp()"
}) \
.whenNotMatchedInsert(values = {
    "sale_id":      "source.SALE_ID",
    "customer_id":  "upper(trim(source.CUSTOMER_ID))",
    "store_id":     "source.STORE_ID",
    "total_amount": "source.AMOUNT",
    "sale_date":    "to_date(source.SALE_DATE, 'DD-MON-YYYY')",
    "year":         "year(to_date(source.SALE_DATE, 'DD-MON-
YYYY'))",
    "month":        "month(to_date(source.SALE_DATE, 'DD-MON-
YYYY'))",
    "status":       "source.STATUS",
    "load_ts":      "current_timestamp()"
}) \
.execute()

print("Merge completed!")

# — GET MERGE STATISTICS —————
# Read operation metrics from merge
history = DeltaTable.forName(spark, "silver_sales").history(1)
last_operation = history.first()

```

```
print(f"Operation: {last_operation['operation']}")
print(f"Metrics: {last_operation['operationMetrics']}")
# Shows: numTargetRowsInserted, numTargetRowsUpdated,
numSourceRows
```

◆ Delta Tables with Streaming Data

```
# DELTA + STREAMING = Perfect combination in Fabric!
```

```
# — READ DELTA TABLE AS STREAM —
# (CDC – Change Data Capture on Delta table!)
# Reads only NEW or CHANGED data since last checkpoint
```

```
df_stream_source = spark.readStream \
    .format("delta") \
    .option("startingVersion", "latest") \
    .table("bronze_sales")
# Processes only new rows appended to bronze_sales!
```

```
# Process stream
df_processed = df_stream_source \
    .filter(F.col("status") == "COMPLETED") \
    .withColumn("load_ts", F.current_timestamp())
```

```
# Write stream to another Delta table
df_processed.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation",
"Files/checkpoints/silver_sales_stream/") \
    .trigger(availableNow=True) \
```

```

        .toTable("silver_sales")

# — WRITE STREAMING DATA TO DELTA —————
# (Real-time ingestion from EventStream/files)

df_realtime = spark.readStream \
    .format("cloudFiles") \
    .option("cloudFiles.format", "json") \
    .option("cloudFiles.schemaLocation",
"Files/schema/payments/") \
    .load("Files/landing/payments/")

df_realtime.writeStream \
    .format("delta") \
    .outputMode("append") \
    .option("checkpointLocation", "Files/checkpoints/payments/")
\
    .trigger(processingTime="30 seconds") \
    .toTable("bronze_payments_realtime")

# KEY: Delta handles concurrent batch + stream writes!
# While stream is writing micro-batches every 30 seconds,
# batch pipeline can also write to same table!
# No conflict → Delta handles it with transaction log!

```

◆ SCD Type 1 and Type 2 (Already Covered — Fabric Version)

```

# SCD OPERATIONS IN FABRIC — USING TABLE NAMES (CLEANER!)

# SCD TYPE 1: Pure SQL approach (clean!)
spark.sql("""

```

```

MERGE INTO dim_customers AS target
USING (
    SELECT * FROM bronze_customers
    WHERE _process_date = '{process_date}'
) AS source
ON target.customer_id = source.CUSTOMER_ID
WHEN MATCHED AND (
    target.email    <> source.EMAIL OR
    target.phone    <> source.PHONE OR
    target.address  <> source.ADDRESS
) THEN UPDATE SET
    target.email      = source.EMAIL,
    target.phone      = source.PHONE,
    target.address    = source.ADDRESS,
    target.updated_at = current_timestamp()
WHEN NOT MATCHED THEN INSERT (
    customer_id, customer_name, email, phone, address,
    customer_tier, created_at, updated_at
) VALUES (
    source.CUSTOMER_ID, source.CUST_NAME, source.EMAIL,
    source.PHONE, source.ADDRESS, 'STANDARD',
    current_timestamp(), current_timestamp()
)
)
""".format(process_date=process_date))

```

SCD TYPE 2: Two-step SQL approach

Step 1: Expire changed records

```

spark.sql(f"""
    UPDATE dim_customers_scd2
    SET is_current = false,
        effective_end = date_sub(current_date(), 1)
    WHERE is_current = true
        AND customer_id IN (
            SELECT b.CUSTOMER_ID
            FROM bronze_customers b

```

```

        JOIN dim_customers_scd2 d
          ON b.CUSTOMER_ID = d.customer_id
         AND d.is_current = true
       WHERE b._process_date = '{process_date}'
          AND (b.EMAIL <> d.email OR b.ADDRESS <> d.address)
    )
    """
)

```

Step 2: Insert new version

```

spark.sql(f"""
    INSERT INTO dim_customers_scd2
    SELECT
        b.CUSTOMER_ID AS customer_id,
        b.CUST_NAME AS customer_name,
        b.EMAIL AS email,
        b.ADDRESS AS address,
        current_date() AS effective_start,
        DATE('9999-12-31') AS effective_end,
        true AS is_current,
        current_timestamp() AS created_at
    FROM bronze_customers b
    LEFT ANTI JOIN dim_customers_scd2 d
      ON b.CUSTOMER_ID = d.customer_id
     AND d.is_current = true
   WHERE b._process_date = '{process_date}'
    """)

```

◆ Time Travel and Versioning

TIME TRAVEL IN FABRIC – GAME CHANGER:

```
# — VIEW TABLE HISTORY —————
display(spark.sql("DESCRIBE HISTORY silver_sales"))
# OR:
display(DeltaTable.forName(spark, "silver_sales").history())

# Output columns:
# version | timestamp          | operation | operationParameters
# 5       | 2024-06-15 01:35   | MERGE     | {predicate: ...,
numRows: 1000}
# 4       | 2024-06-14 01:33   | MERGE     | {predicate: ...,
numRows: 987}
# 3       | 2024-06-13 01:31   | WRITE     | {mode: Append, ...}
# 2       | 2024-06-12 23:00   | UPDATE    | {predicate: ...,
numRows: 5}
# 1       | 2024-06-01 01:00   | WRITE     | {mode: Overwrite,
...}
# 0       | 2024-05-31 12:00   | CREATE    | ...

# — QUERY PREVIOUS VERSIONS —————
# By version number:
df_v3 = spark.read.format("delta") \
    .option("versionAsOf", "3") \
    .table("silver_sales")

# By timestamp:
df_yesterday = spark.read.format("delta") \
    .option("timestampAsOf", "2024-06-14 00:00:00") \
    .table("silver_sales")

# SQL:
display(spark.sql("""
    SELECT * FROM silver_sales VERSION AS OF 3
    WHERE region_code = 'N'
    LIMIT 10
    """))
```

```

# — REAL WORLD: INVESTIGATE DATA ISSUE —————
# "Our North India revenue dropped 40% yesterday – what
happened?"

# Compare today vs yesterday
df_today = spark.read.table("silver_sales") \
    .filter(F.col("sale_date") == "2024-06-15") \
    .filter(F.col("region_code") == "N")

df_yesterday = spark.read.format("delta") \
    .option("timestampAsOf", "2024-06-14 23:59:59") \
    .table("silver_sales") \
    .filter(F.col("sale_date") == "2024-06-14") \
    .filter(F.col("region_code") == "N")

print(f"Today (June 15):
{df_today.agg(F.sum('total_amount')).first()[0]:, .0f}")
print(f"Yesterday (June 14):
{df_yesterday.agg(F.sum('total_amount')).first()[0]:, .0f}")

# — ROLLBACK ENTIRE TABLE —————
# Pipeline wrote wrong data in version 5
# Need to restore version 4

spark.sql("RESTORE TABLE silver_sales TO VERSION AS OF 4")
# Table is back to state after version 4!
# Version 5 data is gone (but version 5 still in history)

# — CONFIGURE DATA RETENTION —————
# How many versions to keep? (for time travel AND VACUUM)
spark.sql("""
    ALTER TABLE silver_sales
    SET TBLPROPERTIES (
        'delta.logRetentionDuration' = 'interval 30 days', --

```

```

keep 30 days of history
    'delta.deletedFileRetentionDuration' = 'interval 7 days'
-- keep 7 days of files
    )
    """
)

```

◆ Implementing Medallion Architecture in Fabric

COMPLETE MEDALLION ARCHITECTURE FOR RETAILMART IN FABRIC:

```

# — BRONZE LAYER —————

def load_bronze(source_data: DataFrame, table_name: str,
process_date: str):
    """
    Bronze Layer: Land exactly as received from source
    No transformations! Only add audit metadata.
    """
    df_bronze = source_data \
        .withColumn("_ingestion_ts", F.current_timestamp()) \
        .withColumn("_source_file", F.input_file_name()) \
        .withColumn("_process_date", F.lit(process_date)) \
        .withColumn("_source_system", F.lit("Oracle_ERP"))

    df_bronze.write \
        .format("delta") \
        .mode("append") \
        .partitionBy("_process_date") \
        .option("mergeSchema", "true") \
        .saveAsTable(f"bronze_{table_name}")

    count = df_bronze.count()

```

```

print(f"✅ Bronze {table_name}: {count:,} rows appended")
return count

# — SILVER LAYER —
def load_silver_sales(process_date: str):
    """
    Silver Layer: Cleaned, validated, standardized
    Apply business rules and quality checks
    """

    # Read from Bronze
    df_bronze = spark.read.table("bronze_sales") \
        .filter(F.col("_process_date") ==
process_date)

    # Validate and clean
    df_clean = df_bronze \
        .filter(F.col("SALE_ID").isNotNull()) \
        .filter(F.col("AMOUNT") > 0) \
        .filter(~F.col("STATUS").isin("CANCELLED", "VOID"))

    # Standardize
    df_silver = df_clean \
        .select(
            F.col("SALE_ID").cast("int").alias("sale_id"),
            F.upper(F.trim("CUSTOMER_ID")).alias("customer_id"),
            F.col("STORE_ID").cast("int").alias("store_id"),

F.upper(F.trim("PRODUCT_CODE")).alias("product_code"),
            F.col("AMOUNT").cast("double").alias("total_amount"),
            F.to_date("SALE_DATE", "DD-MON-
YYYY").alias("sale_date"),
            F.upper("REGION_CODE").alias("region_code"),
            F.lower("PAYMENT_MODE").alias("payment_mode"),
            "STATUS"
        ) \

```

```

        .withColumn("year",      F.year("sale_date")) \
        .withColumn("month",    F.month("sale_date")) \
        .withColumn("gst_amount", F.round(F.col("total_amount") *
0.18, 2)) \
        .withColumn("net_amount", F.col("total_amount") -
F.col("gst_amount")) \
        .withColumn("load_ts",   F.current_timestamp())

# Upsert to Silver (no duplicates!)
DeltaTable.forName(spark, "silver_sales").alias("t").merge(
    df_silver.alias("s"),
    "t.sale_id = s.sale_id"
).whenMatchedUpdateAll().whenNotMatchedInsertAll().execute()

print(f"✅ Silver sales updated for {process_date}")

```

— GOLD LAYER —————

```

def load_gold_store_performance(process_date: str):
    """
    Gold Layer: Business-ready aggregations
    Optimized for Power BI Direct Lake
    """
    df = spark.sql(f"""
        SELECT
            s.sale_date,
            s.store_id,
            st.store_name,
            st.city,
            st.state,
            s.region_code,
            COUNT(s.sale_id)           AS transactions,
            SUM(s.total_amount)        AS gross_revenue,
            SUM(s.net_amount)          AS net_revenue,
            SUM(s.gst_amount)          AS gst_collected,
            AVG(s.total_amount)        AS avg_ticket,

```

```

        COUNT(DISTINCT s.customer_id) AS unique_customers,
        SUM(CASE WHEN s.payment_mode='cash' THEN
s.total_amount ELSE 0 END) AS cash_revenue,
        SUM(CASE WHEN s.payment_mode<>'cash' THEN
s.total_amount ELSE 0 END) AS digital_revenue
    FROM silver_sales s
    JOIN dim_stores st ON s.store_id = st.store_id
    WHERE s.sale_date = '{process_date}'
    GROUP BY s.sale_date, s.store_id, st.store_name,
             st.city, st.state, s.region_code
    """)

# Write to Gold (overwrite today's partition only)
df.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", f"sale_date = '{process_date}'")
\
    .saveAsTable("gold_store_performance")

# Optimize for Power BI Direct Lake (V-ORDER!)
spark.sql("OPTIMIZE gold_store_performance ZORDER BY
(store_id, sale_date)")

print(f"✅ Gold store_performance loaded for {process_date}")

# — RUN COMPLETE MEDALLION PIPELINE —————
def run_daily_medallion(process_date: str):
    print(f"\n{'='*60}")
    print(f"🚀 Medallion Pipeline: {process_date}")
    print(f"{'='*60}")

    # Bronze loads (from Files/ to Tables/)
    df_raw_sales =
spark.read.parquet(f"Files/raw/sales/{process_date}/")

```

```

load_bronze(df_raw_sales, "sales", process_date)

df_raw_customers =
spark.read.parquet(f"Files/raw/customers/{process_date}/")
load_bronze(df_raw_customers, "customers", process_date)

# Silver transforms
load_silver_sales(process_date)
load_silver_customers(process_date)

# Gold aggregations
load_gold_store_performance(process_date)
load_gold_customer_360(process_date)

print(f"\n✅ Complete! {process_date} processed
successfully")

run_daily_medallion(process_date)

```

◆ V-Order in Fabric

V-ORDER: Fabric's Secret Weapon for Power BI Performance

WHAT IS V-ORDER?

Microsoft's proprietary optimization applied to Parquet files

Similar to Delta's OPTIMIZE but specifically designed for

Direct Lake mode in Power BI

V-ORDER does:

→ Sorts data within each Parquet row group optimally

→ Applies additional compression (beyond standard Snappy)

```

# → Enables Power BI to skip unnecessary data pages
# → Result: 2-5x faster Power BI report loading!

# ENABLE V-ORDER when writing to Gold layer:
spark.conf.set("spark.sql.parquet.vorder.enabled", "true")
spark.conf.set("spark.microsoft.delta.optimizeWrite.enabled",
"true")

# Now writes are V-ORDER optimized
df_gold.write \
    .format("delta") \
    .mode("overwrite") \
    .saveAsTable("gold_store_performance")
# Files written with V-ORDER → Power BI Direct Lake is 3x faster!

# APPLY V-ORDER to existing table via OPTIMIZE:
spark.sql("OPTIMIZE gold_store_performance")
# If V-ORDER is enabled in Spark config → OPTIMIZE applies it!

# VERIFY V-ORDER on a table:
df_details = spark.sql("DESCRIBE DETAIL gold_store_performance")
display(df_details.select("format", "properties"))

# V-ORDER BEST PRACTICES:
# ✅ ALWAYS enable for Gold layer tables (Power BI reads these)
# ✅ Enable for Silver layer if analysts query directly
# ⚠️ Not needed for Bronze layer (raw ingestion – PBI doesn't
read this)
# ⚠️ Slight overhead during write – worth it for read
performance!

# OPTIMIZE WRITE (goes with V-ORDER):
spark.conf.set("spark.microsoft.delta.optimizeWrite.enabled",
"true")
# This tells Spark to write optimally-sized Parquet files

```



```
# Avoids the small file problem automatically during writes!

# COMPLETE GOLD WRITE CONFIGURATION:
spark.conf.set("spark.sql.parquet.vorder.enabled", "true")
spark.conf.set("spark.microsoft.delta.optimizeWrite.enabled",
"true")
spark.conf.set("spark.microsoft.delta.optimizeWrite.binSize",
"1073741824") # 1GB bin

df_gold.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", f"sale_date = '{process_date}'") \
    .option("vorder", "true") \
    .saveAsTable("gold_store_performance")

print("✅ Gold table written with V-ORDER optimization")
print("    Power BI Direct Lake will be significantly faster!")
```

11 Fabric Data Warehouse — Deep Dive

◆ Introduction to Data Warehousing in Fabric

WHAT IS A DATA WAREHOUSE?

Simple definition:

"A central repository of integrated data from one or more disparate sources, structured for fast analytical querying"

WHY SEPARATE DATA WAREHOUSE from Lakehouse?

Lakehouse: optimized **for** big data (Spark)
data engineers use this
complex transformations

Warehouse: optimized **for SQL** analytics (T-SQL)
business analysts use this
familiar **SQL** queries
structured, governed data

FABRIC WAREHOUSE **UNIQUE** FEATURES:

- ✓ T-SQL compatible (familiar **for SQL** developers!)
- ✓ Serverless (**no** cluster management)
- ✓ Built **on** Delta Lake + OneLake (**open** format)
- ✓ **Cross**-database queries (query Lakehouse + Warehouse together!)
- ✓ Automatic statistics (**no** manual ANALYZE **TABLE**)
- ✓ Direct Lake **for** Power BI (fastest reports)
- ✓ **Versioning** (Delta **time** travel **on** warehouse tables)
- ✓ Role-based security (**column/row** level)

◆ Data Warehouse Fundamentals

```
-- STAR SCHEMA for RetailMart Warehouse
-- (Industry standard for data warehouse design)
```

```
-- DIMENSION TABLES (describe the what/who/where/when)
CREATE TABLE [dim].[dim_date] (
    date_key          INT          NOT NULL,  -- surrogate key:
20240615
    full_date         DATE         NOT NULL,
    day_of_week       INT,
```

```

        day_name          VARCHAR(10),          -- Monday,
Tuesday...
        day_of_month      INT,
        month_number      INT,
        month_name        VARCHAR(10),          -- January,
February...
        quarter           INT,
        year              INT,
        is_weekend        BIT,
        is_holiday        BIT,
        fiscal_period     VARCHAR(10)
    );

CREATE TABLE [dim].[dim_customer] (
    customer_key          INT                NOT NULL,  -- surrogate key
(auto-increment)
    customer_id           VARCHAR(20)        NOT NULL,  -- natural/business
key
    customer_name         VARCHAR(200),
    email_masked          VARCHAR(200),        -- masked for
privacy!
    customer_tier         VARCHAR(20),
    city                  VARCHAR(100),
    state                 VARCHAR(100),
    region_code          VARCHAR(10),
    registration_date     DATE,
    is_active             BIT
);

CREATE TABLE [dim].[dim_product] (
    product_key           INT                NOT NULL,
    product_code          VARCHAR(50)        NOT NULL,
    product_name          VARCHAR(200),
    category              VARCHAR(100),
    sub_category          VARCHAR(100),

```

```

    brand            VARCHAR(100),
    cost_price       DECIMAL(18,2),
    list_price       DECIMAL(18,2)
);

CREATE TABLE [dim].[dim_store] (
    store_key        INT            NOT NULL,
    store_id         VARCHAR(20)    NOT NULL,
    store_name       VARCHAR(200),
    city            VARCHAR(100),
    state           VARCHAR(100),
    region_code     VARCHAR(10),
    store_type       VARCHAR(50),
    opening_date     DATE,
    manager_name     VARCHAR(200),
    sq_footage       INT
);

-- FACT TABLE (the measurements/events)
CREATE TABLE [fact].[fact_sales] (
    sale_key         BIGINT         NOT NULL, -- surrogate key
    date_key         INT            NOT NULL, -- FK to dim_date
    customer_key     INT,           -- FK to
dim_customer
    product_key      INT,           -- FK to dim_product
    store_key        INT,           -- FK to dim_store
    quantity         INT,
    unit_price       DECIMAL(18,2),
    gross_amount     DECIMAL(18,2),
    discount_amount  DECIMAL(18,2),
    net_amount       DECIMAL(18,2),
    gst_amount       DECIMAL(18,2),
    payment_mode     VARCHAR(50),
    transaction_time TIME
);

```

```
-- WHY SURROGATE KEYS?
-- Natural keys can change (customer moves → new address)
-- Surrogate keys NEVER change (just an auto-increment number)
-- Joins are on INT (fast!) not VARCHAR (slow!)
```

◆ Data Warehouses in Microsoft Fabric — Practical

```
-- CREATE FABRIC WAREHOUSE:
-- Workspace → + New → Warehouse
-- Name: WH_RetailMart_Analytics
```

```
-- FABRIC WAREHOUSE SQL FEATURES:
```

```
-- CREATE SCHEMA
```

```
CREATE SCHEMA dim;
CREATE SCHEMA fact;
CREATE SCHEMA staging;
CREATE SCHEMA reporting;
```

```
-- CREATE TABLE (same T-SQL as SQL Server!)
```

```
CREATE TABLE fact.fact_sales (
    sale_key      BIGINT IDENTITY(1,1),
    date_key      INT NOT NULL,
    customer_key  INT,
    product_key   INT,
    store_key     INT,
    gross_amount  DECIMAL(18,2),
    net_amount    DECIMAL(18,2),
    gst_amount    DECIMAL(18,2),
    quantity      INT
```

```

);

-- INSERT INTO (for staging loads)
INSERT INTO staging.stg_sales
SELECT
    CONVERT(INT, FORMAT(sale_date, 'yyyyMMdd')) AS date_key,
    c.customer_key,
    p.product_key,
    s.store_key,
    f.total_amount AS gross_amount,
    f.net_amount,
    f.gst_amount,
    f.quantity
FROM LH_Sales_Prod.dbo.silver_sales f      -- Lakehouse table!
JOIN dim.dim_customer c ON f.customer_id = c.customer_id
JOIN dim.dim_product p  ON f.product_code = p.product_code
JOIN dim.dim_store s    ON f.store_id = s.store_id
WHERE f.sale_date = CONVERT(DATE, '2024-06-15')

-- CROSS-DATABASE QUERY (AMAZING FEATURE!):
-- Query Lakehouse and Warehouse in SAME SQL query
SELECT
    w.store_name,
    w.city,
    l.gross_revenue AS lakehouse_revenue,      -- from Lakehouse
    SUM(f.gross_amount) AS warehouse_revenue -- from Warehouse
FROM WH_RetailMart_Analytics.fact.fact_sales f
JOIN WH_RetailMart_Analytics.dim.dim_store w ON f.store_key =
w.store_key
JOIN LH_Sales_Prod.dbo.gold_store_performance l -- Lakehouse!
    ON w.store_id = l.store_id
    AND l.sale_date = '2024-06-15'
WHERE CONVERT(DATE, CONVERT(VARCHAR, f.date_key)) = '2024-06-15'
GROUP BY w.store_name, w.city, l.gross_revenue

```

```
-- NO DATA MOVEMENT! Both live in OneLake. Query runs across both!
```

◆ Querying and Transforming Data

```
-- ANALYTICAL QUERIES IN FABRIC WAREHOUSE:
```

```
-- 1. MONTH OVER MONTH GROWTH:
```

```
WITH monthly_revenue AS (  
    SELECT  
        d.year,  
        d.month_number,  
        d.month_name,  
        SUM(f.gross_amount) AS revenue  
    FROM fact.fact_sales f  
    JOIN dim.dim_date d ON f.date_key = d.date_key  
    GROUP BY d.year, d.month_number, d.month_name  
)  
SELECT  
    year,  
    month_name,  
    revenue,  
    LAG(revenue) OVER (ORDER BY year, month_number) AS  
prev_month_revenue,  
    ROUND(  
        (revenue - LAG(revenue) OVER (ORDER BY year,  
month_number)) /  
        NULLIF(LAG(revenue) OVER (ORDER BY year, month_number),  
0) * 100,  
        1  
    ) AS mom_growth_pct
```

```

FROM monthly_revenue
ORDER BY year, month_number;

-- 2. TOP 10 PRODUCTS BY REVENUE:
SELECT TOP 10
    p.product_name,
    p.category,
    SUM(f.gross_amount) AS total_revenue,
    SUM(f.quantity) AS units_sold,
    ROUND(AVG(f.gross_amount / NULLIF(f.quantity, 0)), 2) AS
avg_unit_price,
    RANK() OVER (ORDER BY SUM(f.gross_amount) DESC) AS
revenue_rank
FROM fact.fact_sales f
JOIN dim.dim_product p ON f.product_key = p.product_key
JOIN dim.dim_date d ON f.date_key = d.date_key
WHERE d.year = 2024
GROUP BY p.product_name, p.category
ORDER BY total_revenue DESC;

-- 3. CUSTOMER COHORT ANALYSIS:
WITH customer_first_purchase AS (
    SELECT
        c.customer_key,
        MIN(d.year * 100 + d.month_number) AS cohort_month
    FROM fact.fact_sales f
    JOIN dim.dim_customer c ON f.customer_key = c.customer_key
    JOIN dim.dim_date d ON f.date_key = d.date_key
    GROUP BY c.customer_key
),
customer_activity AS (
    SELECT
        cf.cohort_month,
        d.year * 100 + d.month_number AS activity_month,
        COUNT(DISTINCT f.customer_key) AS active_customers

```



```
FROM fact.fact_sales f
JOIN dim.dim_date d ON f.date_key = d.date_key
JOIN customer_first_purchase cf ON f.customer_key =
cf.customer_key
GROUP BY cf.cohort_month, d.year * 100 + d.month_number
)
SELECT
    cohort_month,
    activity_month,
    active_customers,
    activity_month - cohort_month AS months_since_acquisition
FROM customer_activity
ORDER BY cohort_month, activity_month;
```

◆ SQL Endpoints and Warehouse Security

```
-- USING SQL ENDPOINT:
```

```
-- SQL Endpoint connection string (auto-provided):
-- Server: <workspace-id>.datawarehouse.fabric.microsoft.com
-- Database: WH_RetailMart_Analytics (or LH_Sales_Prod for
lakehouse)
-- Authentication: Azure Active Directory
```

```
-- Use in Power BI Desktop:
-- Get Data → Azure → Synapse Analytics (SQL DW)
-- Server: [above connection string]
-- Database: WH_RetailMart_Analytics
-- Mode: Import or DirectQuery
```

```
-- SECURITY IN WAREHOUSE:
```

```

-- Grant SELECT to BI team
GRANT SELECT ON SCHEMA::reporting TO [bi-team@retailmart.com];

-- Grant column-level access (hide sensitive columns):
-- (No direct column DENY in Fabric yet – use views!)

-- Create secure view (hide PAN/phone for BI team):
CREATE OR ALTER VIEW reporting.vw_customer_safe AS
SELECT
    customer_key,
    customer_name,
    LEFT(email, 3) + '***@***.com' AS email_masked,
    customer_tier,
    city,
    state,
    region_code
FROM dim.dim_customer;

-- Grant view access (not base table):
GRANT SELECT ON reporting.vw_customer_safe TO [bi-
team@retailmart.com];
DENY SELECT ON dim.dim_customer TO [bi-
team@retailmart.com];
-- BI team sees masked data. DE team sees everything.

-- ROW LEVEL SECURITY:
-- Regional managers see only their region
CREATE SCHEMA security;

CREATE FUNCTION security.fn_region_predicate(@region_code
VARCHAR(10))
RETURNS TABLE
WITH SCHEMABINDING
AS
RETURN SELECT 1 AS result

```

```
WHERE @region_code = USER_NAME()  -- user name must match region
code!

OR IS_MEMBER('dw_admin') = 1;

CREATE SECURITY POLICY fact.RegionSalesFilter
ADD FILTER PREDICATE security.fn_region_predicate(region_code)
ON fact.fact_sales
WITH (STATE = ON);

-- North India manager (username: 'N') sees ONLY North India
data!
-- Admin sees everything.
```

◆ Copying Data from Lakehouse to Warehouse

```
# COMMON PATTERN: Lakehouse (processing) → Warehouse (reporting)
```

```
# OPTION 1: Pipeline Copy Activity (simple, reliable)
# Create Pipeline in Fabric workspace
# Source: Lakehouse table (gold_store_performance)
# Sink: Warehouse table (reporting.store_performance)

# Copy Activity configuration:
# Source:
#   Lakehouse: LH_Sales_Prod
#   Table: gold_store_performance
#   Filter: WHERE sale_date = '@{pipeline().parameters.p_date}'
#
# Sink:
#   Warehouse: WH_RetailMart_Analytics
#   Schema: reporting
```

```

# Table: store_performance
# Write behavior: Upsert
# Key columns: [sale_date, store_id]

# OPTION 2: Cross-database INSERT (SQL approach)
# Run this in Fabric Warehouse SQL editor:
"""

INSERT INTO reporting.store_performance
SELECT *
FROM LH_Sales_Prod.dbo.gold_store_performance -- Lakehouse!
WHERE sale_date = '2024-06-15'
      AND NOT EXISTS (
          SELECT 1 FROM reporting.store_performance sp
          WHERE sp.sale_date = '2024-06-15'
                AND sp.store_id = gold_store_performance.store_id
      );
"""

# OPTION 3: Direct shortcut (best approach!)
# Create shortcut from Warehouse to Lakehouse table
# Warehouse → Tables → right-click → New shortcut
# → OneLake → select LH_Sales_Prod.Lakehouse →
gold_store_performance
#
# NOW: Warehouse has shortcut to Lakehouse table
# NO data movement!
# Power BI connects to Warehouse
# Warehouse queries Lakehouse data via shortcut
# Reports are always fresh!
# This is the recommended approach in Fabric!

```

◆ Getting and Processing Data in an Event Stream

REAL-TIME INTELLIGENCE WORKFLOW:

SOURCES → EVENTSTREAM → EVENTHOUSE (KQL DB) → QUERIES/ALERTS

WHAT IS EVENTSTREAM?

- Capture **real-time** data **from** multiple sources
- Process **and** route data **to** destinations
- **No** code required! Visual pipeline **for** streaming

CREATE EVENTSTREAM:

Workspace → + **New** → Eventstream
Name: ES_RetailMart_Payments

SOURCE TYPES:

- Azure Event Hubs (payment events **from** app)
- Azure IoT Hub (store IoT sensors)
- Kafka (**on**-premise streaming)
- Custom App (**any** REST push)
- Sample data (**for** learning!)

CONNECT SOURCE (Azure Event Hubs):

- + Source → Azure Event Hubs
- Namespace: retailmart-eventhubs
- Event Hub: payment-transactions
- Consumer **Group**: fabric-consumer
- Authentication: Connection String (**from** Key Vault)

PROCESS IN EVENTSTREAM (visual transforms):

- **Filter**: **WHERE** amount > 0
- Aggregate: tumbling **window** (1 min)
GROUP BY payment_mode → **SUM**(amount)
- Derived **column**:

```
fraud_risk = IF amount > 100000 AND hour < 5 THEN 'HIGH'
ELSE 'LOW'
```

DESTINATIONS:

- Eventhouse (KQL database – for real-time queries)
- Lakehouse (for batch/historical)
- Warehouse (for SQL analytics)
- Another Eventstream (fan-out)
- Custom endpoint (webhook)

◆ KQL Databases and Kusto Query Language

KQL DATABASE = Time-series optimized database

WHAT IS KQL?

Kusto Query Language = SQL-like language for time-series data

Much faster than SQL for time-series analytics

Used in: Azure Data Explorer, Log Analytics, Fabric RT Intel

WHY KQL FOR REAL-TIME?

Traditional SQL:

```
SELECT COUNT(*), payment_mode
FROM payments
WHERE timestamp BETWEEN '10:00' AND '10:05'
GROUP BY payment_mode
```

→ Might take 10+ seconds on large streaming table!

KQL:

```
payments
| where timestamp between (datetime(10:00) ..
datetime(10:05))
```

```
| summarize count() by payment_mode  
→ Returns in MILLISECONDS even on billions of rows!
```

KQL is designed for time-series from ground up!

KQL SYNTAX BASICS:

```
// — BASIC KQL QUERIES —————  
  
// SELECT first 10 rows (like SELECT TOP 10)  
payments  
| take 10  
  
// FILTER rows (like WHERE)  
payments  
| where amount > 10000  
| where payment_mode == "card"  
| where timestamp > ago(1h) // last 1 hour!  
  
// PROJECT = SELECT specific columns  
payments  
| project payment_id, customer_id, amount, timestamp  
  
// COUNT rows  
payments  
| count  
  
// SUMMARIZE = GROUP BY with aggregations  
payments  
| summarize  
    total_revenue = sum(amount),  
    transaction_count = count(),  
    avg_amount = avg(amount),  
    max_amount = max(amount)  
by payment_mode
```

```
// ORDER BY (sort)
payments
| summarize total = sum(amount) by region_code
| order by total desc

// TOP N
payments
| summarize total = sum(amount) by store_id
| top 10 by total desc

// — TIME-SERIES SPECIFIC —————

// Last 1 hour of transactions
payments
| where timestamp > ago(1h)
| count

// Group by time buckets (every 5 minutes)
payments
| where timestamp > ago(24h)
| summarize
    txn_count = count(),
    total_amount = sum(amount)
  by bin(timestamp, 5m) // 5-minute buckets!
| order by timestamp asc

// Revenue trend by hour
payments
| where timestamp between (ago(7d) .. now())
| summarize hourly_revenue = sum(amount) by bin(timestamp, 1h),
region_code
| render timechart // RENDERS CHART directly in KQL editor!

// High-value transactions in last 10 minutes
```



```
payments
| where timestamp > ago(10m)
| where amount > 100000
| project payment_id, customer_id, amount, timestamp, payment_mode
| order by amount desc
```

```
// — JOINS IN KQL —————
// Join payments with customer risk table
payments
| where timestamp > ago(1h)
| join kind=leftouter (
    customer_risk // another KQL table
    | project customer_id, risk_score, risk_level
) on customer_id
| where risk_score > 0.8 // high risk customers
| project payment_id, customer_id, amount, risk_score, risk_level
```

```
// — FRAUD DETECTION QUERY —————
payments
| where timestamp > ago(10m)
| extend is_high_value = amount > 100000
| extend hour_of_day = hourofday(timestamp)
| extend is_unusual_hour = hour_of_day < 5 or hour_of_day > 23
| extend fraud_risk_score =
    case(
        is_high_value and is_unusual_hour, 3.0,
        is_high_value, 2.0,
        is_unusual_hour, 1.0,
        0.0
    )
| where fraud_risk_score >= 2
| project payment_id, customer_id, amount, hour_of_day,
    fraud_risk_score, timestamp
| order by fraud_risk_score desc
```

13 Activators in Fabric

◆ Complete Guide to Data Activators

DATA ACTIVATOR = "If this happens → do that" for data

WHAT IT REPLACES:

Old way: custom Azure Function + Logic App + Power Automate

New way: Fabric Activator (no code, built-in!)

WHERE IT GETS DATA FROM:

- Eventstream (real-time)
- Power BI reports (detect when visual crosses threshold!)
- Lakehouse (poll on schedule)

WHAT IT CAN DO WHEN TRIGGERED:

- Send email
- Send Teams message
- Start a Fabric pipeline
- Call a webhook (custom action)
- Power Automate flow

CREATE ACTIVATOR:

Workspace → + New → Activator (or Reflex)

Name: ACT_RetailMart_Alerts

SCENARIO 1: Real-time fraud alert

Source: Eventstream (ES_RetailMart_Payments)

Object: payment_transactions table

Property to monitor: fraud_risk_score

Condition: fraud_risk_score >= 2.0

Trigger: for each event (real-time)

Action: Send email to fraud-team@retailmart.com

Body: "High-risk payment detected: {payment_id} for ₹{amount}"

SCENARIO 2: Revenue alert from Power BI

Source: Power BI Report (Sales Dashboard)

Object: Daily Revenue visual

Property: Total Revenue

Condition: Total Revenue FALLS BELOW ₹50,00,000 by 3 PM

Trigger: Once when condition first met

Action: Teams message to channel "Revenue-Alerts"

Body: "⚠ Revenue below target! Current: {revenue}"

SCENARIO 3: Inventory reorder trigger

Source: Lakehouse (poll every hour)

Object: gold_inventory_levels table

Property: current_stock

Condition: current_stock < safety_stock_level

Group by: product_code (detect per product!)

Action: Start Pipeline → PL_Create_Purchase_Order

Pass: {product_code}, {current_stock}, {safety_stock}

◆ Creating Activator Objects and Rules

ACTIVATOR CONFIGURATION WALKTHROUGH:

STEP 1: Create Activator item in workspace

STEP 2: Connect Data Source

→ Eventstream tab: Select your eventstream

→ OR: Power BI tab: Select report + visual

STEP 3: Define OBJECT

Object = the "thing" you're monitoring
e.g., each unique payment_id is an object

Property to track: amount, fraud_score, timestamp

Activate by: Real-time (each event) or Scheduled (hourly poll)

STEP 4: Define TRIGGER CONDITIONS

Simple condition:

Amount > 100,000

Pattern-based:

Amount increases by > 50% in last hour
(compares current value to value 1 hour ago)

Time-based:

Revenue NOT received by 2 PM for a store

Combined (AND/OR):

(Amount > 100,000) AND (hour_of_day < 5)

STEP 5: Define ACTIONS

Send email:

To: fraud-team@retailmart.com

Subject: 🚨 Fraud Alert: Payment {payment_id}

Body: Amount: ₹{amount}, Customer: {customer_id},

Time: {timestamp}, Risk: {fraud_score}

Teams message:

Team: RetailMart-Operations

Channel: Fraud-Alerts

Message: Suspicious payment detected [click to investigate]

Start pipeline:

Pipeline: PL_Investigate_Fraud

Parameters: payment_id={payment_id}

STEP 6: Activate

→ Toggle ON

→ Activator starts monitoring in real-time!

STEP 7: Monitor

Activator → History tab

→ See all triggers: when fired, what action taken

→ Success/failure of each action

14 Power BI in Fabric — Complete Integration Guide

◆ Connecting Microsoft Fabric with Power BI

POWER BI + FABRIC = Most powerful BI stack available

THREE MODES OF POWER BI + FABRIC:

1. IMPORT MODE (traditional):

PBI Desktop → copies data to PBI → fast queries

✗ Data gets stale (manual refresh or scheduled)

✗ Limited to ~1GB per model

✓ Fastest query performance

✓ Works offline

2. DIRECTQUERY MODE (traditional):

PBI → queries source every time user interacts

✓ Always fresh data

✗ Slow (every click = new query)

- ✗ Limited SQL support
- ✗ Data source must handle load

3. DIRECT LAKE MODE (NEW in Fabric!):

PBI reads Delta Parquet files directly from OneLake

- ✓ Always fresh (reads latest files)
- ✓ Fast (columnar Parquet + V-ORDER optimization)
- ✓ No data size limits (reads massive datasets!)
- ✓ No data movement or import
- ✓ Reduces storage cost (same data, no copy)
- ⚠ Requires Fabric capacity (not Power BI Pro)

◆ Creating Dashboards and Reports

POWER BI REPORT TYPES IN FABRIC:

1. REPORT (detailed, interactive):

Multiple pages, many visuals
Drill-through, cross-filter
Export to PDF/Excel
Embedded in Teams/SharePoint

2. DASHBOARD (high-level, real-time):

Single page
Pinned tiles from multiple reports
Real-time data tiles
Mobile-optimized

3. PAGINATED REPORT (pixel-perfect):

For printing/PDF export
Invoice, statement formats

Precise layouts

FABRIC-SPECIFIC REPORT CREATION:

- Workspace → LH_Sales_Prod → Semantic Model tab
- + **New** Report (opens browser-based Power BI!)
- Build report **in** Fabric browser (**no** desktop needed!)

BUILD A SALES DASHBOARD:

Page 1: Executive Summary

RetailMart Sales Dashboard - June 2024				
₹12.5Cr	Total Revenue	142,850	Transactions	
₹87.7K	Avg Ticket	500	Active Stores	
[Revenue by Region - Bar Chart]				
N:	35%	S:	28%	E: 20% W: 17%
[Daily Revenue Trend - Line Chart]				
Shows 30-day trend with target line				
[Top 10 Stores - Table]				
Rank	Store	City	Revenue	vs Target

Page 2: Product Analysis

Page 3: Customer Analytics

Page 4: Store Operations

DIRECT LAKE SEMANTIC MODEL (MOST POWERFUL):

- When** Lakehouse attached **to** Semantic Model:
- Gold layer tables automatically available
- **No** data movement!

- Reports ALWAYS show latest data
- Lakehouse updated at 2 AM → Report shows new data at 2:01 AM!

◆ All Power BI Desktop Connections with Fabric

CONNECTION TYPE 1: Power BI Desktop with Fabric Lakehouse

Power BI Desktop → Get Data → Microsoft Fabric → Lakehouse

OR via SQL endpoint:

Get Data → Azure → Azure Synapse Analytics (SQL DW)

Server: <workspace-id>.datawarehouse.fabric.microsoft.com

Database: LH_Sales_Prod ← your lakehouse name!

Authentication: Microsoft Account (Azure AD)

- See Tables/ as SQL tables
- Can join multiple tables
- Build semantic model
- Publish to Fabric workspace

CONNECTION TYPE 2: Power BI Desktop with OneLake

Get Data → Azure → Azure Data Lake Storage Gen2

URL: [https://onelake.dfs.fabric.microsoft.com/
<workspace-name>/<lakehouse-name>.Lakehouse/](https://onelake.dfs.fabric.microsoft.com/<workspace-name>/<lakehouse-name>.Lakehouse/)

Authentication: Organizational Account

- Browse OneLake like ADLS
- Read Parquet/CSV/JSON files directly
- Power Query (M) transforms available

CONNECTION TYPE 3: Power BI Desktop with Warehouse

Get Data → Microsoft Fabric → Warehouse

OR:

Get Data → SQL Server

Server: <workspace-id>.datawarehouse.fabric.microsoft.com

Database: WH_RetailMart_Analytics

- Full T-SQL query capability
- Can write custom SQL queries
- Best for complex transformations in SQL

CONNECTION TYPE 4: Power BI Desktop with Synapse

Get Data → Azure Synapse Analytics SQL

Server: <synapse-workspace>.sql.azuresynapse.net

Database: RetailMartDW (Dedicated Pool)

- Access Synapse Dedicated SQL Pool
- Use if you have existing Synapse setup

CONNECTION TYPE 5: Power BI Cloud with OneLake

From Power BI Service (browser):

New Report → Select data source

- OneLake data hub
- Browse all available Fabric items
- Select Lakehouse or Warehouse
- Build report directly in browser!
- No Power BI Desktop needed!

BEST PRACTICE RECOMMENDATION:

For large enterprise BI:

- Build Semantic Model in Fabric (Direct Lake)
- Power BI Desktop: build reports → publish to workspace
- Reports stay in Fabric workspace
- Users access via Power BI Service (browser)
- Mobile via Power BI app

This gives: fresh data + fast queries + central governance

◆ OneLake Connections with Power BI

ONELAKE DATA HUB – The central catalog:

Power BI Service → OneLake Data Hub (left nav)

- Shows ALL endorsed datasets across your Fabric tenant
- Filter by: workspace, type, endorsement
- Click any item → see description, tables, reports
- "Create Report" → start building immediately

ENDORSEMENT LEVELS:

No endorsement → just created, might be draft

Promoted → team recommends using this

Certified → officially validated by data team

RETAILMART SETUP:

Only certified Gold layer datasets for BI team

Silver: promoted (for analysts who know data)

Bronze: no endorsement (raw, not for general use)

This prevents BI team **from** accidentally **using** raw/uncleaned data **for** executive reports!

SENSITIVITY LABELS (Microsoft Purview integration!):

Each Fabric item can have:

- Public: anyone can see
- Internal: company employees **only**
- Confidential: restricted access
- Highly Confidential: very restricted

Power BI exports INHERIT sensitivity labels!

Download confidential data → file **is** labeled confidential

- Opens **with** appropriate restrictions **in** Windows
- Automatic data loss prevention (DLP)!

1 5 Azure DevOps — Repos in Fabric

◆ Complete DevOps Integration

FABRIC + AZURE DEVOPS = Enterprise-grade development

WHAT GETS VERSIONED IN GIT:

Item Type	Stored as
-----------	------------------

Notebooks	.py or .ipynb files
Pipelines	pipeline.json
Dataflows	dataflow.json
Semantic Models	.tmdl folder (new format)
Reports	report.json + pages

NOT versioned (*data*):

Lakehouse *data* ← just code *is* versioned, not *actual data*

Warehouse tables ← schema definition could be scripted

GIT BRANCHING STRATEGY *for* Fabric:

main branch → Production workspace

↑ PR + approval

dev/staging branch → Test workspace

↑ PR + code review

feature/* *branches* → *Development workspace*

DEVELOPER WORKFLOW:

1. Create feature branch from main:

git checkout -b feature/add-fraud-detection

2. Link Dev workspace to feature branch:

Workspace Settings → Git → Branch: feature/add-fraud-detection

3. Build notebook in Dev workspace

4. Changes auto-sync to feature branch in Git

5. Create Pull Request to dev/staging:

Azure DevOps → Repos → + New Pull Request

6. Code review:

Reviewer sees actual diff (Python code changes!)

Not just "notebook changed" – sees WHAT changed!

7. Merge to dev/staging → auto-deploys to Test workspace
(using Fabric deployment pipeline!)

8. Testing in Test workspace → approve

9. Create PR to main → merge → deploys to Production

CI PIPELINE (Azure DevOps):

On PR creation:

- ✓ Lint Python code (flake8)
- ✓ Run unit tests (pytest)
- ✓ Validate JSON files (pipelines, dataflows)
- ✓ Check for hardcoded secrets (security scan!)
- ✓ Check notebook outputs cleared (no data in notebooks!)

CD PIPELINE (Azure DevOps + Fabric Deployment):

On merge to staging branch:

- Deploy to Test workspace (automated)
- Run smoke tests

On merge to main:

- Manual approval gate (team lead approves)
- Deploy to Production workspace
- Send notification "Deployment successful"

◆ Integrating Fabric Components with Repos

PRACTICAL GIT SETUP:

STEP 1: Create Azure DevOps Repo

- dev.azure.com → Your Project → Repos
- Create: fabric-retailmart
- Create branches: main, dev, feature/*

STEP 2: Connect Dev Workspace to Git

Workspace → Settings → Git Integration
Provider: Azure DevOps
Org: retailmart-devops
Project: DataPlatform
Repo: fabric-retailmart
Branch: dev
Root folder: /workspaces/RetailMart_Sales/
→ Connect → Initial sync

STEP 3: Commit your first notebook

Make change to notebook
Workspace → Source Control (git icon)
→ Modified: "NB_Transform_Sales"
→ Add commit message
→ Commit → changes in Azure DevOps repo!

STEP 4: See what changed (magical!)

Azure DevOps → Repos → Commits → your commit
→ Click modified file
→ DIFF VIEW:

OLD: `.withColumn("gst", F.col("amount") * 0.18)`
NEW: `.withColumn("gst", F.round(F.col("amount") * 0.18, 2))`

Reviewer can see EXACTLY what logic changed!
No more "I changed the notebook" – see the PROOF!

DEPLOYMENT PIPELINE IN FABRIC:

Workspace → Deployment Pipelines (if admin created one)

Create Pipeline:

Stage 1: Development

→ Source: RetailMart_Sales_Dev workspace

Stage 2: Testing

→ Target: RetailMart_Sales_Test workspace

Stage 3: Production

→ Target: RetailMart_Sales_Prod workspace

DEPLOY PROCESS:

Diff shows what's different between stages

Deploy button → copies changed items

Rule: only DEFINITIONS copy, not DATA

Notebooks: code copies ✓

Lakehouse: schema can copy ✓

Lakehouse data: NEVER copies ✗ (stays in each environment)

16 SDLC and Agile Methodology

◆ Software Development Life Cycle in Fabric Projects

SDLC FOR FABRIC DATA PROJECTS:

PHASE 1: REQUIREMENTS GATHERING (Week 1-2)

Stakeholders:

→ Business: "We need daily store performance reports by 9 AM"

→ Data: "Source is Oracle DB + CSV files from FTP"

→ IT: "Compliance: no PII in reports for BI team"

Deliverables:

✓ Data dictionary (what each field means)

✓ Source-to-target mapping document

✓ Report mockups approved by business

✓ Data quality rules documented

✓ SLA definition (report ready by 9 AM = pipeline finishes by 8 AM)

PHASE 2: DESIGN (Week 3)

Architecture decisions:

- Lakehouse: bronze/silver/gold Medallion
- Warehouse: star schema for BI team
- Direct Lake: for Power BI reports
- Schedule: 1 AM pipeline start, done by 6 AM

Deliverables:

- ✓ Architecture diagram
- ✓ Star schema design (Fabric Warehouse)
- ✓ Lakehouse folder structure
- ✓ Pipeline orchestration design
- ✓ Security matrix (who sees what)

PHASE 3: DEVELOPMENT (Week 4-8)

Sprints:

- Sprint 1: Bronze ingestion (Oracle → Lakehouse Files)
- Sprint 2: Silver transformation (Files → Tables)
- Sprint 3: Gold aggregations + Warehouse load
- Sprint 4: Power BI reports + testing

Each sprint = 2 weeks

Daily standups (15 min)

Sprint review with business (demo!)

Sprint retrospective (what to improve)

PHASE 4: TESTING (Week 9-10)

Unit testing: individual notebook functions

Integration testing: end-to-end pipeline run

UAT: business validates report numbers

Performance testing: pipeline finishes by 6 AM?

Security testing: BI team cannot see PII?

PHASE 5: DEPLOYMENT (Week 11)

- Deployment pipeline: Dev → Test → Prod
- Parallel run (old + new system simultaneously)
- Business validates new reports match old reports
- Cut over: switch to new system
- Monitor first week closely

PHASE 6: MAINTENANCE (Ongoing)

- Monitor daily pipeline runs
- Handle failures (alerts → investigate → fix → deploy)
- Enhance based on business requests
- Monthly performance review

◆ Agile Methodology for Fabric Projects

AGILE SPRINT STRUCTURE:

SPRINT CADENCE (2 weeks):

Week 1, Day 1 (Monday):

SPRINT PLANNING (2 hours)

Team: Data Engineer 1, Data Engineer 2, Tech Lead

- Review backlog (user stories)
- Pick stories for this sprint
- Break into tasks (estimate in hours)
- Create Azure DevOps Work Items

Week 1-2 (Daily):

STANDUP (15 minutes, 10 AM)

Each person answers:

1. "Yesterday I completed: [task]"
2. "Today I'll work on: [task]"
3. "Blockers: [anything blocking me]"

NOT a status meeting! Just sync and unblock.

Week 2, Day 9 (Thursday):

SPRINT REVIEW (1 hour)

- Demo to business stakeholders
- Show working notebooks/reports
- Get feedback
- "Yes/No/Change" from business

Week 2, Day 10 (Friday):

SPRINT RETROSPECTIVE (45 min, team only)

Three questions:

- What went WELL? (keep doing)
- What needs IMPROVEMENT? (change)
- What should we TRY? (experiment)

Action items tracked in next sprint!

RETAILMART SPRINT EXAMPLE:

Sprint 3 User Stories:

Story 1: As a BI analyst, I want a daily store revenue report

Acceptance: shows all 500 stores, refreshes by 9 AM

Estimate: 3 days

Story 2: As a fraud team, I want real-time alerts when amount >

1L

Acceptance: alert within 60 seconds of transaction
Estimate: 2 days

Story 3: As a data engineer, I want pipeline monitoring dashboard

Acceptance: shows run status, duration, error count
Estimate: 1 day

Total: 6 days in 10-day sprint

Buffer: 4 days for bugs, meetings, reviews (40% buffer!)

40% buffer is CRITICAL – always underestimate in planning!

AZURE DEVOPS BOARDS for Fabric:

Project: RetailMart Fabric

Board: Kanban

Columns: Backlog | In Progress | Review | Done

Each task linked to:

- Git commit (proof of work!)
- Pull Request (code review)
- Test results

◆ Applying Agile Principles in Data Engineering

5 AGILE PRINCIPLES FOR DATA ENGINEERS:

1. WORKING SOFTWARE OVER DOCUMENTATION

- Deploy a pipeline that works, even if docs are incomplete
- Document as you go, not before you build
- Runnable notebook > 50-page spec document

2. CUSTOMER COLLABORATION OVER CONTRACTS

- Weekly demos to business team
- Get feedback early (before you build wrong thing!)
- Change direction based on feedback – it's OK!

RetailMart example:

Week 2: demo regional revenue chart to CFO

CFO: "I need this in ₹ Crores not ₹ Lakhs"

- Change in 10 minutes, not 3 weeks!

vs Waterfall: build for 6 months, CFO sees it, says "wrong format"

- Costly rework!

3. RESPONDING TO CHANGE OVER FOLLOWING PLAN

- Source system adds a new column → update schema (don't panic!)
- Business changes KPI definition → update notebook
- Agile: change is expected and welcomed
- Schema drift enabled for this reason!

4. WORKING DATA PRODUCT OVER COMPREHENSIVE PIPELINE

- Deliver simple, working version first (MVP)
- Enhance incrementally

Example:

Sprint 1: Load sales data, basic count dashboard (MVP!)

Sprint 2: Add customer dimension, tier analysis

Sprint 3: Add store dimension, geographic drill-down

Sprint 4: Real-time updates, fraud detection

Business gets VALUE from Sprint 1!

Not waiting 4 months for "complete" solution.

5. FREQUENT SMALL DEPLOYMENTS

- Deploy **every** sprint (**every 2** weeks)
- Small changes = small risk
- Git + Fabric Deployment Pipeline enables this!

vs once-a-quarter "big bang" deployment → high risk!

17 End-to-End Project: Migrating from On-Premises to Cloud

◆ Complete Migration Approach

RETAILMART: ON-PREMISES TO MICROSOFT FABRIC

CURRENT **STATE** (On-Premises):

Oracle DB → reports via **SSRS** (SQL Server Reporting)

SQL Server → Excel **extracts** (manual)

SAP ERP → CSV **exports** (manual)

Power BI Desktop → manual **refresh** (not published)

Problems:

- ✗ Reports ready at **11** AM (too late!)
- ✗ Manual **processes** (error-prone)
- ✗ No single source of truth
- ✗ Cannot **scale** (Oracle license cost exploding)
- ✗ No ML/AI capability

TARGET **STATE** (Microsoft Fabric):

All sources → Fabric **Lakehouse** (automated, midnight)

Lakehouse → Fabric **Warehouse** (dimensional model)

Warehouse → Power BI Direct **Lake** (reports ready by **6** AM!)

EventStream → Real-time fraud **alerts** (within seconds!)

ML model → Demand **forecasting** (Fabric Data Science)

MIGRATION APPROACH: LIFT, SHIFT, MODERNIZE

Phase 1: LIFT (replicate as-is)

- Copy Oracle data to **Lakehouse** (same structure)
- Validate row counts match
- No transformation yet
- Build confidence that data is correct

Phase 2: SHIFT (move workloads)

- Build Medallion architecture
- Bronze: raw Oracle data
- Silver: cleaned, standardized
- Gold: business-ready aggregations
- Parallel run: both old and **new systems**
- Business validates: **"same numbers?"**

Phase 3: MODERNIZE (improve)

- Better **performance** (Fabric vs Oracle **for** reporting)
- New **capabilities** (real-time, ML, AI)
- Better **governance** (column masking, audit logs)
- Self-service **BI** (business can build their own reports!)
- Decommission on-premises systems

◆ Data Ingestion Using Data Factory in Fabric

FABRIC PIPELINE: Migrate Oracle to Lakehouse

```

# Pipeline: PL_Migrate_Oracle_to_Fabric

# STEP 1: Lookup active tables from control table
# (same control table concept as ADF section!)

# STEP 2: ForEach table → Copy to Lakehouse Files/
# Source: Oracle (via on-premises data gateway)
# Sink: LH_Sales_Prod.Lakehouse/Files/raw/oracle/{table_name}/

# STEP 3: Notebook activity → Files to Tables
# Runs: NB_Load_Bronze_From_Files

# NB_Load_Bronze_From_Files:
from pyspark.sql import functions as F

# List all raw files for today
raw_files = mssparkutils.fs.ls(f"Files/raw/oracle/")

for file_info in raw_files:
    if file_info.isDir:
        table_name = file_info.name.rstrip("/")

        # Read from Files
        df_raw =
spark.read.parquet(f"Files/raw/oracle/{table_name}/")

        # Add audit columns
        df_bronze = df_raw \
            .withColumn("_source_system",
F.lit("Oracle_ECOM_Prod")) \
            .withColumn("_ingestion_ts",    F.current_timestamp())
\
            .withColumn("_process_date",    F.lit(process_date)) \
            .withColumn("_row_hash",        F.md5(F.concat_ws("|",
*df_raw.columns)))

```

```
# Write to Bronze table
df_bronze.write \
    .format("delta") \
    .mode("append") \
    .partitionBy("_process_date") \
    .option("mergeSchema", "true") \
    .saveAsTable(f"bronze_{table_name}")

print(f"✅ bronze_{table_name}: {df_bronze.count():,} rows")
```

◆ Data Processing Using Fabric Notebooks

```
# NOTEBOOK: NB_Transform_Silver_to_Gold
```

```
# Parameters
```

```
process_date = "2024-06-15"
```

```
from pyspark.sql import functions as F
```

```
from pyspark.sql.window import Window
```

```
from delta.tables import DeltaTable
```

```
# — READ SILVER TABLES —
```

```
df_sales = spark.read.table("silver_sales")
```

```
df_customers = spark.read.table("silver_customers")
```

```
df_products = spark.read.table("silver_products")
```

```
df_stores = spark.read.table("silver_stores")
```

```
# — BUILD GOLD: DAILY EXECUTIVE SUMMARY —
```

```
df_gold_exec = df_sales \
```



```

    .filter(F.col("sale_date") == process_date) \

.join(F.broadcast(df_customers.select("customer_id", "customer_tier",
    on="customer_id", how="left") \

.join(F.broadcast(df_products.select("product_code", "category", "brand",
    on="product_code", how="left") \

.join(F.broadcast(df_stores.select("store_id", "store_name", "region_code",
    on="store_id", how="left") \
    .groupBy(
        "sale_date", "region_code", "store_name",
        "customer_tier", "category"
    ) \
    .agg(
        F.sum("total_amount").alias("gross_revenue"),
        F.sum("net_amount").alias("net_revenue"),
        F.sum("gst_amount").alias("gst_collected"),
        F.count("sale_id").alias("transactions"),
        F.countDistinct("customer_id").alias("unique_customers"),
        F.avg("total_amount").alias("avg_ticket")
    ) \
    .withColumn("load_ts", F.current_timestamp())

# Write Gold (V-ORDER for Power BI performance!)
spark.conf.set("spark.sql.parquet.vorder.enabled", "true")
spark.conf.set("spark.microsoft.delta.optimizeWrite.enabled",
"true")

df_gold_exec.write \
    .format("delta") \
    .mode("overwrite") \
    .option("replaceWhere", f"sale_date = '{process_date}'") \
    .saveAsTable("gold_executive_summary")

```

```
# Optimize for BI queries
spark.sql("""
    OPTIMIZE gold_executive_summary
    ZORDER BY (region_code, sale_date, customer_tier)
""")

print(f"✅ Gold layer ready for {process_date}")
print(f"    Power BI reports will show {process_date} data!")

mssparkutils.notebook.exit(f"SUCCESS:{process_date}")
```

◆ Scheduling Jobs and Monitoring

SCHEDULING IN FABRIC:

METHOD 1: Pipeline Schedule (most common)

- Pipeline → Schedule → Daily at 1:00 AM IST
- Runs master pipeline PL_RetailMart_Daily_ETL
- All notebooks called from pipeline
- Centralized monitoring in one place

METHOD 2: Notebook Schedule (simple)

- Notebook → Run → Schedule
- For standalone notebooks
- Simple use cases

METHOD 3: Spark Job Definition Schedule

- Create Spark Job Definition (for .py files)
- Schedule like a pipeline
- Better for production Python scripts
- Can set specific cluster config

MONITORING IN FABRIC:

PIPELINE MONITORING:

Workspace → Pipeline → Run History

Run History					
Date	Time	Status	Duration	Triggered by	
2024-06-16	01:00	✓ Success	4h 23m	Schedule	
2024-06-15	01:00	✓ Success	3h 51m	Schedule	
2024-06-14	01:00	✗ Failed	0h 12m	Schedule	
2024-06-14	06:30	✓ Success	3h 45m	Manual (rerun)	

Click any run → see each activity:

- ✓ LKP_GetTables (2 sec) → 25 tables found
- ✓ FE_CopyTables (45 min) → 25 tables copied
- ✓ NB_TransformSilver (2h 15m) → 142,850 rows
- ✓ NB_BuildGold (1h 10m) → 12 gold tables
- ✓ SendEmail (1 sec) → notification sent

FABRIC CAPACITY METRICS APP:

Admin Portal → Capacity Metrics

- CU usage over time
- Which workspace uses most CUs
- Peak vs off-peak usage
- Throttling events
- Use this to right-size your capacity!

ALERTING:

Set up in Azure Monitor or Fabric Activator:

- Pipeline run failed → Teams message to DE team

→ Pipeline duration > 6 hours → alert (something's slow!)
→ CU usage > 80% capacity → scale up warning

◆ Implementing Reports Using Fabric

COMPLETE REPORTING SETUP:

STEP 1: Fabric Gold table ready (from pipeline)

gold_executive_summary, gold_store_performance,
gold_customer_360, gold_product_ranking

STEP 2: Create Semantic Model

Lakehouse → gold layer tables → New Semantic Model

Name: SM_RetailMart_Analytics

Include tables:

- ✓ gold_executive_summary
- ✓ gold_store_performance
- ✓ gold_customer_360
- ✓ dim_date (calendar dimension)

Define relationships:

gold_executive_summary[sale_date] → dim_date[full_date]

Define measures (DAX):

Total Revenue = SUM(gold_executive_summary[gross_revenue])

Revenue YoY = CALCULATE([Total Revenue],
SAMEPERIODLASTYEAR(dim_date[full_date]))
YoY Growth % = DIVIDE([Total Revenue] - [Revenue YoY], [Revenue
YoY]) * 100

STEP 3: Create Reports (in browser!)

Semantic Model → New Report

Build visuals (Direct Lake mode – no import!):

- Card: "₹12.5 Cr Total Revenue" (Total Revenue measure)
- Line Chart: Revenue trend by date
- Map: Revenue by state (bubble map)
- Bar Chart: Top 10 stores
- Table: Store performance with conditional formatting

STEP 4: Publish and Share

- Reports saved in Fabric workspace automatically
- Share with BI team (Viewer role)
- Teams integration: embed report in Teams channel
- Mobile: accessible via Power BI mobile app

STEP 5: Set Refresh (if using Import mode)

Semantic Model → Settings → Scheduled refresh

- Every day at 6:00 AM
- Report ready when business arrives at 9 AM!

(With Direct Lake: NO refresh needed!)

Data updated in Lakehouse → report shows it immediately!)

RESULT:

Pipeline runs 1 AM → data in Gold by 6 AM

Report opens at 9 AM → shows live data from 6 AM run

No manual refresh, no Excel download, no email attachments!

Business gets data in their browser, always fresh!

18 Resume Preparation

◆ Building a Strong Resume for Data Engineering Role

MICROSOFT FABRIC DATA ENGINEER RESUME:

TECHNICAL SKILLS SECTION (what to highlight):

Cloud Platforms:	Microsoft Azure, Microsoft Fabric
Data Engineering:	Azure Data Factory, Fabric Pipelines, Dataflow Gen2
Big Data:	Apache Spark, PySpark, Databricks, Synapse Analytics Spark Pool
Data Storage:	Azure Data Lake Gen2, OneLake, Delta Lake, Azure Blob Storage
Data Warehouse:	Fabric Data Warehouse, Azure Synapse Analytics, SQL Server
Streaming:	Azure EventStream, Fabric Real-time Intelligence, KQL
BI & Reporting:	Power BI (Direct Lake, DirectQuery, Import), DAX, Power Query
Programming:	Python, PySpark, SQL (T-SQL, Spark SQL), KQL (Kusto Query Language)
DevOps:	Azure DevOps, Git, CI/CD Pipelines, Fabric Deployment Pipelines
Governance:	Microsoft Purview, Sensitivity Labels, Column Masking, Row Security

STRONG RESUME BULLET POINTS:

✗ WEAK: "Worked with Microsoft Fabric"

✓ STRONG:

- Designed and implemented end-to-end Medallion Architecture (Bronze/Silver/Gold) in Microsoft Fabric Lakehouse, processing

500GB daily data from 6 source systems using PySpark and Delta Lake

→ Reduced report delivery time from 11 AM to 6 AM

- Built metadata-driven Fabric Pipeline that dynamically loaded 25+ Oracle database tables using control table pattern, replacing 25 separate pipelines with one reusable solution
 - Saved 40% pipeline development time
- Implemented Direct Lake Power BI semantic model on Fabric Gold layer, eliminating daily data refresh and enabling real-time report access for 200+ business users
 - Reduced BI infrastructure cost by 60%
- Developed real-time fraud detection using Fabric EventStream and KQL queries, detecting high-risk transactions within 60 seconds and triggering automatic alerts via Data Activator
 - Prevented estimated ₹50L monthly fraud losses
- Established CI/CD pipeline for Fabric notebooks using Azure DevOps and Fabric Deployment Pipelines across Dev/Test/Production environments with automated unit tests
 - Reduced deployment errors by 90%, deployment time from days to 30 minutes

PROJECTS TO MENTION:

RetailMart On-Premises to Fabric Migration:

- Scale: 500 stores, 50,000 daily transactions, 5 years history
- Tech: Fabric Pipelines + Lakehouse + Warehouse + Power BI
- Outcome: Reports delivered 5 hours earlier, 100% automated

19 Interview Guidelines

◆ Microsoft Fabric Interview Questions & Answers

TOP FABRIC INTERVIEW QUESTIONS:

Q1: What **is** Microsoft Fabric? How **is** it different **from** Azure Synapse?

A: Microsoft Fabric **is** a unified SaaS analytics platform launched **in 2023**

that integrates data engineering, data science, real-time analytics,

data warehousing, **and** business intelligence **in** one platform **on** top **of**

OneLake – a **single** storage layer.

Key differences **from** Synapse:

→ Fabric = SaaS (managed entirely **by** Microsoft, no infrastructure)

→ Synapse = PaaS (you manage some infrastructure **like** Dedicated Pools)

→ OneLake: Fabric has truly unified storage; Synapse has linked storage

→ Power BI: natively built **into** Fabric; Synapse **is** separate

→ Pricing: Fabric uses capacity-based (predictable); Synapse uses multiple meters

→ Direct Lake: Fabric's *unique Power BI mode (doesn't exist in Synapse)*

Q2: Explain OneLake **and** why it's *important*.

A: OneLake **is** Fabric's *unified storage layer – ONE logical data lake*

per Microsoft tenant (built **on** ADLS Gen2 underneath).

Importance:

- Eliminates data silos (everyone works on same data)
- Shortcuts: virtual links to external data without copying
- Open format (Delta Parquet) – no vendor lock-in
- All Fabric experiences (Lakehouse, Warehouse, Spark, Power BI)

read from SAME underlying files

- Zero data movement between experiences!

Impact: Reduced storage costs, eliminated data inconsistency, simplified architecture from 4 tools to 1 platform.

Q3: What is Direct Lake mode in Power BI?

A: Direct Lake is a new Power BI connection mode where Power BI reads

Delta Parquet files DIRECTLY from OneLake (Lakehouse/Warehouse storage).

vs Import: Import copies data to Power BI memory (fast but stale)

vs DirectQuery: Queries source on every click (fresh but slow)

Direct Lake: reads Parquet files natively

- Speed of Import (columnar Parquet + V-ORDER optimization)
- Freshness of DirectQuery (reads latest files)
- No size limits (can handle billions of rows)
- Requires Fabric capacity (not available in Power BI Pro)

Enable by: Creating semantic model on Lakehouse tables, Power BI automatically uses Direct Lake mode!

Q4: What is V-ORDER in Fabric?

A: V-ORDER is Microsoft's proprietary write-time optimization for

Parquet files **in** Fabric. It sorts data within Parquet row groups

optimally **and** applies additional compression beyond standard Snappy.

Enable: `spark.conf.set("spark.sql.parquet.vorder.enabled", "true")`

Impact: **2-5x** faster Power BI Direct Lake report loading

Use: Enable **for** ALL Gold layer tables (Power BI reads these)

Not needed **for**: Bronze layer (raw ingestion)

Q5: Explain Medallion Architecture **in** Fabric context.

A: Three-layer data architecture pattern:

BRONZE: Raw landing zone

- Exact copy **of** source data + audit metadata columns
- Append-only, never modified
- Stored **as** Delta tables **in** Lakehouse Tables/
- Purpose: audit trail, reprocessing capability

SILVER: Cleansed **and** conformed

- Data quality rules applied
- Standardized column names/types
- Basic business logic
- Joined **with** reference data
- Delta MERGE **for** upsert (**handles** SCD Type **1**)

GOLD: Business-ready aggregations

- Purpose-built **for** specific use cases (BI, ML, etc.)
- Denormalized **for** query performance
- V-**ORDER** enabled **for** Power BI Direct Lake
- ZORDER applied **for** analytical queries

In Fabric: All 3 layers in ONE Lakehouse (Tables/)
Power BI connects to Gold via Direct Lake!

Q6: What is a Shortcut in Microsoft Fabric?

A: Shortcut is a symbolic link feature in OneLake that creates a virtual reference to data stored elsewhere WITHOUT copying it.

Source types:

- Another OneLake location (different workspace/lakehouse)
- Azure Data Lake Storage Gen2
- Azure Blob Storage
- Amazon S3 or Google Cloud Storage

Use cases:

- Data Science team shortcuts to Data Engineering's *gold data*
- Warehouse shortcuts to Lakehouse tables (no data movement!)
- Bring external ADLS data into Fabric without migration

Benefit: Zero storage cost for shortcuts,
always fresh (reads original source)!

Q7: What is Dataflow Gen2 and how does it differ from Gen1?

A: Dataflow Gen2 is Power Query (M language) based data ingestion and transformation tool in Fabric.

vs Gen1:

- Gen2 can output directly to Fabric Lakehouse/Warehouse
- Gen2 uses Fast Copy backend (much faster for large data)
- Gen2 better monitoring and error handling
- Gen2 supports more sources (300+ connectors)
- Gen2 can split: some transforms in Power Query, some in

Spark

When to use:

- ✓ Non-coders who know Excel Power Query
- ✓ Simple ingestion + transformation
- ✓ Many connectors (Salesforce, SAP, etc.)
- ✗ Complex Spark transformations → use notebooks

Q8: How do you implement CI/CD for Fabric?

A: Three-component approach:

1. Git Integration (source control):

- Workspace → Settings → Git Integration → Azure DevOps
- All notebooks, pipelines, dataflows stored in Git as files
- Feature branch → PR → code review → merge

2. Azure DevOps CI Pipeline:

- On PR: lint Python, run unit tests, validate JSONs
- Ensures code quality before merge

3. Fabric Deployment Pipeline (built-in CD):

- Dev workspace → Test workspace → Prod workspace
- One-click deployment (or automated via REST API)
- Definitions deploy, not data
- Environment-specific config via parameter substitution

Full flow: commit → PR → review → merge → CI → deploy to Test → approve → deploy to Prod → notify team

Q9: What is Fabric Activator (Reflex)?

A: Activator is Fabric's no-code alerting and automation engine.

Monitors:

- EventStream (real-time data)
- Power BI reports (visual thresholds)
- Lakehouse data (scheduled polling)

Triggers **when**:

- Value crosses a threshold
- Value changes **by** more than X%
- Expected data doesn't *arrive*

Actions:

- Send email/Teams notification
- Start a Fabric pipeline
- **Call** a webhook

Replaces: Azure Logic Apps + Power Automate + Azure Functions
for data-driven automation scenarios.

Q10: How does Fabric handle real-time data?

A: Fabric Real-Time Intelligence stack:

1. EventStream: capture events **from Event** Hubs, IoT Hub, Kafka, **custom** apps. Route **to** destinations. Add transforms.
2. Eventhouse (KQL Database): time-series optimized database. Designed **for** millisecond query response **on** billions **of** events.
3. Kusto Query Language (KQL): SQL-**like** but time-series native.
100x faster than SQL **for event** analytics.
render keyword creates instant charts!
4. Real-time Dashboard: Live dashboards refreshing every few seconds.

Built **on** KQL queries.

5. Activator: triggers actions **when** real-time conditions met.

RetailMart example:

Payment API → EventHub → EventStream → Eventhouse

KQL query: detect fraud pattern **in** real-time (< **1** second latency)

Activator: fraud detected → Teams alert + block payment pipeline

20 Mock Interview Sessions

◆ Scenario-Based Technical Interview

MOCK INTERVIEW FORMAT:

Interviewer: "You're a senior data engineer at a retail company. The business wants near-real-time inventory alerts when stock falls below safety levels. Currently, inventory data is in on-premise SAP. Design the solution using Microsoft Fabric."

STRONG ANSWER STRUCTURE:

Step **1**: Clarify Requirements

"Before I design, let me ask:

- How near-real-time? (seconds, minutes, hours)
- What action when stock falls? (email, PO, block sale?)
- How often does SAP inventory update?

→ What's acceptable latency for alerts?"

(Asking requirements = senior behavior!)

Assume: 30-minute latency OK, email + Teams alert,
SAP updates every 30 minutes, 5-minute alert tolerance.

Step 2: Architecture Design

"Given these requirements, here's my design:

SAP (on-premise, 30-min batch updates)

|

On-premises Data Gateway (Fabric gateway on corp server)

|

Fabric Pipeline (every 30 minutes triggered)

→ Copy Activity: SAP → Lakehouse Files (CSV export via SAP connector)

|

Fabric Notebook (triggered by pipeline)

→ Read CSV from Files → transform → write to Delta table

→ Table: silver_inventory_levels

|

Fabric Activator (monitors silver table)

→ Object: each product_code

→ Property: current_stock

→ Condition: current_stock < safety_stock_level

→ Action 1: Email supply-chain@retailmart.com

→ Action 2: Send Teams to #inventory-alerts channel

→ Action 3: Trigger pipeline to create SAP purchase order

|

Power BI Direct Lake (dashboard for operations team)

→ Shows current inventory levels, flagged items

→ Refreshes when Spark writes new data (no scheduled refresh!)"

Step 3: Optimization and Alternatives

"If 30 minutes is too slow, I'd:

- Use SAP Change Document interface for CDC
- Stream via Kafka → Fabric EventStream
- KQL Database stores events
- Activator monitors EventStream in real-time
- Alert latency: < 60 seconds!"

Step 4: Risk Mitigation

"Key risks:

- On-premises Data Gateway: single point of failure
 - Install second gateway node for HA
- SAP extract performance: large table might be slow
 - Use delta extraction (only changed records)
- False alerts: safety stock might fluctuate
 - Add cooling period: 'only alert once per 4 hours per product'"

This answer shows:

- ✅ Requirements clarification (senior!)
- ✅ Architecture design (technical depth)
- ✅ Real-time alternatives (advanced knowledge)
- ✅ Risk awareness (experience)
- ✅ Practical constraints (not just theory)

BEHAVIORAL QUESTION:

"Tell me about a time a pipeline failed in production"

STRONG ANSWER (STAR format):

Situation: "At RetailMart, our nightly Oracle load failed one Monday at 2 AM, 3 hours before reports needed"

Task: "I was on-call and had to fix it before 9 AM business reports"

Action: "I checked Azure Monitor alerts – saw error:

'Oracle connection timeout'.
Checked SHIR status → Node 1 offline (server rebooted).
Connected to corporate VPN → restarted SHIR on Node 1.
Verified SHIR healthy → re-triggered failed pipeline run.
While pipeline reran, I checked: why did server reboot?
→ Windows Update ran at 2 AM (IT policy).
I added a second SHIR node for HA – future updates won't cause outage."

Result: "Pipeline completed by 5 AM.
Reports ready by 9 AM – business never knew!
Added monitoring alert for SHIR node count < 2.
No repeat outage since then."

KEY: Shows problem-solving, root cause analysis,
proactive improvement, calm under pressure!

Microsoft Fabric — Complete Summary

MICROSOFT FABRIC — COMPLETE GUIDE

FOUNDATION	OneLake (unified storage, Delta Parquet)
	Workspaces, Capacity (F SKUs), Shortcuts

DATA FACTORY	Pipelines (same as ADF!)
	Dataflow Gen2 (Power Query + Lakehouse)
	On-premises Data Gateway (replaces SHIR)
DATA ENGINEERING	Lakehouse (Files/ + Tables/ structure)
	Notebooks (PySpark, SQL, Scala, .NET Spark)
	Apache Spark (with Starter Pools!)
	MSSparkUtils/notebookutils
	All file formats (CSV, JSON, Parquet, Avro, ORC)
DELTA LAKE	ACID, MERGE/UPDATE/DELETE, Time Travel
	SCD Type 1 & 2, Streaming, Deduplication
	Medallion Architecture (Bronze/Silver/Gold)
	V-ORDER (Power BI optimization!)
DATA WAREHOUSE	T-SQL, Star Schema, Cross-DB queries!
	Row/Column security, Shortcuts from LH
	Semantic models, Power BI Direct Lake

REAL-TIME INTEL	EventStream, Eventhouse, KQL Database
	Kusto Query Language, Real-time Dashboards

ACTIVATOR	If-This-Then-That for data
	EventStream + Power BI + Lakehouse sources
	Email, Teams, Pipeline, Webhook actions

POWER BI	Direct Lake (NEW! Best of Import+DQ)
	All connection modes
(Desktop/Cloud/OneLake)	
	Sensitivity Labels, Endorsement

DEVOPS & SDLC	Git Integration (Azure DevOps/GitHub)
	Deployment Pipelines (Dev→Test→Prod)
	Agile sprints, CI/CD, Unit testing

E2E PROJECT	Oracle/SAP → Fabric Pipeline → Lakehouse
	Bronze→Silver→Gold Medallion
	Gold → Warehouse → Power BI Direct Lake
	Real-time: EventStream → KQL → Activator

|| INTERVIEW PREP | 15 Q&A, Scenario-based, STAR method
||
|| | Resume bullet points, Behavioral questions
||

THE BIG PICTURE:

Microsoft Fabric **is WHERE** the market **is** heading **in 2024-2026**.
If you know: ADF + Databricks + Synapse + Power BI already
Learning Fabric = applying same concepts **in ONE** unified platform.

Your competitive advantage:

- Know WHY **each** tool **exists** (not just HOW **to** use it)
- Understand OneLake **as** the foundation **of** everything
- Direct Lake = best Power BI performance (sell this **to** clients!)
- V-**ORDER** = **3x** faster BI (simple **to** implement, huge impact)
- **Real-time** Intelligence = modern streaming solution

Microsoft Fabric Course — 100% Complete!

Every concept from the PDF covered in depth: Big Data intro, Fabric platform overview, all components (OneLake, Data Factory, Data Engineering, Data Science, Real-time Intelligence, Warehouse, Power BI, Activator), Lakehouse architecture, all notebook types, Delta Lake operations (SCD1/SCD2, Time Travel, Medallion), V-ORDER, Fabric Warehouse with cross-database queries, KQL/EventStream/Eventhouse, Activator, all Power BI connection modes, DevOps/Git integration, Deployment Pipelines, SDLC, Agile methodology, E2E migration project, Resume preparation, Interview Q&A, and Mock Interview scenarios.

 **The complete Azure Data Engineering Course guide is now finished!** ADF → Databricks → Synapse → Microsoft Fabric — all covered with RetailMart real-world scenarios, senior engineer patterns, and practical code examples.

19/03/2026, 16:35:59